Appunti di Linguaggi Formali e Compilatori

Alessio Novel

Videolezioni....3
Introduzione....3
Grammatiche generative....6
Analisi lessicale....11
Espressioni regolari....12
Automi finiti non deterministici: NFA....17
Conversione di un'espressione regolare in un NFA....20
Automi finiti deterministici....29
Convertire un NFA in un DFA....32
La costruzione per sottoinsiemi....33
Dimensione contro velocità....35
Minimizzazione dei DFA....36
Proprietà dei linguaggi regolari....43
Pumping lemma per linguaggi regolari....44
Lexer e generatori di lexer....48
Flex....56
Analisi sintattica....57
Grammatiche libere dal contesto....58
Derivazione....61
Operatori di precedenza....63
Altre fonti di ambiguità....66
Proprietà dei linguaggi liberi (dal contesto)....68
Pumping lemma per linguaggi liberi....72
Analisi sintattica....79
Parsing predittivo....80
....82
Parsing predittivo rivisitato....83
....85
LL(1) parsing....87
Riscrivere una grammatica per il parsing....91
SLR parsing....99
Costruzione delle tabelle di parsing SLR....105
Utilizzare le regole di precedenza nelle tabelle di parsing LR....117
Analisi semantica....126
Ambiti (scopes) e Tabelle dei Simboli....126
Tabella dei simboli....127
Type checking....131

Lo spazio di progettazione dei tipi....131
Attributi....133
Ambienti per il type checking....134
Type checking delle espressioni....134
Type checking delle dichiarazioni di funzioni....136
Il type checking di un programma....137
Type checking avanzato....138
Analisi semantica....140
Alberi di sintassi astratta....158
Tabella dei simboli....165
Generazione codice intermedio....168
Scegliere un linguaggio intermedio....171
Il linguaggio intermedio....173
Syntax-directed translation....174
Esempi....174

Videolezioni

Videolezioni

Introduzione

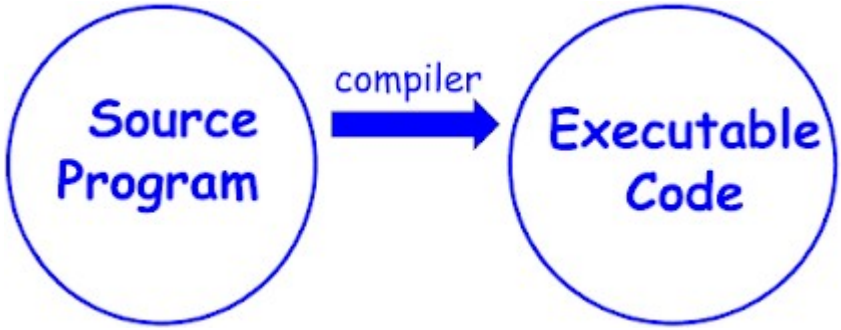
Gli obiettivi di questo corso comprendono l'acquisizione di conoscenze relative ai concetti, alle tecniche e agli algoritmi necessari per realizzare un compilatore. Inoltre, verrà approfondito lo studio dei linguaggi formali e delle metodologie per riconoscere un linguaggio formale.

I programmi di un computer sono scritti in un linguaggio detto **linguaggio macchina**. Visto che il linguaggio macchina è difficile e complesso, la programmazione viene effettuata tramite **linguaggi ad alto livello**.
Ovviamente serve un “ponte” tra i due linguaggi, visto che i computer capiscono solamente il linguaggio macchina, e questo ponte è il **compilatore**.

Usare un linguaggio di programmazione ad alto livello è vantaggioso per molti aspetti:

- La notazione è più vicina al modo in cui una persona pensa alla soluzione
- Il compilatore rileva facilmente errori “ovvi”
- I programmi risultano essere più corti
- I programmi sono portabili, ovvero possono essere facilmente tradotti in diversi linguaggi macchina

Ovviamente questo richiede un overhead di tempo dato dalla traduzione, ma un buon compilatore è in grado di eseguire la traduzione velocemente, soprattutto per programmi ben strutturati.



Scrivere un compilatore è una pratica molto complessa e lunga da effettuare. Infatti lo sviluppo di *gcc* sta andando avanti da più di 20 anni ormai e già 2 anni fa consisteva di più di 2 milioni di linee di codice.
Il processo di compilazione infatti trasforma poche linee di codice ad alto livello in tante linee di codice macchina e deve linkare tutte le librerie per poi generare il codice binario.

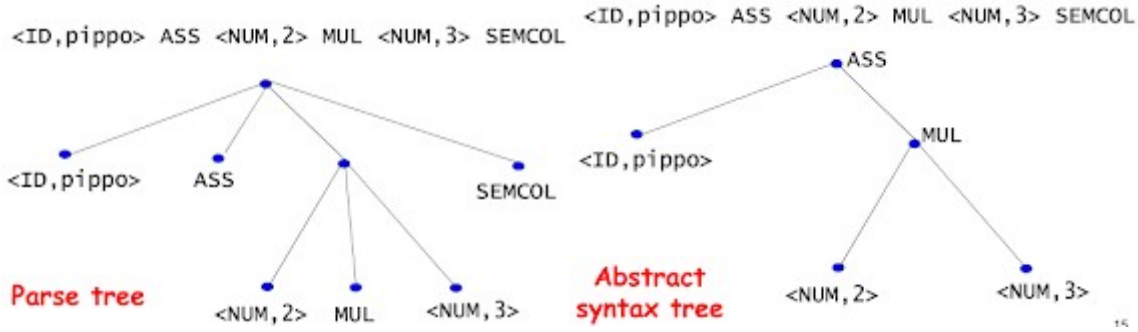
Per questo motivo è buona pratica strutturare il lavoro in diverse fasi e team.
Un ottimo modo per scrivere un compilatore è quello di dividere la compilazione in diverse fasi eseguite in sequenza.

Una comune divisione delle varie fasi è la seguente:

- **Lexical analysis**
  Il testo (scritto in una specifica grammatica) viene letto e diviso in **tokens** (gli elementi minimi/di base dell'analisi lessicale), ognuno dei quali corrisponde ad un simbolo nel linguaggio di programmazione (nome di una variabile, parola chiave, numero).

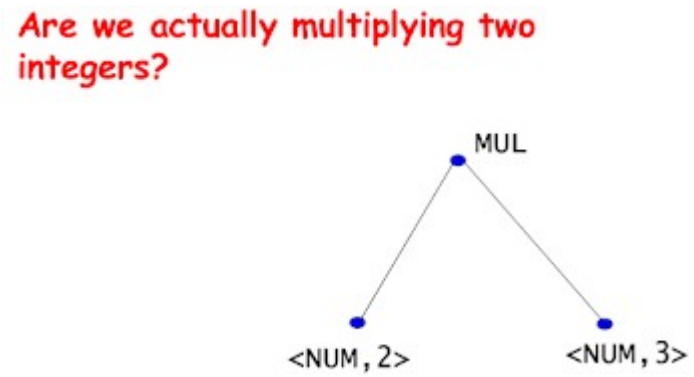
  - E.g. translates
  - pippo = 2*3;
  - Into
  - <ID,pippo> ASS <NUM,2> MUL <NUM,3> SEMCOL
  - ID,ASS,NUM,MUL,SEMCOL are tokens

- **Syntax analysis**
  In questa fase vengono presi i token generati dall'analisi lessicale e vengono organizzati in una struttura ad albero (**parse tree** o **abstract syntax tree**) che riflette la struttura del programma. Negli alberi creati c'è una nozione di parentesizzazione.
  Questa fase è anche chiamata **parsing**.
  Il testo iniziale può essere sintatticamente legale o meno (ad esempio non può usare keyword come nome delle variabili) e deve rispettare la grammatica (per un assegnamento la forma è: *IDENTIFIER ASSIGN EXPRESSION SEMICOLON*).
  Il compilatore deve accertarsi che il programma in input sia scritto correttamente in un determinato linguaggio, ovvero controllare che sia sintatticamente corretto.

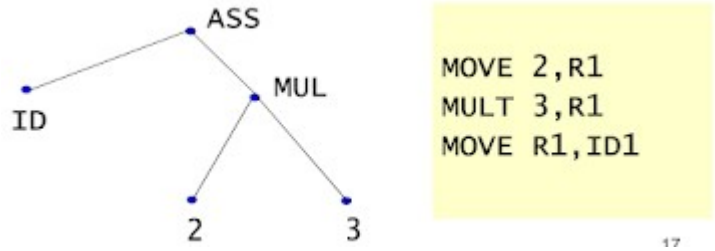


- **Semantic analysis** (o **type checking**)
  Questa fase analizza l'albero della sintassi per determinare se il programma viola determinati requisiti di consistenza, ad esempio se una variabile è stata usata senza essere stata dichiarata oppure se il tipo della variabile è usato in un contesto sbagliato.
  È in questa fase che un numero intero viene convertito in un numero reale nel caso di

una moltiplicazione tra reali.



- **Intermediate code generation**
  Il programma (l'albero) viene esplorato e tradotto in un semplice linguaggio macchina-indipendente intermedio.



- **Register allocation**
  I nomi delle variabili simboliche utilizzati nel codice intermedio vengono tradotti in numeri, ciascuno dei quali corrisponde a un registro nel codice macchina di destinazione.
- **Machine code generation** (o **target code generation**)
  Il linguaggio intermedio viene tradotto in linguaggio assembly (una rappresentazione testuale del codice macchina) per un'architettura di macchina specifica.
  I possibili target non sono solamente le macchine fisiche, ma possono anche essere macchine virtuali oppure un altro linguaggio (un esempio pratico consiste nel compilare da un linguaggio "homemade" a C per avere e sfruttare i vantaggi di gcc).
- **Assembly and linking**
  Il codice in linguaggio assembly viene tradotto in rappresentazione binaria e vengono determinati gli indirizzi di variabili, funzioni, ecc.

Il **frontend** consiste nelle prime tre fasi, ovvero analisi lessicale, analisi sintattica e analisi semantica. Poi c'è una parte intermedia che consiste nel codice intermedio (che andiamo comunque ad inserire nel frontend) e poi c'è il **backend** che consiste nell'allocazione dei registri, nella generazione del codice macchina e nell'assembly and linking.
Questa divisione viene fatta perchè:

- La traduzione da codice intermedio a codice macchina ha un grande impatto prestazionale sulla macchina su cui viene fatto
- Modularità: complessivamente con N frontend (linguaggi) e K backend (macchine) si ottengono $N * K$ compilatori, quindi è un modo per ottimizzare la fase di scrittura dei compilatori.

Ogni fase si affida alla fase precedente così da essere meno complessa, quindi ad esempio il controllo del tipo può presupporre l'assenza di errori di sintassi e la generazione del codice può presupporre l'assenza di errori di tipo.

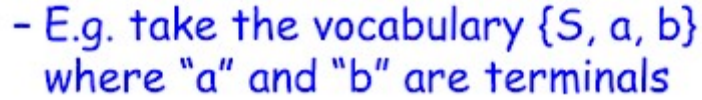
Generalmente nei corsi di compilatori ci si concentra principalmente sul frontend, questo perché trattare il backend richiederebbe, oltre ad essere vincolati da un riferimento ad un'architettura particolare, molto tempo.

Grammatiche generative

Le **grammatiche generative** (generative grammars) si chiamano così perché sono degli strumenti che permettono di generare dei linguaggi partendo da pochi caratteri. I linguaggi generati possono essere potenzialmente infiniti.

Non vedremo un linguaggio o una grammatica specifica e reale, ma tratteremo le nozioni in maniera generale, successivamente analizzeremo come è fatto un linguaggio e una grammatica realistica.

Un **linguaggio** è definito sopra un alfabeto (ad esempio i token prodotti dall'analisi lessicale). I simboli nell'alfabeto sono detti **terminali**.
Una grammatica generativa poi definisce ricorsivamente insiemi di stringhe, ognuno dei quali è denotato da un nome. Questi insiemi sono detti **non-terminali** (o variabili).
L'insieme di tutti i simboli è detto **vocabolario**.
Ovviamente l'insieme dei non-terminali è disgiunto dall'insieme dei terminali.



Uno dei simboli non-terminali viene scelto per denotare il linguaggio descritto dalla grammatica, e viene detto **simbolo di partenza** (start symbol).



Gli insiemi di stringhe sono descritti da un certo numero di **produzioni**. Ogni produzione descrive le possibili stringhe che sono contenute nell'insieme denotato da un non-terminale. Le produzioni possono essere viste come le regole per riscrivere delle stringhe in altre stringhe.
Ovviamente c'è il vincolo che ogni stringa da rimpiazzare deve contenere almeno un non-terminale.



Questi sono gli ingredienti che formano una **grammatica generativa**.
Il linguaggio generato da una grammatica generativa consiste nell'insieme di stringhe, composto da soli terminali, ottenute utilizzando solamente le regole di riscrittura (produzioni) di quella grammatica.

Per verificare la correttezza sintattica di un programma bisogna controllare che ogni parola appartenga al linguaggio generato dalla grammatica relativa (al linguaggio).

Dunque per ottenere un linguaggio generato bisogna partire dal simbolo di partenza e applicare le regole di riscrittura tante volte quanto è possibile e in ogni combinazione possibile. Ogni riscrittura è chiamata **passo di derivazione** (derivation step).

L'idea di base della derivazione, dunque, è considerare le produzioni come regole di riscrittura: ogni volta che abbiamo un non-terminale, possiamo sostituirlo con il righthand di qualsiasi produzione in cui il non-terminale appare sul lefthand.

Vediamo ora un esempio di derivazione (la notazione per indicare un passo di derivazione è ⇒):

$$\{S \rightarrow aSb,\ S \rightarrow ab\}$$

$$S \Rightarrow aSb \Rightarrow aabb$$

Questa è una derivazione a due passi (2-step derivation) e abbiamo ottenuto $aabb$, ovvero una stringa composta solamente da terminali. Dunque $aabb$ **appartiene** al linguaggio generato dalla grammatica data.

$$\{S \rightarrow aSb,\ S \rightarrow ab\}$$

$$S \Rightarrow aSb \Rightarrow aaSbb$$

In questa derivazione a due passi invece abbiamo ottenuto una stringa $aaSbb$ la quale contiene un non-terminale, dunque $aaSbb$ **non appartiene** al linguaggio generato dalla grammatica data.

Proviamo ora a descrivere il linguaggio che viene generato da una data grammatica:

$$\{S \rightarrow aSb,\ S \rightarrow ab\}$$

Questa grammatica genera il linguaggio $\{a^n b^n \mid n > 0\}$ (la notazione a^n significa n occorrenze di a), in quanto possiamo solamente aggiungere tante a quante b fino a che non ci fermiamo, e sicuramente ci sarà almeno una a e una b.

Finora abbiamo visto esempi facenti uso di vocabolari che contengono un solo non-terminale. Quando più simboli non-terminali vengono usati in un vocabolario, dobbiamo definire bene quale è lo start symbol. Per convenzione, se non viene specificato nulla, il simbolo non-terminale sul lefthand side della prima produzione è lo start symbol.

La notazione consiste nell'utilizzare le lettere maiuscole per i non-terminali e le lettere minuscole per i terminali.
Esiste poi un carattere speciale, ovvero ε, che viene usato per denotare la stringa vuota:

- La lunghezza di ε è 0
- $\varepsilon \equiv \varepsilon\,\varepsilon$

- $\varepsilon \equiv b^0$ per qualsiasi terminale b

Le produzioni che hanno la stringa vuota sul righthand side sono dette **produzioni vuote**.

Vediamo ora un'altra grammatica:

$$S \rightarrow aAb$$
$$aA \rightarrow aaAb$$
$$A \rightarrow \varepsilon$$

Analizzandolo un po' osserviamo che il linguaggio generato da questa grammatica è sempre $\{a^n b^n \mid n>0\}$.

Giungiamo dunque alla conclusione che **uno stesso linguaggio può essere generato da differenti grammatiche**, di conseguenza a livello pratico si può scegliere la grammatica più semplice o quella più "conveniente" (vedremo successivamente cosa si intende per conveniente, ovvero quando conviene usare una grammatica piuttosto che un'altra).

Ecco altri esempi di linguaggi generati da certe grammatiche:

Example 2

$S \rightarrow AB$
$A \rightarrow aA$
$A \rightarrow a$
$B \rightarrow Bb$
$B \rightarrow b$

- Generated language: $\{a^n b^m \mid n,m>0\}$

Example 3

$S \rightarrow aSBc$
$S \rightarrow abc$
$cB \rightarrow Bc$
$bB \rightarrow bb$

- Generated language: $\{a^n b^n c^n \mid n>0\}$

Example 4

$S \rightarrow AB$
$A \rightarrow a$

- Generated language: $\emptyset$

Example 5

$S \rightarrow \varepsilon$

- Generated language: $\{\varepsilon\}$
- $\{\varepsilon\} \neq \emptyset$

Notiamo da questi esempi che:

- Se un linguaggio non riesce a generare nessuna stringa che sia composta esclusivamente da caratteri non-terminali, allora il linguaggio generato consiste nell'insieme vuoto $\varnothing$

- Se invece un linguaggio generato è composto solamente dalla stringa vuota ε allora il linguaggio generato consiste in $\{\varepsilon\}$, che è **diverso dall'insieme vuoto**

Example 6

$S \rightarrow aSb$
$S \rightarrow \varepsilon$

- Generated language:
  - $\{a^n b^n \mid n>0\} \cup \{\varepsilon\} = \{a^n b^n \mid n \geq 0\}$

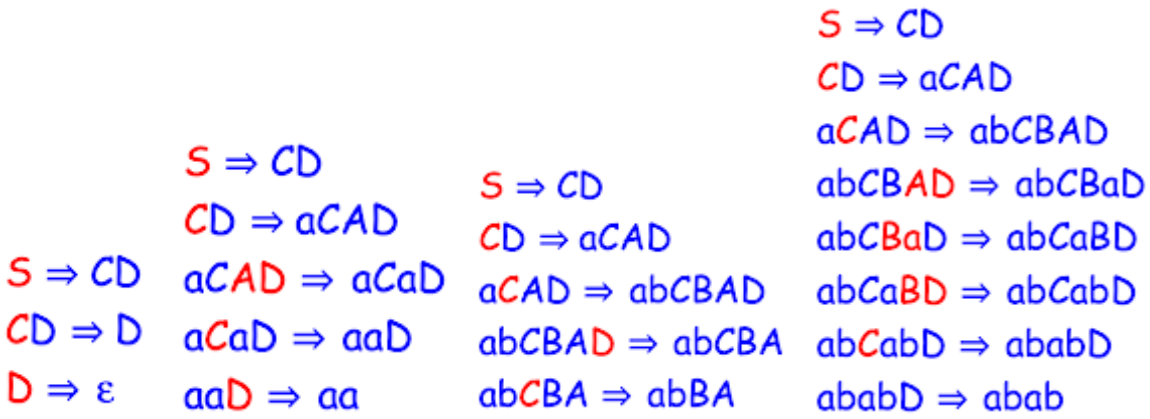
Da questo esempio vediamo che, vista la proprietà di ε, invece di usare l'operatore di unione, possiamo includere la stringa vuota ε al linguaggio semplicemente ampliando il range degli esponenti dei terminali, includendo lo 0.

Quando abbiamo più produzioni con lo stesso lefthand side possiamo unire le produzioni usando l'abbreviazione di pipe |, che significa proprio che lo stesso lefthand side può essere derivato in diverse maniere.

Vediamo ora un esempio di grammatica più complessa e proviamo a dire quale è il linguaggio generato:

$S \rightarrow CD$
$C \rightarrow aCA \mid bCB$
$AD \rightarrow aD$
$BD \rightarrow bD$
$Aa \rightarrow aA$
$Ab \rightarrow bA$
$Ba \rightarrow aB$
$Bb \rightarrow bB$
$C \rightarrow \varepsilon$
$D \rightarrow \varepsilon$

Ed ecco alcuni esempi di derivazione:



Il linguaggio generato da questa grammatica è

$$\{ww \mid w \text{ è una qualsiasi stringa composta dai caratteri } a \text{ e } b\}$$

Abbiamo visto in quest'ultimo esempio che può succedere di intraprendere una combinazione di passi di derivazione che ci porti ad una stringa che possiede dei caratteri non-terminali ma a cui non possiamo applicare ulteriori produzioni. In quel caso dobbiamo eseguire backtrack e scegliere altre produzioni.

In maniera formale, una **grammatica** è una **tupla** (V, T, S, P):

- V: il vocabolario, composto da caratteri sia terminali sia non-terminali
- T: l'insieme dei caratteri terminali
- S: lo start symbol (scelto dall'insieme V ¿)
- P: l'insieme delle produzioni

Le **grammatiche** dunque sono sempre **finite** (consistono in una tupla ben definita), però nonostante questo, come abbiamo già detto, esse possono comunque generare **linguaggi infiniti**.

La notazione dei simboli è la seguente:

- Lettere maiuscole all'inizio dell'alfabeto $A, B, \ldots$
  Rappresentano caratteri non-terminali (appartengono all'insieme V ¿)
- Lettere maiuscole alla fine dell'alfabeto $X, Y, \ldots$
  Rappresentano qualsiasi carattere singolo, sia terminale che non-terminale (appartengono all'insieme V)
- Lettere minuscole all'inizio dell'alfabeto $a, b, \ldots$
  Rappresentano caratteri terminali (appartengono all'insieme T)
- Lettere minuscole all'inizio dell'alfabeto greco $\alpha, \beta, \ldots$
  Rappresentano stringhe composte da zero o più elementi del vocabolario, sia terminali che non-terminali (appartengono all'insieme V^*)
- $w, w_0, \ldots$ con w che sta per word
  Rappresentano stringhe composte da zero o più terminali (appartengono all'insieme T^*)

Una **produzione** ha la forma

$$\delta \rightarrow \alpha$$

dove δ appartiene all'insieme V^+ (stringa composta da uno o più elementi del vocabolario, sia terminali che non-terminali) e **deve** contenere almeno un non-terminale.
δ è detto **driver** (conduttore) della produzione, α è detto **body** (corpo) della produzione.

Il **linguaggio generato** dalla grammatica $G=(V,T,S,P)$ è denotato da $L(G)$:

$$L(G)=\{w \mid w \in T^* \ \&\& \ S \Rightarrow^* w\}$$

Usiamo T^* perchè w può essere equivalente a ε.
La notazione $\Rightarrow^*$ significa che usiamo zero o più passi di derivazione.

Vediamo ora la **gerarchia** delle grammatiche. Le grammatiche possono essere classificate e divise in sottoclassi in base alla forma delle produzioni.
Nelle **grammatiche context-free** o semplicemente **grammatiche libere** ogni produzione ha la forma

$$A \rightarrow \alpha$$

Ovvero il driver è un carattere non-terminale invece il body è una stringa qualsiasi.
Le grammatiche context-dependent sono più difficili da analizzare.
Nel caso dei linguaggi di programmazione si fa riferimento ad un sottoinsieme delle grammatiche libere, ovvero quelle che possono essere analizzate in complessità lineare.

Un **linguaggio** L è detto **context-free** (o libero) *se e solo se* esiste una grammatica libera G tale che

$$L=L(G)$$

Dunque basta trovare una grammatica libera che generi il linguaggio per poter dire che il linguaggio è libero. Noi ci concentreremo sui linguaggi liberi (per l'analisi sintattica) e su un sottoinsieme di essi, ovvero i linguaggi regolari (per l'analisi lessicale).

Il vantaggio di usare grammatiche e linguaggi context-free è il poter trattare con facilità le **derivazioni canoniche** e gli **alberi di derivazione** (vedremo questi concetti più avanti)

Uno **step di derivazione rightmost** (o **leftmost**) è uno step di derivazione in cui viene applicata una produzione al simbolo non-terminale più a destra possibile (o più a sinistra).
Una **derivazione canonica** di una parola di un linguaggio consiste in una derivazione in cui tutti gli step di derivazione sono rightmost (oppure tutti leftmost).
L'utilità delle derivazioni canoniche è che si ha a disposizione un modo per confrontare delle possibili derivazioni tra di loro.

Analisi lessicale

La parola "lessicale" nel senso tradizionale significa "relativo alle parole". In termini di linguaggi di programmazione, le parole sono oggetti come nomi di variabili, numeri, parole chiave, ecc. Tali parole sono tradizionalmente chiamate **token**.

Un **analizzatore lessicale**, o in breve **lexer**, prenderà come input una stringa di singole lettere e dividerà questa stringa in token. Inoltre, filtrerà tutto ciò che separa i token (il cosiddetto spazio bianco), ovvero i caratteri di layout (spazi, caratteri di nuova riga, ecc.) e i commenti.

Lo scopo principale dell'analisi lessicale è quello di facilitare la vita alla successiva fase di analisi sintattica. In teoria, il lavoro svolto durante l'analisi lessicale può essere reso parte integrante dell'analisi sintattica, e nei sistemi semplici ciò avviene spesso. Tuttavia, ci sono delle ragioni per mantenere le fasi separate:

- **Efficienza**
  Un lexer può eseguire le parti semplici del lavoro più velocemente di quanto possa fare un parser più generale. Inoltre, la dimensione di un sistema diviso in due può essere inferiore a quella di un sistema combinato. Ciò può sembrare paradossale ma, come vedremo, c'è in gioco un fattore non lineare che può rendere un sistema separato più piccolo di un sistema combinato.
- **Modularità**
  La descrizione sintattica del linguaggio non deve essere ingombrata da piccoli dettagli lessicali come spazi bianchi e commenti.
- **Tradizione**
  I linguaggi sono spesso progettati tenendo presente fasi lessicali e sintattiche separate, e i documenti standard di tali linguaggio tipicamente separano gli elementi lessicali e sintattici dei linguaggi.

Di solito non è molto difficile scrivere a mano un lexer: prima leggi oltre lo spazio bianco iniziale, poi, in sequenza, provi per vedere se il token successivo è una parola chiave, un numero, una variabile o quant'altro. Tuttavia, questo non è un ottimo modo per gestire il problema: potresti leggere ripetutamente la stessa parte dell'input mentre provi ogni possibile token e in alcuni casi potrebbe non essere chiaro dove finisce il token. Inoltre, un lexer scritto a mano può essere complesso e difficile da mantenere. Pertanto, i lexer sono normalmente costruiti da **generatori di lexer**, che trasformano le specifiche leggibili dall'uomo di token e spazi bianchi in programmi efficienti.
Vedremo la stessa strategia generale nel capitolo sull'analisi sintattica: le specifiche in una notazione ben definita e leggibile dall'uomo vengono trasformate in programmi efficienti.
Per l'analisi lessicale, le specifiche sono tradizionalmente scritte utilizzando **espressioni regolari**: una notazione algebrica per descrivere insiemi di stringhe. I lexer generati appartengono a una classe di programmi estremamente semplici chiamati **automi finiti**.

Questo capitolo descriverà le espressioni regolari e gli automi finiti, le loro proprietà e come le espressioni regolari possono essere convertite in automi finiti. Infine, discutiamo alcuni aspetti pratici dei generatori di lexer.

Espressioni regolari

L'insieme di tutte le costanti intere o l'insieme di tutti i nomi delle variabili sono insiemi di stringhe, in cui le singole lettere sono prese da un particolare alfabeto. Un tale insieme di

stringhe è chiamato linguaggio. Per gli interi, l'alfabeto consiste delle cifre 0-9 e per i nomi delle variabili l'alfabeto contiene sia lettere che cifre (e forse qualche altro carattere, come il carattere di sottolineatura).

Dato un alfabeto, descriveremo insiemi di stringhe mediante **espressioni regolari**, una notazione algebrica compatta e facile da usare e comprendere per gli esseri umani. L'idea è che le espressioni regolari che descrivono semplici insiemi di stringhe possano essere combinate per formare espressioni regolari che descrivono insiemi di stringhe più complessi.

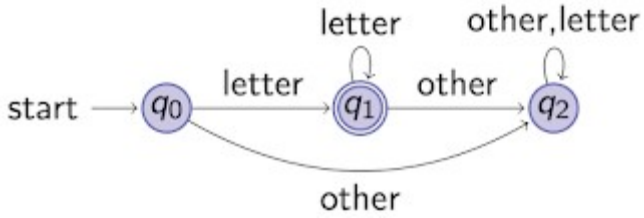
Prendiamo per esempio il linguaggio $L = \{ a^n b^n \mid n > 0 \}$, che è un linguaggio libero.
Uno stack sembra una struttura ideale per riconoscere le parole appartenenti a L:

- Leggi la stringa un simbolo alla volta
- Esegui il push delle a sullo stack
- Esegui il pop per ogni b che incontri e rifiuta ulteriori a
- Controlla se lo stack è vuoto

Il punto è: abbiamo davvero bisogno di tutto quel “conteggio” per generare, ad esempio, stringhe arbitrarie sull'alfabeto inglese?
La risposta è no, infatti, ad esempio, data la grammatica con alfabeto quello inglese e le seguenti produzioni:

$$S \to a \mid b \mid \ldots \mid z \mid aS \mid bS \mid \ldots \mid zS$$

è sufficiente utilizzare una macchina a stati per generare e per verificare le parole appartenenti al linguaggio:



Una **grammatica regolare** è una grammatica libera che ha solamente produzioni nelle seguenti forme:

- $A \to a$
- $A \to aB$
- $A \to \varepsilon$

Dunque le grammatiche regolari sono una specializzazione (sottoinsieme) delle grammatiche libere.

Un'**espressione regolare** è una sequenza di caratteri che definisce un insieme di stringhe. Le espressioni regolari vengono spesso utilizzate per cercare o manipolare testi in modo flessibile.

Le grammatiche regolari e le espressioni regolari sono correlate in quanto entrambe sono utilizzate per descrivere e generare **linguaggi regolari**.

Un linguaggio regolare è un insieme di stringhe che può essere descritto o riconosciuto da un automa a stati finiti o da un'espressione regolare.

Gli **automi a stati finiti** possono essere:

- Deterministici
- Non deterministici

Vediamo meglio le **espressioni regolari**.
Fissiamo un alfabeto A ed un numero di operatori. Le espressioni regolari sono definite induttivamente nella seguente maniera:

- **Casi base**
  - Ogni $a \in A$ è un'espressione regolare
  - ε è un'espressione regolare
- **Passo induttivo**: Se r_1 e r_2 sono espressioni regolari, allora
  - *Alternanza*: $r_1 \mid r_2$ (letto $r_1 \vee r_2$) è un'espressione regolare
  - *Concatenazione*: $r_1 \cdot r_2$ (letto $r_1 r_2$) è un'espressione regolare
  - *Stella di Kleen*: r_1^* (letto r_1 *star*) è un'espressione regolare
  - *Parentesi*: (r_1) è un'espressione regolare

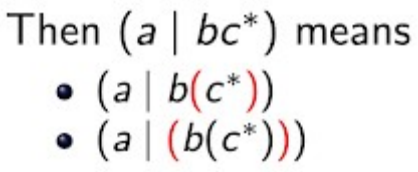
Vediamo ora quale è il **linguaggio denotato** da un'espressione regolare.
Data un'espressione regolare r sull'alfabeto A, il linguaggio denotato da r, scritto $L(r)$, è definito anch'esso induttivamente sulla struttura di r:

- **Casi base**
  - $L(a) = \{a\}$ per ogni $a \in A$
    Ovvero l'insieme costituito dalla stringa di una lettera a.
  - $L(\varepsilon) = \{\varepsilon\}$
    Ovvero il linguaggio contenente la stringa vuota.
- **Passo induttivo**
  - Se $r = r_1 \mid r_2$, allora $L(r) = L(r_1) \cup L(r_2)$
    Ovvero l'insieme costituito dalle stringhe appartenenti ad entrambi i linguaggi.
  - Se $r = r_1 r_2$, allora $L(r) = \{w_1 w_2 \mid w_1 \in L(r_1) \wedge w_2 \in L(r_2)\}$
    Ovvero l'insieme costituito dalle stringhe costruite concatenando una stringa del primo linguaggio con una stringa del secondo linguaggio.
  - Se $r = r_1^*$, allora $L(r) = \{\varepsilon\} \cup \{w_1 w_2 \ldots w_k \mid k \geq 1 \wedge \forall i : 1 \leq i \leq k,\ w_i \in L(r_1)\}$
    Ovvero l'insieme costituito dalla concatenazione di un numero qualsiasi di stringhe nel linguaggio di r_1.
  - Se $r = (r_1)$, allora $L(r) = L(r_1)$

Quando combiniamo diversi simboli di costruzione non è chiaro a priori come siano raggruppate le diverse sottoespressioni. Possiamo sempre usare le parentesi per rendere esplicito il

raggruppamento di simboli, esistono però delle **regole di precedenza**. La convenzione è la seguente:

- La stella Kleene ha la massima precedenza ed è associativa a sinistra.
- La concatenazione ha la seconda precedenza più alta ed è associativa a sinistra e distributiva sull'alternanza.
- L'alternanza ha la precedenza più bassa ed è commutativa e associativa a sinistra.



Spesso useremo abbreviazioni aggiuntive per comodità. Se vogliamo descrivere costanti intere non negative, possiamo farlo dicendo che si tratta di una o più cifre, che viene espresso dalla espressione regolare

$$(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)(0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9)^*$$

Il gran numero di diverse cifre rende questa espressione piuttosto prolissa. Peggiora ancora quando arriviamo ai nomi delle variabili, dove dobbiamo elencare tutte le lettere dell'alfabeto (sia maiuscole che minuscole).

Pertanto, introduciamo un'abbreviazione per insiemi di lettere. Sequenze di lettere tra parentesi quadre rappresentano l'insieme di queste lettere. Ad esempio, usiamo $[ab01]$ come abbreviazione per $a \mid b \mid 0 \mid 1$. Inoltre, possiamo utilizzare la notazione degli intervalli - per abbreviare $[0123456789]$ a $[0\text{-}9]$. Possiamo combinare diversi intervalli all'interno di una parentesi quadra e ad esempio scrivere $[a\text{-}zA\text{-}Z]$ per indicare tutte le lettere dell'alfabeto sia minuscole che maiuscole.

Quando utilizziamo gli intervalli, dobbiamo essere consapevoli dell'ordine dei simboli coinvolti. Per le cifre e le lettere usate sopra, di solito non c'è confusione. Tuttavia, se scriviamo, ad esempio, $[0\text{-}z]$ non è immediatamente chiaro cosa si intenda. Quando si utilizza tale notazione nei generatori di lexer, di solito si utilizzano insiemi di caratteri ASCII standard o ISO 8859-1, con l'ordine di simboli implicito. Per evitare confusione, utilizzeremo la notazione degli intervalli solo per intervalli di cifre o lettere dell'alfabeto.

Tornando all'esempio delle costanti intere sopra, ora possiamo scriverlo molto più brevemente come

$$[0\text{-}9][0\text{-}9]^*$$

Dato che s^* denota zero o più occorrenze di s, abbiamo dovuto scrivere l'insieme di cifre due volte per descrivere che sono consentite una o più cifre. Una tale ripetizione non nulla è piuttosto comune, quindi introduciamo un'altra abbreviazione, s^+, per indicare una o più occorrenze di s. Con questa notazione, possiamo abbreviare la nostra descrizione delle costanti intere a

$$[0\text{-}9]^+$$

Su una nota simile, è comune poter avere zero o un'occorrenza di qualcosa (ad esempio, un segno opzionale per un numero). Pertanto, introduciamo l'abbreviazione $s?$ per $s \mid \varepsilon$.
I simboli + e *?* hanno la stessa precedenza di *.

Dobbiamo sottolineare che queste abbreviazioni sono proprio questo. Non aggiungono nulla all'insieme di lingue che possiamo descrivere, ma rendono possibile descrivere una lingua in modo più compatto. Nel caso di s^+, può persino fare una differenza esponenziale: se + è annidato in profondità n, l'espansione ricorsiva di s^+ a $s\ s^*$ produce $2^n - 1$ occorrenze di * nell'espressione regolare espansa.

$(r\|s)\|t = r\|s\|t = r\|(s\|t)$	| is associative
$s\|t = t\|s$	| is commutative
$s\|s = s$	| is idempotent
$s? = s\|\varepsilon$	by definition
$(rs)t = rst = r(st)$	concatenation is associative
$s\varepsilon = s = \varepsilon s$	ε is a neutral element for concatenation
$r(s\|t) = rs\|rt$	concatenation distributes over |
$(r\|s)t = rt\|st$	concatenation distributes over |
$(s^*)^* = s^*$	* is idempotent
$s^*s^* = s^*$	0 or more twice is still 0 or more
$ss^* = s^+ = s^*s$	by definition

Ecco alcuni esempi di linguaggi denotati da un’espressione regolare:

- $\mathcal{L}(a \mid b) = \{a, b\}$
- $\mathcal{L}((a \mid b)(a \mid b)) = \{aa, ab, ba, bb\}$
- $\mathcal{L}(a^*) = \{a^n \mid n \geq 0\}$
- $\mathcal{L}(a \mid a^*b) = \{a\} \cup \{a^n b \mid n \geq 0\}$

- $(a \mid b \mid \ldots \mid z)(a \mid b \mid \ldots \mid z)^*$ denotes the set of words over the alphabet
- $(0 \mid 1)^*0$ denotes all the even binary numbers
- $b^*(abb^*)^*(a \mid \epsilon)$ denotes the set of words over $\{a, b\}$ with no consecutive occurrences of a
- $(a \mid b)^*aa(a \mid b)^*$ denotes the set of words over $\{a, b\}$ with at least one occcurrence of two consecutive a's

Automi finiti non deterministici: NFA

Nel nostro tentativo di trasformare le espressioni regolari in programmi efficienti, utilizziamo un trampolino di lancio: gli **automi finiti non deterministici** (NFA). Per la loro natura non deterministica, questi non sono così vicini alle “macchine reali” come vorremmo, quindi vedremo in seguito come questi possano essere trasformati in **automi finiti deterministici** (DFA), che sono facilmente ed efficientemente eseguibili su hardware normale.
Gli automi a stati finiti vengono utilizzati per decidere se una parola appartiene alla lingua denotata da un'espressione regolare.
Usiamo le abbreviazioni **FA** per automa finito, **NFA** per automa finito non deterministico e (più avanti in questo capitolo) **DFA** per automa finito deterministico.

Un **automa finito** è, in senso astratto, una macchina che ha un numero finito di **stati** e un numero finito di **transizioni** tra questi. Una transizione tra stati è solitamente etichettata da un carattere dell'alfabeto di input, ma utilizzeremo anche transizioni contrassegnate con ε, le cosiddette ε-transizioni (transizioni nulle).

Un automa finito può essere utilizzato per decidere se una stringa di input è un membro di un particolare insieme di stringhe. Per fare ciò, selezioniamo uno degli stati dell'automa come **stato iniziale**. Iniziamo da questo stato e in ogni passaggio possiamo eseguire una delle seguenti operazioni:

- Seguire una ε-transizione verso un altro stato
- Leggere un carattere dall'input e seguire una transizione etichettata da quel carattere.

Quando tutti i caratteri dell'input vengono letti, vediamo se lo stato corrente è contrassegnato come **accettante**. Se è così, la stringa che abbiamo letto dall'input è una parola appartenente al linguaggio definito dall'automa.

Potremmo avere la possibilità di scegliere tra diverse azioni ad ogni passo: possiamo scegliere tra una ε-transizione o una transizione su un carattere alfabetico e, se ci sono più transizioni con lo stesso simbolo, possiamo scegliere una tra queste. Ciò rende l'automa **non deterministico**, poiché la scelta dell'azione non è determinata esclusivamente osservando lo

stato corrente e l'input. Può darsi che alcune scelte portino ad uno stato di accettazione mentre altre no. Ciò, tuttavia, non significa che la stringa a volte sia nella lingua e a volte no: includeremo una stringa nella lingua se è possibile fare una qualsiasi sequenza di scelte che faccia sì che la stringa conduca ad uno stato di accettazione.

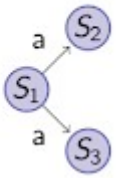
Possiamo definire formalmente un automa finito non deterministico come una tupla $(S, A, move_n, s_0, F)$, dove:

- S è un insieme di stati
- A è un alfabeto tale che $\varepsilon \notin A$
- $s_0 \in S$ è lo stato iniziale
- $F \subseteq S$ è l'insieme degli stati finali (o stati accettanti)
- $move_n : S \times (A \cup \{\varepsilon\}) \to 2^S$ è la funzione di transizione

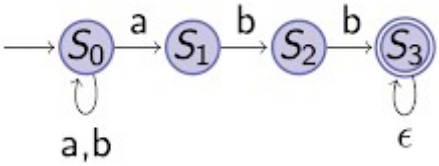
Utilizzeremo principalmente la notazione grafica tramite un grafo diretto per descrivere gli automi finiti:

- I nodi rappresentano gli stati
- Lo stato iniziale è identificato da una freccia in arrivo
- Gli stati finali sono identificati da un doppio cerchio
- Gli archi (etichettati) rappresentano la funzione di transizione

E.g., suppose $\text{move}_n(S_1, a) = \{S_2, S_3\}$, then



Un esempio di NFA è il seguente:



La rappresentazione in tabella della funzione di transizione di questo NFA è la seguente:

	ϵ	a	b
S_0	$\emptyset$	$\{S_0, S_1\}$	$\{S_0\}$
S_1	$\emptyset$	$\emptyset$	$\{S_2\}$
S_2	$\emptyset$	$\emptyset$	$\{S_3\}$
S_3	$\{S_3\}$	$\emptyset$	$\emptyset$

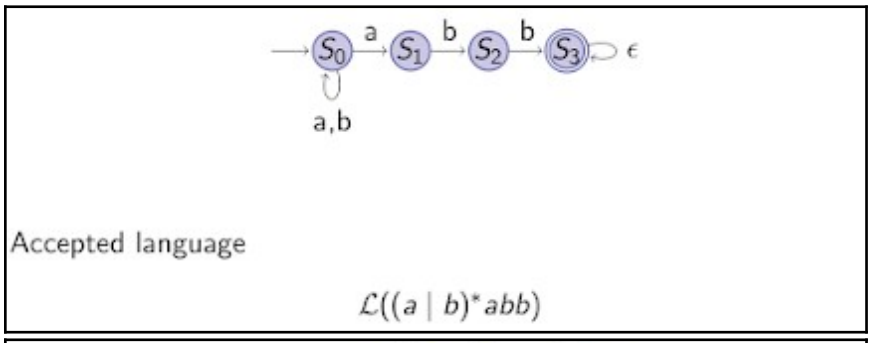
Si dice che un NFA N **accetta** (o **riconosce**) la parola w se e solo se esiste almeno un percorso che descrive w partendo dallo stato iniziale e arrivando in uno qualunque degli stati finali.
Ricordiamo che

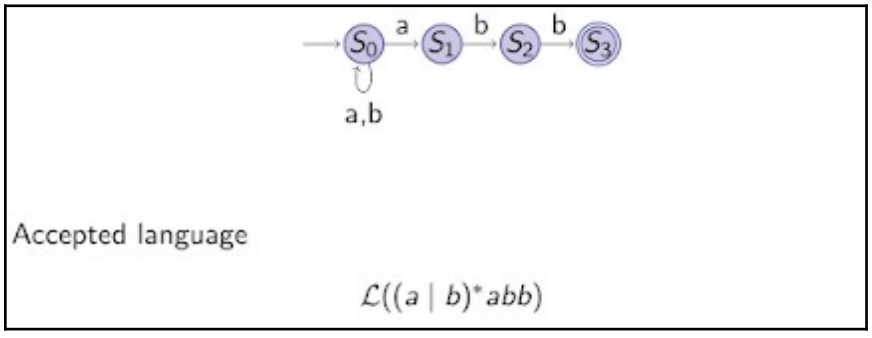
- $\varepsilon\,\varepsilon$ descrive ε
- $a\,\varepsilon$ descrive a
- $\varepsilon\,a$ descrive a

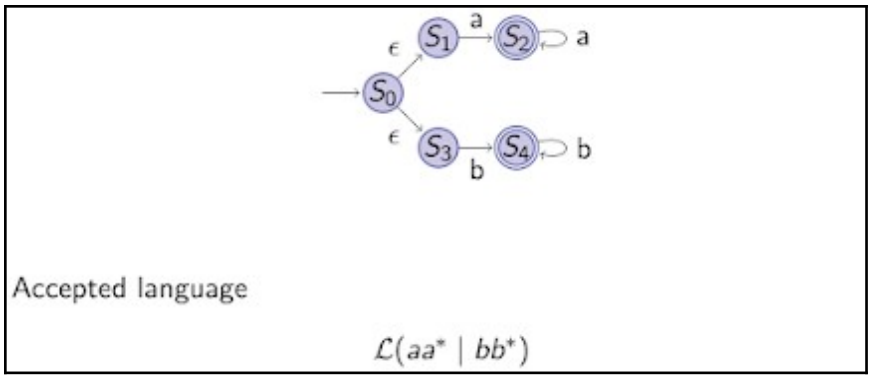
Il linguaggio **accettato** (o **riconosciuto**) dall' NFA N, scritto $L(N)$, è l'insieme di tutte le stringhe accettate da N.

Un programma che decide se una stringa è accettata da un dato NFA dovrà controllare tutti i possibili percorsi per vedere se qualcuno di questi accetta la stringa. Ciò richiede di tornare indietro fino a quando non viene trovato un percorso di successo o di seguire simultaneamente tutti i percorsi possibili, entrambi troppo dispendiosi in termini di tempo per rendere gli NFA adatti a riconoscitori efficienti. Utilizzeremo quindi gli NFA solo come trampolino di lancio tra le espressioni regolari e i DFA più efficienti. Utilizziamo questo trampolino di lancio perché rende la costruzione più semplice rispetto alla costruzione diretta di un DFA da un'espressione regolare.

Vediamo un po' di esempi:







Conversione di un'espressione regolare in un NFA

Costruiremo un NFA **compositivamente** da un'espressione regolare, ovvero costruiremo l'NFA per un'espressione regolare composta tramite gli NFA costruiti dalle sue sottoespressioni.
Per essere precisi, da ciascuna sottoespressione costruiremo un NFA e quindi combineremo questi NFA in NFA più grandi.
Costruire una NFA per un'espressione regolare significa ottenere una NFA tale che il linguaggio descritto da essa coincida con il linguaggio descritto dall'espressione regolare da cui siamo partiti.

In particolare noi vedremo la **costruzione di Thompson**, che è appunto un algoritmo che partendo da una espressione regolare r costruisce un NFA N in modo tale che

$$L(N)=L(r)$$

La costruzione degli NFA per le espressioni regolari segue la struttura dell'espressione regolare creando prima degli NFA intermedi per le sottoespressioni e poi unendoli per formare un NFA per l'intera espressione regolare:

- **Base**: r è ε oppure un carattere dell'alfabeto
    - Definiamo un NFA che descrive $L(\varepsilon)$
    - Definiamo un NFA che descrive $L(a)$
- **Passo induttivo**: r corrisponde a $r_1 \mid r_2$ oppure $r_1 r_2$ oppure r_1* oppure (r_1)
  Dati due NFA N_1 e N_2 tali che $L(N_1)=L(r_1)$ e $L(N_2)=L(r_2)$
    - Definiamo un NFA che descrive $L(r_1 \mid r_2)$
    - Definiamo un NFA che descrive $L(r_1 r_2)$
    - Definiamo un NFA che descrive $L(r_1^*)$
    - Definiamo un NFA che descrive $L((r_1))$

Vedremo che ogni passo della costruzione introduce al massimo 2 nuovi stati, dunque l'NFA generato avrà al massimo $2k$ stati, dove k è il numero di simboli e degli operatori dell'espressione regolare.

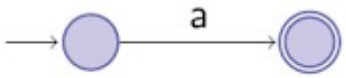
Altre caratteristiche importantissime sono che in ogni NFA intermedio nella costruzione di Thompson:

- C'è esattamente uno stato finale
- Non ci sono archi entranti nello stato iniziale
- Non ci sono archi uscenti dallo stato finale

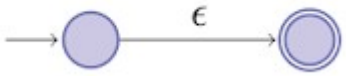
Queste caratteristiche sono fondamentali perché, come vedremo, altrimenti l'algoritmo di Thompson non genererebbe degli NFA corretti.

Nei casi base, gli NFA che generiamo sono i seguenti

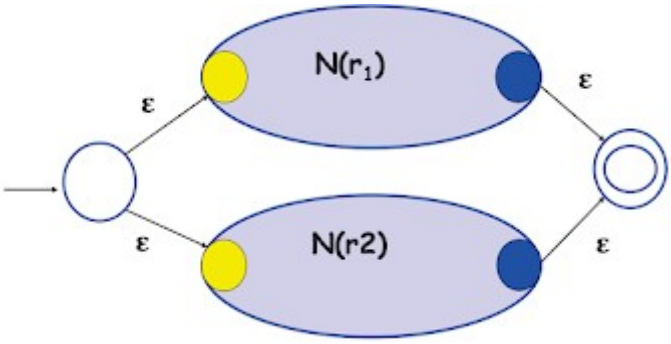
$r = a$



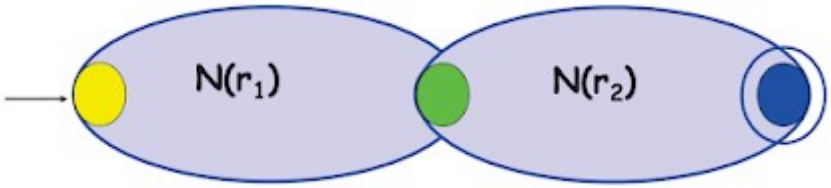
$r = \epsilon$



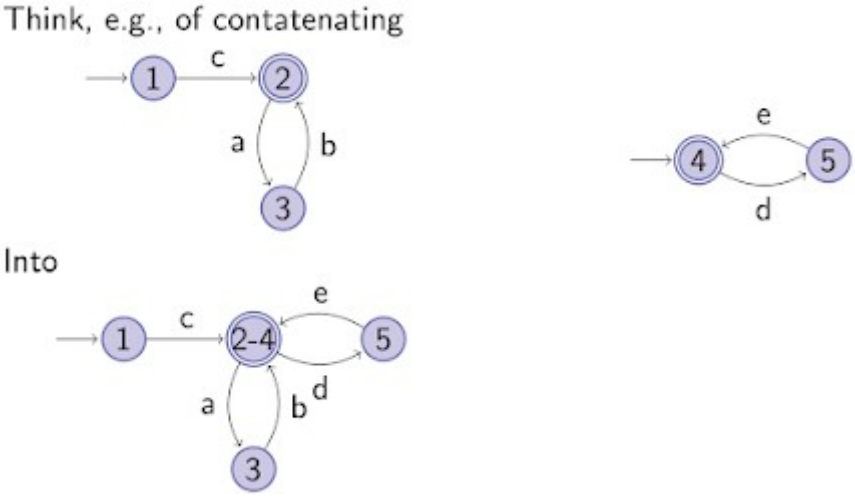
Per l'operatore di **alternanza** abbiamo $r = r_1 \mid r_2$, sappiamo per induzione di avere due NFA intermedi "funzionanti", uno per r_1 e uno per r_2, e sappiamo che entrambi hanno uno stato iniziale e uno stato finale.
Per costruire un NFA per r creiamo un nuovo stato iniziale e aggiungiamo due ε-transizioni verso i due stati iniziali degli NFA intermedi. Successivamente creiamo un nuovo stato finale e facciamo arrivare ad esso due ε-transizioni che partono dai due stati finali degli NFA intermedi (che non saranno più finali nel nuovo NFA più grande).



Per l'operatore di **concatenazione** abbiamo $r = r_1 r_2$, sappiamo per induzione di avere due NFA intermedi "funzionanti", uno per r_1 e uno per r_2, e sappiamo che entrambi hanno uno stato iniziale e uno stato finale.
Per costruire un NFA per r facciamo coincidere lo stato finale di r_1 e lo stato iniziale di r_2 e prendiamo come stato iniziale lo stato iniziale del NFA intermedio di r_1 e come stato finale lo stato finale del NFA intermedio di r_2.

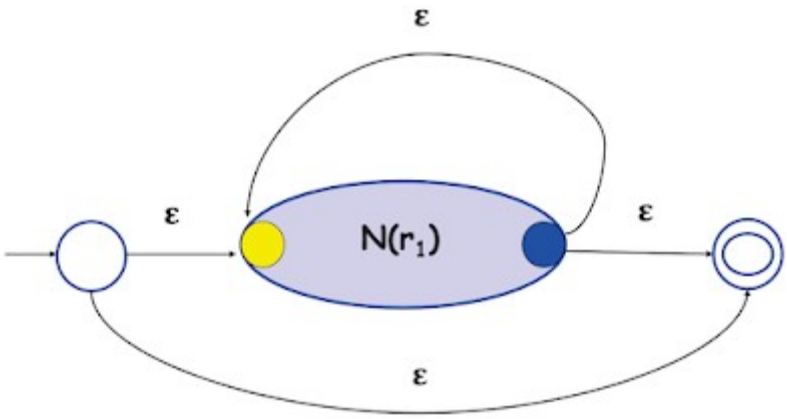


Osservazione: La costruzione è più efficiente fatta in questa maniera piuttosto che collegare il stato finale di $N(r_1)$ con il stato iniziale di $N(r_2)$ con una ε-transizione (vedremo il motivo successivamente, quando parleremo di ε-chiusure).
I vincoli "*Non ci sono archi entranti nello stato iniziale*" e "*Non ci sono archi uscenti dallo stato finale*" garantiscono proprio che nella concatenazione eseguita in questo modo, l'automa costruito descriva esattamente $L(r_1 r_2)$.
Ecco infatti un esempio dove i vincoli non vengono rispettati e l'automa costruito non descrive il linguaggio desiderato:

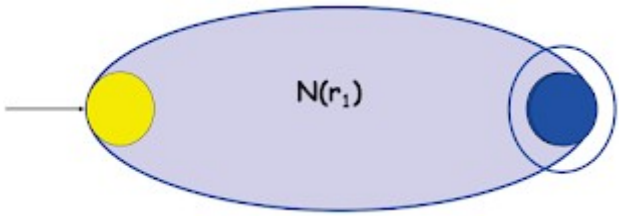


Per l'operatore di **Kleene star** abbiamo $r = r_1^*$, sappiamo per induzione di avere un NFA intermedio "funzionante" r_1 e sappiamo che ha uno stato iniziale e uno stato finale.
Per costruire un NFA per r dobbiamo:

- Creare un nuovo stato finale in cui entri una ε-transizione uscente dallo stato finale del NFA intermedio
- Creare un nuovo stato iniziale da cui escono due ε-transizioni: una entrante nello stato iniziale del NFA intermedio e una che vada direttamente nel nuovo stato finale (questo allo scopo di descrivere la parola ε, che viene incluso dall'operatore Kleene star)
- Aggiungere una ε-transizione dallo stato finale del NFA intermedio allo stato iniziale del NFA intermedio (questo per creare l'iteratività dell'operatore Kleene star)



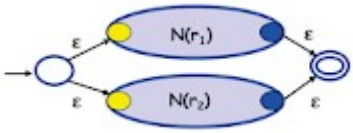
Per l'operatore di **parentesi** abbiamo $r = (r_1)$, sappiamo per induzione di avere un NFA intermedio "funzionante" r_1 e sappiamo che ha uno stato iniziale e uno stato finale.
In questa situazione, manteniamo intatto lo stesso NFA poiché questo operatore riguarda esclusivamente una questione di precedenza.



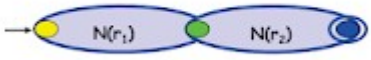
La costruzione consiste nell'iterare questo processo fino a che non viene considerata tutta l'espressione regolare. Si noti che anche se è possibile che un NFA abbia diversi stati accettanti, un NFA costruito utilizzando questo metodo ne avrà sempre solo uno.

Riassumendo:

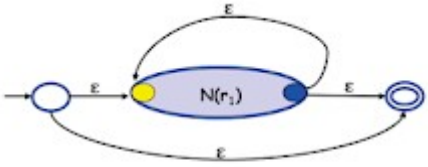
$r = r_1 \mid r_2$



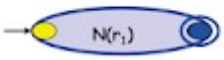
$r = r_1 r_2$



$r = r_1^*$



$r = (r_1)$



Vediamo subito che, considerando anche i casi base, ad ogni passo vengono aggiunti al massimo 2 stati (nel caso di alternanza, Kleene star e dei casi base) e al massimo 4 archi (sempre nel caso di alternanza e Kleene star).

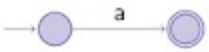
Analizziamo ora la **complessità** della costruzione di Thompson.
Costruiamo un NFA con n nodi e m archi. Ogni passo aggiunge al massimo 2 stati e 4 archi, dunque ogni passo ha un tempo costante. Ci sono esattamente $|r|$ passi, dove $|r|$ è la lunghezza dell'espressione regolare contando sia i caratteri che gli operatori.
Dunque:

- La complessità spaziale è $n+m$ che è uguale a $O(|r|)$
- La complessità temporale è $O(|r|)$

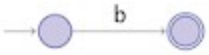
Vediamo un esempio di applicazione della costruzione di Thompson:

Take $r = (a \mid b)^* abb$

$a = r_1$

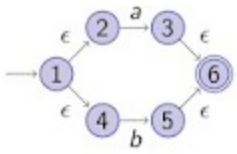


$b = r_2$



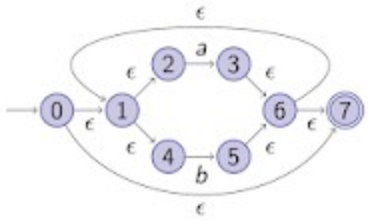
$a \mid b = r_1 \mid r_2 = r_3$

Apply Thompson's construction for alternation to the automata for r_1 and r_2, and get the automaton for r_3



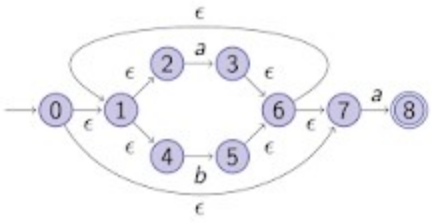
$(a \mid b) = (r_3) = r_4 \qquad (a \mid b)^* = r_4{}^* = r_5$

Apply Thompson's construction for parentheses and for Kleene star to the automaton for r_3, and get the automaton for r_5



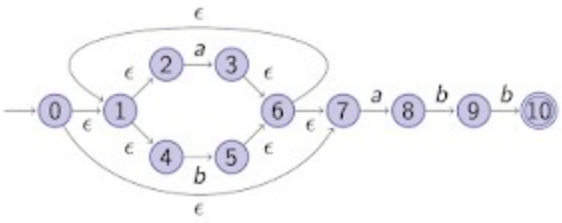
$(a \mid b)^* a = r_5 r_1 = r_6$

Apply Thompson's construction for concatenation to the automata for r_5 and for r_1, and get the automaton for r_6

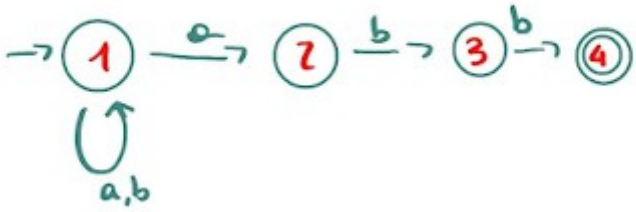


$r = (a \mid b)^* abb$

In two more steps



Notiamo che questo NFA è molto complicato e “grande” e che se avessimo provato a costruirlo a mano avremmo potuto ottenere un NFA molto più piccolo e semplice come, ad esempio, questo:

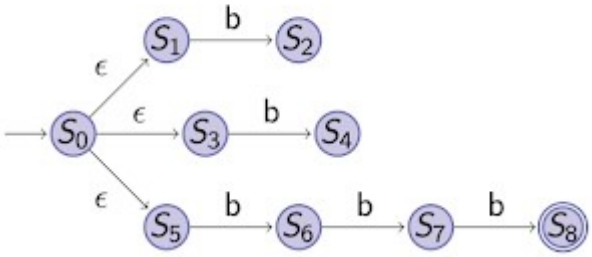


Dunque la costruzione di Thompson non è per niente efficiente, però è un metodo che riesce a generare sempre un NFA corretto per qualsiasi espressione regolare (anche molto complessa) data in input.

Vediamo ora la **simulazione degli NFA**, ovvero data una parola w e un NFA N, stabilire se $w \in L(N)$. Per farlo bisogna vedere se esiste un percorso nel NFA che esegue lo spelling della parola w partendo da uno stato iniziale e arrivando in uno stato finale.

Un esempio molto semplice è il seguente:

Let $w = bbb$



Notiamo che leggere bbb da S_0 è equivalente al leggere bbb da qualsiasi degli stati $\{S_0, S_1, S_3, S_5\}$

Definiamo quindi la ε**-chiusura**.
Sia $(S, A, move_n, s_0, F)$ una NFA, t uno stato in S e T un sottoinsieme di S:

- $\varepsilon\text{-}closure(\{t\})$ consiste nell'insieme degli stati di S che sono raggiungibili da t tramite zero o più ε-transizioni
- $\varepsilon\text{-}closure(T) = ¿\, t \in T \,¿ \bigcup (\varepsilon\text{-}closure(\{t\}))$

Per calcolare $\varepsilon\text{-}closure()$ abbiamo bisogno delle seguenti strutture dati:

- Uno stack per inserire gli stati
- Un array booleano *alreadyOn* di dimensione $|S|$ per verificare in tempo costante se uno stato è presente nello stack
- Un array bidimensionale per rappresentare la funzione di transizione $move_n$. Ogni entry (t, x) è una lista concatenata contenente tutti gli stati in $move_n(t, x)$

La funzione per calcolare la $\varepsilon\text{-}closure(\{t\})$ è la seguente:

```
foreach i=1,..., |S| do alreadyOn[i] = false ;
function closure(t,stack)
    push t onto stack ;
    alreadyOn[t] = true ;
    foreach u ∈ move_n(t, ε) do
        if not alreadyOn[u] then
            closure (u,stack) ;
```

Il funzionamento è il seguente:

- Aggiungiamo il nodo t allo stack (perchè per definizione di chiusura, t appartiene alla chiusura di t) e accendiamo il suo bit nell'array
- Per ogni nodo a cui possiamo arrivare tramite una ε-transizione controlliamo se l'abbiamo già considerato o meno, se non l'abbiamo ancora considerato chiamiamo la funzione ricorsivamente su di esso

Dunque le operazioni che vengono eseguite sono le seguenti:

```
1 push t onto stack;
2 set alreadyOn[t] bit;
3 find next u ∈ move_n(t, ε);
4 test alreadyOn[u] bit;
```

le quali hanno tutte tempo costante, dunque per calcolare la complessità dobbiamo capire quante volte queste operazioni vengono eseguite:

- 1 e 2 vengono eseguite ad ogni invocazione della funzione (sia che sia quella iniziale sia che sia una ricorsiva), e visto che possiamo chiamare la funzione al massimo per ogni stato, ogni stato verrà inserito nello stack al massimo una volta (il bit *alreadyOn* è inizialmente falso, viene impostato a *true* e non verrà più cambiato).
  Dunque $O(n)$
- 3 e 4 vengono eseguite per ogni nodo u raggiungibile tramite una ε-transizione dal nodo t, dunque nel caso peggiore, in cui ogni nodo entra nello stack, verranno eseguite per ogni arco (assumendo che ogni arco sia una ε-transizione).
  Dunque $O(m)$

Nel complesso, assumendo che l'NFA abbia n stati e m archi, la funzione è $O(n+m)$.

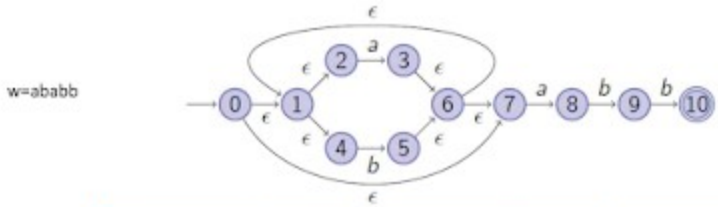
Vediamo ora come funziona l'algoritmo che esegue la simulazione di un NFA.

```
input  : NFA N = (S, A, move_n, s_0, F), w$
output : "yes" if w ∈ L(N), "no" otherwise
states = ε-closure({s_0}) ;
symbol = nextchar() ;
while symbol ≠ $ do
    states = ε-closure(⋃_{t∈states} move_n(t, symbol)) ;
    symbol = nextchar() ;
if states ∩ F ≠ ∅ then
    return "yes" ;
else
    return "no" ;
```

Dunque:

- Si parte con *states* che coincide con tutti gli stati appartenenti alla ε-chiusura di s_0 (ovvero lo stato iniziale del NFA) e si considera il primo carattere
- Fino a che non si arriva alla fine della parola da “simulare”
    - Si trovano gli stati a cui si può arrivare partendo da uno degli stati in *states* tramite una transizione denotata dal carattere corrente
    - Si calcola la ε-chiusura di questi stati e si assegnano i nuovi stati a *states*
    - Si scorre al prossimo carattere
- Se alla fine almeno uno degli stati in *states* è uno stato finale, allora la parola appartiene al linguaggio denotato dal NFA, altrimenti no

Ecco un esempio di esecuzione:



states	symbol	U_t move(t,symbol)	ε-closure
T0 = {0,1,2,4,7}	a	{3,8}	{1,2,3,4,6,7,8}
T1 = {1,2,3,4,6,7,8}	b	{5,9}	{1,2,4,5,6,7,9}
T2 ={1,2,4,5,6,7,9}	a	{3,8}	T1
T1	b		T2
T2	b	{5,10}	{1,2,4,5,6,7,10}
T3={1,2,4,5,6,7,10}	$		

Per quanto riguarda la complessità, ovviamente domina il ciclo while, e all’interno del ciclo while domina la riga $L1$.

```
while symbol ≠ $ do
L1    states = ε-closure(⋃_{t∈states} move_n(t, symbol)) ;
      symbol = nextchar() ;
```

Per implementare la riga $L1$ dobbiamo utilizzare le seguenti strutture dati:

- Due stack per gli stati
    - *currentStack* per gli stati attuali (gli "stati" sul lato destro dell'assegnazione alla riga $L1$)
    - *nextStack* per gli stati successivi (gli "stati" sul lato sinistro dell'assegnazione alla riga $L1$)
- Un array booleano *alreadyOn* di dimensione $|S|$ per verificare in tempo costante se uno stato è presente nello stack
- Un array bidimensionale per rappresentare la funzione di transizione $move_n$. Ogni entry (t, x) è una lista concatenata contenente tutti gli stati in $move_n(t, x)$

Espandendo la riga $L1$ abbiamo:

```
foreach t on currentStack do
    foreach u ∈ move_n(t, symbol) do
        if not alreadyOn[u] then
            closure(u,nextStack);
    pop t from currentStack;
foreach s on nextStack do
    pop s from nextStack ;
    push s on currentStack ;
    alreadyOn[s] = false ;
```

Dove il primo ciclo all'interno ha la popolazione di *nextStack* e il secondo ciclo ha lo swap da *nextStack* a *currentStack*.
Si vede ad occhio che il secondo ciclo ha una complessità minore rispetto al primo.

Assumiamo che l'NFA abbia n stati e m archi.
I cicli for:

- Popolare *nextStack* costa $O(n+m)$ (costo di ε-chiusura), che sarebbe moltiplicato per un fattore t che possiamo ignorare
- Lo swap su *currentStack* costa $O(n)$

Dunque un ciclo while costa $O(n+m)$, ed essendo le iterazioni $|w|$ abbiamo che la funzione totale costa $O(|w|(n+m))$.

In particolare, se l'NFA è stato costruito tramite l'algoritmo di Thompson, allora $(n+m)$ è $O(|r|)$ e quindi la simulazione costa $O(|w||r|)$.

Dunque, per fare un riepilogo, per accettare linguaggi regolari tramite NFA:

- Prendi un'espressione regolare r.
- Applica la costruzione di Thompson, ottenendo un NFA con un tempo di esecuzione di $O(|r|)$.
- Simula l'NFA con un tempo di esecuzione di $O(|w||r|)$.

Nel complesso dunque, il tempo di esecuzione è di $O(|w||r|)$.

Automi finiti deterministici

Gli automi non deterministici, come accennato in precedenza, non sono così vicini a "la macchina" come vorremmo. Pertanto, ora introduciamo una forma più restrittiva di automa finito: l'**automa finito deterministico**, o **DFA** per abbreviare. I DFA sono NFA, ma obbediscono a una serie di restrizioni aggiuntive:

- Non ci sono ε-transizioni.

- Non possono esserci due transizioni con etichette identiche che escono dallo stesso stato.

Questo significa che non abbiamo mai una scelta tra diversi stati successivi: lo stato attuale e il simbolo di input successivo determinano in modo univoco la transizione (o l'assenza di essa). Per questo motivo, questi automi vengono chiamati deterministici.

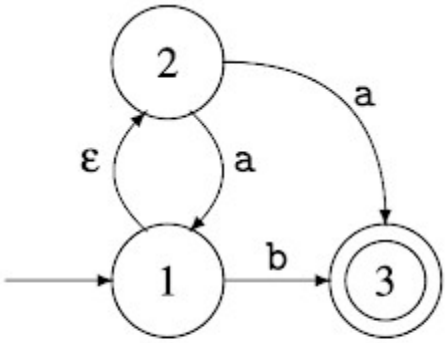


Figure 2.3: Example of an NFA

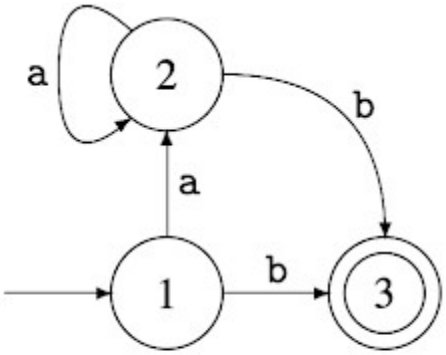


Figure 2.8: Example of a DFA

Passiamo quindi dalla definizione usata per gli NFA, che è la seguente:

$$(S, \mathcal{A}, \text{move}_n, s_0, F)$$
$$\textit{where } \text{move}_n : S \times (\mathcal{A} \cup \{\epsilon\}) \to 2^S$$

alla definizione per i DFA, che è la seguente:

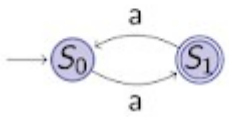
$$(S, \mathcal{A}, \text{move}_d, s_0, F)$$
$$\textit{where } \text{move}_d : S \times \mathcal{A} \to S$$

Vediamo dunque come non sono accettate le ε-transizioni nei DFA e come la funzione di transizione ci ritorni sempre un singolo stato (e non un insieme di stati come negli NFA).

La relazione di transizione $move_d$ in un DFA è una funzione, e spesso la scriviamo come tale: $move_d(s, c)$ è lo stato (se esiste) che si raggiunge dallo stato s tramite una transizione sul simbolo c. Se non esiste tale transizione, $move_d(s, c)$ è indefinito:

- Se $move_d$ è **totale**, allora da ogni stato c'è **esattamente** una a-transizione per ogni simbolo a appartenente all'alfabeto A.
- Se $move_d$ è **parziale**, allora da ogni stato c'è **al massimo** una a-transizione per ogni simbolo a appartenente all'alfabeto A.

- Take the alphabet $\{a\}$
- An example of DFA with total transition function is



- An example of DFA with partial transition function is



È molto facile implementare un DFA: una tabella bidimensionale può essere indicizzata incrociando stato e simbolo per ottenere lo stato successivo (o un'indicazione che non c'è una transizione), implementando essenzialmente la funzione di transizione tramite una ricerca nella tabella. Un'altra tabella (unidimensionale) può indicare quali stati sono accettanti.

I DFA hanno la stessa potenza espressiva degli NFA: un DFA è un caso speciale di NFA e qualsiasi NFA può (come vedremo tra poco) essere convertito in un DFA equivalente. Tuttavia, ciò ha un costo: il DFA risultante può essere esponenzialmente più grande rispetto all'NFA di partenza. Nella pratica (cioè, nella descrizione dei token per un linguaggio di programmazione), l'aumento di dimensioni è di solito modesto, motivo per cui la maggior parte degli analizzatori lessicali si basa sui DFA.

Vediamo ora la **simulazione** di un DFA. Visto che non si può “scegliere”, il costo è lineare con la lunghezza della parola da verificare.

Il linguaggio riconosciuto da un DFA D, indicato con $L(D)$, è l'insieme di parole w tali che:

- O c'è un percorso che rappresenta $w = a_1 \cdots a_k$ con $k \geq 1$ dallo stato iniziale di D ad alcuni dei suoi stati finali.
- Oppure lo stato iniziale è anche finale e $w = \varepsilon$.

Vediamo come cambia il procedimento per rispondere alla domanda “$w \in L(D)$?” in base a se la funzione di transizione è totale oppure parziale:

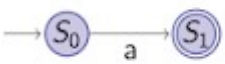
- **Totale**
  - Partendo dallo stato iniziale, segui il percorso che rappresenta la parola w.
  - Se lo stato raggiunto è finale, restituisci "sì".
  - Altrimenti, restituisci "no".
- **Parziale**
  - Partendo dallo stato iniziale, inizia a seguire il percorso che rappresenta $w = a_1 \cdots a_k$.
  - Se per qualche a_i non c'è uno stato “bersaglio”, restituisci "no".

- Se w è finita e lo stato raggiunto è finale, restituisci "sì".
- Altrimenti, restituisci "no".

Dunque nel caso della funzione di transizione parziale c'è la possibilità di fermarsi prima di aver finito la parola, ovvero quando non esistono transizioni etichettate col carattere corrente.
Inoltre i DFA con funzione di transizione parziale occupano meno memoria ovviamente.

Vediamo come passare da una funzione di transizione parziale ad una funzione di transizione totale.
Sia D un DFA con una funzione di transizione parziale. Dobbiamo definire un DFA D' con una funzione di transizione totale tale che $L(D') = L(D)$.
Le operazioni da fare sono le seguenti:

- Aggiungere un **pozzo** (sink) agli stati di D.
- Fare sì che il pozzo sia la destinazione di tutte le transizioni non definite.
- Per ogni simbolo nell'alfabeto a, aggiungere una a-transizione auto-loop sul pozzo.





Convertire un NFA in un DFA

Mostreremo come gli NFA possono essere convertiti in DFA in modo tale che, combinando ciò con la conversione delle espressioni regolari in NFA illustrata precedente, possiamo convertire qualsiasi espressione regolare in un DFA.

La conversione avviene simulando contemporaneamente tutti i percorsi possibili in un NFA. Ciò significa che operiamo con insiemi di stati NFA: quando abbiamo diverse scelte per uno stato successivo, prendiamo tutte le scelte contemporaneamente e formiamo un insieme di stati successivi possibili. L'idea è che tale insieme di stati NFA diventerà uno stato singolo DFA. Per qualsiasi simbolo dato, formiamo l'insieme di tutti gli stati successivi possibili nell'NFA, ottenendo così una singola transizione (etichettata da tale simbolo) che va da un insieme di stati NFA a un altro insieme. Di conseguenza, la transizione diventa deterministica nel DFA formato dagli insiemi di stati NFA.
Passare da una struttura non deterministica ad una deterministica costa tanto, dunque se la parola è abbastanza corta o se la struttura verrà utilizzata relativamente poche volte conviene utilizzare direttamente la struttura non deterministica senza eseguire la conversione.

Le ε-transizioni complicano un po' la costruzione: ogni volta che ci troviamo in uno stato NFA, possiamo sempre scegliere di seguire una ε-transizione senza leggere alcun simbolo. Di conseguenza, dato un simbolo, uno stato successivo può essere trovato sia seguendo una transizione con quel simbolo sia facendo prima qualsiasi numero di ε-transizioni e poi una transizione con il simbolo. Gestiamo ciò nella costruzione estendendo prima l'insieme di stati NFA con quelli che puoi raggiungere da questi usando solo ε-transizioni (la ε-chiusura). Poi, per

ogni simbolo di input possibile, seguiamo le transizioni con tale simbolo per formare un nuovo insieme di stati NFA. Chiaramente utilizzeremo le ε-chiusure per fare ciò.

La costruzione per sottoinsiemi

La costruzione è chiamata "costruzione per sottoinsiemi", poiché ogni stato nel DFA è un sottoinsieme degli stati dell'NFA.

Dunque l'obiettivo è quello, di dato l'NFA N, costruire un DFA D tale che $L(D)=L(N)$.
Come abbiamo già detto, l'idea è quella di utilizzare la ε-chiusura per mappare sottoinsiemi di stati dell'NFA in un singolo stato del DFA.

Questa è la prima istanza di una costruzione che vedremo nuovamente più avanti nel corso (analizzatori sintattici):

- Definisci lo stato iniziale del DFA e aggiungilo alla collezione.
- Popola la collezione di tutti i suoi stati mentre definisci la funzione di transizione nella seguente maniera:

```
foreach state T already collected do
    foreach symbol a in the alphabet do
        compute the plausible target of the a-transition from T ;
        call it T′;
        if T′ is already in the collection then
            nothing else to do ;
        else
            add T′ to the collection ;
```

La funzione completa è la seguente:

```
input  : NFA N = (S, A, move_n, S_0, F)
output : DFA D = (R, A, move_d, T_0, E) such that L(D) = L(N)
T_0 = ε-closure({S_0});
R = {T_0} ;
set T_0 as unmarked ;
while some T ∈ R is unmarked do
    mark T ;
    foreach a ∈ A do
        T′ = ε-closure(⋃_{t∈T} move_n(t, a)) ;
        if T′ ≠ ∅ then
            move_d(T, a) = T′ ;
            if T′ ∉ R then
                add T′ to R ;
                set T′ as unmarked ;
foreach T ∈ R do
    if (T ∩ F) ≠ ∅ then set T ∈ E ;
```

Una piccola spiegazione:

- Lo stato iniziale del DFA è la ε-chiusura dell'insieme che contiene solo lo stato iniziale dell'NFA, cioè gli stati che sono raggiungibili dallo stato iniziale tramite ε-transizioni.

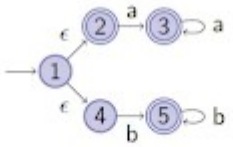
- Una transizione nel DFA avviene trovando l'insieme di stati dell'NFA che costituiscono lo stato del DFA, seguendo tutte le transizioni (sullo stesso simbolo) nell'NFA da tutti questi stati NFA e infine combinando gli insiemi risultanti di stati e chiudendo questo sotto ε-transizioni.
- Uno stato nel DFA è uno stato accettante (finale) se almeno uno degli stati dell'NFA che contiene è accettante.

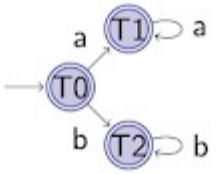
Vediamo ora la **complessità** di questo algoritmo.
L'operazione dominante in termini di complessità è il calcolo della ε-chiusura rispetto all'insieme dei nodi raggiungibili dal vecchio insieme tramite una a-transizione:

```
while some T ∈ R is unmarked do
    ...;
    foreach a ∈ A do
        T' = ε-closure(⋃_{t∈T} move_n(t, a)) ;
        ⋮
```

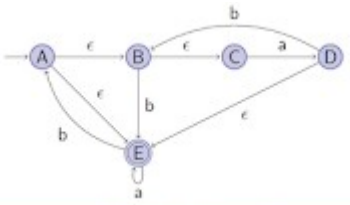
Supponiamo che l'NFA di partenza abbia n stati e m archi, e il DFA finale abbia n_d stati.
Il ciclo *while* viene ripetuto n_d volte. Il ciclo *foreach* viene ripetuto $|A|$ volte (la dimensione dell'alfabeto).
Sappiamo che il calcolo della ε-chiusura costa invece $O(n+m)$
Quindi, la costruzione del sottoinsieme richiede $O(n_d \cdot |A| \cdot (n+m))$.

Vediamo un po' di esempi di applicazione dell'algoritmo:

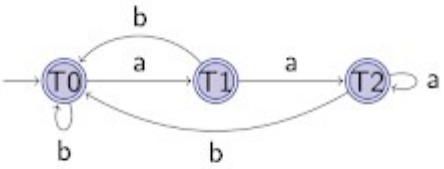


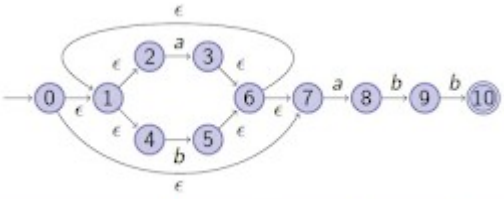


	a	b
T0 = ε-closure({1}) = {1,2,4} % initial, final	ε-closure({3}) = T1	ε-closure({5}) = T2
T1 = {3} % final	ε-closure({3}) = T1	---
T2 = {5} % final	---	ε-closure({5}) = T2

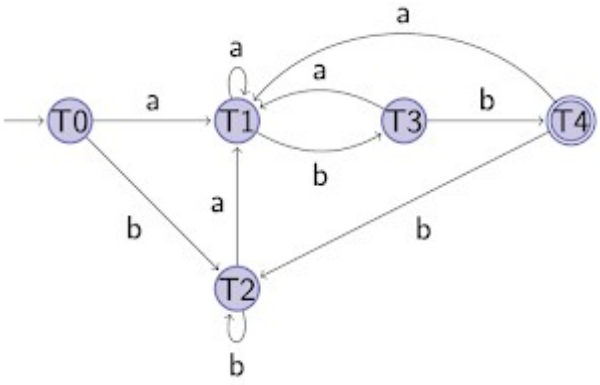


	a	b
T0 = ε-closure({A}) = {A,B,C,E} % initial, final	ε-closure({D,E}) = T1	ε-closure({A,E}) = {A,B,C,E} = T0
T1 = {D,E} % final	ε-closure({E}) = T2	ε-closure({A,B}) = {A,B,C,E} = T0
T2 = {E} % final	ε-closure({E}) = T2	ε-closure({A}) = T0

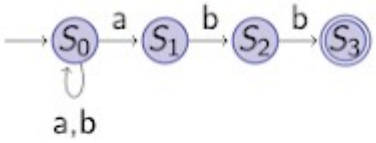


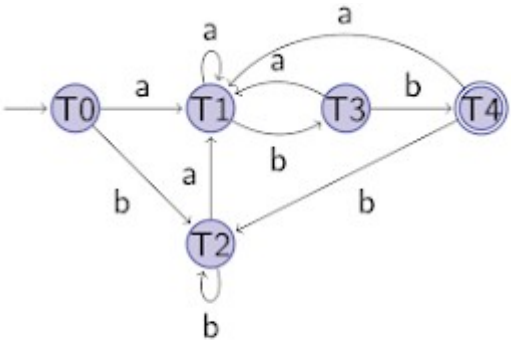


	a	b
T0 = ε-closure({0}) = {0,1,2,4,7} % initial	ε-closure({3,8}) = T1	ε-closure({5}) = T2
T1 = {1,2,3,4,6,7,8}	ε-closure({3,8}) = T1	ε-closure({5,9}) = T3
T2 = {1,2,4,5,6,7}	ε-closure({3,8}) = T1	ε-closure({5}) = T2
T3 = {1,2,4,5,6,7,9}	ε-closure({3,8}) = T1	ε-closure({5,10}) =T4
T4 = {1,2,4,5,6,7,10} % final	ε-closure({3,8}) = T1	ε-closure({5}) = T2



Prendiamo in considerazione l'ultimo DFA. Questo DFA, sebbene più "semplice" del rispettivo NFA, sembra comunque molto grande e complesso. È possibile infatti trovare un DFA molto più semplice che descrive esattamente lo stesso linguaggio:





Both automata recognize $\mathcal{L}((a \mid b)^* abb)$

Vedremo successivamente come **minimizzare** un DFA.

Dimensione contro velocità

Poiché gli stati nel DFA costruito sono insiemi (non vuoti) di stati dell'NFA, potrebbero potenzialmente esserci $2^n - 1$ stati in un DFA costruito da un NFA con n stati. Non è troppo

difficile costruire classi di NFAs che si espandono in modo esponenziale in questo modo quando convertite in DFAs. Dato che siamo principalmente interessati a NFAs che sono costruiti da espressioni regolari, potremmo chiederci se questi non possano essere in una classe adeguatamente semplice che non comporti DFA di dimensioni esponenziali. Purtroppo, questo non è il caso. Proprio come possiamo costruire una classe di NFAs che si espandono in modo esponenziale, possiamo costruire una classe di espressioni regolari in cui i DFAs equivalenti più piccoli sono esponenzialmente più grandi. Questo però accade raramente quando usiamo espressioni regolari o NFAs per descrivere i token nei linguaggi di programmazione.

È possibile evitare l'esplosione delle dimensioni operando direttamente sulle espressioni regolari o sugli NFAs quando si testano le stringhe per l'inclusione nei linguaggi che questi definiscono. Tuttavia, c'è un costo in termini di velocità per farlo. Un DFA può essere eseguito in tempo $k \cdot |w|$, dove $|w|$ è la lunghezza della stringa di input w e k è una piccola costante indipendente dalle dimensioni del DFA. Le espressioni regolari e gli NFAs possono essere eseguiti in un tempo vicino a $c \cdot |N| \cdot |w|$, dove $|N|$ è la dimensione dell'NFA (o dell'espressione regolare) e la costante c di solito è più grande di k. Tutto sommato, i DFAs sono molto più veloci da usare rispetto agli NFAs o alle espressioni regolari, quindi è solo quando le dimensioni del DFA sono un vero problema che si dovrebbe considerare l'uso diretto di NFAs o espressioni regolari.

Minimizzazione dei DFA

I DFA costruiti da espressioni regolari tramite NFAs spesso non sono minimali (come abbiamo visto precedentemente), anche se raramente capita che siano tanto distanti dall'essere minimali. Tuttavia, minimizzare un DFA non è particolarmente difficile e può essere fatto abbastanza velocemente, quindi molti generatori di lexer eseguono la **minimizzazione**.

La minimizzazione consiste nel, dato il DFA D, costruire un DFA minimale D' tale che

$$L(D') = L(D)$$

L'intuizione è quella di **partizionare** gli stati del DFA in classi equivalenti:
Due stati s e t sono equivalenti se e solo se, per ogni x:

la simulazione di x da s ha successo **se e solo** se la simulazione di x da t ha successo

L'algoritmo che vedremo richiede un DFA con **funzione di transizione totale**, perché utilizza l'inverso della funzione e richiede che sia definita per ogni stato.

La definizione formale di equivalenza è:
Dato un DFA con funzione di transizione totale $D = (S, A, move_d, s_0, F)$, allora $s, t \in S$ sono equivalenti se e solo se vale quanto segue:

$$move_d^*(s, x) \in F \ \ se \ \ e \ \ solo \ \ se \ \ move_d^*(t, x) \in F \ \ per \ \ ogni \ \ x \in A^*$$

dove la funzione di transizione multi-step $move_d*$ è definita tramite induzione sulla lunghezza delle stringhe come segue:

- $move_d*(s,\varepsilon)=s$
- $move_d*(s,wa)=move_d(move_d*(s,w),a)$

Una proprietà interessante dei DFA è che ogni linguaggio regolare (un linguaggio che può essere espresso da un'espressione regolare, NFA o DFA) ha un **unico DFA minimale**. Quindi, possiamo decidere l'equivalenza di espressioni regolari (o NFAs o DFAs) convertendo entrambi in DFA minimali e confrontando i risultati.

Come accennato prima, la minimizzazione dei DFA avviene collassando gli stati equivalenti. Tuttavia, decidere se due stati sono equivalenti non è fatto solo testando se le loro transizioni immediate sono identiche, poiché le transizioni verso stati diversi possono essere equivalenti se gli stati di destinazione si rivelano equivalenti. Pertanto, utilizziamo una strategia in cui supponiamo inizialmente che tutti gli stati siano equivalenti e li distinguiamo solo se possiamo dimostrarli diversi. Utilizziamo le seguenti regole per questo:

- Uno stato accettante non è equivalente a uno stato non accettante.
- Se due stati s_1 e s_2 hanno transizioni sullo stesso simbolo c verso gli stati t_1 e t_2 che abbiamo già dimostrato essere diversi, allora s_1 e s_2 sono diversi. Questo si applica anche quando solo uno tra s_1 e s_2 ha una transizione definita su c.

Dato un DFA D sull'alfabeto A con stati S, dove $F \subseteq S$ è l'insieme degli stati accettanti, costruiamo un DFA minimale D_{min} in cui ogni stato è un gruppo di stati di D.
Minimizziamo il DFA D nel seguente modo:

1. Partiamo con due partizioni/blocchi: l'insieme degli stati accettanti $B_1=F$ e l'insieme degli stati non accettanti $B_2=S¿$. Questi sono non contrassegnati.
   Scegliamo questa come partizione iniziale perché $s \in B_1$ sicuramente non è equivalente a $t \in B_2$ in quanto $move_d*(s,\varepsilon) \in F$ mentre $move_d*(t,\varepsilon) \notin F$
2. Verifica se i blocchi contengono stati equivalenti. Se non lo fanno, affina ulteriormente i blocchi dividendoli.
3. Ripeti i passaggi di verifica e affinamento fino a quando la partizione non può essere ulteriormente affinata. I gruppi rimanenti sono gli stati del DFA minimale

Lo stato iniziale del DFA minimale è il gruppo che contiene lo stato iniziale originale e qualsiasi gruppo di stati accettanti è uno stato accettante nel DFA minimale.

Concentriamoci ora sul **partizionamento**.
Se tutti gli stati in $B_i=\{s_1,\ldots,s_k\}$ sono equivalenti, allora per ogni $a \in A$, gli stati di destinazione della a-transizione da $s_1,\ldots,s_k$ dovrebbero appartenere allo stesso blocco.

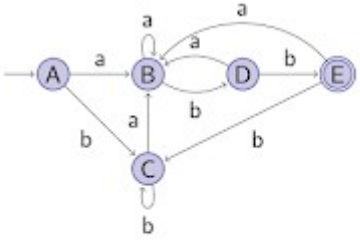
Il blocco B_i può essere diviso rispetto a (a, B_j) se per alcuni $s, t \in B_i$, si ha che $move_d(s, a) \in B_j$ e $move_d(t, a) \notin B_j$.
Dividere il blocco B_i rispetto a (a, B_j) equivale a sostituire B_i con i due blocchi:

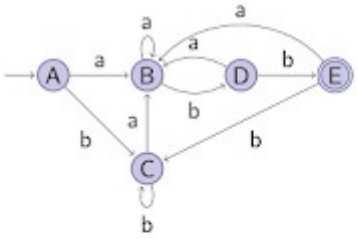
- $\{s \in B_i \mid move_d(s, a) \in B_j\}$
- $\{s \in B_i \mid move_d(s, a) \notin B_j\}$

Dunque l'algoritmo di partizionamento è il seguente:

$B_1 = F$;
$B_2 = S \setminus F$;
$P = \{B_1, B_2\}$;
while *Some* $B_i \in P$ *can be split wrt some* (a, B_j) **do**
    Update P by removing B_i and adding the two blocks
    $\{s \in B_i \mid \text{move}_d(s, a) \in B_j\}$ and
    $\{s \in B_i \mid \text{move}_d(s, a) \notin B_j\}$

Un esempio di partizionamento è il seguente:





Initial Partition: $B_1 = \{E\}$ $B_2 = \{A, B, C, D\}$

- $\text{move}_d(D, b) = E \in B_1$ and $\text{move}_d(A, b) = C \notin B_1$
- Then we can split B_2 wrt (b, B_1)

Init	$B_1 = \{E\}$; $B_2 = \{A, B, C, D\}$
split B_2	$B_1 = \{E\}$; $B_{21} = \{D\}$; $B_{22} = \{A, B, C\}$
split B_{22}	$B_1 = \{E\}$; $B_{21} = \{D\}$; $B_{221} = \{B\}$; $B_{222} = \{A, C\}$

Dunque per eseguire la **minimizzazione** bisogna:

- Eseguire la raffinazione della partizione su un DFA con funzione di transizione totale.
- Ogni blocco è temporaneamente considerato uno stato del DFA minimo.
- Lo stato iniziale del DFA minimo è quello rappresentato dal blocco che contiene s_0.
- Gli stati finali del DFA minimo sono quelli rappresentati dai blocchi che contengono gli stati finali del DFA originale.
- La funzione di transizione del DFA minimo è derivata dalla funzione $move_d$ originale nel seguente modo:
    - Se $s \in B_i$ e $move_d(s, a) \in B_j$, allora c'è una a-transizione nel DFA minimo dallo stato rappresentato da B_i allo stato rappresentato da B_j.

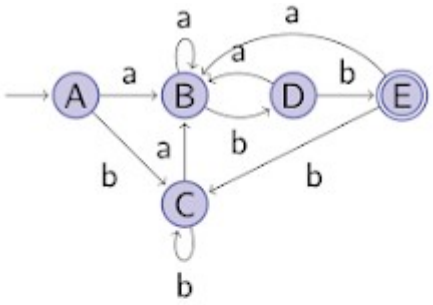
- Se uno degli stati temporanei è morto, rimuoverlo, e rimuovere tutte le transizioni verso e da esso (ad esempio se abbiamo messo un pozzo per fare diventare la funzione di transizione totale).

Nota: Il DFA minimo può avere una **funzione di transizione parziale**.

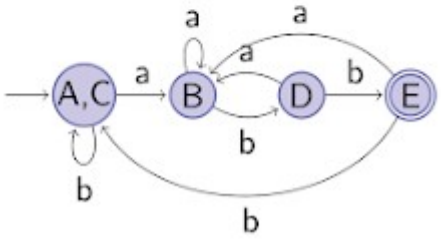
Uno **stato morto** è uno stato dal quale non si può raggiungere alcuno stato accettante.

Non parleremo di complessità per questo algoritmo perchè non vedremo le strutture dati, però esistono implementazioni che lavorano in $O(n \log n)$.

Ecco un esempio:

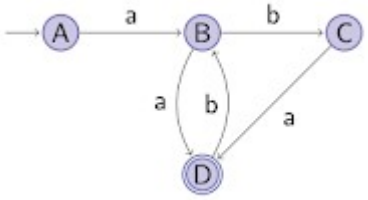


$\{E\}; \{D\}; \{B\}; \{A, C\}$

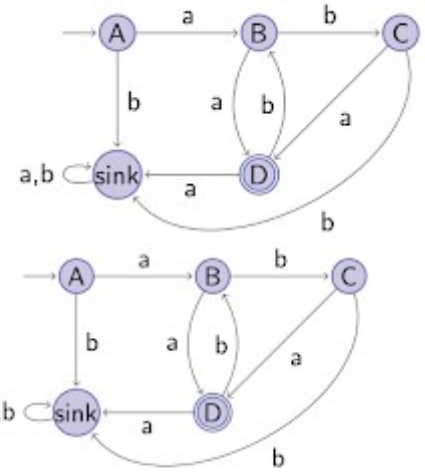


Vediamo ora un esempio in cui inseriamo un pozzo per far diventare la funzione di transizione totale:

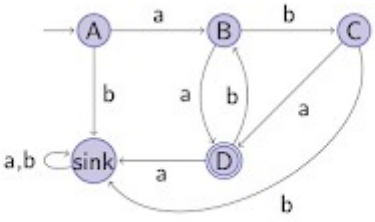
- Minimize the DFA



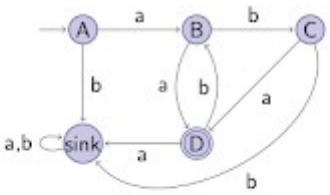
- The DFA has partial transition function, then take



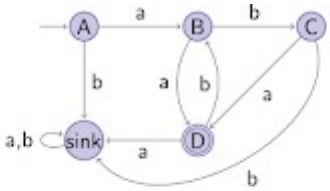
- Initial partition:
  $B_1 = \{D\}$; $B_2 = \{A, B, C, sink\}$
- Split B_2 by $\text{move}_d(A, a) \in B_2$ and $\text{move}_d(B, a) \notin B_2$
- First refinement:
  $B_1 = \{D\}$; $B_{21} = \{A, sink\}$; $B_{22} = \{B, C\}$



- First refinement:
  $B_1 = \{D\}$; $B_{21} = \{A, sink\}$; $B_{22} = \{B, C\}$
- Split B_{22} by $\text{move}_d(B, b) \in B_{22}$ and $\text{move}_d(C, b) \notin B_{22}$
- Second refinement:
  $B_1 = \{D\}$; $B_{21} = \{A, sink\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$



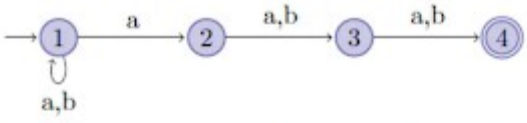
- Second refinement:
  $B_1 = \{D\}$; $B_{21} = \{A, sink\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$
- Split B_{21} by $\text{move}_d(A, a) \in B_{221}$ and $\text{move}_d(sink, a) \notin B_{221}$
- Last refinement:
  $B_1 = \{D\}$; $B_{211} = \{A\}$; $B_{212} = \{sink\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$



- Last refinement:
  $B_1 = \{D\}$; $B_{211} = \{A\}$; $B_{212} = \{sink\}$; $B_{221} = \{B\}$ $B_{222} = \{C\}$
- Remove the sink and all the transitions to/from it, get the original DFA
- The original DFA was already minimal

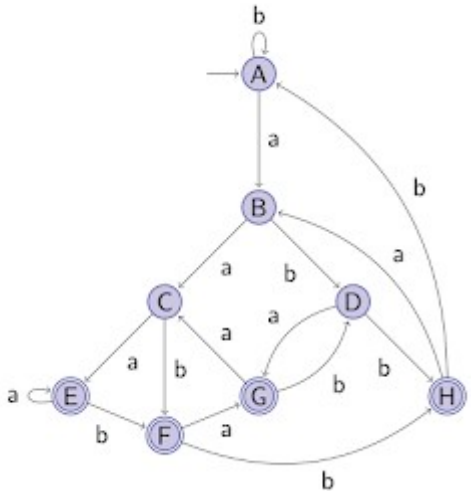
Vediamo che c'è uno stato composto solo dal pozzo, ed essendo uno stato morto (dal pozzo non si può raggiungere nessun stato finale) viene rimosso alla fine del procedimento.

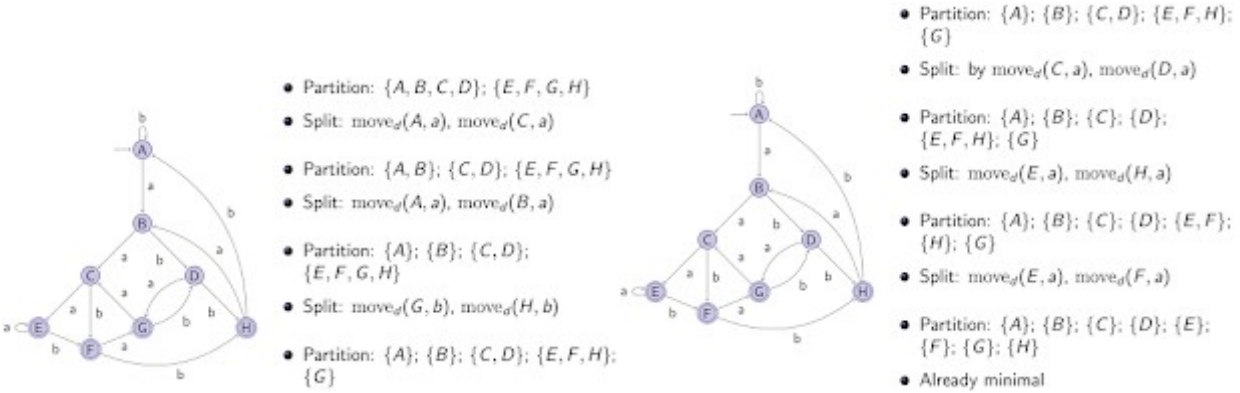
Vediamo ora un esempio in cui trasformiamo un NFA in un DFA (subset construction) e poi proviamo a minimizzarlo (partizionamento).



		a	b
A={1}	initial	B	A
B={1,2}		C	D
C={1,2,3}		E	F
D={1,3}		G	H
E={1,2,3,4}	final	E	F
F={1,3,4}	final	G	H
G={1,2,4}	final	C	D
H={1,4}	final	B	A

DFA resulting from the subset construction





In questo esempio, dopo la subset construction otteniamo un DFA con funzione di transizione totale ma non è sempre così.

Visto che ogni blocco contiene uno stato diverso, allora possiamo dire che il DFA di partenza è già minimo. Notiamo che in questo caso il DFA (e soprattutto il DFA minimo) è molto più grande del NFA da cui siamo partiti.

Vediamo ora quale è il **numero di stati in un DFA nel caso peggiore**.

Abbiamo il seguente lemma:

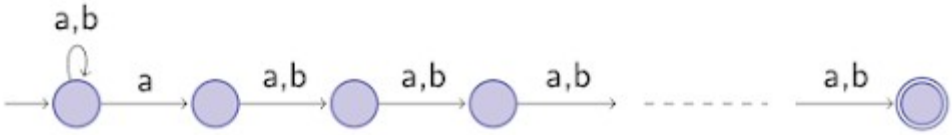
Per ogni $n \in N^+$, esiste un NFA con $n+1$ stati il cui DFA equivalente minimale ha almeno 2^n stati e una funzione di transizione totale.

In altre parole, esistono degli NFA i quali trasformati in DFA vedono crescere il numero di stati in maniera **esponenziale**.

Vediamo la *dimostrazione* per questo lemma.

Prendiamo in considerazione il linguaggio L: $L = L\left((a \mid b)^* a (a \mid b)^{n-1}\right)$, ovvero il linguaggio delle parole su $\{a, b\}$ che nella n-esima posizione a partire dal fondo possiede una a.

C'è un NFA che accetta il linguaggio L e ha esattamente $n+1$ stati:



Supponiamo, per assurdo, che esista un DFA minimale D che accetta L e ha $k < 2^n$ stati.

Ci sono esattamente 2^n parole distinte sull'insieme $\{a, b\}$ di lunghezza n.

Allora esistono due percorsi in D:

- La cui lunghezza è n
- Che rappresentano rispettivamente le parole w_1 e w_2 con $w_1 \neq w_2$
- Che **condividono almeno un nodo** (perché il DFA ha meno di 2^n stati)

Altrimenti, se non esistessero questi due percorsi, gli stati sarebbero come minimo 2^n.

Allora, per alcuni x_1, x_2, x, vale o:

- $w_1 = x_1 ax$ e $w_2 = x_2 bx$ oppure
- $w_1 = x_1 bx$ e $w_2 = x_2 ax$

Ovvero andiamo a pescare la prima posizione dal fondo per la quale le due parole sono differenti.
Senza perdita di generalità, supponiamo che $w_1 = x_1 ax$ e $w_2 = x_2 bx$.

Allora possiamo definire $w_1' = x_1 a b^{n-1} \in L(D)$.
Quindi lo stato raggiunto da w_1' in D è finale.
Ma questa è una contraddizione in quanto quello stato non può essere finale perché viene raggiunto anche da $x_2 b b^{n-1}$, che non appartiene al linguaggio $L(D)$.
Tutto il lemma gioca sul fatto che un DFA ha bisogno esattamente di tutti i possibili cammini che siano composti da $n-1$ alternanze di a e di b per esprimere lo stesso linguaggio espresso dall'NFA.
Dunque l'assurdità sta nel fatto che uno stato dovrebbe essere contemporaneamente finale e non finale, e questo perché abbiamo assunto che il DFA minimo abbia meno di 2^n nodi.

Proprietà dei linguaggi regolari

Abbiamo parlato dei linguaggi regolari come della classe di linguaggi che possono essere descritti da espressioni regolari o automi finiti. Tuttavia, questo di per sé potrebbe non fornire una comprensione chiara di ciò che è possibile e di ciò che non è possibile descrivere con un linguaggio regolare. Pertanto, adesso elencheremo alcune proprietà dei linguaggi regolari e forniremo alcuni esempi di linguaggi regolari e non regolari, oltre a dare regole empiriche informali che possono (a volte) essere utilizzate per decidere se un linguaggio è regolare.

In primo luogo, ribadiamo che le espressioni regolari, gli NFA (automi a stati finiti non deterministici) e i DFA (automi a stati finiti deterministici) hanno esattamente la stessa potenza espressiva: tutti possono descrivere tutti i linguaggi regolari e solo quelli. Alcuni linguaggi, tuttavia, possono avere descrizioni molto più brevi in una di queste forme rispetto alle altre.

Abbiamo già argomentato che da un'espressione regolare possiamo costruire un NFA le cui dimensioni sono lineari rispetto alle dimensioni dell'espressione regolare, e che la conversione di un NFA in un DFA può potenzialmente comportare un aumento esponenziale delle dimensioni. Dato che i DFA sono anche NFA, gli NFA sono chiaramente almeno altrettanto compatti (e a volte molto più compatti) dei DFA. Allo stesso modo, possiamo vedere che gli NFA sono almeno altrettanto compatti (fino a un piccolo fattore costante) rispetto alle espressioni regolari. Ma non abbiamo ancora considerato se il contrario è vero: un NFA può essere convertito in un'espressione regolare di dimensioni proporzionali. Purtroppo, la risposta è no: esistono classi di NFA (e persino DFA) che richiedono espressioni regolari che sono esponenzialmente più grandi per descriverli. Tuttavia, questo è principalmente di interesse accademico, poiché raramente dobbiamo effettuare conversioni in questa direzione.

Se siamo interessati solo a stabilire se un linguaggio è regolare piuttosto che alle dimensioni della sua descrizione, non importa quale formalismo scegliamo, quindi possiamo scegliere il formalismo che ci si addice meglio in ogni caso. A volte è più semplice descrivere un linguaggio regolare usando un DFA o un NFA invece di un'espressione regolare. Per i token dei linguaggi di programmazione, però, le espressioni regolari sono tipicamente molto adatte.

La subset construction mappa insiemi di stati di un NFA a stati di un DFA. Dato che ci sono $2^n - 1$ insiemi non vuoti di *n* stati di un NFA, il DFA risultante può potenzialmente avere un numero di stati esponenzialmente superiore rispetto all'NFA.

Pumping lemma per linguaggi regolari

Il "pumping lemma" per i linguaggi regolari è un teorema fondamentale nella teoria dei linguaggi formali. Esso fornisce una condizione che deve essere verificata da un linguaggio affinché possa essere considerato regolare.

Lemma: Sia L un linguaggio regolare. Allora:

- $\exists\, p \in N^+$ (detta "**pumping length**") tale che
- $\forall\, z \in L$ tale che $|z| > p$
- $\exists\ u, v, w$ tali che
  - $z = uvw$ e
  - $|uv| \leq p$ e
  - $|v| > 0$ e
  - $\forall\, i \in N,\ u v^i w \in L$

Dimostrazione:

- Sia L un linguaggio regolare.
- Allora esiste un DFA $D = (S, A, move_d, s_0, F)$ tale che $L = L(D)$.
- Sia $p = |S| - 1$.
- Allora tutti i percorsi da s_0 a uno stato finale che attraversano ogni stato al massimo una volta hanno lunghezza limitata da p (se la lunghezza della stringa è p, allora abbiamo visitato $p + 1 = |S|$ stati)
- Quindi, data $|z| > p$, per alcuni $a_1, \ldots, a_p$ e per qualche z', possiamo scomporre z in questa maniera $z = a_1 \cdots a_p z'$ e almeno uno stato, diciamo s^*, viene attraversato più di una volta lungo il cammino $a_1 \cdots a_p$.
- Allora esiste un ciclo in D che va da s^* a s^* e che rappresenta $a_{i+1} \cdots a_j$ per alcuni i e j tali che $i < j \leq p$.
- Allora prendendo come
  - $u = a_1 \cdots a_i$
  - $v = a_{i+1} \cdots a_j$
  - $w = \begin{cases} z' & \text{if } j = p \\ a_{j+1} \cdots a_p z' & \text{if } j < p \end{cases}$

- Allora:
  - $|uv| \leq p$ (perchè siamo sicuri di incontrare la seconda ricorrenza di s^* di entro p transizioni)
  - La lunghezza di v è almeno 1 (per definizione, un ciclo ha almeno una transizione, ed essendo un DFA allora sicuramente leggerà almeno un carattere)
  - $u\,v^i\,w$ è accettata da D per ogni $i \in N$

A cosa serve questo lemma?
Se un linguaggio è regolare, allora vale *thesis*:

$$\exists p \in \mathbb{N}^+.\ \forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w.\ P$$
where
$$P \equiv (z = uvw \text{ and } |uv| \leq p \text{ and } |v| > 0 \text{ and } \forall i \in \mathbb{N}. uv^i w \in \mathcal{L})$$

Dunque il lemma non può assolutamente essere usato per dimostrare che un certo linguaggio sia regolare.
È usato invece per verificare che un linguaggio non è regolare, infatti

- Assumiamo che L sia un linguaggio regolare
- Mostriamo che *thesis* è falso, ovvero che $not\,(thesis)$ è vero
- Per contraddizione possiamo concludere che L non è un linguaggio regolare

Dunque la $not\,(thesis)$ è:

$$\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w.\ Q$$
where
$$Q \equiv (z = uvw \text{ and } |uv| \leq p \text{ and } |v| > 0) \text{ implies}$$
$$(\exists i \in \mathbb{N}. uv^i w \notin \mathcal{L})$$

Proviamo che il linguaggio $L = \{a^n b^n \mid n > 0\}$ non è regolare.
Intuitivamente, notiamo che lo stato in cui ci si trova dopo aver letto 2 occorrenze di a non può essere lo stesso stato in cui ci si trova dopo aver letto 3 occorrenze di a, questo perché non è possibile tener conto di quante a sono state lette, e dunque il numero di stati dovrebbe essere infinito in quanto n è infinito.
Vediamo come dimostrarlo tramite il pumping lemma.
Supponiamo che L sia un linguaggio regolare, e sia p un generico numero intero positivo.
Consideriamo $z = a^p b^p$, e quindi $|z| > p$.
Osserviamo che $\forall u, v, w \quad if\,(z = uvw \ \wedge\ |uv| \leq p \ \wedge\ |v| > 0)$:

- Allora v contiene solamente a, poichè $|uv| \leq p$
- E v contiene almeno una a, poichè $|v| > 0$

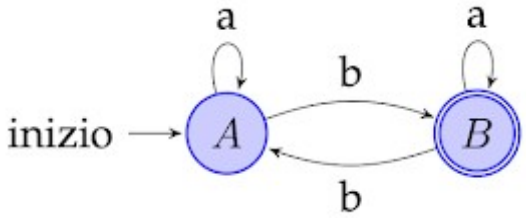
Quindi, per qualche $j > 0$, $u\,v^2 w = a^p a^j b^p$
Allora $u\,v^2 w \notin L$, il che contraddice il Pumping Lemma per i linguaggi regolari.

La proprietà più basilare di un DFA è che è finito: ha un numero finito di stati e nessun altro posto in cui memorizzare informazioni. Questo significa, ad esempio, che qualsiasi linguaggio

che richieda un conteggio illimitato non può essere regolare. Un esempio di ciò è il linguaggio $\{a^n b^n \mid n>0\}$, cioè, qualsiasi sequenza di a seguita da una sequenza dello stesso numero di b. Se dobbiamo decidere l'appartenenza a questo linguaggio con un DFA che legge l'input da sinistra a destra, dobbiamo, al momento in cui abbiamo letto tutte le a, sapere quant'è il loro numero, così da poterlo confrontare con il numero di b. Ma poiché un automa finito non può fare conteggi arbitrariamente elevati, il linguaggio non è regolare. Un linguaggio simile che non è regolare è il linguaggio delle parentesi ben annidate. Tuttavia, se limitiamo la profondità di annidamento delle parentesi a una costante n, possiamo riconoscere questo linguaggio con un DFA che ha $n+1$ stati (da 0 a n), dove lo stato i corrisponde a i parentesi aperte non abbinate. Lo stato 0 è sia lo stato iniziale che l'unico stato accettante.

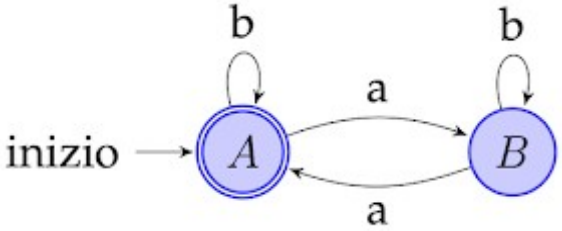
Vediamo ora un po' di linguaggi e vediamo se sono regolari o meno.

Il linguaggio $L1$ è definito come l'insieme di parole sull'alfabeto $\{a,b\}$ con un numero dispari di occorrenze della lettera b. $L1$ è regolare?
La risposta è si, infatti possiamo costruire il seguente automa finito che lo rappresenta:



Inoltre abbiamo anche l'espressione regolare $a^* b a^* (a^* b a^* b a^*)^*$ descrive questo linguaggio in quanto sicuramente accetterò parole composte da una b e un numero arbitrario di a prima e dopo e poi nel caso in cui volessi aumentare il numero di b, sono obbligato a farlo a coppie con la possibilità di interporre un numero arbitrario di a prima, in mezzo o alla fine.
Un'altra soluzione possibile è $(b a^* b \mid a)^* b a^*$.

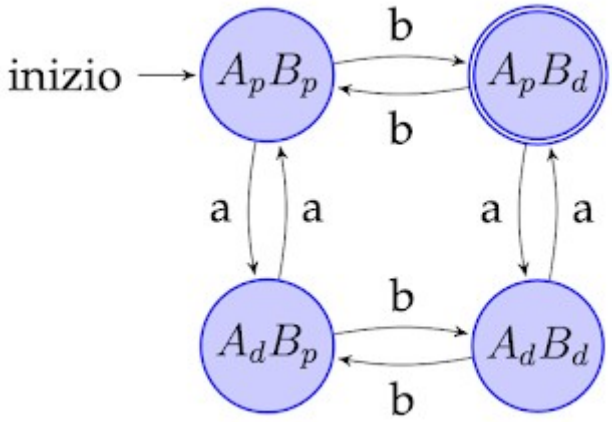
Il linguaggio $L2$ è definito come l'insieme di parole sull'alfabeto $\{a,b\}$ con un numero pari di occorrenze della lettera a. $L2$ è regolare?
La risposta è si, infatti possiamo costruire il seguente automa finito che lo rappresenta:



Un'espressione regolare che denota questo linguaggio è $(b^* a b^* a b^*)^*$.

Il linguaggio $L3$ è definito come l'insieme di parole sull'alfabeto $\{a,b\}$ con un numero pari di occorrenze della lettera a e un numero dispari di occorrenze della lettera b. $L3$ è regolare?

Se si prova ad unire i due automi visti precedentemente si potrebbe pensare che $L3$ non sia regolare in quanto non si riesce a trovare un modo per unirli adeguatamente. Possiamo però pensare a una macchina a stati in maniera diversa da come le abbiamo viste fino ad ora, con ogni stato che rappresenta tutte le combinazioni possibili di quante a e quante b abbiamo letto:



Lo stato iniziale ovviamente deve essere quello dove abbiamo letto 0 a e 0 b, dunque entrambi pari, mentre lo stato finale quello con a pari e b dispari.

Un altro modo per arrivare direttamente alla soluzione senza dover fare un ragionamento di questo tipo, ma pensando invece all'intersezione delle soluzioni che abbiamo visto per $L1$ e $L2$ è: immaginiamo di avere una parola w a disposizione e di poterci contemporaneamente muovere su entrambi i cammini degli automi descritti come soluzioni di $L1$ e di $L2$. Continuiamo a spostarci eseguendo le transizioni coerentemente alla parola w scelta, fino a che non arriviamo al carattere terminatore. A questo punto sono dati due casi:

1. se mi trovo in entrambi gli automi su uno stato finale allora $w \in L$
2. se almeno uno dei due stati su cui mi trovo non è uno stato finale allora $w \notin L$

Vediamo ora le **proprietà di chiusura dei linguaggi regolari**.
Lemma
I linguaggi regolari sono chiusi rispetto a

- **Unione** (come i linguaggi liberi)
  Ciò significa che se hai due linguaggi regolari, puoi unirli (o combinarli) per ottenere un altro linguaggio regolare. Ad esempio, se hai il linguaggio A che consiste nelle stringhe abc e def, e il linguaggio B che consiste in 123 e 456, l'unione di A e B sarebbe un altro linguaggio regolare che contiene tutte e quattro le stringhe: abc, def, 123 e 456.
  Se abbiamo espressioni regolari s e t per due linguaggi, l'espressione regolare $s|t$ descrive l'unione di questi linguaggi.
- **Concatenazione** (come i linguaggi liberi)
  Ciò significa che se hai due linguaggi regolari, puoi concatenarli (o metterli uno dietro l'altro) per ottenere un altro linguaggio regolare. Ad esempio, se hai il linguaggio A che consiste nelle stringhe abc e def, e il linguaggio B che consiste in 123 e 456, la concatenazione di A e B sarebbe un altro linguaggio regolare che consiste in tutte le possibili combinazioni di una stringa da A seguita da una stringa da B, ad esempio

$abc123$ e $¿456$.
Allo stesso modo dell'unione, i linguaggi regolari sono chiusi rispetto a questa operazione in quanto corrisponde ad un operatore di base delle espressioni regolari.

- **Complemento**
  Ciò significa che se hai un linguaggio regolare, puoi prendere il suo complemento (cioè tutte le stringhe che non sono nel linguaggio originale) e ottenere un altro linguaggio regolare. Ad esempio, se hai un linguaggio che contiene tutte le stringhe di lunghezza dispari, il suo complemento conterrà tutte le stringhe di lunghezza pari.
  Utilizzando l'automa che denota un linguaggio, vi è la possibilità di applicare una particolare operazione per trasformarlo nel suo complementare? Sì, è possibile: basta semplicemente rendere finali tutti quegli stati che prima non lo erano e, simmetricamente, non finali tutti quelli che prima lo erano.
  Relativamente a questo procedimento è però necessario fare una precisazione: tale metodo funziona solamente se il DFA presenta una funzione di transizione totale: intuitivamente, è necessario riconoscere tutte quelle parole che non appartengono al linguaggio regolare in quanto queste apparterranno al linguaggio del DFA complementare; se per assurdo si avesse la mancanza di una x-transizione (non fondamentale nel primo DFA) non si avrebbe la possibilità di costruire il set completo delle parole.
- **Intersezione**
  I linguaggi regolari sono chiusi rispetto all'intersezione. Ciò significa che se hai due linguaggi regolari, puoi trovare l'intersezione (o l'insieme di tutte le stringhe che sono contemporaneamente in entrambi i linguaggi) e ottenere un altro linguaggio regolare.
  Questo è dimostrabile tramite De Morgan:
  $$\mathcal{L}_1 \cap \mathcal{L}_2 = \neg(\neg(L1 \cap L2)) = \neg(\neg\mathcal{L}_1 \cup \neg\mathcal{L}_2)$$

Lexer e generatori di lexer

Nelle sezioni precedenti abbiamo visto come possiamo convertire una descrizione di linguaggio scritta come espressione regolare in una rappresentazione eseguibile in modo efficiente (un DFA). Quello che vogliamo è qualcosa in più, cioè un programma che effettua l'**analisi lessicale**, ovvero un **lexer**:

- Un lexer deve essere in grado di distinguere tra diversi tipi di token, ad esempio numeri, variabili e parole chiave. Ognuno di questi è descritto dalla propria espressione regolare.
- Un lexer non verifica se l'intero input è incluso nei linguaggi definiti dalle espressioni regolari. Invece, deve suddividere l'input in pezzi (token), ciascuno dei quali è incluso in uno dei linguaggi.
- Se ci sono diverse modalità per suddividere l'input in token legali, il lexer deve decidere quale di queste utilizzare.

Ricordiamoci che lo scopo dell'analisi lessicale è quello di fornire l'input per l'analisi sintattica.

"C99 grammar.y" contiene la specifica della grammatica per il linguaggio di programmazione C nel suo standard del 1999 (C99).

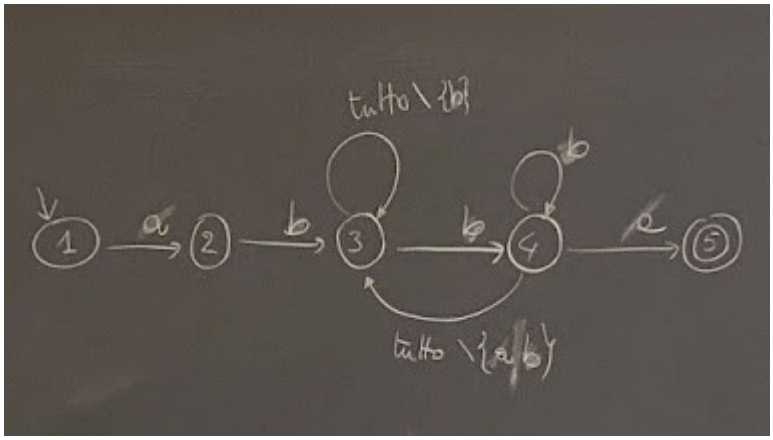
Questo file specifica le regole sintattiche che definiscono come il codice C deve essere strutturato, comprese le espressioni, le dichiarazioni, le istruzioni e così via.

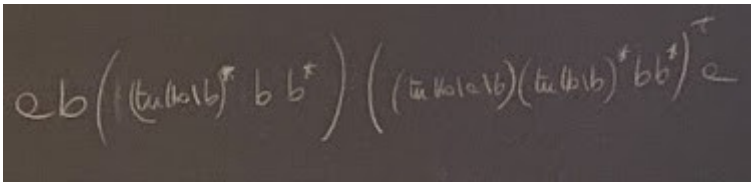
Un programma che prende un insieme di definizioni di token (ognuna composta da un'espressione regolare e un nome del token) e genera un lexer è chiamato **generatore di lexer**.

Le tipiche scelte per i token sono:

1. Un token per ogni parola chiave (ad esempio $while$, if, …)
2. Token per gli operatori (o per classi di operatori)
3. Un token per gli identificatori
4. Token per i simboli di punteggiatura

Scrivere le espressioni regolari per questi token è abbastanza semplice, per i commenti però è leggermente più complicato, ecco un esempio di DFA e di espressione regolare che riconosce i commenti di C, che sono della forma /* $commento$ */ con commento che può comprendere i caratteri / e * esclusa la specifica ricorrenza di fine commento */.
Ridenominiamo / con a e * con b per questioni di semplicità di lettura:



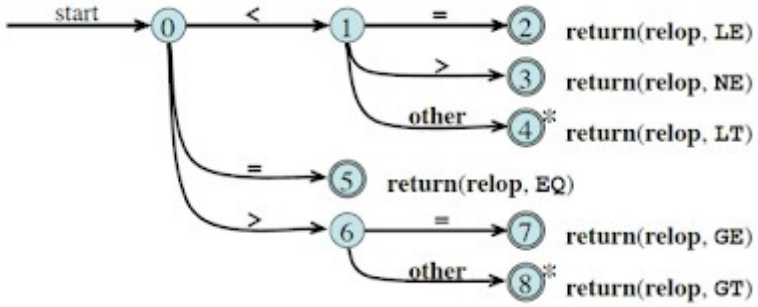


L'obiettivo del lexer è quindi riconoscere i cosiddetti lessemi, ossia quelle parti del programma che corrispondono ai token, e ritornarli.
Ciascuno di questi, di solito, viene ritornato sotto forma di coppia
$<token\text{-}name>:<token\text{-}value>$, dove:

- $<token\text{-}name>$ è il nome scelto per denotare quel preciso token
- $<token\text{-}value>$ è tipicamente un puntatore a una entry della symbol table, in cui si va a salvare tutte le informazioni relative a quel preciso token di tipo $<token\text{-}name>$

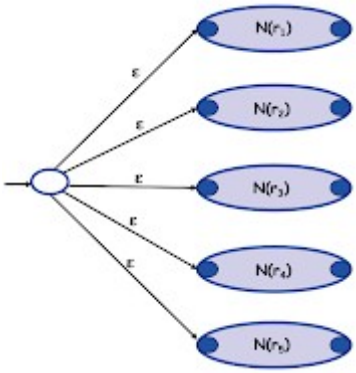
I **lessemi** sono descritti da espressioni regolari e vengono riconosciuti da una macchina a stati che può intraprendere azioni appropriate quando riconosce parole.

Ecco un esempio di DFA per un lexer:



Le * sugli stati 4 e 8 significa che abbiamo letto dei caratteri “in più”, vedremo dopo in che senso e come comportarsi in questa situazione.

L'approccio più semplice sarebbe generare un DFA per ciascuna definizione di token e applicare i DFA uno alla volta all'input. Tuttavia, ciò può essere piuttosto lento, quindi invece genereremo, dall'insieme di definizioni di token, un singolo DFA che testa contemporaneamente tutti i token. Questo non è difficile da fare: se i token sono definiti dalle espressioni regolari $r_1, r_2, \ldots, r_n$, allora l'espressione regolare $r_1 \mid r_2 \mid \ldots \mid r_n$ descrive l'unione dei linguaggi $r_1, r_2, \ldots, r_n$ e il DFA costruito da questa espressione regolare combinata riconoscerà contemporaneamente tutti i tipi di token.



Tuttavia, desideriamo anche distinguere tra diversi tipi di token, quindi dobbiamo essere in grado di sapere quale dei molti token è stato riconosciuto dal DFA. Il modo più semplice per farlo è il seguente:

1. Costruire gli NFA $N_1, N_2, \ldots, N_n$ per ciascuna delle $r_1, r_2, \ldots, r_n$.
2. Contrassegnare gli stati accettanti degli NFA con il nome dei token che accettano.
3. Combinare gli NFA in un singolo NFA aggiungendo uno stato iniziale nuovo che ha ε-transizioni verso ciascuno degli stati iniziali degli NFA.
4. Convertire il NFA combinato in un DFA.
5. Ogni stato accettante del DFA è composto da un insieme di stati NFA, alcuni dei quali sono stati accettanti che abbiamo contrassegnato per tipo di token nel passo 2. Questi contrassegni vengono utilizzati per contrassegnare gli stati accettanti del DFA in modo che ciascuno indichi i tipi di token che accetta.

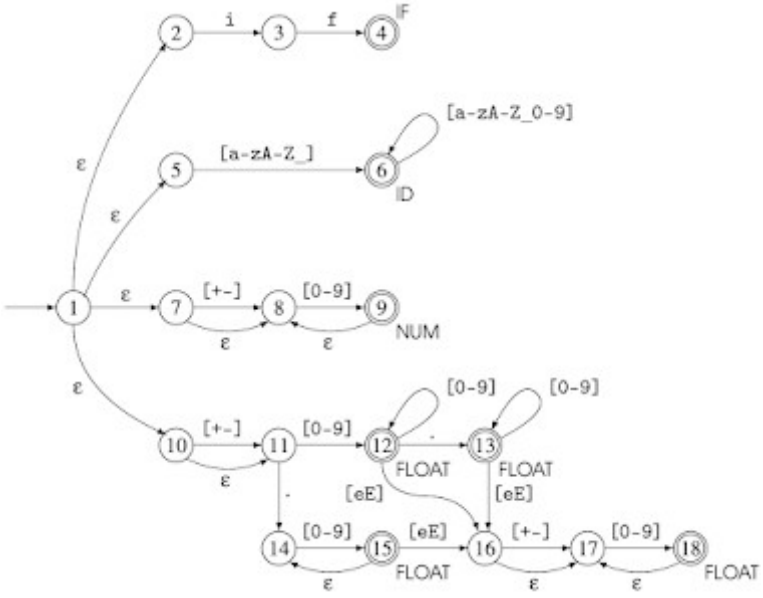
Se lo stesso stato accettante nel DFA può accettare diversi tipi di token, è perché questi si sovrappongono. Questo non è insolito, poiché le parole chiave di solito si sovrappongono con i nomi delle variabili e una descrizione di costanti in virgola mobile può includere anche costanti intere. In tali casi, possiamo fare una delle due cose:

- Consentire al generatore di lexer di generare un errore e richiedere all'utente di assicurarsi che i token siano disgiunti.
- Consentire all'utente del generatore di lexer di scegliere quale dei token è preferito.

Può essere abbastanza difficile (sebbene sempre possibile) con le espressioni regolari definire, ad esempio, l'insieme di nomi che non sono parole chiave. Di conseguenza, è comune lasciare al lexer la scelta secondo una lista prioritizzata. Normalmente, l'ordine in cui i token sono definiti nell'input al generatore di lexer indica la priorità (i token definiti precedentemente hanno la precedenza su quelli definiti successivamente). Di conseguenza, le parole chiave di solito vengono definite prima dei nomi delle variabili, il che significa che, ad esempio, la stringa *if* viene riconosciuta come parola chiave e non come nome di variabile. Quando uno stato accettante in un DFA contiene stati NFA accettanti con marchi diversi, viene utilizzato il marchio corrispondente al token con la priorità più alta (definito inizialmente). Quindi, possiamo semplicemente cancellare tutti tranne un marchio da ciascuno stato accettante. Questa è una soluzione molto semplice ed efficace al problema.

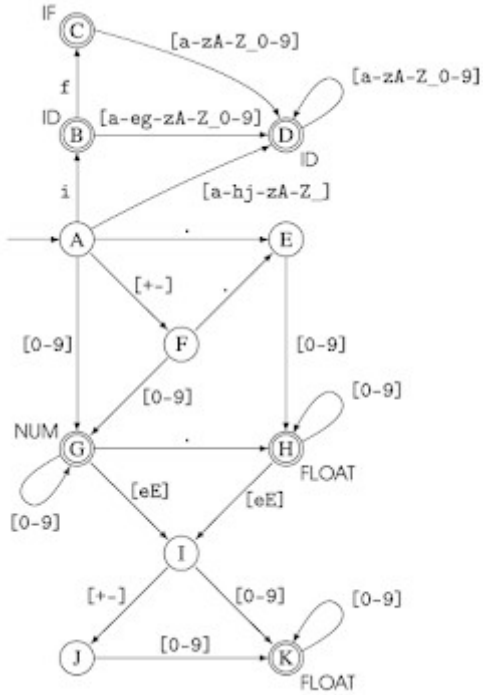
Quando abbiamo descritto la minimizzazione dei DFA, abbiamo utilizzato due gruppi iniziali: uno per gli stati accettanti e uno per gli stati non accettanti. Ora che ci sono diversi tipi di stati accettanti (uno per ciascun token), dobbiamo utilizzare un gruppo per ciascun token, quindi avremo un totale di $n+1$ gruppi iniziali quando abbiamo n token diversi.

Per illustrare la regola di precedenza, la figura mostra un NFA ottenuto combinando gli NFA per i nomi delle variabili, la parola chiave *if*, i numeri interi e i numeri in virgola mobile, come descritto dalle espressioni regolari.



Gli NFA individuali sono (versioni semplificate di) ciò che si ottiene dal metodo descritto nella costruzione di Thompson. Quando una transizione è contrassegnata da un insieme di caratteri, è un modo abbreviato per un insieme di transizioni ciascuna contrassegnata da un singolo carattere. Gli stati accettanti sono contrassegnati con i nomi dei token come descritto in precedenza.

Il DFA minimizzato corrispondente è mostrato nella figura:



Nota che lo stato G è una combinazione degli stati 9 e 12 dell'NFA, quindi può accettare sia NUM che $FLOAT$, ma poiché gli interi hanno la priorità sui numeri in virgola mobile, abbiamo contrassegnato G solo con NUM.

Facciamo alcune osservazioni:

- Il carattere ε è un'astrazione usata da noi, infatti non appartiene all'alfabeto di nessun linguaggio
- Esiste un algoritmo che dato in input un automa a stati finiti ritorna un'espressione regolare, noi non lo vedremo però è importante sapere che esiste

Divisione del flusso di input.
Come accennato, il lexer deve suddividere l'input in token. Ciò può essere fatto in diversi modi. Ad esempio, la stringa $if\ 17$ può essere suddivisa in molti modi diversi:

- Come un unico token, che è il nome della variabile $if\ 17$.
- Come il nome della variabile $if\ 1$ seguito dal numero 7.
- Come la parola chiave if seguita dal numero 17.
- Come la parola chiave if seguita dai numeri 1 e 7.
- Come il nome della variabile i seguito dal nome della variabile $f\ 17$.
- E molti altri.

Una convenzione comune prevede che venga scelto **il prefisso più lungo** dell'input che corrisponde a un qualsiasi token. Di conseguenza, verrà scelta la prima delle suddivisioni possibili di $if\ 17$ descritte precedentemente. Si noti che il principio del match più lungo ha la precedenza sull'ordine di definizione dei token. Quindi, anche se la stringa inizia con la parola chiave if, che ha una priorità più alta rispetto ai nomi delle variabili, viene scelto il nome della variabile perché è più lungo.
Questo viene fatto perché, come si può vedere dal primo lexer che abbiamo visto, se scriviamo <= intendiamo minore o uguale e se ci fermassimo a < per costruire il token sarebbe scorretto in quanto non potremmo mai riconoscere il token <=.

Linguaggi moderni come C, Java o SML seguono questa convenzione, e lo fanno anche la maggior parte dei generatori di lexer, ma alcuni (principalmente linguaggi più datati) come FORTRAN non lo fanno. Quando vengono utilizzate altre convenzioni, i lexer devono essere scritti a mano per gestire tali convenzioni o è necessario aggirare le convenzioni utilizzate dal generatore di lexer. Alcuni generatori di lexer consentono all'utente di avere un certo controllo sulle convenzioni utilizzate.

Il principio della corrispondenza del prefisso più lungo viene gestito facendo leggere al DFA finché può, fino a quando non raggiunge la fine dell'input o non è definita nessuna transizione sul prossimo simbolo in input. Se lo stato corrente in questo punto è di accettazione, siamo fortunati e possiamo semplicemente produrre in output il token corrispondente. Se non lo è, dobbiamo tornare all'ultima volta in cui eravamo in uno stato di accettazione e produrre in output il token indicato da questo. I caratteri letti da allora vengono reinseriti nel flusso di input. Il lexer deve quindi conservare i simboli che ha letto dall'ultima volta in cui si trovava in uno stato di accettazione in modo che possano essere reinseriti nell'input in situazioni del genere. Se non siamo alla fine del flusso di input, riavviamo il DFA (nel suo stato iniziale) sul rimanente input per trovare i token successivi.

A titolo di esempio, considera l'analisi lessicale della stringa $3e\text{-}y$ con il DFA precedente. Raggiungiamo lo stato di accettazione G dopo aver letto la cifra 3. Tuttavia, possiamo continuare a fare transizioni legali verso lo stato I su e e poi verso lo stato J su - (poiché potrebbero essere l'inizio della parte dell'esponente di un numero reale). È solo quando, nello stato J, scopriamo che non c'è una transizione su y che ci rendiamo conto che non è il caso. Dobbiamo ora tornare all'ultimo stato di accettazione (G) e produrre in output il numero 3 come primo token e reinserire - ed e nel flusso di input, in modo che possiamo continuare con $e\text{-}y$ quando cerchiamo i token successivi.

Dunque per eseguire il pattern matching basata su NFA:

- Simula l'NFA
- Le azioni sono associate agli stati finali
- Trova la corrispondenza più lunga: continua a simulare finché non è possibile effettuare ulteriori spostamenti
- Se il set di stati raggiunto ha azioni associate, esegui la "prima" azione
- Altrimenti

- Guarda all'indietro la sequenza di stati
- Scegli il primo set di stati contenente almeno uno stato finale
- Esegui la "prima" azione
- Aggiorna il puntatore al buffer di input di conseguenza

La corrispondenza di modelli basata su DFA ottenuti tramite costruzione di sottoinsiemi è analogo.

Errori lessicali

Se nessun prefisso della stringa di input forma un token valido, si è verificato un errore lessicale. In questo caso, il lexer di solito segnalerà un errore. A questo punto, potrebbe smettere di leggere l'input o potrebbe tentare un'ulteriore analisi lessicale saltando i caratteri fino a quando non trova un prefisso valido.
Lo scopo dell'approccio successivo è cercare ulteriori errori lessicali nella stessa stringa di input, in modo che l'utente possa correggerne diversi prima di eseguire nuovamente il lexer. Tuttavia, alcuni di questi errori successivi potrebbero non essere veri errori ma potrebbero essere causati dal lexer che non salta abbastanza caratteri (o ne salta troppi) dopo aver trovato il primo errore. Ad esempio, se l'inizio di un commento è malformato, il lexer potrebbe cercare di interpretare i contenuti del commento come token individuali, e se la fine di un commento è malformata, il lexer leggerà fino alla fine del successivo commento (se presente) prima di continuare, saltando quindi troppo testo.
Quando il lexer trova un errore, il consumatore dei token prodotti dal lexer (ad esempio, il resto del compilatore) di solito non può produrre da sé un risultato valido. Tuttavia, il compilatore potrebbe cercare altri errori nell'input rimanente, consentendo nuovamente all'utente di individuare diversi errori in un singolo ciclo di modifica-compilazione. Di nuovo, alcuni degli errori successivi potrebbero essere effettivamente errori spurii causati dagli errori lessicali, quindi l'utente dovrà fare congetture sulla validità di ogni messaggio di errore tranne il primo, poiché solo il primo messaggio di errore è garantito essere un vero errore. Tuttavia, è stato dimostrato che tale recupero degli errori, quando l'input è così grande che riavviare il lexer dall'inizio dell'input comporta un notevole overhead di tempo, aiuta la produttività individuando più errori in meno tempo. Meno comunemente, il lexer potrebbe lavorare interattivamente con un editor di testo e riavviare dal punto in cui è stato individuato un errore dopo che l'utente ha cercato di correggerlo.

Parliamo ora di **generatori di lexer**.
Un generatore di lexer utilizzerà tipicamente una notazione per le espressioni regolari simile a quella descritta precedentemente, ma potrebbe richiedere che i caratteri dell'alfabeto siano messi tra virgolette per distinguerli dai simboli utilizzati per costruire le espressioni regolari. Ad esempio, un * inteso per corrispondere a un simbolo di moltiplicazione nell'input viene distinto da un * utilizzato per indicare la ripetizione mettendo tra virgolette il simbolo *, ad esempio come `'*'`. Inoltre, alcuni generatori di lexer estendono le espressioni regolari in vari modi, ad esempio, consentendo di specificare un insieme di caratteri elencando quelli che non fanno parte dell'insieme. Questo è utile, ad esempio, per specificare i simboli all'interno di un commento fino ai caratteri di terminazione.

L'input per il generatore di lexer conterrà normalmente un elenco di espressioni regolari che ciascuna denota un token. Ciascuna di queste espressioni regolari ha un'**azione** associata. L'azione descrive cosa viene passato al consumatore (ad esempio, il parser), tipicamente un elemento di un tipo di dato token, che descrive il tipo di token (NUM, ID, ecc.) e talvolta informazioni aggiuntive come il valore di un token numerico, il nome di un token identificatore e, forse, la posizione del token nel file di input. Le informazioni necessarie per costruire tali valori sono tipicamente fornite dal generatore di lexer attraverso funzioni di libreria o variabili che possono essere utilizzate nelle azioni.

Normalmente, il generatore di lexer richiede che spazi bianchi e commenti siano definiti da espressioni regolari. Le azioni per queste espressioni regolari sono tipicamente vuote, il che significa che spazi bianchi e commenti vengono semplicemente ignorati.
Un'azione può essere più di una semplice restituzione di un token. Se, ad esempio, un linguaggio ha un gran numero di parole chiave, un DFA che le riconosce tutte individualmente può essere piuttosto grande. In tali casi, le parole chiave non vengono descritte come espressioni regolari separate nella definizione del lexer, ma vengono invece trattate come casi speciali del token identificatore. L'azione per gli identificatori cercherà il nome in una tabella di parole chiave e restituirà il tipo di token appropriato (o un token identificatore se il nome non è una parola chiave). Una strategia simile può essere utilizzata se il linguaggio consente agli identificatori di oscurare le parole chiave.

Un altro utilizzo di azioni di lexer non triviali è per i commenti nidificati. In principio, un'espressione regolare (o automa finito) non può riconoscere commenti nidificati arbitrariamente, ma utilizzando un contatore globale, le azioni per i token di commento possono tener traccia del livello di nidificazione. Se sono consentite sequenze di escape (per definire, ad esempio, caratteri di controllo) nelle costanti di stringa, le azioni per i token di stringa tradurranno tipicamente la stringa contenente queste sequenze in una stringa in cui sono state sostituite dai caratteri che rappresentano.

A volte i generatori di lexer permettono diversi punti di inizio. Nell'esempio visto precedentemente, tutte le espressioni regolari condividono lo stesso stato iniziale. Tuttavia, un singolo lexer può essere utilizzato, ad esempio, sia per i token nel linguaggio di programmazione che per i token nell'input a quel linguaggio. Spesso ci sarà una buona quantità di condivisione tra questi insiemi di token (i token consentiti nell'input possono, ad esempio, essere un sottoinsieme dei token consentiti nei programmi). Quindi, è utile consentire loro di condividere un NFA, poiché ciò risparmierà spazio. Il DFA risultante avrà diversi stati iniziali. Uno stato di accettazione può ora avere più di un nome di token associato, purché provengano da diversi insiemi di token (corrispondenti a diversi punti di inizio).
Oltre all'uso di questa caratteristica per diverse fonti di testo (programma e input), può essere utilizzata localmente all'interno di un singolo testo per leggere token molto complessi. Ad esempio, i commenti nidificati e le stringhe con formati complessi (con sequenze di escape non banali) possono essere più facili da gestire se viene utilizzata questa funzionalità.

Flex

QUESTO CAPITOLO È FUORI DAL PROGRAMMA

I generatori lessicali sono dei software che, date in input delle espressioni regolari e delle azioni da compiere quando vengono riconosciute parole appartenenti ai rispettivi linguaggi denotati dalle espressioni regolari, creano l'automa.

L'idea di creare un generatore di analizzatori lessicali è nata in concomitanza con la prima definizione e implementazione del compilatore del C, e l'idea era di evitare di dover scrivere ogni volta, per ogni programma, un analizzatore lessicale e uno sintattico a mano.

Flex (Fast lex), il primo della sua specie, è uno strumento ampiamente utilizzato per generare analizzatori lessicali, noti anche come lexer, all'interno di compilatori e altri strumenti che richiedono l'analisi del testo. Si tratta di un generatore di lexer che accetta specifiche scritte in un linguaggio di definizione dei token e produce un programma C che funge da scanner per riconoscere i token all'interno di un testo in ingresso. Flex semplifica la creazione di analizzatori lessicali, offrendo una soluzione efficace e efficiente per riconoscere e suddividere il testo in token.

Ci sono strumenti simili a lex e flex disponibili per molti dei linguaggi principali, incluso Java.

Vediamo come utilizzare lo strumento flex per creare un analizzatore lessicale:

1. Si inizia scrivendo un file chiamato "file.l" che conterrà le specifiche per Flex, dove si definiranno i pattern dei token e le azioni associate ad essi.
2. Successivamente, il file "file.l" viene compilato utilizzando Flex tramite il comando `flex file.l`. Questo processo genera un file di output denominato "lex.yy.c", che conterrà il codice sorgente C dell'analizzatore lessicale basato sulle specifiche definite in "file.l".
3. Una volta ottenuto il file "lex.yy.c", viene compilato utilizzando il compilatore gcc tramite il comando `gcc lex.yy.c -lfl`. Questo passaggio crea l'eseguibile dell'analizzatore lessicale.
4. Infine, per testare l'analizzatore lessicale appena creato, si esegue il comando `./a`, che avvia il programma e consente di interagire con l'analizzatore, testando la sua capacità di riconoscere e dividere il testo in token sulla base delle specifiche definite in "file.l".

I file in flex sono strutturati in tre parti: preambolo, patterns e epilogo:

```
.....           (preamble)
%%
.....           (patterns)
%%
.....           (epilogue)
```

Vediamo dove viene inserito cosa:

```
%{ code
}%

shorthands for patterns

%%
pattern-1 {action-1};
pattern-2 {action-2};
....

%%
user routines, copied into lex.yy.c
```

Nel preambolo (scritto in C) vengono definite macro, incluse librerie e abbreviazioni per i pattern, come ad esempio "lettera" per le lettere. Qui vengono anche dichiarate e inizializzate le variabili che saranno utilizzate nelle azioni e nell'epilogo.

Nei pattern invece funziona cosi:
pattern-1 rappresenta un'espressione regolare, mentre action-1 definisce l'azione da eseguire una volta riconosciuto un elemento nel linguaggio identificato da pattern-1.
Come per ogni software, ci sono convenzioni specifiche per fornire l'input, ad esempio utilizzando [a-z0-9] come abbiamo già visto in precedenza.

Nell'epilogo (che è facoltativo), con yylex(), richiamo il lexer appena creato, e successivamente posso eseguire diverse operazioni, simili a un normale programma in C.

È importante notare che anche flex rispetta le regole della corrispondenza più lunga e della prima corrispondenza trovata.

Analisi sintattica

La fase di analisi lessicale suddivide l'input in token, mentre lo scopo dell'analisi sintattica (nota anche come **parsing**) è di ricombinare questi token. Non tornando a una lista di caratteri, ma piuttosto in qualcosa che riflette la struttura del testo. Questo "qualcosa" è tipicamente una struttura dati chiamata albero sintattico del testo. Come suggerisce il nome, si tratta di una struttura ad albero. Le foglie di quest'albero sono i token trovati dall'analisi lessicale e se le foglie vengono lette da sinistra a destra, la sequenza è la stessa dell'input. Quindi, ciò che è importante nell'albero sintattico è come queste foglie vengono combinate per formare la struttura dell'albero e come vengono etichettati i nodi interni dell'albero.
Oltre a trovare la struttura del testo in input, l'analisi sintattica deve anche respingere testi non validi segnalando errori di sintassi.
Poiché l'analisi sintattica è meno locale di natura rispetto all'analisi lessicale, sono necessari metodi più avanzati. Tuttavia, utilizziamo la stessa strategia di base: una notazione adatta per la comprensione umana viene trasformata in una notazione a basso livello simile a quella delle

macchine, adatta per un'esecuzione efficiente. Questo processo è chiamato **generazione del parser**.

La notazione che utilizziamo per la manipolazione umana sono le **grammatiche libere dal contesto** (context-free grammars), che è una notazione ricorsiva per descrivere insiemi di stringhe e imporre una struttura su ciascuna di tali stringhe. In alcuni casi, questa notazione può essere tradotta quasi direttamente in programmi ricorsivi, ma è spesso più conveniente generare **automi a pila**. Questi sono simili agli automi finiti utilizzati per l'analisi lessicale, ma possono utilizzare anche uno stack, che consente di conteggiare e di effettuare corrispondenze non locali di simboli.
Vedremo due modi di generare tali automi:

- Il primo di questi, **LL(1)**, è relativamente semplice, ma funziona solo per una classe di grammatiche piuttosto limitata.
- La costruzione **SLR**, che presenteremo in seguito, è più complessa ma accetta una classe più ampia di grammatiche.

Purtroppo, nessuna di queste funziona per tutte le grammatiche libere dal contesto. Esistono strumenti che gestiscono tutte le grammatiche libere dal contesto, ma possono comportare una penalità di velocità notevole, motivo per cui la maggior parte dei generatori di parser limita la classe di grammatiche di input.

Grammatiche libere dal contesto

Come le espressioni regolari, le grammatiche libere dal contesto descrivono insiemi di stringhe, cioè linguaggi. Inoltre, una grammatica libera dal contesto definisce anche la struttura delle stringhe nel linguaggio che essa definisce. Un linguaggio è definito su un certo alfabeto, ad esempio l'insieme di token prodotti da un analizzatore lessicale o l'insieme di caratteri alfanumerici. I simboli nell'alfabeto sono chiamati **terminali**.

Una grammatica libera dal contesto definisce in modo ricorsivo diversi insiemi di stringhe. Ogni insieme è indicato da un nome, chiamato **non-terminale**. L'insieme di non-terminali è disgiunto dall'insieme di terminali. Uno dei non-terminali viene scelto per indicare il linguaggio descritto dalla grammatica. Questo è chiamato il **simbolo di avvio** della grammatica. Gli insiemi sono descritti da un certo numero di **produzioni**. Ogni produzione descrive alcune delle possibili stringhe che sono contenute nell'insieme indicato da un non-terminale.

Una produzione ha la forma:

$$N \to X_1 \dots X_n$$

dove N è un non-terminale e $X_1 \dots X_n$ sono zero o più simboli, ognuno dei quali è o un terminale o un non-terminale. Il significato inteso di questa notazione è affermare che l'insieme indicato da N contiene stringhe ottenute concatenando stringhe dagli insiemi indicati da $X_1 \dots X_n$. In questo contesto, un terminale indica un insieme singoletto (contenente un solo elemento), proprio come i caratteri dell'alfabeto nelle espressioni regolari. Quando non c'è rischio di confusione, considereremo un non-terminale equivalente all'insieme di stringhe che esso denota.

Ad esempio la produzione

$$A \to aA$$

afferma che l'insieme indicato da A contiene tutte le stringhe formate mettendo una a davanti a una stringa presa dall'insieme indicato da A. Assieme alla produzione $A \to a$, queste due produzioni indicano che A contiene tutte le sequenze non vuote di a e quindi (in assenza di altre produzioni) è equivalente all'espressione regolare a^+.

Possiamo definire una grammatica equivalente all'espressione regolare a^* con le due produzioni:

- $B \to \varepsilon$
- $B \to aB$

dove la prima produzione indica che la stringa vuota fa parte dell'insieme B.
Le produzioni con lati destri vuoti (o con ε) vengono chiamate **produzioni vuote**.

Finora, non abbiamo descritto alcun insieme che non potrebbe essere altrettanto ben descritto utilizzando espressioni regolari. Le grammatiche libere dal contesto, tuttavia, sono in grado di esprimere linguaggi molto più complessi. Abbiamo notato precedentemente che il linguaggio $\left(a^n b^n \mid n \geq 0\right)$ non è regolare. Tuttavia, può essere facilmente descritto dalla grammatica:

- $S \to \varepsilon$
- $S \to aSb$

La seconda produzione assicura che le a e le b siano abbinate simmetricamente intorno al centro della stringa, quindi si verificano in uguale quantità.

Gli esempi sopra hanno utilizzato solo un non-terminale per grammatica. Quando vengono utilizzati diversi non-terminali, è necessario specificare chiaramente quale di essi è il simbolo di avvio. Per convenzione (se non viene specificato nulla di diverso), il non-terminale sul lefthand della prima produzione è il simbolo di avvio.

A volte viene utilizzata una notazione abbreviata in cui tutte le produzioni dello stesso non-terminale vengono combinate in una singola regola, utilizzando il simbolo di alternativa | delle espressioni regolari per separare i righthand.

Ogni sottoespressione dell'espressione regolare è numerata e la sottoespressione s_i viene assegnata a un non-terminale N_i. Le produzioni per N_i dipendono dalla forma di s_i come mostrato nella tabella.

Form of s_i	Productions for N_i
ε	$N_i \rightarrow$
a	$N_i \rightarrow$ a
$s_j s_k$	$N_i \rightarrow N_j N_k$
$s_j \mid s_k$	$N_i \rightarrow N_j$ $N_i \rightarrow N_k$
s_j*	$N_i \rightarrow N_j N_i$ $N_i \rightarrow$
s_j+	$N_i \rightarrow N_j N_i$ $N_i \rightarrow N_j$
$s_j?$	$N_i \rightarrow N_j$ $N_i \rightarrow$

Come accennato in precedenza, un'espressione regolare può essere sistematicamente riscritta come una grammatica libera dal contesto utilizzando un non-terminale per ogni sottoespressione nell'espressione regolare e utilizzando una o due produzioni per ogni non-terminale. Quindi, se possiamo pensare a un modo di esprimere un linguaggio come un'espressione regolare, è facile crearne una grammatica.
Tuttavia, vogliamo anche utilizzare le grammatiche per descrivere linguaggi non regolari. Un esempio sono i tipi di espressioni aritmetiche che fanno parte della maggior parte dei linguaggi di programmazione (e si trovano anche sulle calcolatrici elettroniche). Tali espressioni possono essere descritte dalla seguente grammatica.

$$\begin{aligned} Exp &\rightarrow Exp + Exp \\ Exp &\rightarrow Exp - Exp \\ Exp &\rightarrow Exp * Exp \\ Exp &\rightarrow Exp / Exp \\ Exp &\rightarrow \textbf{num} \\ Exp &\rightarrow (Exp) \end{aligned}$$

Nota che, come menzionato precedentemente, le parentesi tonde di chiusura non possono essere descritte dalle espressioni regolari, in quanto queste non possono "contare" il numero di parentesi aperte non abbinate in un punto specifico della stringa. Tuttavia, se non avessimo parentesi nel linguaggio, esso potrebbe essere descritto dall'espressione regolare:

$$num\left((+\mid -\mid *\mid /)\ num\right)^*$$

Tuttavia, la descrizione regolare non è utile se si desidera che gli operatori abbiano diverse precedenze, poiché tratta l'espressione come una stringa piatta piuttosto che come una struttura. Esamineremo la struttura nelle sezioni successive. La maggior parte delle costruzioni dei linguaggi di programmazione sono facilmente espresse da grammatiche libere dal contesto. Infatti, la maggior parte dei linguaggi moderni sono progettati in questo modo.

Quando si scrive una grammatica per un linguaggio di programmazione, di solito si inizia dividendo le costruzioni del linguaggio in diverse categorie sintattiche. Una categoria sintattica è un sotto-linguaggio che incarna un particolare concetto. Esempi di comuni categorie sintattiche nei linguaggi di programmazione sono:

- Le **espressioni** vengono utilizzate per esprimere il calcolo di valori.
- Le **istruzioni** esprimono azioni che si verificano in una sequenza specifica.

- Le **dichiarazioni** esprimono proprietà di nomi utilizzati in altre parti del programma.

Ogni categoria sintattica è indicata da un non-terminale principale, ad esempio $\exp$ dalla grammatica precedente. Potrebbero essere necessari ulteriori non-terminali per descrivere una categoria sintattica o fornire struttura ad essa, come vedremo, e le produzioni per una categoria sintattica possono fare riferimento a non-terminali per altre categorie sintattiche. Ad esempio, le istruzioni possono contenere espressioni, quindi alcune delle produzioni per le istruzioni utilizzano il non-terminale principale per le espressioni. Una grammatica semplice per le istruzioni potrebbe assomigliare alla seguente grammatica, che fa riferimento al non-terminale $\exp$.

$$
\begin{array}{lcl}
Stat & \rightarrow & \textbf{id} := Exp \\
Stat & \rightarrow & Stat \ ; \ Stat \\
Stat & \rightarrow & \textbf{if} \ Exp \ \textbf{then} \ Stat \ \textbf{else} \ Stat \\
Stat & \rightarrow & \textbf{if} \ Exp \ \textbf{then} \ Stat
\end{array}
$$

Derivazione

Finora, abbiamo fatto ricorso a intuizioni ricorsive quando descriviamo l'insieme di stringhe che una grammatica produce. Introduciamo di seguito il concetto di derivazione. Un vantaggio aggiuntivo di questo approccio è che, come vedremo in seguito, l'analisi sintattica è strettamente correlata alla derivazione.

L'idea fondamentale della derivazione è considerare le produzioni come regole di riscrittura: ogni volta che abbiamo un non-terminale, possiamo sostituirlo con il righthand di qualsiasi produzione in cui il non-terminale compare sul lefthand. Possiamo farlo ovunque in una sequenza di simboli (terminali e non-terminali) e ripeterlo fino a quando abbiamo solo terminali. La sequenza risultante di terminali è una stringa nel linguaggio definito dalla grammatica. Formalmente, definiamo la relazione di derivazione $\Rightarrow$ tramite tre regole:

1. $\alpha N \beta \Rightarrow \alpha \gamma \beta$ se c'è una produzione $N \rightarrow \gamma$
2. $\alpha \Rightarrow \alpha$
3. $\alpha \Rightarrow \gamma$ se c'è una β tale che $\alpha \Rightarrow \beta$ e $\beta \Rightarrow \gamma$

dove α, β e γ sono sequenze (possibilmente vuote) di simboli della grammatica (terminali e non-terminali). La prima regola afferma che utilizzare una produzione come regola di riscrittura (ovunque in una sequenza di simboli della grammatica) è un passo di derivazione. La seconda afferma che la relazione di derivazione è riflessiva, cioè che una sequenza deriva se stessa. La terza regola descrive la transitività, cioè che una sequenza di derivazioni è essa stessa una derivazione.

Possiamo utilizzare la derivazione per definire formalmente il **linguaggio** che una grammatica libera dal contesto genera:

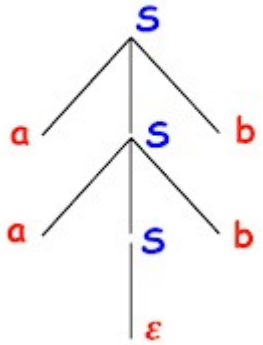
Dato una grammatica libera dal contesto G con simbolo di avvio S, simboli terminali T e produzioni P, il linguaggio $L(G)$ che G genera è definito come l'insieme di stringhe di simboli

terminali che possono essere ottenute per derivazione da S usando le produzioni P, cioè l'insieme $\{w \in T^* \mid S \Rightarrow^* w\}$

Una derivazione che riscrive sempre il non-terminale più a sinistra viene chiamata derivazione più a sinistra (o **leftmost derivation**). Allo stesso modo, una derivazione che riscrive sempre il non-terminale più a destra viene chiamata derivazione più a destra (o **rightmost derivation**). Tuttavia, dato che i passaggi di derivazione sono locali, l'ordine non ha importanza. Quindi, potremmo benissimo decidere di riscrivere sempre il non-terminale più a sinistra.

Possiamo rappresentare una derivazione come un **albero**: La radice dell'albero è il simbolo di avvio della grammatica, e ogni volta che riscriviamo un non-terminale, aggiungiamo come suoi figli i simboli sul lato destro della produzione che è stata utilizzata. Le foglie dell'albero (la frontiera dell'albero) sono terminali che, letti da sinistra a destra, formano la stringa derivata. Se un non-terminale viene riscritto utilizzando una produzione vuota, viene mostrato un ε come suo figlio. Anche questo è un nodo foglia, ma viene ignorato quando si legge la stringa sulla frontiera dell'albero.

$S \rightarrow aSb \mid \varepsilon$



Quando scriviamo un tale albero sintattico, l'ordine di derivazione è irrilevante: otteniamo lo stesso albero sia per la derivazione più a sinistra, la derivazione più a destra o qualsiasi altro ordine di derivazione. Conta solo la scelta della produzione per la riscrittura di ciascun non-terminale.
L'albero sintattico aggiunge **struttura** alla stringa che deriva. È questa struttura che sfruttiamo nelle fasi successive del compilatore.

Per la compilazione, facciamo la derivazione all'indietro: iniziamo con una stringa e vogliamo produrre un albero sintattico. Questo processo è chiamato **analisi sintattica** o **parsing**.

Anche se l'ordine di derivazione non ha importanza quando si costruisce un albero sintattico, la scelta della produzione per quel non-terminale conta. Ovviamente, scelte diverse possono portare a diverse stringhe derivate, ma può anche accadere che siano possibili diverse strutture sintattiche per la stessa stringa.
Quando una grammatica permette diverse strutture sintattiche per alcune stringhe, diciamo che la grammatica è **ambigua**.

Una **grammatica** G è **ambigua** se e solo se esiste una parola $w \in L(G)$ che può essere generata da due derivazioni canoniche differenti (o entrambe rightmost o entrambe leftmost). Se il nostro unico scopo con la grammatica è descrivere insiemi di stringhe, l'ambiguità non è un problema. Tuttavia, quando vogliamo utilizzare la grammatica per imporre struttura alle stringhe, è meglio che la struttura sia sempre la stessa. Pertanto, è una caratteristica desiderabile che una grammatica sia non ambigua. Nella maggior parte (ma non in tutti) i casi, una grammatica ambigua può essere riscritta in una grammatica non ambigua che genera lo stesso insieme di stringhe, oppure possono essere applicate regole esterne per decidere quale degli alberi sintattici possibili è quello "corretto".

Come facciamo a sapere se una grammatica è ambigua? Se possiamo trovare una stringa e mostrare due alberi sintattici alternativi per essa, questo è una prova di ambiguità. Tuttavia, potrebbe essere difficile trovare una tale stringa e, quando la grammatica è non ambigua, è ancora più difficile dimostrarlo. In realtà, il problema è formalmente indecidibile, ovvero non esiste un metodo che possa rispondere alla domanda "Questa grammatica è ambigua?" per tutte le grammatiche.

Nelle sezioni successive vedremo metodi per costruire parser a partire dalle grammatiche. Questi metodi hanno la proprietà di funzionare solo su grammatiche non ambigue, quindi la costruzione riuscita di un parser è una prova di non ambiguità. Tuttavia, i metodi possono anche fallire su alcune grammatiche non ambigue, quindi non possono essere utilizzati per dimostrare l'ambiguità.

Nella prossima sezione, vedremo modi per riscrivere una grammatica per eliminare alcune fonti di ambiguità. Queste trasformazioni preservano il linguaggio generato dalla grammatica. Utilizzando tali trasformazioni (e altre che vedremo in seguito), possiamo creare un ampio insieme di grammatiche equivalenti, cioè grammatiche che generano lo stesso linguaggio (anche se possono imporre diverse strutture sulle stringhe del linguaggio).

Date due grammatiche, sarebbe bello poter dire se sono equivalenti. Sfortunatamente, non esiste un metodo noto in grado di decidere questo in tutti i casi, ma, a differenza dell'ambiguità, al momento della scrittura, non si sa se possa o meno esistere teoricamente un tale metodo. A volte, l'equivalenza può essere dimostrata ad esempio per induzione sull'insieme di stringhe che le grammatiche producono. La tesi contraria può essere dimostrata trovando un esempio di una stringa che una grammatica può generare ma l'altra no. Ma in alcuni casi, dobbiamo accettare affermazioni di equivalenza sulla fiducia o rinunciare a decidere la questione.

Operatori di precedenza

Come detto precedentemente, possiamo descrivere le espressioni aritmetiche tradizionali mediante la grammatica.

$$
\begin{aligned}
Exp &\rightarrow Exp + Exp \\
Exp &\rightarrow Exp - Exp \\
Exp &\rightarrow Exp * Exp \\
Exp &\rightarrow Exp / Exp \\
Exp &\rightarrow \textbf{num} \\
Exp &\rightarrow (Exp)
\end{aligned}
$$

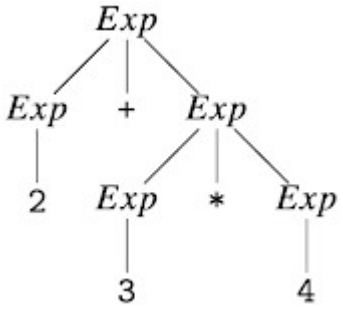
Nota che *num* è un terminale che denota tutte le costanti intere e che, in questo contesto, le parentesi sono simboli terminali (a differenza delle espressioni regolari, in cui vengono utilizzate per imporre struttura alle espressioni regolari).

Questa grammatica è ambigua, come evidenziato, ad esempio, dalla produzione

$$\exp \rightarrow \exp + \exp$$

Questa ambiguità non sorprende, poiché siamo abituati al fatto che un'espressione come $2+3*4$ può essere letta in due modi: o come la moltiplicazione della somma di 2 e 3 per 4 o come l'addizione di 2 al prodotto di 3 e 4. Le semplici calcolatrici elettroniche sceglieranno la prima di queste interpretazioni (poiché effettuano sempre i calcoli da sinistra a destra), mentre le calcolatrici scientifiche e la maggior parte dei linguaggi di programmazione sceglieranno la seconda, poiché utilizzano una gerarchia di precedenze degli operatori che impone che il prodotto debba essere calcolato prima della somma. La gerarchia può essere sovrascritta con parentesizzazione esplicita, ad esempio $(2+3)*4$.

La maggior parte dei linguaggi di programmazione utilizza la stessa convenzione delle calcolatrici scientifiche, quindi vogliamo rendere questo esplicito nella grammatica. Idealmente, vorremmo che l'espressione $2+3*4$ generi il seguente l'albero sintattico, che riflette le precedenze degli operatori raggruppando le sottoespressioni: quando si valuta un'espressione, le sottoespressioni rappresentate dai sottoalberi dell'albero sintattico vengono valutate prima che l'operatore più esterno venga applicato.



Un possibile modo di risolvere l'ambiguità è utilizzare regole di precedenza durante l'analisi sintattica per selezionare tra i possibili alberi sintattici. Molti generatori di parser permettono questo approccio, come vedremo successivamente. Tuttavia, alcuni metodi di parsing richiedono che le grammatiche siano non ambigue, quindi dobbiamo esprimere la gerarchia degli operatori direttamente nella grammatica stessa. Per chiarire ciò, definiamo prima alcuni concetti:

- Un operatore $\oplus$ è associativo a sinistra se l'espressione $a \oplus b \oplus c$ deve essere valutata da sinistra a destra, cioè come $(a \oplus b) \oplus c$.

- Un operatore $\oplus$ è associativo a destra se l'espressione $a \oplus b \oplus c$ deve essere valutata da destra a sinistra, cioè come $a \oplus (b \oplus c)$.
- Un operatore $\oplus$ è non associativo se le espressioni della forma $a \oplus b \oplus c$ sono illegali.

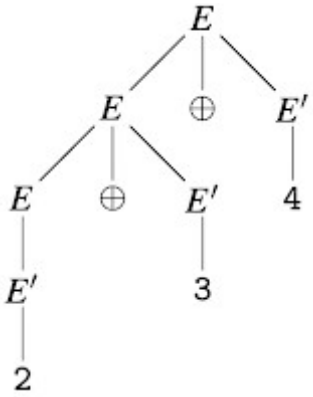
Se abbiamo una grammatica ambigua come:

- $E \to E \oplus E$
- $E \to num$

Possiamo riscriverla in una grammatica non ambigua che genera la struttura corretta. Poiché questo dipende dall'associatività di $\oplus$, utilizziamo diverse regole di riscrittura per diverse associatività.
Se $\oplus$ è associativo a sinistra, rendiamo la grammatica sinistra-ricorsiva avendo un riferimento ricorsivo solo a sinistra del simbolo dell'operatore:

- $E \to E \oplus E'$
- $E \to E'$
- $E' \to num$

Ora, l'espressione $2 \oplus 3 \oplus 4$ può essere analizzata solo come:



Gestiamo l'associatività a destra in modo simile: rendiamo la produzione problematica destra-ricorsiva:

- $E \to E' \oplus E$
- $E \to E'$
- $E' \to num$

Gli operatori non associativi sono gestiti da produzioni non ricorsive:

- $E \to E' \oplus E'$
- $E \to E'$
- $E' \to num$

Si noti che quest'ultima trasformazione in realtà cambia il linguaggio generato dalla grammatica, poiché rende le espressioni del tipo $num \oplus num \oplus num$ illegali.

Finora abbiamo affrontato solo casi in cui un operatore interagisce con se stesso. Questo può essere facilmente esteso al caso in cui diversi operatori con la stessa precedenza e associatività interagiscono tra loro.
Gli operatori con la stessa precedenza devono avere la stessa associatività affinché ciò funzioni, poiché mescolare produzioni ricorsive a sinistra e a destra per lo stesso non-terminale rende la grammatica ambigua.

In generale, non c'è un modo ovvio di risolvere l'ambiguità in un'espressione come $1+2 \oplus num$, in cui + è associativo a sinistra e $\oplus$ è associativo a destra (o viceversa). Di conseguenza, la maggior parte dei linguaggi di programmazione (e la maggior parte dei generatori di parser) richiede che gli operatori dello stesso livello di precedenza abbiano la stessa associatività.

Dobbiamo anche gestire gli operatori con precedenze diverse. Questo viene fatto utilizzando un non-terminale per ciascun livello di precedenza. L'idea è che se un'espressione utilizza un operatore di un certo livello di precedenza, allora le sue sottoespressioni non possono utilizzare operatori di livelli di precedenza inferiori (a meno che questi siano all'interno di parentesi). Di conseguenza, le produzioni per un non-terminale corrispondente a un particolare livello di precedenza si riferiscono solo a non-terminali che corrispondono a livelli di precedenza uguali o superiori, a meno che parentesi o costrutti di delimitazione simili non chiariscano l'uso di questi.

La seguente grammatica mostra come queste regole vengono utilizzate per creare una versione non ambigua della grammatica delle espressioni aritmetiche vista prima:

$$
\begin{array}{lcl}
Exp & \rightarrow & Exp + Exp2 \\
Exp & \rightarrow & Exp - Exp2 \\
Exp & \rightarrow & Exp2 \\
Exp2 & \rightarrow & Exp2 * Exp3 \\
Exp2 & \rightarrow & Exp2 \,/\, Exp3 \\
Exp2 & \rightarrow & Exp3 \\
Exp3 & \rightarrow & \mathbf{num} \\
Exp3 & \rightarrow & (\, Exp\,)
\end{array}
$$

Altre fonti di ambiguità

La maggior parte delle potenziali ambiguità nelle grammatiche dei linguaggi di programmazione proviene dalla sintassi delle espressioni e può essere gestita sfruttando le regole di precedenza come mostrato nella sezione precedente. Un altro classico esempio di ambiguità è il problema del "dangling-else".

Linguaggi imperativi come Pascal o C spesso consentono che la parte *else* di un'istruzione condizionale sia opzionale, come mostrato nella grammatica

$$
\begin{aligned}
Stat &\rightarrow \textbf{id} := Exp \\
Stat &\rightarrow Stat \ ; \ Stat \\
Stat &\rightarrow \textbf{if} \ Exp \ \textbf{then} \ Stat \ \textbf{else} \ Stat \\
Stat &\rightarrow \textbf{if} \ Exp \ \textbf{then} \ Stat
\end{aligned}
$$

Il problema è che non è chiaro come analizzare, ad esempio,

$$if \ \ p \ \ then \ \ if \ \ q \ \ then \ \ s1 \ \ else \ \ s2$$

Secondo la grammatica, l'*else* può corrispondere in modo egualmente valido a qualsiasi *if*. La convenzione comune è che un *else* corrisponda all'*if* più vicino non ancora associato, il che, nell'esempio, farebbe corrispondere l'*else* al secondo *if*.

Come possiamo rendere ciò chiaro nella grammatica? Possiamo trattare *if*, *then* e *else* come una sorta di operatori associativi a destra, poiché questo farebbe sì che essi si raggruppino a destra, facendo sì che un *if* - *then* corrisponda all' *else* più vicino. Tuttavia, le trasformazioni della grammatica mostrate nella sezione precedenti non possono essere applicate direttamente alla grammatica precedente, poiché le produzioni per le condizioni non hanno la forma corretta.

Invece, facciamo uso dell'osservazione seguente: quando un *if* e un *else* si abbinano, tutti gli *if* che si verificano tra di essi devono avere *else* corrispondenti. Questo può essere facilmente dimostrato assumendo diversamente e concludendo che ciò porta a una contraddizione.

Di conseguenza, creiamo due non-terminali: uno per le condizioni abbinate (cioè con parte *else*) e uno per le condizioni non abbinate (cioè senza parte *else*). Il risultato è mostrato nella seguente grammatica

$$
\begin{aligned}
Stat &\rightarrow Stat2 \ ; \ Stat \\
Stat &\rightarrow Stat2 \\
Stat2 &\rightarrow Matched \\
Stat2 &\rightarrow Unmatched \\
Matched &\rightarrow \texttt{if} \ Exp \ \texttt{then} \ Matched \ \texttt{else} \ Matched \\
Matched &\rightarrow \textbf{id} := Exp \\
Unmatched &\rightarrow \texttt{if} \ Exp \ \texttt{then} \ Matched \ \texttt{else} \ Unmatched \\
Unmatched &\rightarrow \texttt{if} \ Exp \ \texttt{then} \ Stat2
\end{aligned}
$$

Questa grammatica risolve anche l'associatività del punto e virgola (a destra) e la precedenza di *if* rispetto al punto e virgola.

Un'alternativa alla riscrittura delle grammatiche per risolvere l'ambiguità è utilizzare una grammatica ambigua e risolvere i conflitti utilizzando regole di precedenza durante l'analisi. Esamineremo questo approccio nelle prossime sezioni.

Tutti i casi di ambiguità devono essere trattati con attenzione: non è sufficiente eliminare l'ambiguità, dobbiamo farlo in modo che si ottenga la struttura desiderata. La struttura delle espressioni aritmetiche è significativa e fa differenza a quale *if* viene associato un *else*.

Proprietà dei linguaggi liberi (dal contesto)

Vediamo ora delle proprietà generali dei linguaggi liberi.
Queste proprietà ci aiuteranno a capire se è possibile generare o meno un determinato linguaggio partendo da una certa grammatica e anche come eseguire la manipolazione della grammatica.

I non-terminali svolgono il ruolo di variabili.
Lemma: Sia $G=(V,T,S,P)$. Allora ogni grammatica G' ottenuta da G rinominando i non-terminali è tale che $L(G)=L(G')$.
Infatti il nome preciso dei non-terminali è del tutto irrilevante ai fini della generazione del linguaggio.

- For instance, any derivation for the grammar

$$\begin{aligned} S &\rightarrow aSbC \mid ab \\ C &\rightarrow cC \mid c \end{aligned}$$

- Can be mimicked using the grammar

$$\begin{aligned} R &\rightarrow aRbD \mid ab \\ D &\rightarrow cD \mid c \end{aligned}$$

- And viceversa

Vediamo ora la **pulizia di una grammatica**.
Lemma: Sia L un linguaggio libero. Allora esiste una grammatica libera G tale che $L(G)=L¿\{\varepsilon¿\}$ e che:

- Non abbia ε-produzioni (ovvero non abbia produzioni della forma $A \rightarrow \varepsilon$)
- Non abbia produzioni unitarie (ovvero non abbia produzioni della forma $A \rightarrow B$)
- Non abbia non-terminali inutili (ovvero non abbia non-terminali che non compaiono mai in nessuna derivazione di alcune stringhe di terminali)

Prendiamo il seguente esempio:

$$\begin{aligned} S &\rightarrow ABC \mid abc \\ A &\rightarrow aB \mid \epsilon \\ B &\rightarrow bA \mid C \\ C &\rightarrow \epsilon \end{aligned}$$

I passi per eliminare le ε-produzioni sono i seguenti:

1. Trovare i non-terminali “annullabili”, ovvero ogni non-terminale A tale che $A \Rightarrow^{*} \varepsilon$
    - Base: Se $A \rightarrow \varepsilon$ è una produzione, allora A è annullabile
    - Iterazione: Se $A \rightarrow Y_1 Y_2 \ldots Y_n$ è una produzione e $Y_1 Y_2 \ldots Y_n$ sono tutti annullabili, allora A è annullabile

- A and C nullable by $A \to \epsilon$, $C \to \epsilon$
- B nullable by $B \to C$ and C nullable
- S nullable by $S \to ABC$ and A, B, C nullable

2. Sostituire ogni produzione $A \to Y_1 Y_2 \dots Y_n$ con una famiglia di produzioni dove le combinazioni di $Y_i s$ annullabili vengono rimosse dal corpo (se tutti gli $Y_i s$ sono annullabili allora non bisogna prendere nella famiglia di produzioni anche $A \to \varepsilon$)

$$
\begin{aligned}
S &\to ABC \mid abc \mid AB \mid AC \mid BC \mid A \mid B \mid C \\
A &\to aB \mid a \mid \epsilon \\
B &\to bA \mid b \mid C \\
C &\to \epsilon
\end{aligned}
$$

3. Eliminare ogni produzione $A \to \varepsilon$

$$
\begin{aligned}
S &\to ABC \mid abc \mid AB \mid AC \mid BC \mid A \mid B \mid C \\
A &\to aB \mid a \\
B &\to bA \mid b \mid C
\end{aligned}
$$

Vediamo l'eliminazione dei non-terminali inutili:

$$
\begin{aligned}
S &\to ABC \mid abc \mid AB \mid AC \mid BC \mid A \mid B \mid C \\
A &\to aB \mid a \\
B &\to bA \mid b \mid C
\end{aligned}
$$

$$
\begin{aligned}
S &\to abc \mid AB \mid A \mid B \\
A &\to aB \mid a \\
B &\to bA \mid b
\end{aligned}
$$

E infine l'eliminazione delle produzioni unitarie:

$$
\begin{aligned}
S &\to abc \mid AB \mid A \mid B \\
A &\to aB \mid a \\
B &\to bA \mid b
\end{aligned}
$$

$$
\begin{aligned}
S &\to abc \mid AB \mid aB \mid a \mid bA \mid b \\
A &\to aB \mid a \\
B &\to bA \mid b
\end{aligned}
$$

Dunque riassumendo abbiamo pulito la grammatica:

- From

$$\begin{aligned} S &\to ABC \mid abc \\ A &\to aB \mid \epsilon \\ B &\to bA \mid C \\ C &\to \epsilon \end{aligned}$$

- We get

$$\begin{aligned} S &\to abc \mid AB \mid aB \mid a \mid bA \mid b \\ A &\to aB \mid a \\ B &\to bA \mid b \end{aligned}$$

Vediamo ora la proprietà per la quale **la classe dei linguaggi liberi è chiusa rispetto all'operazione di unione insiemistica**, ovvero:

- Se L_1 e L_2 sono linguaggi liberi, allora $L_1 \cup L_2$ è un linguaggio libero

Dimostrazione:

- Siano L_1 e L_2 due linguaggi liberi
- Allora esistono due grammatiche libere $G_1=(V_1,T_1,S_1,P_1)$ e $G_2=(V_2,T_2,S_2,P_2)$ tali che $L_1=L(G_1)$ e $L_2=L(G_2)$
- Possiamo assumere che $(V_1\{T¿¿1) \cup (V_2\{T¿¿2)=\varnothing$, ovvero che G_1 e G_2 non condividono alcun non-terminale (visto che possiamo rinominarli a piacere)
- Sia $G=(V_1 \cup V_2 \cup \{S\}, T_1 \cup T_2, S, P_1 \cup P_2 \cup \{S \to S_1 | S_2\})$, con S nuovo simbolo non appartenente a $V_1 \cup V_2$
- Allora $L(G)$ è un linguaggio libero e $L(G)=L(G_1) \cup L(G_2)$

Verifichiamo che G è libera:

- Le produzioni di G sono quelle in $P_1 \cup P_2 \cup \{S \to S_1 | S_2\}$
- Le produzioni in P_1 e P_2 sono tutte nella forma $A \to w$ (perchè provengono da grammatiche libere)
- Le produzioni $S \to S_1$ e $S \to S_2$ hanno anche loro la forma $A \to w$
- Dunque G è una grammatica libera

Verifichiamo che vale $L(G)=L(G_1) \cup L(G_2)$:

- $w \in L(G)$
- Se e solo se esiste una derivazione $S \Rightarrow^* w$
- Se e solo se $S \Rightarrow S_1 \Rightarrow^* w$ oppure $S \Rightarrow S_2 \Rightarrow^* w$
- Se e solo se $S_1 \Rightarrow^* w$ oppure $S_2 \Rightarrow^* w$
- Se e solo se $w \in L(G_1)$ oppure $w \in L(G_2)$

- Se e solo se $w \in L(G_1) \cup L(G_2)$

Come mai insistiamo sul fatto che G_1 e G_2 non condividano non-terminali? Dobbiamo farlo per evitare di mescolare le produzioni di una grammatica con le produzioni dell'altra e ottenere derivazioni errate. Ecco un esempio:

- Take

$$\mathcal{G}_1: \begin{array}{lcl} S_1 & \to & aA \\ A & \to & a \end{array} \qquad \mathcal{G}_2: \begin{array}{lcl} S_2 & \to & bA \\ A & \to & b \end{array}$$

- Then $\mathcal{L}(\mathcal{G}_1) = \{aa\}$ and $\mathcal{L}(\mathcal{G}_2) = \{bb\}$.
- However, if we just put everything together and define

$$\mathcal{G}: \begin{array}{lcl} S & \to & S_1 \mid S_2 \\ S_1 & \to & aA \\ S_2 & \to & bA \\ A & \to & a \mid b \end{array}$$

- Then $\mathcal{L}(\mathcal{G}) = \{aa, ab, ba, bb\} \neq \mathcal{L}(\mathcal{G}_1) \cup \mathcal{L}(\mathcal{G}_2)$.
- Take

$$\mathcal{G}_1: \begin{array}{lcl} S_1 & \to & aA \\ A & \to & a \end{array} \qquad \mathcal{G}_2: \begin{array}{lcl} S_2 & \to & bA \\ A & \to & b \end{array}$$

- Then $\mathcal{L}(\mathcal{G}_1) = \{aa\}$ and $\mathcal{L}(\mathcal{G}_2) = \{bb\}$.
- The symbol A plays distinct roles in $\mathcal{G}_1$ and in $\mathcal{G}_2$
- To reflect this distinction, rather take

$$\mathcal{G}_1: \begin{array}{lcl} S_1 & \to & aA \\ A & \to & a \end{array} \qquad \mathcal{G}_2: \begin{array}{lcl} S_2 & \to & bA' \\ A' & \to & b \end{array}$$

- No mix up of productions now, and, by contruction, $\mathcal{L}(\mathcal{G}) = \{aa, bb\}$

Vediamo ora la proprietà per la quale **la classe dei linguaggi liberi è chiusa rispetto all'operazione di concatenazione**, ovvero:

- Se L_1 e L_2 sono linguaggi liberi, allora $\{w_1 w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2\}$ è un linguaggio libero

Dimostrazione:

- Siano L_1 e L_2 due linguaggi liberi
- Allora esistono due grammatiche libere $G_1 = (V_1, T_1, S_1, P_1)$ e $G_2 = (V_2, T_2, S_2, P_2)$ tali che $L_1 = L(G_1)$ e $L_2 = L(G_2)$
- Possiamo assumere che $(V_1 \{T ¿¿ 1) \cup (V_2 \{T ¿¿ 2) = \varnothing$, ovvero che G_1 e G_2 non condividono alcun non-terminale (visto che possiamo rinominarli a piacimento)
- Sia $G = (V_1 \cup V_2 \cup \{S\}, T_1 \cup T_2, S, P_1 \cup P_2 \cup \{S \to S_1 S_2\})$, con S nuovo simbolo non appartenente a $V_1 \cup V_2$

- Allora $L(G)$ è un linguaggio libero e $L(G)=\{w_1 w_2 \mid w_1 \in L(G_1) \wedge w_2 \in L(G_2)\}$

La classe dei linguaggi liberi non è però chiusa rispetto all'operazione di intersezione (come vedremo dopo).

Ricordando il lemma della grammatica pulita:
Lemma: Sia L un linguaggio libero. Allora esiste una grammatica libera G tale che $L(G)=L¿\{\varepsilon¿\}$ e che:

- Non abbia ε-produzioni (ovvero non abbia produzioni della forma $A \to \varepsilon$)
- Non abbia produzioni unitarie (ovvero non abbia produzioni della forma $A \to B$)
- Non abbia non-terminali inutili (ovvero non abbia non-terminali che non compaiono mai in nessuna derivazione di alcune stringhe di terminali)

Possiamo facilmente notare che ogni produzione $A \to \beta$ in G è tale che o β è un singolo terminale oppure $|\beta| \geq 2$, in ogni caso β non è annullabile.

Pumping lemma per linguaggi liberi

Abbiamo precedentemente visto il pumping lemma per i linguaggi regolari, vediamolo ora per i linguaggi liberi.

Il pumping lemma riguarda la presenza in ogni linguaggio context-free infinito (e in particolare in ogni linguaggio regolare infinito) di successioni di stringhe che presentano regolarità ben definite. In particolare ogni stringa sufficientemente lunga di un tale linguaggio contiene una o più porzioni che possono essere ripetute un numero arbitrario di volte ottenendo ancora una stringa appartenente al linguaggio.

Lemma: Sia L un linguaggio libero. Allora:

- $\exists\, p \in N^+$ (detta "**pumping length**") tale che
- $\forall\, z \in L$ tale che $|z| > p$
- $\exists\ u, v, w, x, y$ tali che
  - $z = uvwxy$ e
  - $|vwx| \leq p$ e
  - $|vx| > 0$ e
  - $\forall\, i \in N,\ u v^i w x^i y \in L$

Dimostrazione:

- Sia L un linguaggio libero
- Il lemma tratta parole che sono più lunghe di p, con $p > 0$, e quindi che sono diverse da ε
- Possiamo quindi considerare una grammatica "pulita" G tale che $L(G)=L¿\{\varepsilon¿\}$
- Trasformiamo G in una grammatica in forma normale di Chomsky G', ovvero dove ogni produzione ha la forma
  - $A \to a$ oppure

- $A \to A_1 A_2$

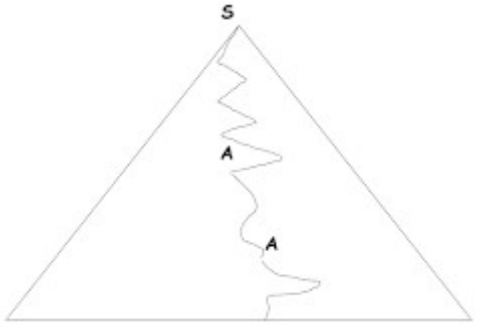
- Example
  - $S \to aSb \mid ab$
  - Both aSb and ab are to be changed. Pick up a new non-terminal for a, say A, and a new non-terminal for b, say B, and transform
  - $S \to ASB \mid AB, \quad A \to a, \quad B \to b$
  - ASB is longer than 2, pick up a new non-terminal for SB, say C, and transform
  - $S \to AC \mid AB, \quad C \to SB, \quad A \to a, \quad B \to b$

- Definiamo k come il numero di non-terminali in G'
- Osserviamo che ogni albero di derivazione per le parole in $L(G')$ ha:
  - Esattamente $2^0 = 1$ nodo al livello 0
  - Al massimo 2^1 nodi al livello 1
  - Al massimo 2^2 nodi al livello 2
  - …
  - Al massimo 2^j nodi al livello j
- Prendiamo $p = 2^{k+1}$
- Prendiamo $z \in L$ tale che $|z| > p$
- Allora l'albero di derivazione per z deve avere almeno $k+2$ livelli
- Allora ogni cammino più lungo dell'albero di derivazione per z ha un terminale all'ultimo livello e attraversa quindi almeno $k+1$ non-terminali
- Quindi c'è almeno un non-terminale che si verifica due o più volte lungo il percorso
- Consideriamo il percorso più lungo nell'albero e la coppia più profonda di occorrenze dello stesso non-terminale lungo quel percorso (ovvero scegli il non-terminale la cui seconda occorrenza viene trovata per prima andando dal basso verso l'alto)

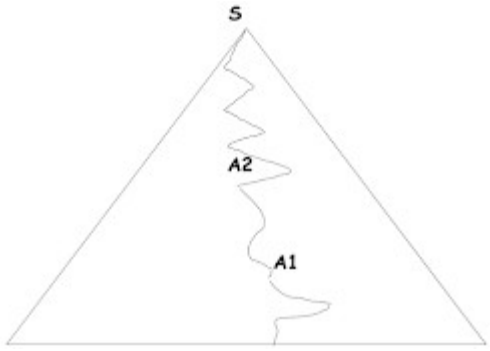
For instance, below the deepest pair is always the pair of As



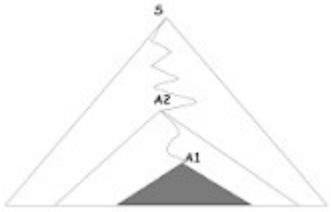
- Sia A il non-terminale della coppia più profonda di occorrenze dello stesso non-terminale lungo il percorso

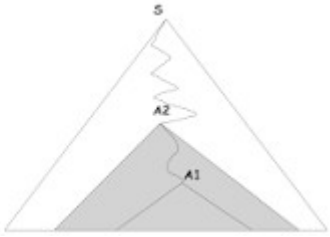


- Chiamiamo A_1 e A_2 le due occorrenze di A per distinguerle

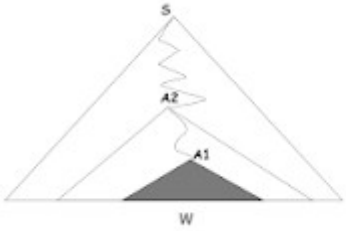


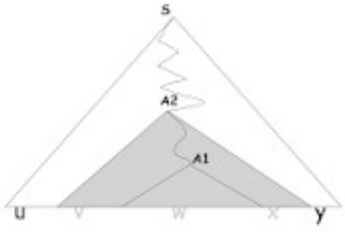
- Quindi ci i sono due sottoalberi distinti con radice in A: il “sottoalbero rosa” (più piccolo) e il “sottoalbero verde” (più grande)



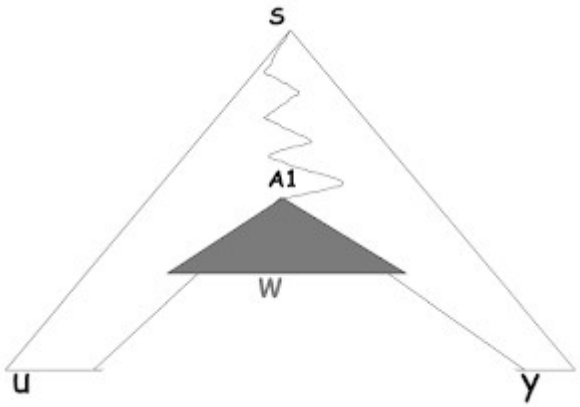


- Allora $z = uvwxy$

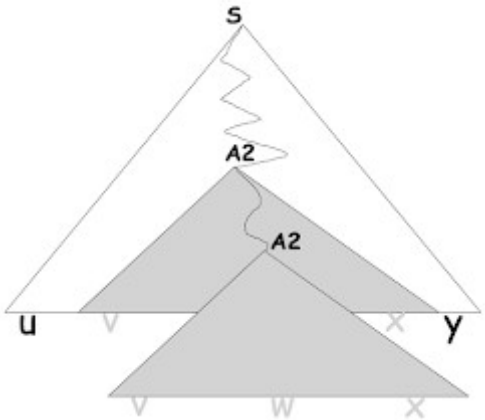




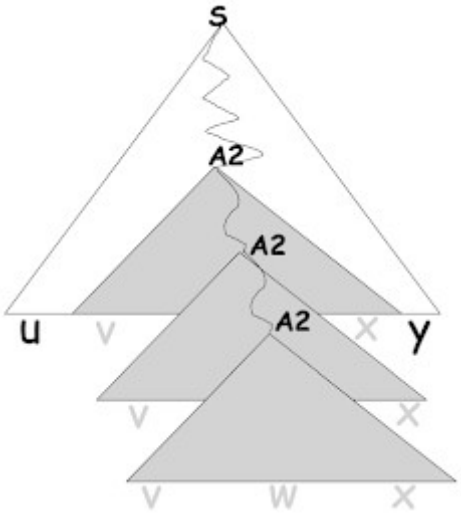
- E quindi $u\,v^0\,w\,x^0\,y \in L$



- E quindi $u\,v^2\,w\,x^2\,y \in L$



- E quindi $u\,v^3\,w\,x^3\,y \in L$



- Quindi $\forall\, i \in N,\, u\,v^i\,w\,x^i\,y \in L$
- $|vwx| \leq p$
    - Poiché abbiamo scelto la coppia più profonda di non-terminali ripetuti (A_1, A_2) lungo il percorso più lungo da S

    - Quindi lungo il percorso più lungo da A_2, sotto A_2, nessun non-terminale può verificarsi più di una volta
    - Quindi il sottoalbero radicato in A_2 ha al massimo $k+1$ livelli
    - Quindi la lunghezza della frontiera (il numero di foglie) del sottoalbero è limitato da $2^{k+1} = p$
- $|vx| > 0$
    - Poichè G' è una grammatica pulita
    - Dunque non può essere $A \Rightarrow^+ A$ ma piuttosto $A \Rightarrow^+ \alpha A \beta$ con almeno un terminale derivato da α o da β, dunque v e x non possono essere entrambe vuote

Questo ci conferma che nei linguaggi liberi si riescono a riconoscere dei bilanciamenti.

Per lo stesso discorso fatto per il pumping lemma nel caso dei linguaggi regolari, il pumping lemma per linguaggi liberi è utile per verificare che un linguaggio **non è libero**:

- Supporre che il linguaggio sia libero
- Dimostrare che la $not(PL\text{-}THESIS)$ è vera

La $PL\text{-}THESIS$ è la seguente:

- $\exists p \in \mathbb{N}^+.\ \forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w, x, y.\ P$
- where
    - $P \equiv P1$ and $P2$ and $P3$ and $P4$
    - $P1 \equiv z = uvwxy$
    - $P2 \equiv |vwx| \leq p$
    - $P3 \equiv |vx| > 0$
    - $P4 \equiv \forall i \in \mathbb{N}. uv^i wx^i y \in \mathcal{L}$

La $not(PL\text{-}THESIS)$ invece è la seguente e proviamo a “semplifcarla”:

- not ($\exists p \in \mathbb{N}^+.\ \forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w, x, y.\ P$)
- $\forall p \in \mathbb{N}^+.$ not ($\forall z \in \mathcal{L}:\ |z| > p.\ \exists u, v, w, x, y.\ P$)
- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.$ not ($\exists u, v, w, x, y.\ P$)
- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w, x, y.$ not (P)
- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w, x, y.$ not ($P1$ and $P2$ and $P3$ and $P4$)

- not ($P1$ and $P2$ and $P3$ and $P4$)
- not (($P1$ and $P2$ and $P3$) and $P4$)
- not ($P1$ and $P2$ and $P3$) or not $P4$
- ($P1$ and $P2$ and $P3$) implies not $P4$
- ($P1$ and $P2$ and $P3$) implies not ($\forall i \in \mathbb{N}. uv^i wx^i y \in \mathcal{L}$)
- ($P1$ and $P2$ and $P3$) implies $\exists i \in \mathbb{N}$. not ($uv^i wx^i y \in \mathcal{L}$)

P_1 *implies* P_2 è equivalente a $not(P_1) \vee P_2$.

Dunque in sintesi, la $not(PL\text{-}THESIS)$ è la seguente:

- $\forall p \in \mathbb{N}^+.\ \exists z \in \mathcal{L}:\ |z| > p.\ \forall u, v, w, x, y.$
  - ($z = uvwxy$ and $|vwx| \leq p$ and $|vx| > 0$)
  - implies
  - $\exists i \in \mathbb{N}. uv^i wx^i y \notin \mathcal{L}$

Dunque operativamente:

- **Qualunque** sia il numero naturale positivo p (dunque lo lasciamo “variabile”)
- **Scegli** una parola z più lunga di p e appartenente al linguaggio (scegliamo quella più conveniente)
- Dimostrare che **qualunque** sia lo spacchettamento di z in $uvwxy$ con $|vxy| \leq p$ e $|vx| > 0$ che viene preso in considerazione, si può scegliere un numero naturale i che sia tale che $u v^i w x^i y \notin L$ (ovvero tale che la versione pumped della parola non appartiene al linguaggio)
  - ... let p be an arbitrary positive integer
    $\forall p$: Any p
  - Take $z = a^p b^p c^p$
    $\exists z$: Choose $z \in \mathcal{L}$ longer than p
  - ... If ($z = uvwxy$ and $|vwx| \leq p$ and $|vx| > 0$) then ....
    $\forall u, v, w, x, y : z = uvwxy$ and $|vwx| \leq p$ and $|vx| > 0$
  - ... $uv^0 wx^0 y \notin \mathcal{L}$
    $\exists i$: Choose a value for the iterator
  - ... contradiction

Vediamo ora qualche esempio sul come utilizzare la $not(PL\text{-}THESIS)$.

Prendiamo in considerazione la seguente grammatica G:

$$\begin{array}{rrcl} \mathcal{G}: & S & \rightarrow & aSBc \mid abc \\ & cB & \rightarrow & Bc \\ & bB & \rightarrow & bb \end{array}$$

Questa grammatica è context-dependent e il linguaggio generato è $L(G)=\{a^n b^n c^n \mid n>0\}$.
Vogliamo verificare che non è libero tramite il pumping lemma. Supponiamo quindi che $L(G)$ sia un linguaggio libero, e sia p un intero positivo arbitrario.
Prendiamo $z=a^p b^p c^p$.
Se

- $z=uvwxy$
- $|vwx|\leq p$
- $|vx|>0$

Allora vx non può avere occorrenze sia di a che di c perché l'ultima occorrenza di a e la prima occorrenza di c sono $p+1$ posizioni distanti.
Infatti, per alcuni k e j positivi

- O $vwx=a^k$
- O $vwx=a^k b^j$
- O $vwx=b^j$
- O $vwx=b^j c^k$
- O $vwx=c^k$

Dunque vx risulta sempre o senza occorrenze di a o senza occorrenze di c, ovvero nessuna a e nessuna c appare contemporaneamente in vx.
Quindi uv^0wx^0y non può avere la forma $a^n b^n c^n$, dunque $uv^0wx^0y \notin L$.
In conclusione, per contraddizione del pumping lemma, L non è libero.

Vediamo un altro esempio con la seguente grammatica:

$$\begin{array}{rrcl} \mathcal{G}: & S & \rightarrow & CD \\ & C & \rightarrow & aCA \mid bCB \mid \epsilon \\ & AD & \rightarrow & aD \\ & BD & \rightarrow & bD \\ & Aa & \rightarrow & aA \\ & Ab & \rightarrow & bA \\ & Ba & \rightarrow & aB \\ & Bb & \rightarrow & bB \\ & D & \rightarrow & \epsilon \end{array}$$

Anche in questo caso G è context-dependent.
Analizziamo questa grammatica:

- Derived strings have a bookmark D initially at rightmost position

$$S \rightarrow CD$$

- Strings can only grow longer by replacing C

$$C \rightarrow aCA \mid bCB \mid \epsilon$$

- Non-terminals close to the rightmost delimiter can be converted to the corresponding terminal

$$AD \rightarrow aD$$
$$BD \rightarrow bD$$

- Non-terminals and terminals can be swapped when the terminal is at the right of the non-terminal

$$Aa \rightarrow aA$$
$$Ab \rightarrow bA$$
$$Ba \rightarrow aB$$
$$Bb \rightarrow bB$$

Abbiamo già visto che il linguaggio generato da questa grammatica è $L(G)=\{ww \mid w \in (a \mid b)^*\}$.
Ci chiediamo di nuovo se questo linguaggio è libero o meno. La risposta è no e per provarlo si agisce in maniera uguale a prima (una buona scelta per z è $z=a^p b^p a^p b^p$).

Dunque:

- $\{a^n b^n c^n \mid n>0\}$ non è libero, visto che non rispetta il pumping lemma
- $\{a^n b^n c^j \mid n,j>0\}$ è libero in quanto è la concatenazione di due linguaggi liberi
  - $\{a^n b^n \mid n>0\}$
  - $\{c^j \mid j>0\}$
- $\{a^j b^n c^n \mid j,n>0\}$ è libero in quanto è la concatenazione di due linguaggi liberi
  - $\{a^j \mid j>0\}$
  - $\{b^n c^n \mid n>0\}$

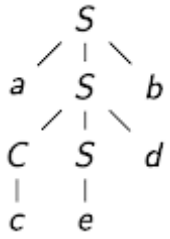
Questo ci conferma il fatto che **la classe dei linguaggi liberi non è chiusa rispetto all'operazione di intersezione**, infatti:

- $L_1=\{a^n b^n c^j \mid n,j>0\}$ è libero
- $L_2=\{a^j b^n c^n \mid j,n>0\}$ è libero
- $L_1 \cap L_2=\{a^n b^n c^n \mid n>0\}$ non è libero (per il pumping lemma)

Analisi sintattica

La fase di analisi sintattica di un compilatore prende una stringa di token prodotti dal lexer e da questa costruisce un albero sintattico per la stringa trovando una derivazione della stringa dal simbolo iniziale della grammatica.

In generale, il **parsing** consiste nel, data una grammatica $G=(V,T,S,P)$ e una stringa w, determinare se w appartiene a $L(G)$ e, in caso affermativo, fornire il suo albero di derivazione. Prendiamo in esempio questo albero di derivazione



dunque la stringa *acedb* appartiene alla grammatica.

Ciò può essere fatto indovinando le derivazioni fino a trovare quella corretta, ma l'indovinare a caso difficilmente è un metodo efficace. Tuttavia, alcune tecniche di parsing si basano sull' "indovinare" le derivazioni, queste si assicurano, guardando la stringa, di indovinare sempre correttamente. Queste tecniche sono chiamate metodi di **parsing predittivi**. I parser predittivi costruiscono sempre l'albero sintattico dall'alto verso il basso e sono quindi chiamati anche **parser top-down** (deterministici), e ritornano la leftmost derivation.

- Top-down: Take, in the given order:
  - $S \to aSb$
  - $S \to CSd$
  - $C \to c$
  - $S \to e$

Altri parser seguono un approccio opposto: cercano parti della stringa di input che corrispondono ai lati destri delle produzioni e riscrivono queste nei non-terminali a sinistra, costruendo contemporaneamente parti dell'albero sintattico. L'albero sintattico viene completato quando la stringa è stata riscritta (attraverso una derivazione inversa) nel simbolo di inizio. Anche qui, desideriamo assicurarci di scegliere sempre le riscritture "corrette", in modo da ottenere un parsing deterministico. Tali metodi sono chiamati metodi di **parsing bottom-up**, e ritornano la rightmost derivation.

- Bottom-up: Take, in reverse order:
  - $C \to c$
  - $S \to e$
  - $S \to CSd$
  - $S \to aSb$

Nelle prossime sezioni esamineremo prima il parsing predittivo e successivamente un metodo di parsing bottom-up chiamato parsing SLR.

Parsing predittivo

Se guardiamo la seguente leftmost derivation

$$
\begin{aligned}
& \underline{T} \\
\Rightarrow\ & \mathtt{a}\underline{T}\mathtt{c} \\
\Rightarrow\ & \mathtt{aa}\underline{T}\mathtt{cc} \\
\Rightarrow\ & \mathtt{aa}\underline{R}\mathtt{cc} \\
\Rightarrow\ & \mathtt{aa}\underline{R}\mathtt{b}R\mathtt{cc} \\
\Rightarrow\ & \mathtt{aa}\underline{R}\mathtt{b}R\mathtt{b}R\mathtt{cc} \\
\Rightarrow\ & \mathtt{aab}\underline{R}\mathtt{b}R\mathtt{cc} \\
\Rightarrow\ & \mathtt{aab}\underline{R}\mathtt{b}R\mathtt{b}R\mathtt{cc} \\
\Rightarrow\ & \mathtt{aabb}\underline{R}\mathtt{b}R\mathtt{cc} \\
\Rightarrow\ & \mathtt{aabbb}\underline{R}\mathtt{cc} \\
\Rightarrow\ & \mathtt{aabbbcc}
\end{aligned}
$$

basata sulla seguente grammatica

$$
\begin{aligned}
T &\rightarrow R \\
T &\rightarrow \mathtt{a}T\mathtt{c} \\
R &\rightarrow \\
R &\rightarrow R\mathtt{b}R
\end{aligned}
$$

vediamo che a sinistra dei non-terminali riscritti ci sono solo terminali. Questi terminali corrispondono a un prefisso della stringa che viene analizzata. In una situazione di analisi, questo prefisso sarà la parte dell'input che è già stata letta. Ora il compito del parser è scegliere la produzione con cui il non-terminale più a sinistra non espanso deve essere riscritto. Il nostro obiettivo è essere in grado di fare questa scelta in modo deterministico basandoci sul prossimo simbolo di input non ancora abbinato, così da evitare di dover fare backtracking.

Se osserviamo la terza riga della derivazione precedente, abbiamo già letto due a e (se la stringa di input è quella mostrata nell'ultima riga) il prossimo simbolo è una b. Dal momento che il lato destro della produzione

$$T \rightarrow aTc$$

inizia con una a, è ovvio che non possiamo utilizzarla. Quindi, possiamo riscrivere T solo utilizzando la produzione

$$T \rightarrow R$$

Non siamo altrettanto fortunati nel passo successivo. Nessuna delle produzioni per R inizia con un simbolo terminale, quindi non possiamo scegliere immediatamente una produzione basandoci su questo. Poiché la grammatica è ambigua, non dovrebbe sorprenderci che non sempre possiamo scegliere in modo univoco. Se invece utilizziamo la grammatica non ambigua

$$
\begin{aligned}
T &\rightarrow R \\
T &\rightarrow \mathtt{a}T\mathtt{c} \\
R &\rightarrow \\
R &\rightarrow \mathtt{b}R
\end{aligned}
$$

possiamo scegliere immediatamente la seconda produzione per R. Quando tutte le b vengono lette e siamo arrivati alla successiva c, scegliamo la produzione vuota per R e abbiniamo il resto dell'input con il resto della stringa derivata.

Se possiamo sempre scegliere una produzione univoca basandoci sul prossimo simbolo di input, siamo in grado di eseguire il parsing predittivo senza backtracking.

FIRST

Nei casi semplici, come quello sopra, tutte le produzioni per un non-terminale iniziano con terminali distinti, tranne al massimo una produzione che non inizia con un terminale. Scegliamo quest'ultima ogni volta che il simbolo di input non corrisponde a nessuno dei simboli terminali che iniziano le altre produzioni. Possiamo estendere il metodo per funzionare anche per grammatiche in cui diverse produzioni iniziano con non-terminali. Dobbiamo solo essere in grado di selezionare tra di esse in base al simbolo di input. In altre parole, se le stringhe che queste produzioni possono derivare iniziano con simboli da insiemi disgiunti, controlliamo se il simbolo di input è in uno di questi insiemi e scegliamo la produzione corrispondente se lo è. Se non lo è, e c'è una produzione vuota, la scegliamo. In caso contrario, segnaliamo un messaggio di errore di sintassi.

Pertanto, definiamo la funzione $FIRST$, che, data una sequenza di simboli grammaticali (ad esempio, il lato destro di una produzione), restituisce l'insieme di simboli con cui le stringhe derivate da quella sequenza possono iniziare:

Un simbolo c è in $FIRST(\alpha)$ se e solo se $\alpha \Rightarrow c\,\beta$ per qualche sequenza (possibilmente vuota) β di simboli grammaticali.

Se $\alpha \Rightarrow^{*} \varepsilon$ allora $\varepsilon \in FIRST(\alpha)$

Descriviamo il calcolo di $FIRST$ tramite un'analisi dei casi sulle possibili forme di sequenze di simboli grammaticali:

$$FIRST(\varepsilon) = \{\varepsilon\}$$
$$FIRST(a) = \{a\}$$
$$FIRST(A) = ¿\, A \to \alpha \,¿\, ❑ FIRST(\alpha)$$

La computazione di $FIRST(Y_1 \dots Y_n)$ è:

```
first(Y1 ... Yn) = ∅;
j = 1 ;
while j ≤ n do
    add first(Yj) \ {ϵ} to first(Y1 ... Yn) ;
    if ϵ ∈ first(Yj) then j = j + 1 ;
    else break ;
if j = n + 1 then add ϵ to first(Y1 ... Yn) ;
```

dove a è un terminale, A è un non-terminale, $\alpha = Y_1 \dots Y_n$ è una sequenza di simboli grammaticali ed ε rappresenta la sequenza vuota di simboli grammaticali.

L'equazione non banale è quella per $\alpha = Y_1 \dots Y_n$. Ovviamente, qualsiasi cosa possa iniziare una stringa derivabile da Y_1 può anche iniziare una stringa derivabile da $Y_1 \dots Y_n$. Tuttavia, se Y_1 è

annullabile, ovvero se $\varepsilon \in FIRST(Y_1)$, una derivazione può procedere come $Y_1 \dots Y_n \Rightarrow Y_2 \dots Y_n \Rightarrow \dots$, quindi qualsiasi cosa sia in $FIRST(Y_2 \dots Y_n)$ è anche in $FIRST(Y_1 \dots Y_n)$. Alla fine, ε appartiene a $FIRST(\alpha)$ se e solo se da ognuno dei simboli $Y_1 \dots Y_n$ posso derivare la ε.

Ecco un esempio:

$$
\begin{array}{lcl}
E & \rightarrow & TE' \\
E' & \rightarrow & +TE' \mid \epsilon \\
T & \rightarrow & FT' \\
T' & \rightarrow & *FT' \mid \epsilon \\
F & \rightarrow & (E) \mid id
\end{array}
$$

Consider the production for E, compute first(E)

	elements in *first*
E	(, *id*
E'	+, ϵ
T	(, *id*
T'	*, ϵ
F	(, *id*

Quando si lavora manualmente con le grammatiche, di solito è abbastanza facile capire per la maggior parte delle produzioni quali sono i loro insiemi $FIRST$. Ad esempio se il lato destro inizia con un terminale, l'insieme $FIRST$ consiste solo in quel simbolo.

Parsing predittivo rivisitato

Siamo ora pronti a costruire parser predittivi per una classe più ampia di grammatiche: se i lati destri delle produzioni per un non-terminale hanno insiemi $FIRST$ disgiunti, possiamo utilizzare il prossimo simbolo di input per scegliere tra le produzioni.
Nella sezione precedente sul parsing predittivo, abbiamo scelto la produzione vuota (se presente) su qualsiasi simbolo che non fosse negli insiemi $FIRST$ delle produzioni non vuote per lo stesso non-terminale. Possiamo estendere questo concetto in modo che, in caso di insiemi $FIRST$ che non corrispondono, possiamo selezionare una produzione se è annullabile, ovvero se $\varepsilon \in FIRST$. L'idea è che una produzione annullabile può derivare la stringa vuota, quindi il simbolo di input non deve essere letto dalla produzione stessa.
Ma se ci sono diverse produzioni annullabili, non abbiamo modo di scegliere tra di loro. Pertanto, non permettiamo che più di una produzione per un non-terminale sia annullabile.

Precedentemente abbiamo detto che i nostri metodi di analisi sintattica rileveranno le grammatiche ambigue. Tuttavia, ciò non è vero con il metodo come enunciato sopra: possiamo ottenere una scelta univoca di produzione anche per alcune grammatiche ambigue. In questo caso, l'analisi sintattica sceglierà semplicemente uno dei possibili alberi sintattici per una data stringa di input. In molti casi, non consideriamo accettabile tale comportamento. In realtà, vorremmo molto che il nostro metodo di costruzione del parser ci avvertisse se per errore scriviamo una grammatica ambigua.

Ancora peggio, le regole per il parsing predittivo come presentate qui potrebbero, anche per alcune grammatiche non ambigue, fornire una scelta deterministica di produzione, ma rifiutare stringhe che in realtà appartengono al linguaggio descritto dalla grammatica. Se, ad esempio, cambiamo la seconda produzione nella grammatica

$$\begin{aligned} T &\rightarrow R \\ T &\rightarrow \mathtt{a}T\,\mathtt{c} \\ R &\rightarrow \\ R &\rightarrow \mathtt{b}R \end{aligned}$$

in:

$$T \rightarrow aTb$$

questo non cambierà le scelte fatte dal parser predittivo per il non-terminale R. Tuttavia, scegliere sempre l'ultima produzione per R su una b porterà al rifiuto erroneo di molte stringhe, compresa ab.

Questo tipo di comportamento è chiaramente inaccettabile. Dovremmo, almeno, ricevere un avviso che ciò potrebbe accadere, in modo da poter riscrivere la grammatica o scegliere un altro metodo di analisi sintattica.
Quindi, aggiungiamo alla nostra costruzione di parser predittivi un test che respingerà tutte le grammatiche ambigue e quelle grammatiche non ambigue che potrebbero causare al parser di fallire erroneamente.

Finora abbiamo semplicemente scelto una produzione annullabile solo se nessun'altra scelta è possibile. Tuttavia, ciò non è sempre corretto, quindi dobbiamo cambiare la regola per dire che scegliamo una produzione $N \rightarrow \alpha$ sul simbolo c se una delle due condizioni seguenti è soddisfatta:

1. $c \in FIRST(\alpha)$
2. $\varepsilon \in FIRST(\alpha)$ e la sequenza Nc può occorrere da qualche parte in una derivazione che inizia dal simbolo iniziale della grammatica

La prima regola è ovvia, ma la seconda richiede una piccola spiegazione: se α è annullabile, possiamo costruire un albero di sintassi per essa senza leggere alcun input, quindi sembra che una produzione annullabile potrebbe essere una scelta valida indipendentemente dal prossimo simbolo di input. Non solo ciò darebbe molte scelte valide di produzione ogni volta che ci sono produzioni sia annullabili che non annullabili per lo stesso non-terminale, ma non è sempre corretto scegliere la produzione annullabile: il parsing predittivo effettua una leftmost derivation, quindi riscriviamo sempre il non-terminale più a sinistra N nella sequenza corrente di simboli grammaticali. Quindi qualsiasi input non corrisposto da N deve essere corrisposto dalla sequenza di simboli grammaticali che segue N nella sequenza corrente. Se ciò non è possibile, abbiamo fatto una cattiva scelta durante la derivazione di N. In particolare, se N deriva la sequenza vuota, il prossimo simbolo di input c dovrebbe iniziare la derivazione della sequenza che segue N. Quindi, almeno la sequenza di simboli che segue N dovrebbe avere una derivazione che inizia con c. Se (usando un diverso ordine di derivazione) deriviamo i simboli dopo N prima di derivare N, dovremmo vedere la sequenza Nc durante la derivazione. Se non

lo facciamo, non possiamo riscrivere N nella sequenza vuota senza rimanere bloccati in seguito quando riscriviamo la sequenza rimanente. Poiché l'ordine di derivazione non cambia l'albero di sintassi, possiamo vedere che se l'alternativo ordine di derivazione si blocca, lo farà anche l'ordine di derivazione più a sinistra. Pertanto, possiamo riscrivere N nella sequenza vuota solo se il prossimo simbolo di input c può occorrere in combinazione con N in una derivazione legale. Vedremo nella prossima sezione come possiamo determinare questo.

Va notato che una produzione annullabile $N \to \alpha$ può essere selezionata validamente se $c \in FIRST(\alpha)$. Anche con la restrizione sulla scelta delle produzioni annullabili, possiamo comunque avere situazioni in cui sia le produzioni annullabili che quelle non annullabili sono scelte valide. Questo include tutte le grammatiche ambigue che non vengono rilevate come ambigue dal metodo originale in cui scegliamo solo le produzioni annullabili se non è possibile effettuare altre scelte.

FOLLOW

Per determinare quando è possibile selezionare produzioni annullabili durante il parsing predittivo, introduciamo gli insiemi di $FOLLOW$ per i non-terminali.

> Un simbolo terminale a appartiene a $FOLLOW(A)$ se e solo se esiste una derivazione dal simbolo iniziale S della grammatica tale che $S \Rightarrow \alpha A a \beta$, dove α e β sono sequenze (possibilmente vuote) di simboli grammaticali.

In altre parole, un terminale a appartiene a $FOLLOW(A)$ se a può seguire A in qualche punto di una derivazione. A differenza di $FIRST(A)$, questa non è una proprietà delle produzioni che hanno A come lefthand, ma delle produzioni che (direttamente o indirettamente) utilizzano A nel loro righthand.

Per gestire correttamente le condizioni di fine stringa, vogliamo rilevare se $S \Rightarrow \alpha A$, cioè se ci sono derivazioni in cui A può essere seguito dalla fine dell'input. Risulta essere facile fare questo aggiungendo una produzione extra alla grammatica:

$$S' \to S\$$$

dove S' è un nuovo non-terminale che sostituisce S come simbolo di avvio e $\$$ è un nuovo simbolo terminale che rappresenta la fine dell'input. Quindi, nella nuova grammatica, $\$$ sarà in $FOLLOW(A)$ esattamente se $S' \Rightarrow \alpha A \$$, il che avviene esattamente quando $S \Rightarrow \alpha A$.

Il modo più semplice per calcolare $FOLLOW$ è generare una collezione di vincoli di insiemi, che vengono successivamente risolti per trovare gli insiemi minimi che rispettano i vincoli.

Una produzione

$$B \to \alpha A \beta$$

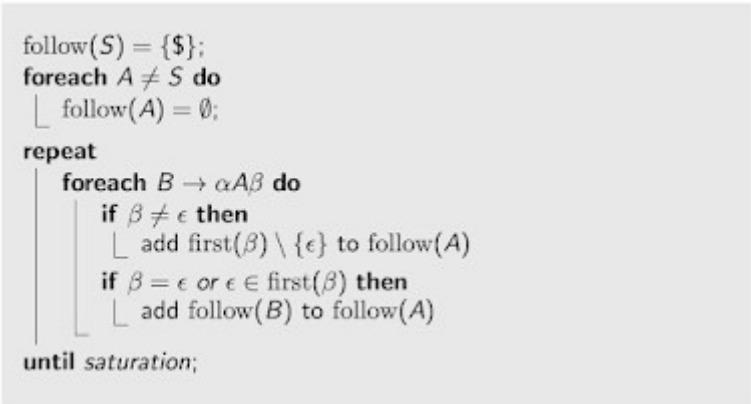
genera il vincolo $FIRST(\beta) \subseteq FOLLOW(A)$, poiché β, ovviamente, può seguire A. Inoltre, se β è annullabile la produzione genera anche il vincolo $FOLLOW(B) \subseteq FOLLOW(A)$ (nota la direzione dell'inclusione). Il motivo è che, se un simbolo c è in $FOLLOW(B)$, allora (per

definizione) c'è una derivazione $S' \Rightarrow \gamma\, Bc\, \delta$. Ma dato che $B \rightarrow \alpha\, A\, \beta$ e β è annullabile, possiamo continuare questo processo con $\gamma\, Bc\, \delta \Rightarrow \gamma\, \alpha\, Ac\, \delta$, quindi anche c è in $FOLLOW(A)$.

I passi necessari per calcolare i set di follow di una grammatica sono i seguenti:

1) Aggiungi un nuovo non-terminale $S' \rightarrow S\,\$$, dove S è il simbolo di partenza della grammatica originale. S' diventa il nuovo simbolo di partenza per la grammatica estesa.
2) Per ogni non-terminale A, individua tutte le occorrenze di A nei lati destri delle produzioni. Per ogni occorrenza, esegui quanto segue:
   2.1) Sia β il resto del lato destro della produzione dopo l'occorrenza di A. Nota che β potrebbe essere vuoto. In altre parole, la produzione ha la forma $B \rightarrow \alpha\, A\, \beta$, dove B è un non-terminale (eventualmente uguale a A), e α e β sono sequenze (eventualmente vuote) di simboli della grammatica. Tieni presente che se il lato destro contiene diverse occorrenze di A, effettuiamo una suddivisione per ogni occorrenza.
   2.2) Definiamo m come l'insieme $FIRST(\beta)$¿. Aggiungiamo il vincolo $m \subseteq FOLLOW(A)$ all'insieme di vincoli. Nel caso in cui β sia vuoto, puoi omettere questo vincolo, poiché non aggiunge nulla.
   2.3) Se β è annullabile, individua il non-terminale B sul lato sinistro della produzione e aggiungi il vincolo $FOLLOW(B) \subseteq FOLLOW(A)$. Se $A = B$, puoi omettere il vincolo, poiché non aggiunge nulla. Nota che se β è vuoto, allora β è annullabile.
3) Risolviamo i vincoli seguendo i seguenti passaggi:
   3.1) Inizia con insiemi vuoti per $FOLLOW(A)$ per tutti i non-terminali N (escluso S').
   3.2) Per ogni vincolo nella forma $m \subseteq FOLLOW(A)$ costruito nel passo 2.1, aggiungi il contenuto di m a $FOLLOW(A)$.
   3.3) Iterando fino a raggiungere un punto fisso, per ogni vincolo nella forma $FOLLOW(B) \subseteq FOLLOW(A)$, aggiungi il contenuto di $FOLLOW(B)$ a $FOLLOW(A)$.

Dunque l'algoritmo per calcolare i $FOLLOW$ è il seguente:



A livello pratico, prima di computare $FOLLOW$ calcolo prima tutti i $FIRST$.

Ecco un esempio

$$
\begin{array}{lcl}
E & \rightarrow & TE' \\
E' & \rightarrow & +TE' \mid \epsilon \\
T & \rightarrow & FT' \\
T' & \rightarrow & *FT' \mid \epsilon \\
F & \rightarrow & (E) \mid id
\end{array}
$$

foreach $B \rightarrow \alpha A \beta$ **do**
  if $\beta \neq \epsilon$ **then**
    add $\text{first}(\beta) \setminus \{\epsilon\}$ to $\text{follow}(A)$
  end
  if $\beta = \epsilon$ *or* $\epsilon \in \text{first}(\beta)$ **then**
    add $\text{follow}(B)$ to $\text{follow}(A)$
  end
end

	first	computation of follow	follow
E	{id, (}	$;)	{$,)}
E'	{ϵ,+}	add follow(E)	{$,)}
T	{id, (}	+; add follow(E); +; add follow(E')	{$,),+}
T'	{ϵ,*}	add follow(T)	{$,),+}
F	{id, (}	*; add follow(T); *; add follow(T')	{$,),+,*}

Se torniamo alla questione del parsing predittivo della grammatica

$$
\begin{array}{lcl}
T & \rightarrow & R \\
T & \rightarrow & \mathtt{a}T\mathtt{c} \\
R & \rightarrow & \\
R & \rightarrow & R\mathtt{b}R
\end{array}
$$

vediamo che per il non-terminale R dovremmo scegliere la produzione vuota su qualsiasi simbolo in $FOLLOW(R)$, cioè $\{\$, c, b\}$, e scegliere la produzione non vuota sui simboli in $FIRST(RbR)$ cioè $\{b\}$. Dato che questi insiemi si sovrappongono (sul simbolo b), non possiamo scegliere in modo univoco una produzione per R in base al prossimo simbolo in input. Pertanto, la revisione della costruzione dei parser predittivi (vedi sotto) rifiuterà questa grammatica come potenzialmente ambigua.

LL(1) parsing

Abbiamo, nelle sezioni precedenti, esaminato come possiamo scegliere le produzioni basandoci sugli insiemi $FIRST$ e $FOLLOW$, cioè utilizzando la regola che scegliamo una produzione $N \rightarrow \alpha$ quando il simbolo di input è c se:

- $c \in FIRST(\alpha)$, oppure
- $\varepsilon \in FIRST(\alpha)$ (ovvero α è annullabile) e $c \in FOLLOW(N)$

Se possiamo sempre **scegliere una produzione in modo univoco** utilizzando queste regole, questo viene chiamato parsing $\mathrm{LL}(1)$ - la prima L indica la direzione di lettura (left-to-right), la seconda L indica l'ordine di derivazione (left) e il 1 indica che c'è una previsione di un simbolo. Una grammatica che può essere analizzata utilizzando il parsing $\mathrm{LL}(1)$ viene chiamata grammatica $\mathrm{LL}(1)$.

Nel resto di questa sezione, vedremo come possiamo implementare analizzatori $\mathrm{LL}(1)$ come programmi. Esistono due metodi di implementazione: il discesa ricorsiva (che non vedremo), dove la struttura della grammatica viene tradotta direttamente nella struttura di un programma, e un approccio basato su tabelle che codifica il processo decisionale in una tabella.

Nel **parsing** $\mathrm{LL}(1)$ **basato su tabelle**, codifichiamo la selezione delle produzioni in una tabella. Un programma semplice e non ricorsivo utilizza questa tabella e uno stack per eseguire l'analisi.

La tabella è indicizzata incrociando i non-terminali e i terminali e contiene per ogni coppia di questo tipo la produzione (se presente) che viene scelta per quel non-terminale quando quel terminale è il prossimo simbolo di input. Questa decisione viene presa nella seguente maniera: la produzione $N \to \alpha$ è presente nella tabella in posizione (N, a) se a è in $FIRST(\alpha)$ o se sia α è annullabile e a è in $FOLLOW(N)$ (come abbiamo detto prima).
Usiamo $\$$ come carattere di fine stringa.

In altre parole la tabella ci dice, in base al non-terminale che stiamo leggendo e in base al simbolo che stiamo puntando in lettura, quale produzione dobbiamo utilizzare per espandere il non-terminale per arrivare (prima o poi) quel simbolo.

Considerando la seguente grammatica:

$$\begin{aligned} E &\to TE' \\ E' &\to +TE' \mid \epsilon \\ T &\to FT' \\ T' &\to *FT' \mid \epsilon \\ F &\to (E) \mid id \end{aligned}$$

La tabella di parsing è la seguente:

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

Dunque:

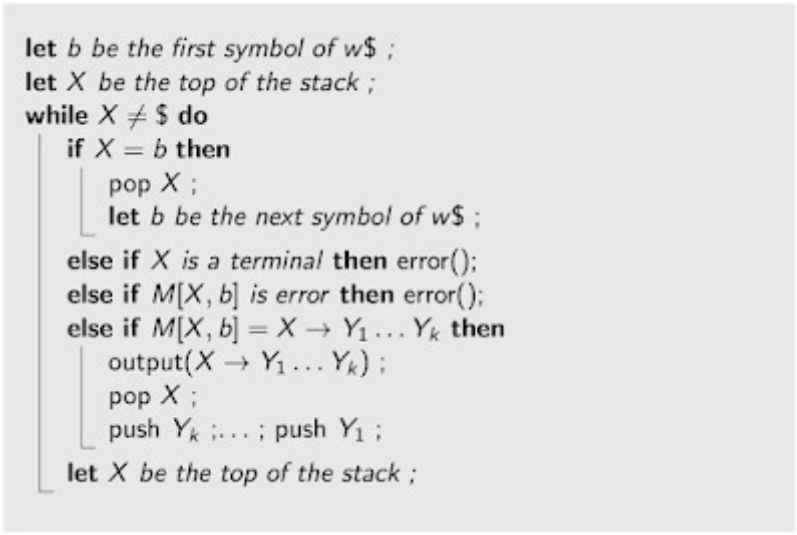
- Abbiamo una parola, diciamo $id + id * id$, e dobbiamo derivarla dalla produzione E.
- Controlliamo l'ingresso nella tabella $[E, id]$.
- L'ingresso contiene $E \to TE'$, quindi facciamo il passo di derivazione $E \Rightarrow TE'$.

E così via.

L'algoritmo per il parsing top-down predittivo che utilizza questa tabella deve:

- Input: stringa w, tabella di parsing top-down M per $G = (V, T, S, P)$
- Output: Derivazione leftmost di w se $w \in L(G)$, altrimenti $error()$
- Inizializzazione: $w\$$ nel buffer di input, $\$S$ nello stack con S sul top

L’algoritmo è mostrato nella figura:



Esso fa uso di uno stack, che contiene in ogni momento (letto dal top al bottom) la parte della derivazione corrente che non è ancora stata abbinata all'input. Quando questo diventa vuoto, l'analisi è completata. Se lo stack non è vuoto e il suo elemento superiore contiene un terminale, quel terminale viene confrontato con l'input e rimosso dallo stack. Altrimenti, la cima dello stack deve essere un non-terminale, che incrociamo nella tabella col simbolo di input successivo. Se l'ingresso nella tabella è vuoto, segnaliamo un errore. Se non è vuoto, rimuoviamo il non-terminale dallo stack e lo sostituiamo con il righthand della produzione nell’entry della tabella. La lista di simboli sul righthand viene inserita in modo che il primo di essi sia sul top dello stack.

Tornando all’esempio di prima, l'esecuzione completa è la seguente:

	id	$+$	$*$	$($	$)$	$\$$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

Implementation: $w = id + id * id$

stack	input	output	leftmost derivation
$\$E$	$id + id * id\$$	$E \to TE'$	$E \Rightarrow TE'$
$\$E'T$	$id + id * id\$$	$T \to FT'$	$\Rightarrow FT'E'$
$\$E'T'F$	$id + id * id\$$	$F \to id$	$\Rightarrow idT'E'$
$\$E'T'id$	$id + id * id\$$		
$\$E'T'$	$+id * id\$$	$T' \to \epsilon$	$\Rightarrow idE'$
$\$E'$	$+id * id\$$	$E' \to +TE'$	$\Rightarrow id+TE'$
$\$E'T+$	$+id * id\$$		
$\$E'T$	$id * id\$$	$T \to FT'$	$\Rightarrow id+FT'E'$
$\$E'T'F$	$id * id\$$	$F \to id$	$\Rightarrow id+idT'E'$
$\$E'T'id$	$id * id\$$		
$\$E'T'$	$*id\$$	$T' \to *FT'$	$\Rightarrow id+id*FT'E'$
$\$E'T'F*$	$*id\$$		
$\$E'T'F$	$id\$$	$F \to id$	$\Rightarrow id+id*idT'E'$
$\$E'T'id$	$id\$$		
$\$E'T'$	$\$$	$T' \to \epsilon$	$\Rightarrow id+id*idE'$
$\$E'$	$\$$	$E' \to \epsilon$	$\Rightarrow id+id*id$
$\$$	$\$$		

L’algoritmo che abbiamo visto verifica solo se l'input è valido. Esso può essere esteso per costruire un albero di sintassi. Questo può essere fatto facendo sì che ciascun non-terminale nello stack punti al suo nodo nell'albero di sintassi parzialmente costruito. Quando il non-terminale viene sostituito da uno dei suoi righthand, i nodi per i simboli sul righthand vengono aggiunti come figli al nodo.

Per costruire la tabella di parsing usiamo il seguente algoritmo:

input : Grammar $\mathcal{G} = (V, T, S, \mathcal{P})$
output : Predictive parsing table M
foreach $A \to \alpha \in \mathcal{P}$ **do**
  add $A \to \alpha$ to $M[A, b]$ for each $b \in \text{first}(\alpha)$
  if $\epsilon \in \text{first}(\alpha)$ **then**
    add $A \to \alpha$ to $M[A, x]$ for each $x \in \text{follow}(A)$
set to *error*() all the empty entries;

Notiamo che $FOLLOW(A)$ può contenere $\$$, quindi usiamo x (anziché b) per fare riferimento ad esso.

Quando un simbolo a permette diverse scelte di produzione per un non-terminale N, diciamo che c'è un conflitto su quel simbolo per quel non-terminale. I conflitti possono essere causati da grammatiche ambigue (infatti tutte le grammatiche ambigue causeranno conflitti), ma ci sono anche grammatiche non ambigue che causano conflitti. Vedremo nella prossima sezione come possiamo riscrivere queste grammatiche per evitare conflitti, ma è importante notare che questo non è sempre possibile: ci sono linguaggi per i quali esistono grammatiche non ambigue ma per cui nessuna grammatica del linguaggio genera una tabella $\text{LL}(1)$ senza conflitti. Tali linguaggi sono detti non-$\text{LL}(1)$. Tuttavia, è importante notare la differenza tra un linguaggio non-$\text{LL}(1)$ e

una grammatica non-$\mathrm{LL}(1)$: un linguaggio può benissimo essere $\mathrm{LL}(1)$ anche se la grammatica usata per descriverlo non lo è.

Ecco un esempio:

LL(1) grammars

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

Is it LL(1)?

$\mathrm{first}(E) = \mathrm{first}(T) = \mathrm{first}(F) = \{(, id\}$

Hence, e.g.,

- $M[E, id]$ contains both $E \rightarrow E + T$ and $E \rightarrow T$

What if we always expand E into $E + T$ when we see the input id?

$E \Rightarrow E + T \Rightarrow E + T + T \Rightarrow E + T + T + T \ldots$

In questo esempio la grammatica non è $\mathrm{LL}(1)$ perchè c'è almeno un conflitto, ovvero quello nella entry $M[E, id]$.

Riscrivere una grammatica per il parsing $\mathrm{LL}(1)$

In questa sezione esamineremo metodi per riscrivere le grammatiche in modo che siano più adatte al parsing $\mathrm{LL}(1)$. In particolare, esamineremo l'**eliminazione della ricorsione a sinistra** e la **fattorizzazione a sinistra**.
Tuttavia, è importante notare che non tutte le grammatiche possono essere riscritte per consentire il parsing $\mathrm{LL}(1)$. In questi casi devono essere utilizzate tecniche di parsing più avanzate.

La seguente grammatica delle espressioni non ambigua

$$\begin{aligned} Exp &\rightarrow Exp + Exp2 \\ Exp &\rightarrow Exp - Exp2 \\ Exp &\rightarrow Exp2 \\ Exp2 &\rightarrow Exp2 * Exp3 \\ Exp2 &\rightarrow Exp2 / Exp3 \\ Exp2 &\rightarrow Exp3 \\ Exp3 &\rightarrow \mathbf{num} \\ Exp3 &\rightarrow (Exp) \end{aligned}$$

non è $\mathrm{LL}(1)$. Il motivo è che tutte le produzioni in exp e $\mathrm{exp}\,2$ hanno gli stessi insiemi $FIRST$. Questo tipo di sovrapposizione avviene sempre quando ci sono produzioni ricorsive a sinistra nella grammatica, poiché l'insieme $FIRST$ di una produzione ricorsiva a sinistra includerà l'insieme $FIRST$ del non-terminale stesso e quindi sarà un insieme superiore rispetto agli insiemi $FIRST$ di tutte le altre produzioni per quel non-terminale.

Se una grammatica G è **left recursive** (ricorsiva a sinistra), allora G non è $\mathrm{LL}(1)$.

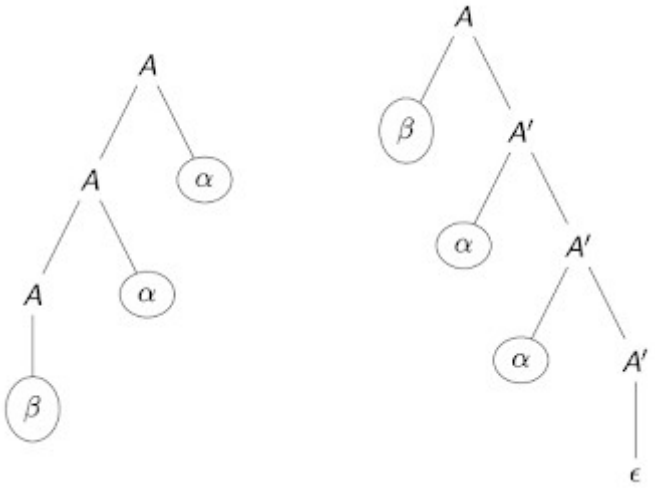
In particolare, una grammatica è **ricorsiva a sinistra** se, per alcuni A e alcuni α, $A \Rightarrow^* A\alpha$.

Per risolvere questo problema, dobbiamo evitare la ricorsione a sinistra nella grammatica.
Iniziamo esaminando la ricorsione a sinistra diretta:
Una grammatica è **ricorsiva direttamente a sinistra** se ha una produzione nella forma $A \to A\alpha$.

Prendiamo prima il caso base, nel quale dobbiamo trasformare la grammatica

$$A \to A\alpha$$
$$A \to \beta$$

con $\alpha \neq \varepsilon$ (altrimenti la produzione non avrebbe senso) e $\beta \neq A\gamma$.
L'intuizione per la trasformazione è la seguente, ovvero "ruotare" l'albero di derivazione, in modo da derivare prima i caratteri a sinistra:



Per farlo dobbiamo sostituire quelle produzioni con:

$$A \to \beta A'$$
$$A' \to \alpha A' \mid \varepsilon$$

Dove A' è un nuovo non-terminale.
Otteniamo dunque una ricorsione a destra per la seconda produzione, che non è un problema.

Quando abbiamo un non-terminale con alcune produzioni ricorsive a sinistra e alcune produzioni che non lo sono, cioè,

$$A \to A\alpha_1 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \ldots \mid \beta_k$$

dove $\alpha_j \neq \varepsilon$ e i β_i non iniziano con A. Osserviamo che ciò genera tutte le sequenze che iniziano con uno dei β_i e continuano con qualsiasi numero (compreso 0) di α_j. In altre parole, la grammatica è equivalente all'espressione regolare

$$(\beta_1 \mid \ldots \mid \beta_k)(\alpha_1 \mid \ldots \mid \alpha_n)^*$$

Applicando lo stesso ragionamento di prima, possiamo ottenere la seguente grammatica:

$$\begin{aligned} A &\rightarrow \beta_1 A' \mid \ldots \mid \beta_k A' \\ A' &\rightarrow \alpha_1 A' \mid \ldots \mid \alpha_n A' \mid \epsilon \end{aligned}$$

Nota che, poiché i β_i non iniziano con A, non c'è ricorsione a sinistra diretta nelle prime k produzioni. Dato che A' è un nuovo non-terminale, gli α_j non possono iniziare con questo, quindi anche le ultime n produzioni non possono avere ricorsione a sinistra diretta. Tuttavia, potrebbe ancora esserci ricorsione a sinistra indiretta se alcuni degli α_j possono essere derivati in ε o se i β_i possono derivare qualcosa che inizia con A. Esamineremo brevemente la ricorsione a sinistra indiretta di seguito.

Sebbene abbiamo eliminato la ricorsione a sinistra diretta, abbiamo anche modificato gli alberi di sintassi che vengono costruiti dalle stringhe analizzate. Pertanto, dopo l'analisi, l'albero di sintassi deve essere ristrutturato per ottenere la struttura descritta dalla grammatica originale.

Come esempio di rimozione della ricorsione a sinistra, prendiamo la grammatica delle espressioni non ambigua vista prima. Questa ha ricorsione a sinistra sia in $\exp$ che in $\exp 2$, quindi applichiamo la trasformazione a entrambe per ottenere la seguente grammatica

$$\begin{aligned} Exp &\rightarrow Exp2\ Exp_* \\ Exp_* &\rightarrow +\ Exp2\ Exp_* \\ Exp_* &\rightarrow -\ Exp2\ Exp_* \\ Exp_* &\rightarrow \\ Exp2 &\rightarrow Exp3\ Exp2_* \\ Exp2_* &\rightarrow *\ Exp3\ Exp2_* \\ Exp2_* &\rightarrow /\ Exp3\ Exp2_* \\ Exp2_* &\rightarrow \\ Exp3 &\rightarrow \mathbf{num} \\ Exp3 &\rightarrow (\ Exp\ ) \end{aligned}$$

La grammatica risultante è ora $\mathrm{LL}(1)$, il che può essere verificato generando una tabella $\mathrm{LL}(1)$ per essa.

La trasformazione vista ora è applicabile solo nel caso semplice in cui non ci sia ricorsione a sinistra indiretta. La ricorsione a sinistra indiretta può presentare diverse forme:

1. Ci sono produzioni mutuamente ricorsive a sinistra

$$\begin{aligned} N_1 &\rightarrow N_2\alpha_1 \\ N_2 &\rightarrow N_3\alpha_2 \\ &\vdots \\ N_{k-1} &\rightarrow N_k\alpha_{k-1} \\ N_k &\rightarrow N_1\alpha_k \end{aligned}$$

2. C'è una produzione $N \rightarrow \alpha\, N\, \beta$ dove α è annullabile.

oppure qualsiasi combinazione delle due. Più precisamente, una grammatica è (direttamente o indirettamente) ricorsiva a sinistra se c'è una sequenza di derivazione non vuota $A \Rightarrow^* A\alpha$, cioè se un non-terminale deriva una sequenza di simboli grammaticali che inizia con lo stesso non-terminale. Se c'è ricorsione a sinistra indiretta, dobbiamo prima riscrivere la grammatica per rendere la ricorsione a sinistra diretta e poi utilizzare la trasformazione sopra descritta.

Riscrivere una grammatica per trasformare la ricorsione a sinistra indiretta in ricorsione a sinistra diretta può essere fatto in modo sistematico, ma il processo è un po' complicato. Non entreremo nei dettagli qui, poiché nella pratica la maggior parte dei casi di ricorsione a sinistra sono ricorsione a sinistra diretta.

Ecco un esempio:

Take

$$\begin{aligned} A &\rightarrow Ba \mid b \\ B &\rightarrow Bc \mid Ad \mid b \end{aligned}$$

Left recursion shows in two derivation steps:

$$A \Rightarrow Ba \Rightarrow Ada$$

To decrease the number of steps of the derivation, replace the production $B \rightarrow Ad$ by

$$B \rightarrow Bad \mid bd$$

Get

$$\begin{aligned} A &\rightarrow Ba \mid b \\ B &\rightarrow Bc \mid Bad \mid bd \mid b \end{aligned}$$

Eliminate immediate left recursion from B and get the grammar

$$\begin{aligned} A &\rightarrow Ba \mid b \\ B &\rightarrow bdB' \mid bB' \\ B' &\rightarrow cB' \mid adB' \mid \epsilon \end{aligned}$$

Ci chiediamo se eliminare la ricorsione a sinistra ci porti sempre ad ottenere una grammatica $\mathrm{LL}(1)$. La risposta è no, infatti vediamo dal seguente esempio che otteniamo una grammatica non-$\mathrm{LL}(1)$ anche dopo aver eliminato la ricorsione a sinistra.

Eliminate left recursion from the **ambiguous** grammar

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

Get the grammar

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

Is it LL(1)?

Not really: the entry $[E', +]$ of the parsing table contains two productions:

- $E' \rightarrow +EE'$
- $E' \rightarrow \epsilon$

In fact:

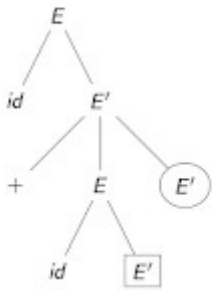
	elements in first	elements in follow
E	$($ id	$\$$ $+$ $*$ $)$
E'	$+$ $*$ ϵ	$\$$ $+$ $*$ $)$

Lesson learnt: Elimination of left recursion does not guarantee to get an LL(1) grammar

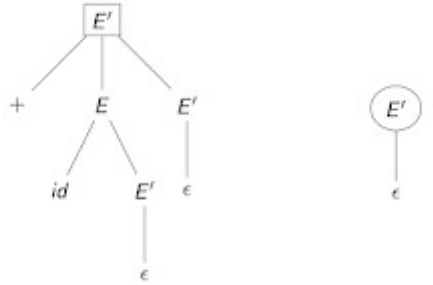
Ci chiediamo poi se eliminare la ricorsione a sinistra porti sempre anche all'eliminazione dell'ambiguità della grammatica. La risposta è anche in questo caso no, infatti prendendo in considerazione sempre la grammatica di prima:

$$\begin{aligned} E &\rightarrow (E)E' \mid idE' \\ E' &\rightarrow +EE' \mid *EE' \mid \epsilon \end{aligned}$$

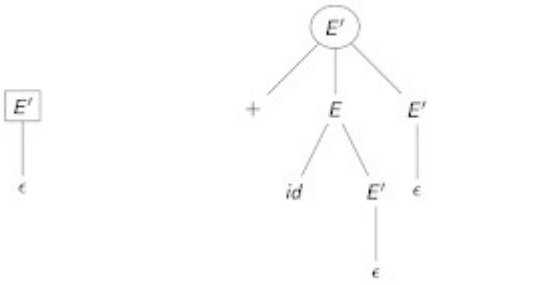
Take the partial derivation tree



Get a derivation of $id + id + id$ by completing the tree with this instantiation of the missing sub-trees



Get a different derivation of $id + id + id$ by completing the tree with this instantiation



Se due produzioni per lo stesso non-terminale iniziano con la stessa sequenza di simboli, è ovvio che hanno insiemi $FIRST$ sovrapposti. Come esempio, nella grammatica

$$
\begin{aligned}
Stat &\rightarrow \textbf{id} := Exp \\
Stat &\rightarrow Stat \ ; \ Stat \\
Stat &\rightarrow \textbf{if} \ Exp \ \textbf{then} \ Stat \ \textbf{else} \ Stat \\
Stat &\rightarrow \textbf{if} \ Exp \ \textbf{then} \ Stat
\end{aligned}
$$

le due produzioni per "if" hanno prefissi sovrapposti.

Se una grammatica G può essere **left factorized**, allora G non è $\mathrm{LL}(1)$.
In particolare, una grammatica può essere left factorized (sottoposta a left factorization) se multiple produzioni per lo stesso non-terminale hanno lo stesso prefisso.

Riscriviamo questo in modo che le produzioni sovrapposte vengano combinate in una singola produzione che contiene il prefisso comune delle produzioni e utilizza un nuovo non-terminale ausiliario per i diversi suffissi:

$$
\begin{aligned}
Stat &\rightarrow \textbf{id} := Exp \\
Stat &\rightarrow \texttt{if} \ Exp \ \texttt{then} \ Stat \ Elsepart \\
\\
Elsepart &\rightarrow \texttt{else} \ Stat \\
Elsepart &\rightarrow
\end{aligned}
$$

In questa grammatica, possiamo scegliere univocamente una delle produzioni per “Stat” in base a un token di input.

Dunque in generale, lo scopo è ritardare il più possibile la scelta tra produzioni con lo stesso prefisso, dunque se abbiamo le produzioni

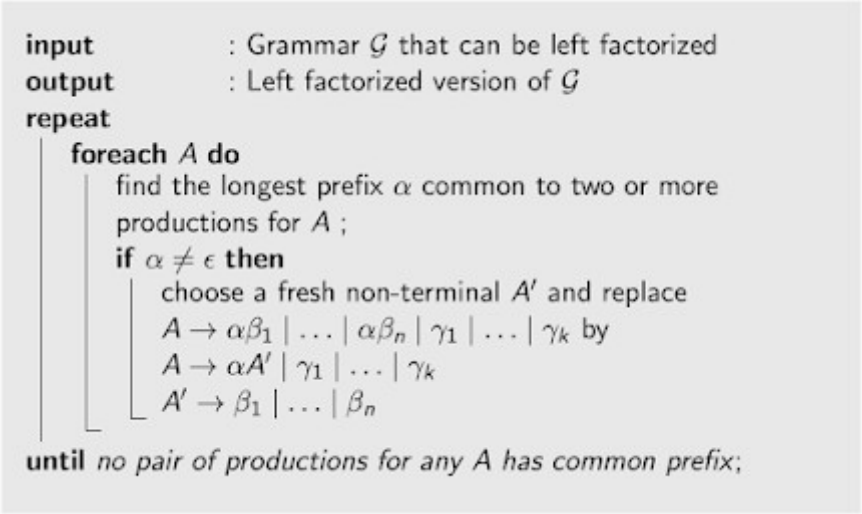
$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$$

le sostituiamo con

$$
\begin{aligned}
A &\rightarrow \alpha A' \\
A' &\rightarrow \beta_1 \mid \beta_2
\end{aligned}
$$

dove A' è un nuovo non-terminale.

L’algoritmo per eseguire la left factorization è il seguente:



Per la maggior parte delle grammatiche, combinare produzioni con prefissi comuni risolve il problema. Tuttavia, nell’esempio che abbiamo visto prima, la grammatica non è ancora $\mathrm{LL}(1)$: non possiamo scegliere in modo univoco una produzione per il non-terminale ausiliario $Elsepart$, poiché *else* è sia in $FOLLOW(Elsepart)$ che nell'insieme $FIRST$ della prima produzione per $Elsepart$. Questo non dovrebbe sorprenderci, poiché, in definitiva, la grammatica è ambigua e le grammatiche ambigue non possono essere $\mathrm{LL}(1)$. La grammatica equivalente non ambigua, ovvero

$$
\begin{array}{lcl}
Stat & \to & Stat2\ ;\ Stat \\
Stat & \to & Stat2 \\
Stat2 & \to & Matched \\
Stat2 & \to & Unmatched \\
Matched & \to & \texttt{if}\ Exp\ \texttt{then}\ Matched\ \texttt{else}\ Matched \\
Matched & \to & \textbf{id}\ :=Exp \\
Unmatched & \to & \texttt{if}\ Exp\ \texttt{then}\ Matched\ \texttt{else}\ Unmatched \\
Unmatched & \to & \texttt{if}\ Exp\ \texttt{then}\ Stat2
\end{array}
$$

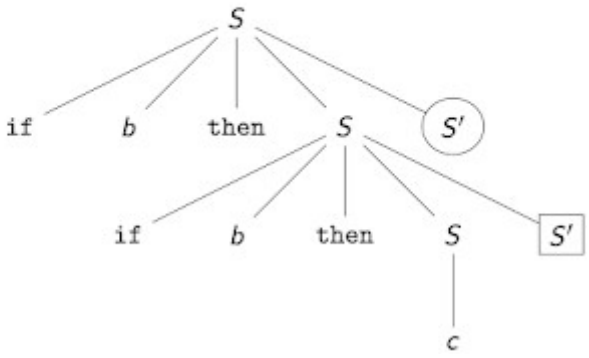
non può essere facilmente riscritta in una forma adatta per $\mathrm{LL}(1)$, quindi nella pratica viene comunque utilizzata la grammatica che abbiamo appena visto e il conflitto viene gestito scegliendo la produzione non vuota per $Elsepart$ ogni volta che si incontra il simbolo *else*, poiché ciò fornisce il comportamento desiderato di far corrispondere un *else* al *if* più vicino. Possiamo ottenere questo rimuovendo la produzione vuota dall'ingresso della tabella per $Elsepart$/*else*, in modo che rimanga solo la produzione non vuota $Elsepart \to else\ \ Stat$.

Tuttavia, pochissimi conflitti $\mathrm{LL}(1)$ causati dall'ambiguità possono essere rimossi in questo modo senza cambiare anche il linguaggio riconosciuto dalla grammatica. Ad esempio, l'ambiguità della precedenza degli operatori non può essere risolta eliminando le voci in conflitto nella tabella $\mathrm{LL}(1)$.

Dunque, come per il caso dell'eliminazione delle left-recursion, eseguire la left-factorization non garantisce né di ottenere una grammatica $\mathrm{LL}(1)$, né di ottenere una grammatica non ambigua, infatti per la grammatica possiamo costruire i seguenti alberi di derivazione per la stessa frase:

$$S \rightarrow \texttt{if } b \texttt{ then } S\, S' \mid c$$
$$S' \rightarrow \texttt{else } S \mid \epsilon$$

Take the partial derivation tree



Get a derivation of "if b then if b then c else c" by completing the tree with this instantiation of the missing sub-trees



Get a different derivation of "if b then if b then c else c" by completing the tree with this instantiation



Come già accennato precedentemente, una strategia comune per evitare l'ambiguità dovuta al dangling else è imporre il cosiddetto legame più interno: ogni else deve corrispondere al then più vicino non abbinato.
Il legame più interno può essere imposto definendo una grammatica che consente solo coppie di then-else abbinati tra le occorrenze di then ed else:

$$S \rightarrow M \mid U$$
$$M \rightarrow \texttt{if } b \texttt{ then } M \texttt{ else } M \mid c$$
$$U \rightarrow \texttt{if } b \texttt{ then } S \mid \texttt{if } b \texttt{ then } M \texttt{ else } U$$

Dunque riassumendo:

- Nessuna grammatica left-recursive è $\mathrm{LL}(1)$.
- Nessuna grammatica che può essere left-factorized è $\mathrm{LL}(1)$.
- Nessuna grammatica ambigua è $\mathrm{LL}(1)$.

E quindi per eseguire la costruzione della tabella di parser LL(1) bisogna:

1. Eliminare l'ambiguità.
2. Eliminare la ricorsione a sinistra.
3. Eseguire la fattorizzazione a sinistra quando necessario.
4. Aggiungere una produzione di start aggiuntiva $S' \rightarrow S\,\$$ alla grammatica.

5. Calcolare $FIRST$ per ogni produzione e $FOLLOW$ per ogni non-terminale.
6. Per il non-terminale N e il simbolo di input c, scegliere la produzione $N \rightarrow \alpha$ quando:
    - $c \in FIRST(\alpha)$, oppure
    - $\varepsilon \in FIRST(\alpha)$ e $c \in FOLLOW(N)$

Lemma:
La grammatica G è $\mathrm{LL}(1)$ se e solo se, se G ha produzioni $A \rightarrow \alpha \mid \beta$, allora

- $FIRST(\alpha) \cap FIRST(\beta) = \varnothing$
- Se $\varepsilon \in FIRST(\alpha)$ allora $FIRST(\beta) \cap FOLLOW(A) = \varnothing$ e, viceversa, se $\varepsilon \in FIRST(\beta)$ allora $FIRST(\alpha) \cap FOLLOW(A) = \varnothing$

SLR parsing

Un problema con il parsing LL(1) è che la maggior parte delle grammatiche richiede una riscrittura estensiva per portarle in una forma che consenta una scelta univoca della produzione. Anche se questa riscrittura può essere in larga parte automatizzata, ci sono comunque un gran numero di grammatiche che non possono essere trasformate automaticamente in grammatiche LL(1).

Esistono diverse tecniche di parsing bottom-up disponibili (SLR(1), LALR(1), LR(1), ...). Queste tecniche condividono molte caratteristiche comuni, tra cui:

- L'aggiunta della produzione $S' \rightarrow S$ dove S è il simbolo iniziale della grammatica e S' è un nuovo non-terminale.
- Il fatto che il controller dell'algoritmo di parsing è un automa a stati finiti.

A seconda della tecnica, gli automi caratteristici codificano informazioni più dettagliate o più approssimate. Più dettagliate sono le informazioni, più grandi sono gli automi caratteristici, più potente è la tecnica di parsing e più ampia è l'insieme di grammatiche analizzate. Qui consideriamo la più semplice tra queste tecniche, **SLR(1)**, che è guidata da automi caratteristici chiamati **LR(0)-automi**.

Gli analizzatori LR sono una classe di metodi bottom-up per il parsing che accettano una classe di grammatiche molto più ampia rispetto al parsing LL(1), anche se ancora non tutte le grammatiche. Il vantaggio principale del parsing LR è che è necessaria una riscrittura minore per ottenere una grammatica in una forma accettabile per il parsing LR rispetto a quanto necessario nel caso del parsing LL(1). Inoltre, come vedremo successivamente, gli analizzatori LR consentono la dichiarazione esterna delle precedenze degli operatori per risolvere l'ambiguità, anziché richiedere che le grammatiche stesse siano non ambigue.

Esamineremo una forma semplice di parsing LR chiamata parsing SLR. Le lettere "SLR" significano "Simple", "Left" e "Right". "Left" indica che l'input viene letto da sinistra a destra e "Right" indica che viene costruita una derivazione rightmost.

Come già detto precedentemente, il bottom-up parsing consiste nel ricostruire, in ordine inverso, la rightmost derivation di una parola dalla frontiera dell'albero fino alla sua radice.

Gli analizzatori LR sono parser bottom-up, sono table-driven e utilizzano due tipi di "azioni" coinvolgenti lo stream di input e uno stack:

- **shift**: Un simbolo viene letto dall'input e spinto nello stack
- **reduce**: Gli N elementi in cima allo stack contengono simboli identici agli N simboli sul lato destro di una produzione specificata. Questi N simboli vengono sostituiti dall'azione di riduzione con il non-terminale sul lato sinistro della produzione specificata. Contrariamente ai parser LL, lo stack contiene i simboli sul lato destro in modo che l'ultimo simbolo sul lato destro sia in cima allo stack.

Se il testo di input non è conforme alla grammatica, ad un certo punto durante il parsing non ci saranno azioni applicabili e il parser si interromperà con un messaggio di errore. In caso contrario, il parser leggerà tutto l'input e lascerà un singolo elemento (il simbolo iniziale della grammatica) nello stack.

Gli analizzatori LR sono anche chiamati *parser shift-reduce*. Come per LL(1), il nostro obiettivo è far dipendere la scelta dell'azione solo dal prossimo simbolo di input e dal simbolo in cima allo stack. Per raggiungere questo obiettivo, costruiamo un DFA (Automa a Stati Finiti Deterministico). Concettualmente, questo DFA legge il contenuto dello stack, partendo dal basso. Se il DFA si trova in uno stato accettante quando raggiunge la cima dello stack, l'azione corretta è la riduzione tramite una produzione determinata dal prossimo simbolo di input e da un marcatore sullo stato accettante del DFA. Se il DFA non si trova in uno stato accettante quando raggiunge la cima dello stack, l'azione corretta è uno shift su uno dei simboli per i quali c'è un arco uscente dallo stato del DFA. Pertanto, ad ogni passo, il DFA legge lo stack dal basso verso l'alto e l'azione è determinata guardando lo stato del DFA e il prossimo simbolo di input.

Consentire al DFA di leggere l'intero stack ad ogni azione non è molto efficiente. Invece, memorizziamo con ogni elemento dello stack lo stato del DFA quando legge quell'elemento. In questo modo, non dobbiamo partire dal fondo dello stack, ma possiamo partire dalla cima dello stack attuale facendo iniziare il DFA nello stato memorizzato anziché nello stato iniziale.

Quando il DFA ha indicato uno shift, il percorso da seguire è semplice: otteniamo lo stato dalla cima dello stack e seguiamo la transizione contrassegnata con il prossimo simbolo di input per trovare il prossimo stato del DFA, e memorizziamo sia il simbolo che il nuovo stato nello stack.

Se il DFA ha indicato una riduzione, rimuoviamo i simboli corrispondenti al lato destro della produzione dallo stack (si fanno tanti pop dallo stack quanto è la lunghezza del body della produzione di cui stiamo facendo il reduce). Poi leggiamo lo stato del DFA dalla nuova cima dello stack. Questo stato del DFA dovrebbe avere una transizione sul non-terminale che è il lato sinistro della produzione, quindi memorizziamo sia il non-terminale che lo stato alla fine della transizione nello stack.

Con queste ottimizzazioni, il DFA deve ispezionare un terminale o un non-terminale solo una volta, nel momento in cui viene inserito nello stack. In tutti gli altri momenti, deve semplicemente leggere lo stato del DFA memorizzato sulla cima dello stack. In realtà, non è necessario memorizzare il simbolo di input corrente o il non-terminale una volta che abbiamo effettuato una transizione su di essi, poiché le future transizioni non dipenderanno da essi: lo stato memorizzato del DFA è sufficiente. Quindi possiamo far sì che ogni elemento dello stack contenga solo lo stato del DFA anziché sia il simbolo che lo stato. Utilizziamo comunque il DFA per determinare la prossima azione, ma ora è sufficiente guardare solo lo stato corrente (memorizzato sulla cima dello stack) e il prossimo simbolo di input (in caso di azione di shift) o non-terminale (in caso di azione di riduzione).

Rappresentiamo il DFA come una tabella, dove incrociamo uno stato del DFA con un simbolo (terminale o non-terminale) e troviamo una delle seguenti azioni:

- **shift n**: leggi il prossimo simbolo di input e spingi lo stato n nello stack.
- **go n:** spingi lo stato n nello stack.
- **reduce p**: riduci con la produzione numerata p.
- **accept**: il parsing è stato completato con successo.
- **error**: è stato rilevato un errore di sintassi.

La tabella di parsing ha dunque una riga per ogni stato dell'automa LR(0) e una colonna per ogni simbolo in $V \cup \{\$\}$.
Il simbolo del dollaro è presente nella tabella di parsing perché può comparire nei FOLLOW dei simboli non-terminali. La sua presenza nella colonna corrispondente è giustificata anche dal fatto che dobbiamo determinare se l'analisi è terminata con successo o meno.

È importante notare che lo stato corrente è sempre situato in cima allo stack. Le azioni di *shift* e *reduce* vengono utilizzate quando uno stato viene incrociato con un simbolo terminale. Le azioni di *go* vengono utilizzate quando uno stato viene incrociato con un non-terminale. Un'azione di *go* può verificarsi solo immediatamente dopo una riduzione, ma non possiamo combinare le azioni di *go* con le azioni di *reduce* nella tabella, poiché lo stato di destinazione di un'azione di *go* dipende dallo stato in cima allo stack dopo che il lato destro della produzione ridotta è stato rimosso: una *reduce* nello stato corrente è immediatamente seguita da un *go* nello stato che viene trovato quando lo stack è svuotato del lato destro della produzione ridotta.

Un esempio di tabella SLR è illustrata nella figura

	a	b	c	$	T	R
0	s3	s4	r3	r3	g1	g2
1				a		
2			r1	r1		
3	s3	s4	r3	r3	g5	g2
4		s4	r3	r3		g6
5			s7			
6			r4	r4		
7			r2	r2		

derivata dalla grammatica

$$
\begin{aligned}
T &\rightarrow R \\
T &\rightarrow \mathtt{a}T\mathtt{c} \\
R &\rightarrow \\
R &\rightarrow \mathtt{b}R
\end{aligned}
$$

attraverso il metodo mostrato che vedremo nella prossima sezione. Le azioni sono state abbreviate alle loro prime lettere, mentre l'errore è rappresentato come una voce vuota.

Si può notare che la sezione a destra della tabella, quella etichettata con i non-terminali, è comunemente definita come la "sezione goto della tabella".
In breve, un'indicazione chiara è che se l'arco è terminale, si possono avere sia shift che reduce (con reduce applicato solo se lo stato è terminale ed è uno stato diverso da quello che dovrebbe leggere il simbolo $); se invece l'arco è non-terminale, si utilizza la funzione goto. L'elemento "accept" viene inserito solo quando si riconosce l'intera parola, quindi negli stati in cui è atteso il simbolo $.

Osservando la parte sinistra della tabella, si trovano i vari terminali (incluso $) e, nelle rispettive voci, si trovano le operazioni di shift, reduce e celle contrassegnate come "error". Nella sezione a destra, invece, sono elencati i non-terminali con le relative operazioni di goto.

L'algoritmo per l'analisi di una stringa utilizzando la tabella è il seguente:

- **Input**: stringa w; tabella di parsing *shift-reduce* M per $G=(V,T,S,P)$
- **Output**: rightmost derivation di w in ordine inverso se $w \in L(G)$, altrimenti $error()$
- **Inizializzazione**: P_0 nello stack degli stati $stSt$; niente nella pila dei simboli $symSt$; $w\,\$$ nel buffer di input

```
let b be the first symbol in the input buffer;
while true do
    let S be the top of stSt;
    if M[S, b] = "Shift T" then
        push b onto symSt;
        push T onto stSt;
        let b be the next symbol in the input buffer;
    else if M[S, b] = "Reduce A → β" then
        pop |β| symbols off symSt; push A onto symSt;
        pop |β| symbols off stSt;
        let temp be the top of stSt;
        push T onto stSt, where T is such that M[temp, A] = "Goto T";
        output "A → β";
    else if M[S, b] = "Accept" then return;
    else error();
```

L'algoritmo mostrato determina semplicemente se una stringa appartiene al linguaggio generato dalla grammatica. Tuttavia, può essere esteso facilmente per costruire un albero di sintassi: ogni elemento dello stack contiene (oltre al numero di stato) una porzione di un albero di sintassi. Durante un'azione di *reduce*, si costruisce un nuovo albero di sintassi parziale utilizzando il non-terminale della produzione ridotta come radice e gli alberi di sintassi memorizzati negli elementi dello stack rimossi come figli. Il nuovo albero e il nuovo stato vengono quindi inseriti nello stack come un singolo elemento.

La seguente figura

input	stack	action
aabbbcc$	0	s3
abbbcc$	03	s3
bbbcc$	033	s4
bbcc$	0334	s4
bcc$	03344	s4
cc$	033444	r3 ($R \to$) ; g6
cc$	0334446	r4 ($R \to$ bR) ; g6
cc$	033446	r4 ($R \to$ bR) ; g6
cc$	03346	r4 ($R \to$ bR) ; g2
cc$	0332	r1 ($T \to R$) ; g5
cc$	0335	s7
c$	03357	r2 ($T \to$ aTc) ; g5
c$	035	s7
$	0357	r2 ($T \to$ aTc) ; g1
$	01	accept

mostra un esempio di analisi della stringa "aabbbcc" utilizzando la tabella precedente. Le sequenze di numeri nella colonna "stack" rappresentano i contenuti dello stack, con il fondo dello stack mostrato a sinistra e la cima dello stack a destra. Ad ogni passaggio, si osserva il prossimo simbolo di input (nella colonna "input" alla sinistra della stringa) e lo stato in cima allo stack (alla fine della sequenza nella colonna "stack"). Si cerca la coppia di simbolo di input e stato nella tabella e si trova un'azione, che viene mostrata nella colonna "action". Quando

l'azione mostrata è una riduzione, viene anche mostrata la riduzione utilizzata (tra parentesi) e, dopo un punto e virgola, anche l'azione di "go" eseguita dopo la riduzione.

Vediamo un altro esempio di parsing SLR:

Parse $w = abbcde$

1. $S \rightarrow aABe$
2. $A \rightarrow Abc$
3. $A \rightarrow b$
4. $B \rightarrow d$

	a	*b*	*c*	*d*	*e*	$	*S*	*A*	*B*
0	s2						1		
1						Acc			
2		s4						3	
3		s6		s7					5
4		r3		r3					
5					s8				
6			s9						
7					r4				
8						r1			
9		r2		r2					

abbcde$	*stSt*	0				
	symSt					
*a*bbcde$	*stSt*	0	2			
	symSt	a				
*ab*bcde$	*stSt*	0	2	4		
	symSt	a	b			
*ab*bcde$	*stSt*	0	2	~~4~~	3	
	symSt	a	~~b~~	A		output $A \rightarrow b$
*abb*cde$	*stSt*	0	2	3	6	
	symSt	a	A	b		
*abbc*de$	*stSt*	0	2	3	6	9
	symSt	a	A	b	c	

abbcde$	stSt	0	2	3	6	9	3	
	symSt	a	A	b	c	A		output $A \to Abc$
abbcde$	stSt	0	2	3	7			
	symSt	a	A	d				
abbcde$	stSt	0	2	3	7	5		
	symSt	a	A	d	B			output $B \to d$
abbcde$	stSt	0	2	3	5	8		
	symSt	a	A	B	e			
abbcde$	stSt	0	2	3	5	8	1	
	symSt	a	A	B	e	S		output $S \to aABe$
abbcde$	stSt	0	1					
	symSt	S			Accept			

L'output, preso nell'ordine inverso, è la sequenza di produzioni da applicare per ottenere la derivazione più a destra di *abbcde*:

Output	Derivation
$S \to aABe$	$S \Rightarrow aABe$
$B \to d$	$\Rightarrow aAde$
$A \to Abc$	$\Rightarrow aAbcde$
$A \to b$	$\Rightarrow abbcde$

Costruzione delle tabelle di parsing SLR

Introduciamo prima gli **automi LR(0)**.
Ad ogni passaggio di parsing, ci spostiamo da uno stato a un altro.
Per ogni produzione $A \to Y_1 \dots Y_n$ (dove $Y_1 \dots Y_n$ possono essere sia terminali che non-terminali), ci sono percorsi nell'automa LR(0) della forma

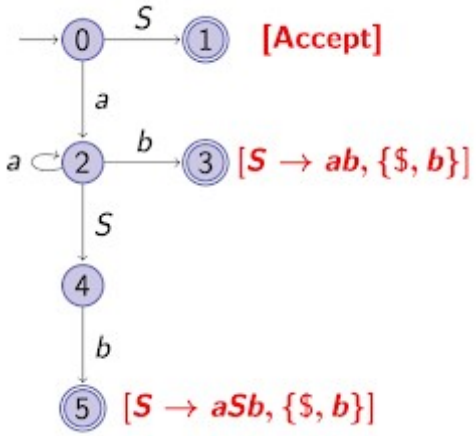


L'intuizione è che se, partendo dallo stato 1, i passaggi di parsing ci portano fino allo stato 2 e se il prossimo simbolo in input è "appropriato", allora dallo stato 1 abbiamo trovato una derivazione da A e ci spostiamo allo stato 3, proprio come se avessimo riconosciuto direttamente A quando eravamo nello stato 1. Questo è ciò che viene chiamato un **passaggio di riduzione**.

In altre parole: Nell'automa, per ogni produzione della grammatica, c'è un sottoinsieme di stati organizzato come segue: c'è uno stato specifico, denominato stato 3 nell'immagine sopra, con un arco etichettato come il "driver" della produzione e un percorso il cui spelling corrisponde esattamente alla produzione del "driver". Lo stato raggiunto attraverso questo percorso è uno stato finale, indicandoci così di eseguire una riduzione per quel percorso. In altre parole, "dato che, partendo da 1, sono arrivato a 2 seguendo il corpo della produzione, eseguo una riduzione per questa produzione e procedo allo stesso modo se, virtualmente, nell'input vedessi A. Mi sposto quindi a 3".

Dunque gli stati finali, nel contesto del parsing bottom-up, indicano che, quando raggiungiamo questi stati, abbiamo individuato l'ultimo stato che corrisponde a una delle produzioni della grammatica (ovvero al corpo di una delle produzioni della grammatica).

Ecco un esempio di automa LR(0) per la grammatica $S \to aSb \mid ab$ (a cui aggiungiamo la produzione $S' \to S$):



I valori nelle parentesi graffe sono i FOLLOW del non-terminale driver della produzione (in questo caso S).

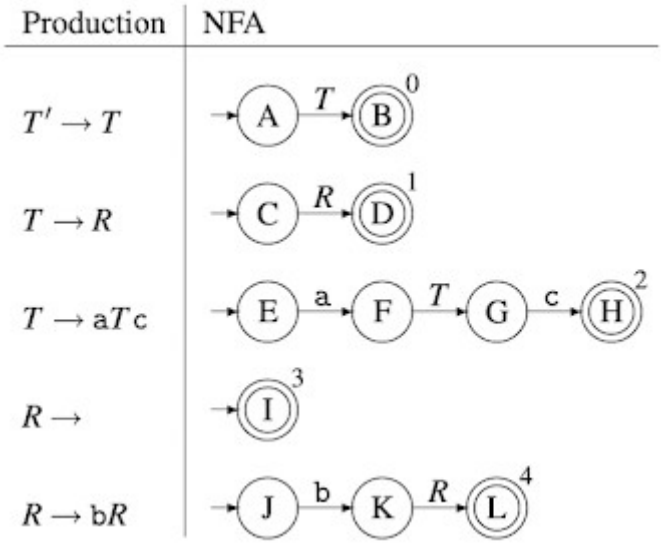
Una tabella di parsing SLR ha come nucleo un DFA. Costruire questo DFA dalla grammatica è simile alla costruzione di un DFA da un'espressione regolare, come mostrato precedentemente: prima costruiamo un NFA utilizzando tecniche simili a quelle già viste e poi convertiamo questo in un DFA usando la costruzione per sottoinsiemi. Prima di costruire l'NFA, però, estendiamo la grammatica con una nuova produzione iniziale.
Applicando questa procedura alla grammatica

$$\begin{aligned} T &\to R \\ T &\to \mathtt{a}T\mathtt{c} \\ R &\to \\ R &\to \mathtt{b}R \end{aligned}$$

otteniamo la grammatica

$$
\begin{array}{lrcl}
0: & T' & \rightarrow & T \\
1: & T & \rightarrow & R \\
2: & T & \rightarrow & \mathtt{a}T\mathtt{c} \\
3: & R & \rightarrow & \\
4: & R & \rightarrow & \mathtt{b}R
\end{array}
$$

Il passo successivo è creare un NFA per ciascuna produzione. Questo viene fatto esattamente come visto precedentemente, trattando sia i terminali che i non-terminali come simboli dell'alfabeto. Lo stato accettante di ciascun NFA è etichettato con il numero della produzione corrispondente. Il risultato è mostrato nella seguente figura

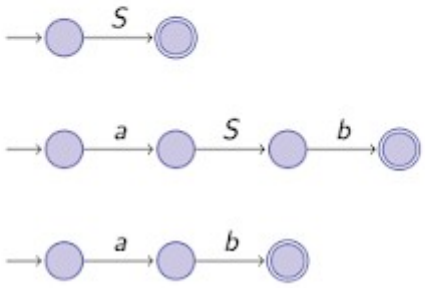


Si noti che abbiamo utilizzato la costruzione ottimizzata per ε (la produzione vuota).

Per il nostro esempio precedente dunque abbiamo:

FIRST STEP: Set up three automata

- An automaton for S' which accepts S, the body of $S' \rightarrow S$
- An automaton for S which accepts aSb, the body of $S \rightarrow aSb$
- An automaton for S which accepts ab, the body of $S \rightarrow ab$



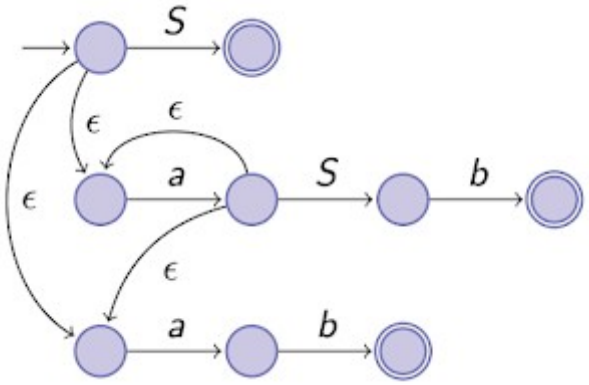
Gli NFA appena visti effettuano transizioni sia su terminali che su non-terminali. Le transizioni tramite terminali corrispondono ad azioni di shift e le transizioni sui non-terminali corrispondono

ad azioni di *go*. Un'azione di *go* avviene dopo una riduzione, mediante la quale alcuni elementi dello stack (corrispondenti al lato destro di una produzione) vengono sostituiti da un non-terminale (corrispondente al lato sinistro di quella produzione).
Tuttavia, prima di poter ridurre su un non-terminale, i simboli che formano il lato destro devono essere presenti nello stack. Quindi ci prepariamo per una successiva transizione su un non-terminale permettendo transizioni che, alla fine, lasceranno i simboli che formano il lato destro della produzione nello stack, così da poterli ridurre al non-terminale e quindi effettuare una transizione su di esso.
Quindi, ogni volta che è possibile una transizione tramite un non-terminale, permettiamo anche transizioni sulle sequenze di simboli sui lati destri delle produzioni per quel non-terminale. Otteniamo ciò aggiungendo ε-transizioni agli NFA: Ogniqualvolta c'è una transizione dallo stato s allo stato t su un non-terminale N, aggiungiamo ε-transizioni da s agli stati iniziali di tutti gli NFA per le produzioni con N sul lato sinistro. Aggiungere le ε-transizioni come frecce nella figura precedente produrrebbe un'immagine molto disordinata, quindi invece annotiamo le transizioni in una tabella separata, mostrata nella seguente figura

state	epsilon-transitions
A	C, E
C	I, J
F	C, E
K	I, J

Ma questo è solo a scopo di presentazione: le transizioni hanno lo stesso significato indipendentemente dal fatto che siano mostrate nell'immagine del DFA o nella tabella.

Tornando al nostro esempio avremmo:



Insieme a queste ε-transizioni, gli NFA nella figura precedente formano un unico NFA combinato. Questo NFA ha come stato iniziale A (ovvero lo stato iniziale dell'NFA per la produzione iniziale aggiunta per S'/T') e uno stato accettante per ciascuna produzione nella grammatica. Ora dobbiamo convertire questo NFA in un DFA utilizzando la costruzione per sottoinsiemi. Invece di mostrare il DFA risultante graficamente, costruiamo una tabella in cui le transizioni sui terminali sono mostrate come azioni di *shift* e le transizioni sui non-terminali come azioni di *go*. Questo renderà la tabella simile alla tabella SLR che abbiamo visto nello scorso

paragrafo, eccetto per il fatto che non sono ancora presenti azioni di *reduce* o *accept*. La seguente figura

DFA state	NFA states	Transitions a	b	c	T	R
0	A, C, E, I, J	s3	s4		g1	g2
1	B					
2	D					
3	F, C, E, I, J	s3	s4		g5	g2
4	K, I, J		s4			g6
5	G			s7		
6	L					
7	H					

mostra il DFA costruito dall'NFA ottenuto aggiungendo ε-transizioni agli NFA precedenti secondo la tabella delle ε-transizioni. Gli insiemi di stati NFA che formano ciascuno stato DFA sono mostrati nella seconda colonna della tabella appena vista. Avremo bisogno di questi sottoinsiemi per aggiungere azioni di *reduce* e *accept*, ma una volta fatto ciò, non ne avremo più bisogno, quindi possiamo rimuoverli dalla tabella finale.

Per aggiungere azioni di *reduce* e *accept*, prima dobbiamo calcolare gli insiemi di $FOLLOW$ per ciascun non-terminale. Per calcolare $FOLLOW$, aggiungiamo un'altra produzione di partenza extra: $T'' \rightarrow T'\$$, per gestire le condizioni di fine testo come descritto nella sezione del $FOLLOW$. Questo ci dà il seguente risultato:

$$\begin{aligned} FOLLOW(T') &= \{\$\} \\ FOLLOW(T) &= \{\mathtt{c}, \$\} \\ FOLLOW(R) &= \{\mathtt{c}, \$\} \end{aligned}$$

Aggiungiamo quindi azioni di *reduce* seguendo la seguente regola: se uno stato DFA s contiene lo stato NFA accettante per una produzione $p : N \rightarrow \alpha$, aggiungiamo l'azione di *reduce* p allo stato s su tutti i simboli in $FOLLOW(N)$. La riduzione sulla produzione 0 (la produzione extra di partenza che è stata aggiunta prima di costruire l'NFA) è indicata come *accept*.

Nella tabella vista prima, lo stato 0 contiene lo stato NFA I, che accetta la produzione 3. Pertanto, aggiungiamo l'azione $r3$ ai simboli c e $\$$ (poiché questi sono in $FOLLOW(R)$). Lo stato 1 contiene lo stato NFA B, che accetta la produzione 0. Aggiungiamo questo al simbolo $\$$ ($FOLLOW(T')$). Come notato in precedenza, questo viene indicato come *accept* (abbreviato come "*a*"). Allo stesso modo, aggiungiamo azioni di *reduce* agli stati 3, 4, 6 e 7. Il risultato è mostrato nella seguente figura

	a	b	c	$	T	R
0	s3	s4	r3	r3	g1	g2
1				a		
2			r1	r1		
3	s3	s4	r3	r3	g5	g2
4		s4	r3	r3		g6
5			s7			
6			r4	r4		
7			r2	r2		

Ecco un riassunto della costruzione SLR:

1. Aggiungi la produzione $S' \to S$, dove S è il simbolo iniziale della grammatica.
2. Crea un NFA per il lato destro di ciascuna produzione.
3. Se uno stato NFA s ha una transizione in uscita su un non-terminale N, aggiungi ε-transizioni da s agli stati iniziali degli NFA per i lati destri delle produzioni per N.
4. Converti l'NFA combinato in un DFA. Usa lo stato iniziale dell'NFA per la produzione aggiunta nel passaggio 1 come stato iniziale per l'NFA combinato.
5. Costruisci una tabella indicizzata dagli stati DFA e dai simboli della grammatica (terminali incluso $\$$ e non-terminali). Aggiungi azioni di *shift* per le transizioni sui terminali e azioni di *go* per le transizioni sui non-terminali.
6. Calcola $FOLLOW$ per ciascun non-terminale. Per questo scopo, aggiungi un'altra produzione di partenza: $S'' \to S'\$$.
7. Quando uno stato DFA contiene uno stato NFA accettante contrassegnato con il numero di produzione p, dove il non-terminale per p è N, trova i simboli in $FOLLOW(N)$ e aggiungi un'azione di *reduce* p nello stato DFA su tutti questi simboli. Se la produzione p è quella aggiunta nel passaggio 1, aggiungi invece un'azione di *accept* anziché una azione di *reduce* p.

Vediamo adesso un'altra tecnica per costruire l'automa LR(0).
Vediamo dunque come costruire l'automa LR(0) da zero, calcolando gli stati come insiemi di **LR(0)-items**.
Un **LR(0)-item** è una produzione della grammatica con un **marker** in una posizione del suo corpo, ad esempio:

$$A \to \alpha \cdot \beta$$

L'intuizione è che il parser si trova in uno stato contenente $A \to \alpha \cdot \beta$ se quello stato viene raggiunto dopo aver elaborato una porzione di stringa che deriva da α mentre la porzione di stringa che deriva da β deve ancora essere elaborata.
NOTA: L'LR(0)-item per una produzione della forma $A \to \varepsilon$ è $A \to \cdot$.

Consideriamo l'LR(0)-item $S' \to \cdot S$. Intuitivamente, questo significa che non abbiamo ancora visto nessun simbolo della parola che stiamo per analizzare, e il parsing avrà successo solo se

la parola che stiamo analizzando deriva da S (o, nel caso più generale, di qualsiasi cosa segua il marker $\cdot$). Quindi, l'item $S' \to \cdot S$ è sicuramente nello stato iniziale dell'automa caratteristico, diciamo P_0. Anche altri items dovrebbero essere presenti in P_0.

Considerando nuovamente la grammatica $S \to aSb \mid ab$, se iniziamo ad analizzare una parola e ci aspettiamo di vedere una derivazione da S, allora ci aspettiamo di vedere ab oppure una derivazione da aSb. Pertanto, gli LR(0)-items

$$S \to \cdot aSb$$
$$S \to \cdot ab$$

dovrebbero essere anch'essi presenti in P_0.

Nel nostro esempio precedente abbiamo cercato di capire intuitivamente quali items è logico inserire negli stati, ma è chiaro che serve ben poca complessità per mettere fuori gioco il nostro intuito. Abbiamo bisogno di un procedimento formale per determinare quali items andranno inseriti nei vari stati, e in questo ci viene in aiuto il concetto di chiusura di un insieme di LR(0)-items:
Consideriamo P come un insieme di LR(0)-items, e P' l'insieme delle produzioni della grammatica estesa con $S' \to S$.
La chiusura $closure(P)$ è il più piccolo insieme di items che soddisfa l'equazione seguente:

$$closure(P) = P \cup \{B \to \cdot\gamma \mid A \to \alpha \cdot B\beta \in closure(P) \wedge B \to \gamma \in P'\}$$

Calcoliamo $closure(P)$ mediante un calcolo del punto fisso:

1. Inizializziamo $closure(P)$ come P.
2. Aggiungiamo gli items $B \to \cdot\gamma$ secondo necessità, fino a raggiungere il punto fisso.

In pratica, iniziamo con P e iterativamente aggiungiamo items $B \to \cdot\gamma$ che soddisfano le condizioni specificate fino a quando non si aggiunge più nulla all'insieme, ottenendo così $closure(P)$.

Vediamo un esempio:

Take

$$\begin{aligned} E &\to E + T \mid T \\ T &\to T * F \mid F \\ F &\to (E) \mid id \end{aligned}$$

Compute $\text{closure}(\{E' \to \cdot E\})$

$\text{closure}(P) = P \cup$
$\{B \to \cdot\gamma$ such that $A \to \alpha \cdot B\beta \in \text{closure}(P)$ and $B \to \gamma \in \mathcal{P}'\}$

- Init: $\text{closure}(\{E' \to \cdot E\}) = \{E' \to \cdot E\}$
- Add $E \to \cdot E + T$ and $E \to \cdot T$
- Add $T \to \cdot T * F$ and $T \to \cdot F$
- Add $F \to \cdot(E)$ and $F \to \cdot id$

L'algoritmo per il calcolo della chiusura è il seguente:

```
function closure(P)
    tag every item in P as unmarked ;
    while there is an unmarked item I in P do
        mark I ;
        if I has the form A → α · Bβ then
            foreach B → γ ∈ P' do
                if B → ·γ ∉ P then
                    add B → ·γ as an unmarked item to P ;
    return P ;
```

Vediamo ora come costruire l'automa LR(0) con quest'altro metodo.
Per costruire l'automa, popoliamo una collezione di stati definendo la funzione di transizione.

1. Definiamo $P_0 = closure(\{S' \to S\})$
2. Supponiamo P sia uno degli stati già presenti nella collezione, non ancora visitati, e prendiamo in considerazione la chiusura del suo kernel. Per ogni Y tale che P contenga almeno un item con il marcatore alla sinistra di Y, c'è una Y-transizione da P a uno stato Q.
    - Il contenuto di Q è determinato dagli item di P. Infatti per ogni item $A \to \alpha \cdot Y \beta$ in P, Q conterrà $A \to \alpha Y \cdot \beta$. Questi items costituiscono il **kernel** di Q.
3. Una volta determinato il kernel di Q, altri items vengono aggiunti tramite la chiusura del kernel. Infatti, poiché Q contiene $A \to \alpha Y \cdot \beta$, conterrà anche tutti gli items in $closure(\{A \to \alpha Y \cdot \beta\})$.

$\{S' \to S\}$ è il kernel di P_0.
C'è però una nota da aggiungere a questo procedimento: può accadere che quando generiamo un nuovo stato Q per transizione da uno stato P ci rendiamo conto che il kernel di questo stato corrisponde al kernel di un altro stato che abbiamo già visitato, diciamo Q'; in questo caso invece che creare un nuovo stato Q, quello che facciamo è collegare (tramite una Y-transizione) P a Q'.

Vediamo quindi un esempio di costruzione di un automa LR(0) con questa tecnica.
Consideriamo la seguente grammatica:

$$\begin{aligned} S &\to aABe \\ A &\to Abc \mid b \\ B &\to d \end{aligned}$$

Lo stato 0 avrà la produzione iniziale (il kernel, in bianco) e a seguire tutte le produzioni per S (la chiusura del kernel, in grigio), dunque:

State 0:

$S' \to \cdot S$

$S \to \cdot aABe$

Gli items nello stato 0 hanno il marker alla sinistra di S e di a. Quindi, se non sono ancora presenti, dobbiamo aggiungere alla collezione altri due stati:

- $\tau(0, S) = Stato\ \ 1$
- $\tau(0, a) = 2$

Con $\tau(n, X)$ intendiamo lo stato a cui si arriva dallo stato n tramite una X-transizione, e usiamo questa sintassi per definire anche le transizioni esistenti nell'automa.

Abbiamo finito con lo stato 0, passiamo quindi allo stato 1. Calcoliamo innanzitutto il kernel, che si ottiene spostando il marker oltre al carattere che ci ha portati lì, dunque $S' \to S\cdot$.

$\tau(0, S) = 1$

$S' \to S \cdot$

Da questo stato non si presentano ulteriori transizioni possibili dato che il marker è arrivato in fondo, quindi passiamo ad analizzare un altro stato.

$\tau(0, a) = 2$

$S \to a \cdot ABe$

$A \to \cdot Abc$

$A \to \cdot b$

Da questo stato dobbiamo aggiungere, se non sono ancora presenti, $\tau(2, A)$ e $\tau(2, b)$, per verificare questa cosa bisogna controllare se non esiste già uno stato con gli stessi item. Facciamo attenzione che in $\tau(2, A)$ possiamo arrivarci tramite due produzioni differenti, quindi il kernel avrà due produzioni.
Quindi dallo stato 2 posso arrivare nei seguenti stati:

$\tau(2, A) = 3$

$S \to aA \cdot Be$

$A \to A \cdot bc$

$B \to \cdot d$

If not there yet, add $\tau(3, B)$, $\tau(3, b)$ and $\tau(3, d)$

$\tau(2, b) = 4$

$A \to b \cdot$

No transition from this one

Dallo stato 3 posso arrivare nei seguenti stati:

$\tau(3, B) = 5$

$\boxed{S \to aAB \cdot e}$ If not there yet, add $\tau(5, e)$

$\tau(3, b) = 6$

$\boxed{A \to Ab \cdot c}$ If not there yet, add $\tau(6, c)$

$\tau(3, d) = 7$

$\boxed{B \to d \cdot}$ No transition from this one

E infine dallo stato 5 e dallo stato 6 posso arrivare nei seguenti stati:

$\tau(5, e) = 8$

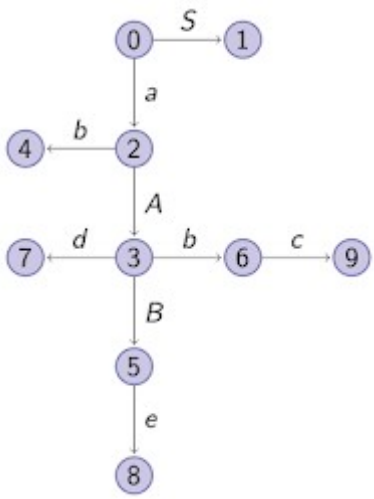
$\boxed{S \to aABe \cdot}$ No transition from this one

$\tau(6, c) = 9$

$\boxed{A \to Abc \cdot}$ No transition from this one either

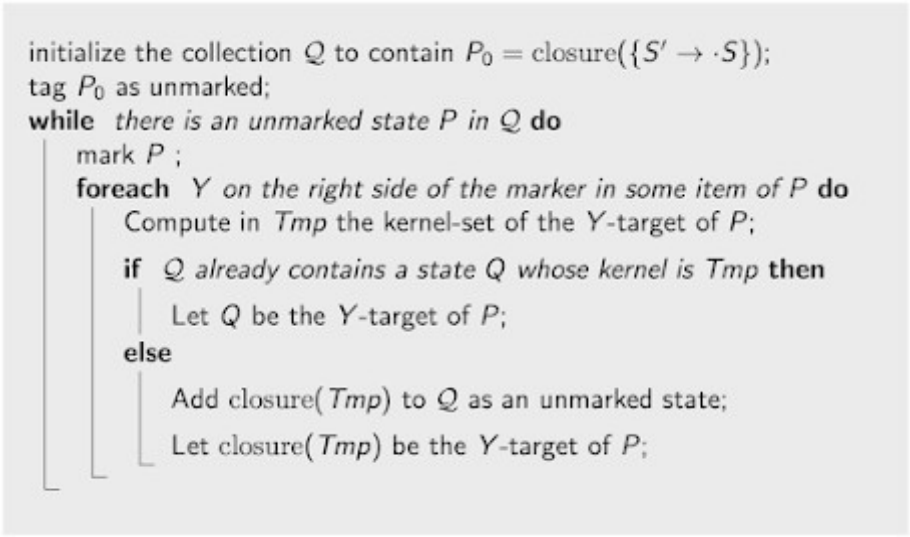
Ottenendo in fine il seguente automa LR(0):

$$
\begin{aligned}
S &\to aABe \\
A &\to Abc \mid b \\
B &\to d
\end{aligned}
$$



È buona pratica segnare gli stati contenenti item di riduzione come finali per distinguerli dagli altri.

L’algoritmo per questa tecnica è il seguente:



Questi automi sono deterministici quando li leggiamo con le frecce nel loro orientamento. Se li leggessimo nel verso opposto, questi non sarebbero deterministici.

Vediamo ora come costruire una tabella di parsing.
I movimenti dell'algoritmo *shift-reduce* sono guidati da una tabella di parsing, come abbiamo già visto.
Le mosse di *shift* dipendono dalla funzione di transizione dell'automa.
La riduzione della produzione $A \to \beta$ avviene quando il parser si trova in uno stato contenente $A \to \beta \cdot$, detto anche **item di riduzione** per $A \to \beta$.
Le riduzioni dovute a $A \to \beta \cdot$ vengono collocate in ciascun elemento di $FOLLOW(A)$.
Dunque per eseguire la tabella di parsing secondo questa tecnica dobbiamo compilare ogni voce (P, Y), dove P è uno stato dell'automa e Y può essere un terminale oppure un non-terminale, secondo le seguenti regole:

- Inserisci *Shift* Q se Y è un terminale e $\tau(P, Y) = Q$.
- Inserisci *Reduce* $A \to \beta$ se P contiene l'item di riduzione per $A \to \beta$ e $Y \in FOLLOW(A)$.
- Imposta *Accept* se P contiene l'item accettante $S' \to S \cdot$ e $Y = \$$.
- Imposta *Goto* Q se Y è un non-terminale e $\tau(P, Y) = Q$.

Se Y è un terminale o $\$$ e nessuna delle condizioni precedenti è valida, inserisce errore (o lascia vuoto). Le celle vuote che hanno per colonna un non-terminale, invece, rimangono semplicemente vuote, in quanto sono dei casi che non è possibile che si verifichino.
Se disponiamo dell'automa e delle posizioni degli item di riduzione, possediamo già tutte le informazioni presenti anche nella tabella di parsing.

Vediamo ora i **conflitti nelle tabelle di parsing SLR**.
L'automa da cui partiamo è un automa deterministico, quindi sicuramente quello che capita è che da ciascuno stato P vi è un'unica transizione etichettata Y qualunque sia il simbolo Y.
Quello che può capitare è che quando vengono aggiunte azioni di *reduce* alle tabelle di parsing SLR, potremmo aggiungerne una in un punto in cui c'è già un'azione di *shift*, oppure potremmo

aggiungere azioni di *reduce* per diverse produzioni diverse nello stesso punto. Quando uno di questi casi si verifica, non abbiamo più una scelta univoca di azione, cioè si presenta un conflitto. La prima situazione è chiamata conflitto *shift-reduce*, mentre l'altro caso è un conflitto *reduce-reduce*. Entrambi possono verificarsi nello stesso punto.

G **è SLR(1) se e solo se la sua tabella di parsing SLR(1) non presenta alcun conflitto.**
I conflitti sono spesso causati da grammatiche ambigue, ma (come nel caso dei parser LL) anche alcune grammatiche non ambigue possono generare conflitti. Se un conflitto è causato da una grammatica ambigua, di solito (ma non sempre) è possibile trovare una grammatica equivalente non ambigua. I metodi per eliminare l'ambiguità sono stati discussi precedentemente. In alternativa, le dichiarazioni di precedenza degli operatori possono essere utilizzate per chiarire una grammatica ambigua, come vedremo nella sezione successiva.

Anche le grammatiche non ambigue possono in alcuni casi generare conflitti nelle tabelle SLR. In certi casi, è ancora possibile riscrivere la grammatica per aggirare il problema, ma in alcuni casi il linguaggio semplicemente non è SLR. Riscrivere una grammatica non ambigua per eliminare i conflitti è in qualche modo un'arte. L'analisi degli stati NFA che formano lo stato DFA problematico spesso aiuta a identificare l'esatta natura del problema, che è il primo passo per risolverlo. A volte, cambiare una produzione da ricorsiva a sinistra a ricorsiva a destra può essere d'aiuto, anche se la ricorsione a sinistra in generale non è un problema per i parser SLR, come lo è per i parser LL(1).

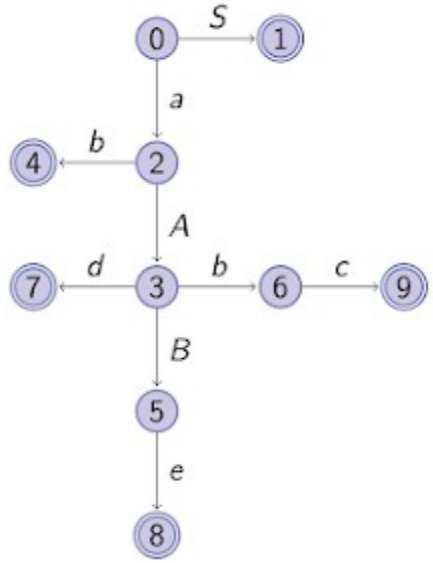
Ecco un esempio di costruzione della tabella:

Construct the SLR(1) parsing table for

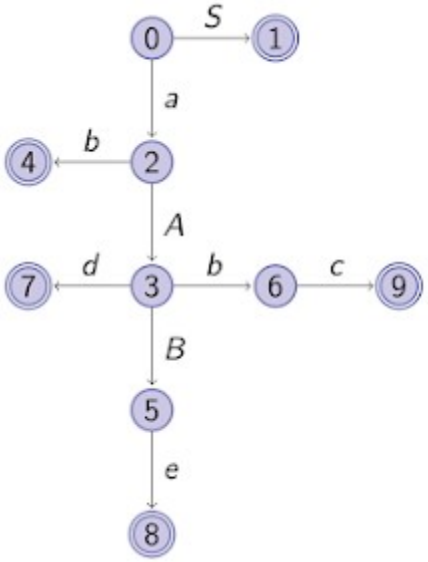
$$
\begin{aligned}
S &\rightarrow aABe \\
A &\rightarrow Abc \mid b \\
B &\rightarrow d
\end{aligned}
$$

LR(0)-automaton for

$$
\begin{aligned}
S &\rightarrow aABe \\
A &\rightarrow Abc \mid b \\
B &\rightarrow d
\end{aligned}
$$



- $4 = \{A \to b\cdot\}$
- $7 = \{B \to d\cdot\}$
- $8 = \{S \to aABe\cdot\}$
- $9 = \{A \to Abc\cdot\}$



1. $S \to aABe$
2. $A \to Abc$
3. $A \to b$
4. $B \to d$

	a	b	c	d	e	$	S	A	B
0	s2						1		
1						Acc			
2		s4						3	
3		s6		s7					5
4		r3		r3					
5					s8				
6			s9						
7					r4				
8						r1			
9		r2		r2					

Dal momento che G dà origine a una tabella di parsing senza creare alcun conflitto, possiamo concludere che G è SLR(1).

Utilizzare le regole di precedenza nelle tabelle di parsing LR

Abbiamo visto, parlando della left-factorization, che il conflitto derivante dall'ambiguità del dangling-else poteva essere risolto eliminando una delle voci nella tabella di analisi LL(1). Risolvere l'ambiguità eliminando azioni in conflitto può essere fatto anche nelle tabelle SLR. In generale, ci sono più casi in cui questo può essere fatto con successo per i parser SLR rispetto ai parser LL(1). In particolare, l'ambiguità nelle grammatiche delle espressioni come nella grammatica

$$
\begin{array}{lcl}
Exp & \to & Exp + Exp \\
Exp & \to & Exp - Exp \\
Exp & \to & Exp * Exp \\
Exp & \to & Exp / Exp \\
Exp & \to & \textbf{num} \\
Exp & \to & (Exp)
\end{array}
$$

può essere eliminata in questo modo in una tabella SLR, ma non in una tabella LL(1). La maggior parte dei generatori di parser LR consente dichiarazioni di precedenza e associatività per i token utilizzati come operatori infissi. Queste dichiarazioni vengono poi utilizzate per eliminare i conflitti nelle tabelle di analisi.

Ci sono diversi vantaggi in questo approccio:

- Le grammatiche delle espressioni ambigue sono più compatte e più facili da leggere rispetto alle grammatiche non ambigue nello stile della sezione sugli operatori di precedenza.
- Le tabelle di analisi costruite da grammatiche ambigue sono spesso più piccole delle tabelle prodotte da grammatiche non ambigue equivalenti.
- Il parsing utilizzando grammatiche ambigue è (leggermente) più veloce, in quanto richiede meno riduzioni del tipo $\exp 2 \rightarrow \exp 3$ *etc*.

Utilizzare le regole di precedenza per eliminare i conflitti è molto semplice. La grammatica precedente genererà diversi conflitti:

1) Un conflitto tra lo shift su + e la riduzione tramite la produzione $\exp \rightarrow \exp + \exp$.
2) Un conflitto tra lo shift su + e la riduzione tramite la produzione $\exp \rightarrow \exp * \exp$.
3) Un conflitto tra lo shift su $*$ e la riduzione tramite la produzione $\exp \rightarrow \exp + \exp$.
4) Un conflitto tra lo shift su $*$ e la riduzione tramite la produzione $\exp \rightarrow \exp * \exp$.

E diversi altri di natura simile coinvolgono - e /, per un totale di 16 conflitti. Affrontiamo ciascuno dei quattro conflitti sopra elencati e vediamo come le regole di precedenza possono essere utilizzate per eliminarli. Utilizzeremo le regole che + e $*$ sono entrambi associativi a sinistra e che $*$ lega più strettamente di +.

1) Questo conflitto sorge dalle espressioni come $a+b+c$. Dopo aver letto $a+b$, il prossimo simbolo di input è un +. Ora possiamo scegliere di ridurre $a+b$, raggruppando intorno alla prima addizione prima della seconda, oppure fare lo shift sul più, che successivamente porterà alla riduzione di $b+c$ e quindi al raggruppamento intorno alla seconda addizione prima della prima. Poiché le regole indicano che + è associativo a sinistra, preferiamo la prima di queste opzioni e, quindi, eliminiamo l'azione di shift dalla tabella e manteniamo l'azione di riduzione.
2) Le espressioni problematiche qui hanno la forma $a * b+c$. Poiché le regole indicano che la moltiplicazione ha una precedenza maggiore rispetto all'addizione, ancora una volta preferiamo la riduzione rispetto allo shift.
3) Nelle espressioni della forma $a+b * c$, le regole indicano nuovamente che la moltiplicazione ha una precedenza maggiore, quindi eseguiamo uno shift per evitare il raggruppamento intorno all'operatore + e, di conseguenza, eliminiamo l'azione di riduzione dalla tabella.
4) Questo caso è identico al caso 1, dove un operatore che, secondo le regole, è associativo a sinistra entra in conflitto con se stesso. Quindi, come nel caso 1, lo gestiamo eliminando lo shift.

In generale, l'eliminazione dei conflitti tramite dichiarazioni di precedenza degli operatori può essere riassunta nelle seguenti regole:

a) Se il conflitto è tra due operatori di diverse priorità, eliminare l'azione con l'operatore a più bassa priorità a favore dell'azione con l'operatore a più alta priorità. L'operatore associato a un'azione di riduzione è l'operatore utilizzato nella produzione che viene ridotta.
b) Se il conflitto è tra operatori della stessa priorità, viene utilizzata l'associatività (che deve essere la stessa, come già indicato precedentemente) degli operatori: se gli operatori sono associativi a sinistra, viene eliminata l'azione di shift e viene mantenuta l'azione di riduzione. Se gli operatori sono associativi a destra, viene eliminata l'azione di riduzione e viene mantenuta l'azione di shift. Se gli operatori non sono associativi, entrambe le azioni vengono eliminate.
c) Se ci sono diversi operatori con precedenza dichiarata nella produzione che viene utilizzata in un'azione di riduzione, l'ultimo di questi viene usato per determinare la precedenza dell'azione di riduzione.

Gli operatori prefissi e postfissi possono essere gestiti in modo simile. L'associatività si applica solo agli operatori infissi, quindi conta solo la precedenza degli operatori prefissi e postfissi.

Si noti che solo i conflitti *shift-reduce* vengono eliminati dalle regole sopra. Alcuni generatori di parser consentono anche di eliminare i conflitti reduce-reduce tramite regole di precedenza (in questo caso, la produzione con l'operatore di precedenza più alta viene preferita), ma questo non è ovviamente altrettanto utile come quanto descritto sopra.

L'ambiguità del dangling-else (che abbiamo già visto) può anche essere eliminata utilizzando le regole di precedenza. Se abbiamo letto $if\ \exp\ then\ Stat$ e il prossimo simbolo è un *else*, vogliamo fare uno shift su *else*, in modo che *else* sia associato al *then*. Se, invece, riducessimo sulla produzione $Stat \rightarrow if\ \exp\ then\ Stat$, perderemmo questa associazione. Dare all' *else* una precedenza maggiore rispetto a *then* o dare loro la stessa precedenza e renderli associativi a destra garantirà che venga eseguito uno shift su *else* quando ne abbiamo bisogno.

Non tutti i conflitti dovrebbero essere eliminati tramite regole di precedenza. Se si aggiungono ciecamente regole di precedenza fino a quando non vengono segnalati conflitti, si rischia di escludere anche stringhe valide, quindi il parser accetterà solo un sottoinsieme del linguaggio previsto. Normalmente, si dovrebbero utilizzare solo dichiarazioni di precedenza per specificare le gerarchie degli operatori, a meno che non si sia analizzato attentamente le azioni del parser e si sia scoperto che non ci sono conseguenze indesiderate nell'aggiungere le regole di precedenza.

Proviamo dunque a costruire una tabella di parsing per una grammatica ambigua come questa:

$$E \rightarrow E + E \mid E * E \mid id$$

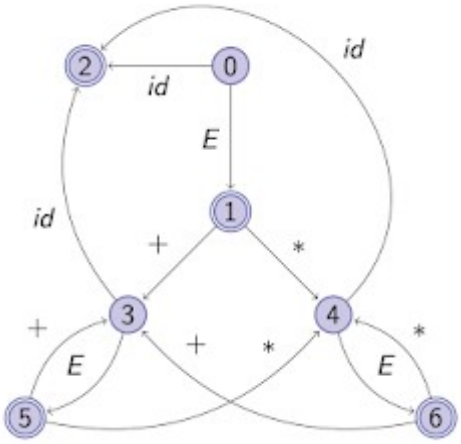
Questa grammatica è ambigua perché non fornisce informazioni alle proprietà del $+$ e del $*$, e in particolare non fornisce informazioni riguardanti il fatto che il $*$ ha la precedenza sul $+$.

Se applichiamo il procedimento che abbiamo visto per generare l'automa LR(0), otteniamo:

2
$E \to id\cdot$

5
$E \to E + E\cdot$
$E \to E \cdot + E$
$E \to E \cdot * E$

6
$E \to E * E\cdot$
$E \to E \cdot + E$
$E \to E \cdot * E$



Dove sulla sinistra abbiamo riportato gli stati finali, ovvero quelli contenenti dei reducing item.

Se andiamo però a costruire la tabella di parsing

1. $E \to E + E$
2. $E \to E * E$
3. $E \to id$

	id	+	*	$	*E*
0	s2				1
1		s3	s4	Acc	
2		r3	r3	r3	
3	s2				5
4	s2				6
5		s3; r1	s4; r1	r1	
6		s3; r2	s4; r2	r2	

notiamo che l'ambiguità della grammatica ha prodotto un effetto collaterale piuttosto sgradevole, infatti la tabella ha dei conflitti.
Dunque la grammatica da cui siamo partiti non è SLR(1).

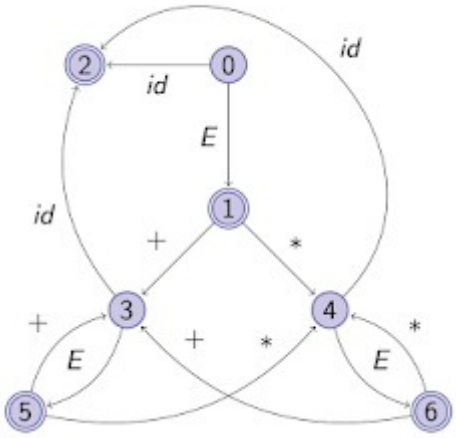
Proviamo dunque a capire come possiamo risolvere questi conflitti (non diversamente da quanto già avevamo fatto nel parsing top-down con le entries multiple defined).
I nostri conflitti sono localizzati solo in quattro situazioni, ovvero quando ci troviamo in uno tra gli stati 5 o 6 e, contemporaneamente, troviamo in lettura un simbolo tra + o $*$.

È facile notare, guardando il nostro automa caratteristico, che se il nostro parser si trova nello stato 5, in testa alla nostra pila dei simboli *symSt*, avremo necessariamente $E + E$.
Il discorso è del tutto analogo quando il nostro parser si trova nello stato 6, dove in testa alla nostra pila dei simboli *symSt*, avremo $E * E$.

	+	*
5	s3; r1	s4; r1
6	s3; r2	s4; r2

If the parser is in state 5 then $E + E$ is on *symSt*

If the parser is in state 6 then $E * E$ is on *symSt*



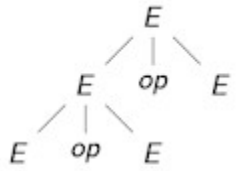
Dunque, quando due simboli collegati da un operatore sono stati già letti (come in $E+E$ o $E * E$) e immediatamente dopo ne leggiamo un terzo, è evidente che si verificano dei conflitti. Questi conflitti sorgono a causa dell'ambiguità della grammatica, la quale non specifica in modo adeguato né l'associatività né la precedenza per i due operatori.

Quindi, come possiamo risolvere questi conflitti fastidiosi? Beh, almeno in questa situazione, la strategia è quella di esaminare attentamente ciascuna cella in cui si presenta un conflitto e decidere manualmente quale azione mantenere. Questa decisione sembra essere nelle nostre mani e, nel nostro caso specifico, faremo in modo che il parser segua le regole comuni di precedenza e associatività per $+$ e $*$.

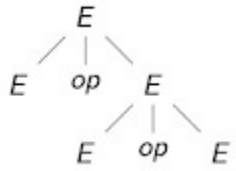
Notiamo che in realtà se abbiamo letto $E+E$ e il prossimo simbolo è $+$, la mossa che scegliamo è ininfluente in quanto l'operatore $+$ è associativo. Faremo però comunque in modo di mantenere l'associatività a sinistra per consistenza.

Guardando ai parse tree della nostra grammatica, possiamo trovare un valido aiuto. Gli alberi stessi, nella loro struttura, trasmettono in modo efficiente le informazioni sull'annidamento dei loro elementi, che può corrispondere direttamente alla parentesizzazione (e quindi alle regole di precedenza tra le espressioni). Di fatto, assumendo, come facciamo noi, che i sottoalberi vengano risolti prima degli alberi genitori, stabiliamo regole di precedenza molto simili a quelle che si otterrebbero con una parentesizzazione.
Consideriamo *op* una generica operazione. Se vogliamo che *op* sia associativa a sinistra:

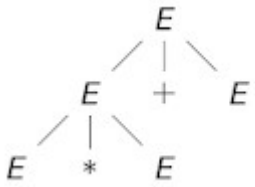
We want

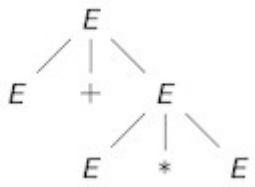


Rather than



Se consideriamo invece due operazioni diverse, in particolare + e $*$, vogliamo preservare la precedenza di $*$ e dunque vogliamo generare i seguenti parse trees:





Certamente, ecco una riformulazione:

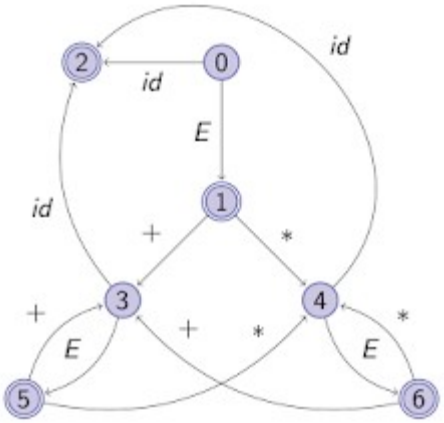
Ora possiamo affrontare i quattro conflitti individuati, decidendo per ciascuna cella l'azione che renderà il parsing conforme alle regole precedentemente stabilite. Con riferimento alla tabella di parsing, ci mettiamo all'opera.

- $M[5,+]$: Qui, con la testa di *symSt* che contiene $E+E$ e il successivo simbolo letto come +, essendo gli operatori associativi a sinistra, optiamo per la parentesizzazione della testa di *symSt* prima di procedere con il nuovo simbolo. La mossa scelta è quindi una riduzione, ovvero $r1$, per $M[5,+]$.
- $M[6,*]$: Simile al caso precedente, qui la testa di *symSt* è $E*E$ e il simbolo successivo è $*$. Poiché anche $*$ è associativo a sinistra, selezioniamo nuovamente la riduzione (in questo caso $r2$) per mantenere la coerenza con la parentesizzazione.
- $M[5,*]$: Tornando allo stato 5, dove *symSt* è ancora $E+E$ ma con il simbolo successivo come $*$, la priorità tra $*$ e + richiede che $*$ sia gestito prima di +. Pertanto, effettuiamo lo shift (in questo caso $s4$) per posticipare la parentesizzazione di *symSt* e procedere con la lettura.
- $M[6,+]$: Qui, con *symSt* come $E*E$ e un + successivo, manteniamo la coerenza con le regole di precedenza (ovvero $*$ prima di +) e scegliamo la riduzione (in questo caso $r2$) per parentesizzare *symSt* prima di leggere +.

Dunque abbiamo risolto tutti i conflitti ottenendo questa situazione:

Done!

	+	*
5	r1	s4
6	r2	r2



È importante sottolineare che di fronte a una tabella di parsing non deterministica, non sempre possiamo decidere in modo deliberato quali voci o azioni mantenere basandoci esclusivamente su un ragionamento. Questo approccio ha senso soprattutto quando consideriamo grammatiche che incorporano operatori. Tendenzialmente si preferisce l'utilizzo di grammatiche ambigue piuttosto di quelle non ambigue in quanto le grammatiche non ambigue tendono a generare un numero maggiori di voci e, di conseguenza, tabelle di parsing di dimensioni più grandi. Solitamente, insieme alla grammatica ambigua vengono anche fornite regole o direttive per stabilire una tabella di parsing deterministica.

Tuttavia questo non è l'unico modo di procedere per eliminare i conflitti, infatti i conflitti sono una conseguenza dell'ambiguità della grammatica, dunque basta generare una grammatica non ambigua prima di iniziare tutto il procedimento per non riscontrare conflitti. Cambiare la grammatica però, in generale, non è banale, in quanto è estremamente importante che la nuova grammatica esprima correttamente le regole di precedenza e associatività che vogliamo.
Ad esempio potevamo considerare la grammatica

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * id \mid id \end{aligned}$$

Notiamo che questa grammatica è organizzata in modo tale che E fa generare una somma, oppure diventa il non-terminale associato ad un'altra operazione.

L'ordine in cui abbiamo organizzato le produzioni non è casuale: abbiamo posizionato l'operatore $*$ in un livello di produzione inferiore a $+$ con uno specifico intento. Poiché la derivazione è rightmost e attraversiamo l'albero dal basso verso l'alto (bottom-up), questa disposizione assicura che le occorrenze di $*$ siano parentesizzate prima di altre operazioni.

Facciamoci alcune domande:

- **Perché non modificare la produzione $E \rightarrow T + E$ invece di mantenere $E \rightarrow E + T$?**
  Se desideriamo associatività a sinistra per l'operatore $+$, la modifica proposta, in cui abbiamo $E \rightarrow T + E$, potrebbe non adempiere a questo requisito. Mantenendo $E \rightarrow E + T$, confermiamo l'associatività a sinistra dell'operatore $+$ in quanto l'espressione sarà

sempre interpretata come $E+T$, garantendo la corretta sequenza di valutazione da sinistra a destra.

- **Perché non modificare la produzione $E \to E * T$ invece di mantenere $E \to E+T$ (invertendo l'operatore anche nell'altra produzione)?**
  Se vogliamo dare precedenza a $*$, la modifica proposta in cui $E \to E * T$ potrebbe non garantire questa precedenza. Mantenendo $E \to E+T$, confermiamo che l'operazione di moltiplicazione avviene tra T e id, fornendo la precedenza richiesta a $*$, poiché questa produzione sarà valutata prima di qualsiasi produzione con il +.

Dunque per stabilire la priorità della moltiplicazione rispetto all'addizione, è fondamentale posizionare l'operazione di somma a un livello meno profondo rispetto a quella della moltiplicazione. Pertanto, è essenziale definire le produzioni per l'addizione partendo dal simbolo di partenza e poi, successivamente, definire le produzioni per la moltiplicazione. Questa scelta riflette il processo di lettura degli alberi di derivazione, che avviene dal basso verso l'alto. Maggiore è la profondità di un'operazione nell'albero di derivazione, maggiore è la sua precedenza nell'esecuzione.

Vediamo un ultimo esempio.

Construct the SLR(1) parsing table for

$$\begin{aligned} S &\to aAd \mid bBd \mid aBe \mid bAe \\ A &\to c \\ B &\to c \end{aligned}$$

0:
$S' \to \cdot S$
$S \to \cdot aAd$
$S \to \cdot bBd$
$S \to \cdot aBe$
$S \to \cdot bAe$

$1 = \tau(0, S)$:
$S' \to S \cdot$

$2 = \tau(0, a)$:
$S \to a \cdot Ad$
$S \to a \cdot Be$
$A \to \cdot c$
$B \to \cdot c$

$3 = \tau(0, b)$:
$S \to b \cdot Bd$
$S \to b \cdot Ae$
$A \to \cdot c$
$B \to \cdot c$

$4 = \tau(2, A)$:
$S \to aA \cdot d$

$5 = \tau(2, B)$:
$S \to aB \cdot e$

$6 = \tau(2, c)$:
$A \to c \cdot$
$B \to c \cdot$

$7 = \tau(3, B)$:
$S \to bB \cdot d$

$8 = \tau(3, A)$:
$S \to bA \cdot e$

$\tau(3, c) = 6$

$10 = \tau(4, d)$:
$S \to aAd \cdot$

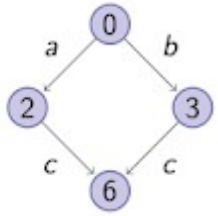
$11 = \tau(5, e)$:
$S \to aBe \cdot$

$12 = \tau(7, d)$:
$S \to bBd \cdot$

$13 = \tau(8, e)$:
$S \to bAe \cdot$

Analizziamo ora un pezzo dell'automa caratteristico derivante da questa analisi

$$
\begin{aligned}
S &\rightarrow aAd \mid bBd \mid aBe \mid bAe \\
A &\rightarrow c \\
B &\rightarrow c
\end{aligned}
$$



Notiamo che lo stato 6 contiene due reducing items:

- $A \rightarrow c \cdot$
- $B \rightarrow c \cdot$

Quindi, avendo $FOLLOW(A) = FOLLOW(B) = \{d, e\}$, ci sono conflitti *reduce-reduce* sia nelle voci $(6, d)$ che $(6, e)$ della tabella di parsing SLR(1), e dunque la grammatica non è SLR(1).

Se riuscissimo a definire in modo più sofisticato e preciso i $FOLLOW$, potremmo determinare quale riduzione effettuare in base al contesto, introducendo regole addizionali per trasformare la tabella di parsing in una forma deterministica. È evidente che questa grammatica genera esattamente quattro stringhe. Tuttavia, la responsabilità non ricade sulla grammatica in sé, poiché non è affatto ambigua (produce solo 4 parole). La criticità risiede piuttosto nel metodo di parsing adottato, che appare eccessivamente grezzo: anziché raggruppare tutto all'interno di uno stato unico (come osservato nello stato 6), non sarebbe preferibile avere due stati distinti?

Dunque, non potrebbero $\tau(2, c)$ e $\tau(3, c)$ rappresentare stati differenti? Dopotutto

- se abbiamo letto ac dovremmo ridurre c ad A solo se il prossimo simbolo è d, e dovremmo ridurre c a B solo se il prossimo simbolo è e.
- Viceversa, se abbiamo letto bc dovremmo ridurre c ad A solo se il prossimo simbolo è e , e dovremmo ridurre c a B solo se il prossimo simbolo è d.

Le grammatiche più avanzate rispetto alle SLR hanno un potere discriminante superiore in quanto riescono a definire in modo più preciso quali simboli richiedono una riduzione basata sul percorso nell'automa caratteristico. Di conseguenza, gli automi utilizzati per queste grammatiche contengono un'ampia quantità di informazioni rispetto a quelli delle grammatiche SLR. Gli automi risultanti, noti come LR(1), presentano tipicamente un maggior numero di stati a causa dell'uso di "items" più dettagliati rispetto agli automi SLR. Questi "items" consistono nell'item originale seguito da un insieme di simboli, un sottoinsieme di ciò che abbiamo precedentemente definito come follow dei non-termini, che indica dove posizionare le riduzioni.

Gli **LR(1)-items** possono fare la differenza.

Gli **automi LR(1)** sono più complessi degli automi LR(0).
Gli stati di questi automi sono insiemi di **LR(1)-items**, che sono coppie della forma

$$[A \rightarrow \alpha \cdot \beta, \Delta]$$

dove Δ, chiamato **insieme di lookahead**, è un sottoinsieme di $T \cup \{\$\}$.

L'operazione di chiusura per gli LR(1)-items affina la $closure()$ già vista.
La chiusura di $\{[A \to \alpha \cdot \beta, \Delta]\}$ propaga i simboli successivi a B in questo item agli insiemi di lookahead degli items $[B \to \cdot \gamma, \Gamma]$ aggiunti per chiudere l'insieme rispetto a B
L'obiettivo è raffinare gli insiemi di $FOLLOW$ utilizzati per posizionare le azioni di riduzione
L'idea è calcolare gli insiemi di $FOLLOW$ localmente (durante la costruzione dell'automa caratteristico) piuttosto che globalmente (sulla grammatica).

Le due principali tecniche di parsing utilizzate sono:

- LR(1), nota anche come LR(1) canonica, che richiede la creazione degli automi caratteristici più grandi possibili (con un elevato numero di stati). Solo pochi linguaggi di programmazione adottano parser di questa classe, data la correlazione diretta tra la dimensione della tabella di parsing e il numero di stati dell'automa. Ad esempio, l'intera famiglia di linguaggi OCaml fa uso di questa tecnica di parsing.
- La tecnica LALR(1), più ampia rispetto a SLR(1), offre un potere di riconoscimento maggiore rispetto agli SLR, pur mantenendo le dimensioni del parser simili a quelle di un SLR(1). Questa strategia consente un vantaggio computazionale mentre amplia la gamma di grammatiche riconosciute rispetto a SLR.

Abbiamo discusso SLR in quanto, a parte i raffinamenti di LR, la dinamica di *shift-reduce* è più comprensibile in SLR. Tuttavia, man mano che il parser diventa più efficiente, è in grado di riconoscere ed analizzare una gamma più ampia di grammatiche. Le classi di grammatiche sono identificate attraverso il parser utilizzato per riconoscerle e la tecnica di parsing impiegata.

Analisi semantica

Ambiti (scopes) e Tabelle dei Simboli

QUESTO CAPITOLO È FUORI DAL PROGRAMMA
Un concetto importante nei linguaggi di programmazione è la capacità di dare nomi agli oggetti come variabili, funzioni e tipi. Ogni oggetto nominato avrà una dichiarazione, dove il nome viene definito come sinonimo dell'oggetto. Questo processo è chiamato **binding** (vincolo o associazione). Ogni nome avrà anche un certo numero di utilizzi, dove il nome viene utilizzato come riferimento all'oggetto a cui è vincolato.

Spesso, la dichiarazione di un nome ha uno scope limitato: una porzione del programma dove il nome sarà visibile. Queste dichiarazioni sono chiamate **dichiarazioni locali**, mentre una dichiarazione che rende il nome dichiarato visibile nell'intero programma è chiamata **globale**. Può accadere che lo stesso nome venga dichiarato in diverse scope nidificate. In questo caso, è normale che la dichiarazione più vicina a un utilizzo del nome sarà quella che definisce quel particolare utilizzo. In questo contesto, "più vicino" è legato all'albero sintattico del programma: lo scope di una dichiarazione sarà un sotto-albero dell'albero sintattico e le dichiarazioni nidificate daranno origine a scope che sono sotto-alberi nidificati. La dichiarazione più vicina di

un nome è quindi la dichiarazione corrispondente al sotto-albero più piccolo che racchiude l'uso del nome. Come esempio, guarda questo blocco di istruzioni in C:

```
{
  int x = 1;
  int y = 2;
  {
    double x = 3.14159265358979;
    y += (int)x;
  }
  y += x;
}
```

Le due righe immediatamente dopo la prima parentesi graffa aperta dichiarano le variabili intere x e y con scope fino alla parentesi graffa di chiusura nell'ultima riga. Un nuovo scope inizia con la seconda parentesi graffa di apertura e viene dichiarata una variabile floating-point x con un valore iniziale vicino a π. Questo avrà uno scope fino alla prima parentesi graffa di chiusura, quindi la variabile x originale non è visibile fino alla fine dello scope interno. L'assegnazione $y+=¿$ aggiungerà 3 a y, quindi il suo nuovo valore sarà 5. Nell'assegnazione successiva $y+=x;$, siamo usciti dallo scope interno, quindi viene ripristinata la x originale. L'assegnazione aggiungerà quindi 1 a y, che avrà il valore finale 6.

Lo scoping basato sulla struttura dell'albero sintattico, come mostrato nell'esempio, è chiamato **binding statico o lessicale** ed è la regola di scoping più comune nei linguaggi di programmazione moderni. Nella restante parte di questo capitolo, assumeremo che venga utilizzato il binding statico. Alcuni linguaggi utilizzano il **binding dinamico**, dove la dichiarazione incontrata più di recente durante l'esecuzione del programma definisce l'uso corrente del nome. Per sua natura, il binding dinamico non può essere risolto a tempo di compilazione, quindi le tecniche descritte nel resto di questo capitolo come utilizzate in un compilatore dovranno essere impiegate a tempo di esecuzione se il linguaggio utilizza il binding dinamico.
Un compilatore dovrà tenere traccia dei nomi e degli oggetti a cui sono legati, in modo che ogni utilizzo di un nome venga attribuito correttamente alla sua dichiarazione. Questo viene tipicamente fatto utilizzando una tabella dei simboli (o ambiente, come a volte viene chiamata).

Tabella dei simboli

Una tabella dei simboli è una struttura dati che associa nomi a informazioni. Abbiamo bisogno di diverse operazioni su una tabella dei simboli per realizzare questo:

- **empty**: Creare una tabella dei simboli vuota, in cui nessun nome è definito.
- **binding**: Essere in grado di associare un nome a un'informazione. Nel caso in cui il nome sia già definito nella tabella dei simboli, la nuova associazione ha la precedenza sulla vecchia.
- **lookup**: Essere in grado di cercare un nome in una tabella dei simboli per trovare l'informazione a cui il nome è associato. Se il nome non è definito nella tabella dei simboli, dobbiamo ricevere una notifica.
- **enter**: Essere in grado di entrare in un nuovo scope.

- **exit**: Essere in grado di uscire da uno scope, ripristinando la tabella dei simboli a come era prima che lo scope fosse entrato.

Queste operazioni sono fondamentali per gestire correttamente gli scope e le associazioni tra nomi e informazioni all'interno di un programma.

Ci sono molte modalità per **implementare** le tabelle dei simboli, ma la distinzione più importante tra queste riguarda il modo in cui vengono gestiti gli scope. Questo può essere fatto utilizzando una struttura dati persistente (o funzionale) o può essere fatto utilizzando una struttura dati imperativa (o aggiornata distruttivamente).

Una **struttura dati persistente** ha la proprietà che nessuna operazione sulla struttura distruggerà la struttura stessa. Concettualmente, viene creata una nuova copia modificata della struttura dati ogni volta che un'operazione la aggiorna, preservando quindi invariata la vecchia struttura. Ciò significa che è banale ripristinare la vecchia tabella dei simboli quando si esce da uno scope, in quanto è stata preservata dalla natura persistente della struttura dati. In pratica, solo una piccola porzione della struttura dati viene copiata quando si aggiorna una tabella dei simboli, la maggior parte è condivisa con la versione precedente.

Nel **metodo imperativo**, esiste solo una copia della tabella dei simboli, quindi sono necessarie azioni esplicite per memorizzare le informazioni necessarie per ripristinare la tabella dei simboli a uno stato precedente. Questo può essere fatto utilizzando uno stack ausiliario. Quando viene apportato un aggiornamento, la vecchia associazione di un nome sovrascritto viene registrata (inserita) nello stack ausiliario. Quando si entra in un nuovo scope, viene inserito uno "marcatore" nello stack ausiliario. Quando si esce dallo scope, le associazioni nello stack ausiliario (fino al marcatore) vengono utilizzate per ripristinare la vecchia tabella dei simboli. Le associazioni e il marcatore vengono rimossi (estratti) dallo stack ausiliario durante questo processo, restituendo lo stack ausiliario allo stato in cui si trovava prima di entrare nello scope.

Di seguito, esamineremo implementazioni semplici di entrambi gli approcci e discuteremo di come approcci più avanzati possano superare alcuni dei problemi di efficienza dei metodi più semplici.

Tabelle dei simboli persistenti semplici

Nei linguaggi funzionali come SML, Scheme o Haskell, le strutture dati persistenti sono la norma più che l'eccezione (motivo per cui le strutture dati persistenti vengono talvolta chiamate strutture dati funzionali). Ad esempio, quando viene aggiunto un nuovo elemento all'inizio di una lista o un elemento viene rimosso dall'inizio della lista, la vecchia lista continua ad esistere e può essere utilizzata altrove. Una lista è un modo naturale per implementare una tabella dei simboli in un linguaggio funzionale: un legame è una coppia di un nome e le relative informazioni associate, e una tabella dei simboli è una lista di tali coppie. Le operazioni sono implementate nel modo seguente:

- **empty**: Una tabella dei simboli vuota è una lista vuota.
- **binding**: Un nuovo legame (coppia nome/informazione) viene aggiunto (consed) all'inizio della lista.

- **lookup**: La lista viene esaminata fino a quando non viene trovata una coppia con un nome corrispondente. Le informazioni associate al nome vengono restituite. Se si raggiunge la fine della lista, viene restituito un'indicazione che questo è avvenuto. Questa indicazione può essere ottenuta sollevando un'eccezione o facendo sì che la funzione di ricerca restituisca un valore speciale che rappresenta "non trovato". Questo richiede un tipo che possa contenere sia informazioni normali che questo valore speciale, cioè un tipo somma (una struttura dati che può rappresentare uno tra diversi tipi alternativi).
- **enter**: La vecchia lista viene ricordata, cioè viene creata un riferimento ad essa.
- **exit**: Viene richiamata la vecchia lista, cioè viene utilizzato il riferimento sopra citato.

Le ultime due operazioni non sono effettivamente operazioni esplicite, poiché la variabile utilizzata per contenere la tabella dei simboli prima di entrare in un nuovo scope conserverà comunque la stessa tabella dei simboli dopo che lo scope è terminato. Quindi tutto ciò di cui c'è bisogno è una variabile per contenere (un riferimento alla) la tabella dei simboli.

Poiché nuovi legami vengono aggiunti all'inizio della lista e la lista viene esaminata dall'inizio alla fine, i legami negli scope interni avranno automaticamente la precedenza sui legami negli scope esterni.

Un altro approccio funzionale alle tabelle dei simboli consiste nell'utilizzare le funzioni: Una tabella dei simboli è naturalmente vista come una funzione dai nomi alle informazioni. Le operazioni sono:

- **empty**: Una tabella dei simboli vuota è una funzione che restituisce un'indicazione di errore (o genera un'eccezione) indipendentemente da quale sia il suo argomento.
- **binding**: Aggiungere un legame del nome n alle informazioni i in una tabella dei simboli t viene fatto definendo una nuova funzione della tabella dei simboli t' in termini di t e del nuovo legame. Quando t' viene chiamata con un nome $n1$ come argomento, confronta $n1$ con n. Se sono uguali, t' restituisce le informazioni i. Altrimenti, t' chiama t con $n1$ come argomento e restituisce il risultato che questa chiamata produce. In Standard ML, possiamo definire una funzione di legame in questo modo:
  $bind\ (n,i,t) = fn\ n1 => if\ n1 = n\ then\ i\ else\ t\ n1$
- **lookup**: La funzione della tabella dei simboli viene chiamata con il nome come argomento.
- **enter**: Viene ricordata la vecchia funzione (referenziata).
- **exit**: Viene richiamata la vecchia funzione (utilizzando un riferimento).

Di nuovo, le ultime due operazioni sono per lo più implicite.

Tabelle dei simboli imperative semplici

Le tabelle dei simboli imperative sono naturali da usare se il compilatore è scritto in un linguaggio imperativo. Una semplice tabella dei simboli imperativa può essere implementata come uno stack, che funziona in modo simile all'implementazione funzionale basata su liste:

- **empty**: Una tabella dei simboli vuota è uno stack vuoto.

- **binding**: Un nuovo legame (coppia nome/informazione) viene inserito in cima allo stack.
- **lookup**: Lo stack viene cercato dall'alto verso il basso fino a quando non viene trovato un nome corrispondente. Le informazioni associate al nome vengono restituite. Se si raggiunge il fondo dello stack, restituiamo invece un'indicazione di errore.
- **enter**: Inseriamo un marcatore in cima allo stack.
- **exit**: Rimuoviamo i legami dallo stack fino a quando non viene trovato un marcatore. Anche il marcatore viene rimosso dallo stack.

È importante notare che poiché la tabella dei simboli stessa è uno stack, non abbiamo bisogno dello stack ausiliario menzionato precedentemente.
Questa struttura non è esattamente una struttura dati persistente, poiché lasciare uno scope distruggerà la sua tabella dei simboli. Per linguaggi semplici, questo potrebbe non essere un problema, poiché uno scope non è necessario dopo essere stato terminato. Tuttavia, caratteristiche del linguaggio come classi, moduli e chiusure lessicali possono richiedere che le tabelle dei simboli persistano anche dopo l'uscita dallo scope. In questi casi, è necessario utilizzare una vera tabella dei simboli persistente o copiare e memorizzare le parti necessarie della tabella dei simboli per un successivo recupero prima di uscire da uno scope.

Problemi di efficienza

Le implementazioni che abbiamo visto qui sopra sono semplici, ma condividono lo stesso problema di efficienza: la ricerca è fatta tramite ricerca lineare, quindi il tempo peggiore per la ricerca è proporzionale alla dimensione della tabella dei simboli. Questo è principalmente un problema in relazione alle librerie: è abbastanza comune che un programma utilizzi librerie che definiscono letteralmente centinaia di nomi.
Una soluzione comune a questo problema è l'hashing: i nomi vengono trasformati in valori numerici (hash), che vengono utilizzati per indicizzare un array. Ogni elemento dell'array è quindi una lista lineare dei legami dei nomi che condividono lo stesso codice hash. Con una tabella hash sufficientemente grande, queste liste saranno tipicamente molto corte, quindi il tempo di ricerca è praticamente costante.
L'utilizzo delle tabelle hash complica leggermente l'entrata e l'uscita dagli scope. Mentre ogni elemento della tabella hash è una lista che può essere gestita come nei casi semplici, farlo per tutti gli elementi dell'array ad ogni entrata e uscita impone un notevole overhead. Invece, è tipico nelle implementazioni imperative utilizzare uno stack ausiliario singolo per registrare tutti gli aggiornamenti alla tabella in modo che possano essere annullati in un tempo proporzionale al numero di aggiornamenti effettuati nello scope locale. Le implementazioni funzionali utilizzano tipicamente tabelle hash persistenti, che eliminano il problema.

Spazi dei nomi condivisi o separati

In alcuni linguaggi (come Pascal), una variabile e una funzione nello stesso scope possono avere lo stesso nome, poiché il contesto d'uso chiarirà se si tratta di una variabile o di una funzione. Si dice che funzioni e variabili abbiano spazi dei nomi separati, il che significa che definire un nome in uno spazio non influisce sullo stesso nome nell'altro spazio. In altri linguaggi (ad esempio C o SML), il contesto non può distinguere (facilmente) le variabili dalle funzioni. Di

conseguenza, dichiarare una variabile locale potrebbe nascondere una funzione dichiarata in uno scope esterno o viceversa. Questi linguaggi hanno uno spazio dei nomi condiviso per variabili e funzioni.
Gli spazi dei nomi possono essere condivisi o separati per tutti i tipi di nomi che possono comparire in un programma, ad esempio variabili, funzioni, tipi, eccezioni, costruttori, classi, selettori di campo, ecc. Quali spazi dei nomi sono condivisi dipende dal linguaggio.
Gli spazi dei nomi separati sono facilmente implementati utilizzando una tabella dei simboli per ogni spazio dei nomi, mentre gli spazi dei nomi condivisi condividono naturalmente una singola tabella dei simboli. Tuttavia, a volte è conveniente utilizzare una singola tabella dei simboli anche se ci sono spazi dei nomi separati. Ciò può essere fatto abbastanza facilmente aggiungendo indicatori dello spazio dei nomi ai nomi. Un indicatore dello spazio dei nomi può essere un prefisso testuale al nome o può essere un tag associato al nome. In entrambi i casi, una ricerca nella tabella dei simboli deve corrispondere sia al nome che all'indicatore dello spazio dei nomi del simbolo cercato con il nome e l'indicatore dello spazio dei nomi dell'ingresso nella tabella.

Type checking

QUESTO CAPITOLO È FUORI DAL PROGRAMMA
La fase di analisi lessicale e di analisi sintattica può respingere molti testi come programmi non corretti. Tuttavia, molti linguaggi hanno requisiti di correttezza sintattica che non possono essere gestiti esclusivamente dalle tecniche viste finora. Questi requisiti possono essere, ad esempio, la correttezza statica dei tipi o il requisito che il matching dei pattern o le istruzioni `case` siano esaustive.

Queste proprietà sono molto spesso non context-free, cioè non possono essere verificate dall'appartenenza a un linguaggio context-free. Di conseguenza, vengono verificate da una fase che (concettualmente) segue l'analisi sintattica (anche se potrebbe essere intercalata con essa). Questi controlli possono avvenire in una fase che non fa nulla altro, oppure possono essere combinati con l'esecuzione effettiva o la traduzione in un altro linguaggio. Spesso, il traduttore può sfruttare o dipendere dalle informazioni sul tipo, il che rende naturale combinare il calcolo dei tipi con la traduzione effettiva. Il type checking può essere fatto durante l'esecuzione, e in questo caso viene chiamato **tipizzazione dinamica**. In questo capitolo, supporremo che il type checking e i controlli correlati vengano effettuati in una fase precedente all'esecuzione o alla traduzione (ossia, **tipizzazione statica**) e supporremo allo stesso modo che qualsiasi informazione raccolta da questa fase sia disponibile nelle fasi successive.

Lo spazio di progettazione dei tipi

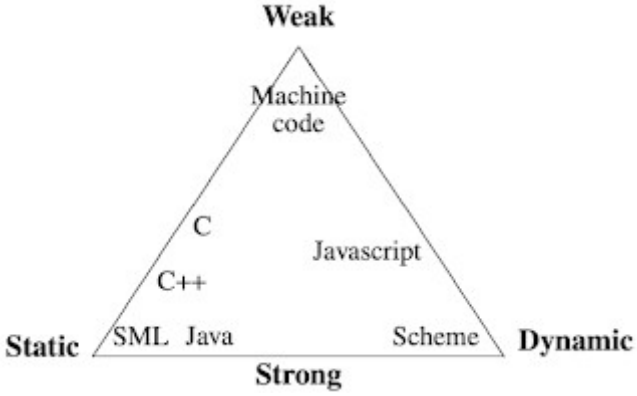
Abbiamo già discusso la differenza tra tipizzazione statica e dinamica, ovvero se i controlli di tipo vengono effettuati prima o durante l'esecuzione di un programma. Inoltre, possiamo distinguere tra linguaggi debolmente e fortemente tipizzati.

La **tipizzazione forte** significa che l'implementazione del linguaggio assicura che ogni volta che viene eseguita un'operazione, gli argomenti dell'operazione siano di un tipo per il quale

l'operazione è definita. Ad esempio, non si cerca di concatenare una stringa e un numero in virgola mobile. Questo è indipendente dal fatto che sia garantito staticamente (prima dell'esecuzione) o dinamicamente (durante l'esecuzione).

In contrasto, un linguaggio **debolmente tipizzato** non fornisce alcuna garanzia che le operazioni vengano eseguite su argomenti che hanno senso per l'operazione. Il linguaggio debolmente tipizzato archetipico è il linguaggio macchina: le operazioni vengono eseguite senza alcun controllo e se c'è qualche concetto di tipo a livello di macchina, è piuttosto limitato: i registri possono essere divisi in interi, virgola mobile e (possibilmente) registri di indirizzi, e la memoria è (se del caso) divisa solo in codice e dati. I linguaggi debolmente tipizzati vengono utilizzati principalmente per la programmazione di sistema, dove è necessario manipolare, spostare, copiare, crittografare o comprimere dati senza preoccuparsi di cosa rappresentino i dati.

Molti linguaggi combinano sia tipizzazione forte che debole o sia tipizzazione statica che dinamica: alcuni tipi vengono verificati prima dell'esecuzione e altri durante l'esecuzione, mentre alcuni tipi non vengono verificati affatto. Ad esempio, C è un linguaggio staticamente tipizzato (poiché non vengono effettuati controlli durante l'esecuzione), ma non tutti i tipi sono verificati. Ad esempio, è possibile memorizzare un intero in una variabile di tipo union e leggerlo come puntatore o numero in virgola mobile. Un altro esempio è Javascript: se provi a moltiplicare due stringhe, l'interprete verificherà se le stringhe contengono sequenze di cifre e, in caso affermativo, "interpreterà" le stringhe come numeri e le moltiplicherà. Questo è un tipo di tipizzazione debole, poiché l'operazione di moltiplicazione viene applicata ad argomenti (stringhe) dove la moltiplicazione non ha senso. Ma invece di, come nel linguaggio macchina, cercare ciecamente di moltiplicare le rappresentazioni in macchina delle stringhe come se fossero numeri, Javascript esegue un controllo dinamico e una conversione per far sì che i valori si conformino all'operazione. Continuerò comunque a chiamare questo comportamento tipizzazione debole, poiché non c'è nulla che indichi che convertire le stringhe in numeri prima della moltiplicazione abbia più senso che moltiplicare semplicemente le rappresentazioni in macchina delle stringhe. Il punto principale è che il linguaggio, invece di segnalare un possibile problema, esegue silenziosamente qualcosa che probabilmente non ha senso.



La figura mostra un diagramma dello spazio di progettazione della tipizzazione statica vs. dinamica e della tipizzazione debole vs. forte, posizionando alcuni linguaggi di programmazione ben noti in questo spazio di progettazione. Si noti che lo spazio di progettazione è mostrato

come un triangolo: se non si verificano mai i tipi, non lo si fa né staticamente né dinamicamente, quindi nell'estremità debole rispetto allo spettro debole vs. forte, la distinzione tra statico e dinamico è priva di significato.

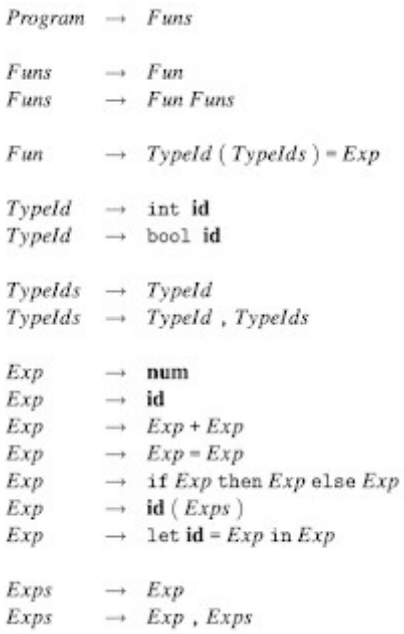
Attributi

La fase di checking opera sull'abstract syntax tree del programma e può effettuare diverse iterazioni su di esso. Tipicamente, ogni iterazione è un attraversamento ricorsivo dell'abstract syntax tree, raccogliendo informazioni o utilizzando informazioni raccolte nelle iterazioni precedenti. Tali informazioni vengono spesso chiamate attributi del syntax tree. Di solito, distinguiamo tra due tipi di attributi: gli **attributi sintetizzati** vengono passati verso l'alto nel syntax tree, dalle foglie fino alla radice. Gli **attributi ereditati**, al contrario, vengono passati verso il basso nel syntax tree. Tuttavia, va notato che le informazioni sintetizzate in un sottoalbero possono essere ereditate da un altro sottoalbero o, in un'iterazione successiva, dallo stesso sottoalbero. Un esempio di questo è una tabella dei simboli: questa è sintetizzata da una dichiarazione ed ereditata dallo scope della dichiarazione. Quando le dichiarazioni sono ricorsive, lo scope può essere lo stesso syntax tree della dichiarazione stessa, nel qual caso una iterazione su questo albero costruirà la tabella dei simboli come attributo sintetizzato mentre una seconda iterazione la utilizzerà come attributo ereditato.

Tipicamente, ogni categoria sintattica (rappresentata da un tipo nella struttura dati per l'abstract syntax tree o da un gruppo di non-terminali correlati nella grammatica) avrà il suo set di attributi. Quando scriviamo un checker come un insieme di funzioni mutuamente ricorsive, ci sarà una o più di tali funzioni per ogni categoria sintattica. Ognuna di queste funzioni prenderà attributi ereditati (compreso il syntax tree stesso) come argomenti e restituirà attributi sintetizzati come risultati.

In questo capitolo ci concentreremo sul type checking e accenneremo brevemente ad altre proprietà che possono essere verificate. I metodi utilizzati per il type checking possono nella maggior parte dei casi essere facilmente modificati per gestire tali altre verifiche.

Utilizzeremo il seguente linguaggio come esempio per il controllo statico dei tipi:



Program → *Funs*

Funs → *Fun*
Funs → *Fun Funs*

Fun → *TypeId* (*TypeIds*) = *Exp*

TypeId → `int` **id**
TypeId → `bool` **id**

TypeIds → *TypeId*
TypeIds → *TypeId* , *TypeIds*

Exp → **num**
Exp → **id**
Exp → *Exp* + *Exp*
Exp → *Exp* = *Exp*
Exp → `if` *Exp* `then` *Exp* `else` *Exp*
Exp → **id** (*Exps*)
Exp → `let` **id** = *Exp* `in` *Exp*

Exps → *Exp*
Exps → *Exp* , *Exps*

Ambienti per il type checking

Per effettuare il type checking del programma, abbiamo bisogno di tabelle dei simboli che associno variabili e funzioni ai loro tipi. Poiché ci sono spazi dei nomi separati per variabili e funzioni, utilizzeremo due tabelle dei simboli, una per le variabili e una per le funzioni. Una variabile è associata a uno dei due tipi: $\int ¿$ o $bool$. Una funzione è associata al suo tipo, che consiste nei tipi dei suoi argomenti e nel tipo del suo risultato.
I tipi delle funzioni sono scritti come una lista tra parentesi dei tipi degli argomenti, seguiti da una freccia e il tipo di risultato, ad esempio $\left(\int, bool\right) \rightarrow \int ¿$ per una funzione che prende due parametri (di tipo $\int ¿$ e $bool$, rispettivamente) e restituisce un intero.
Assumeremo che le tabelle dei simboli siano persistenti, quindi non è necessaria alcuna azione esplicita per ripristinare la tabella dei simboli per lo scope esterno quando si esce da uno scope interno. Non è necessario conservare le tabelle dei simboli per gli scope interni una volta che questi sono usciti (quindi un comportamento simile a uno stack va bene).

Type checking delle espressioni

Quando effettuiamo il type checking delle espressioni, le tabelle dei simboli per le variabili e le funzioni sono attributi ereditati. Il tipo ($\int ¿$ o $bool$) dell'espressione viene restituito come attributo sintetizzato. Per rendere la presentazione indipendente da una struttura dati specifica per la sintassi astratta, permetteremo alla funzione del controllore dei tipi di utilizzare una notazione simile alla sintassi concreta per scopi di matching dei pattern. Tuttavia, è importante considerarla come una sintassi astratta, quindi tutti i problemi di ambiguità, ecc., sono stati risolti.
Per i terminali (nomi delle variabili e costanti numeriche) con attributi, assumiamo che ci siano funzioni predefinite per estrarli. Pertanto, "id" ha associata una funzione "getname" che estrae il

nome dell'identificatore. Allo stesso modo, "num" ha una funzione "getvalue" che restituisce il valore del numero. Quest'ultima non è necessaria per il controllo statico dei tipi.

Per ciascun non-terminale, definiamo una o più funzioni che prendono un sottoalbero della sintassi astratta e attributi ereditati come argomenti e restituiscono gli attributi sintetizzati. Nella figura

$Check_{Exp}(Exp, vtable, ftable) =$ case Exp of	
num	`int`
id	$t = lookup(vtable, getname(\textbf{id}))$ *if* $t = unbound$ *then* **error**(); `int` *else* t
Exp_1 + Exp_2	$t_1 = Check_{Exp}(Exp_1, vtable, ftable)$ $t_2 = Check_{Exp}(Exp_2, vtable, ftable)$ *if* $t_1 =$ `int` *and* $t_2 =$ `int` *then* `int` *else* **error**(); `int`
Exp_1 = Exp_2	$t_1 = Check_{Exp}(Exp_1, vtable, ftable)$ $t_2 = Check_{Exp}(Exp_2, vtable, ftable)$ *if* $t_1 = t_2$ *then* `bool` *else* **error**(); `bool`
`if` Exp_1 `then` Exp_2 `else` Exp_3	$t_1 = Check_{Exp}(Exp_1, vtable, ftable)$ $t_2 = Check_{Exp}(Exp_2, vtable, ftable)$ $t_3 = Check_{Exp}(Exp_3, vtable, ftable)$ *if* $t_1 =$ `bool` *and* $t_2 = t_3$ *then* t_2 *else* **error**(); t_2
id ($Exps$)	$t = lookup(ftable, getname(\textbf{id}))$ *if* $t = unbound$ *then* **error**(); `int` *else* $((t_1, \ldots, t_n) \rightarrow t_0) = t$ $[t'_1, \ldots, t'_m] = Check_{Exps}(Exps, vtable, ftable)$ *if* $m = n$ *and* $t_1 = t'_1, \ldots, t_n = t'_n$ *then* t_0 *else* **error**(); t_0
`let` **id** = Exp_1 `in` Exp_2	$t_1 = Check_{Exp}(Exp_1, vtable, ftable)$ $vtable' = bind(vtable, getname(\textbf{id}), t_1)$ $Check_{Exp}(Exp_2, vtable', ftable)$

$Check_{Exps}(Exps, vtable, ftable) =$ case $Exps$ of	
Exp	$[Check_{Exp}(Exp, vtable, ftable)]$
Exp , $Exps$	$Check_{Exp}(Exp, vtable, ftable)$ :: $Check_{Exps}(Exps, vtable, ftable)$

mostriamo la funzione di type checking per le espressioni. La funzione per il type checking delle espressioni è chiamata $\mathrm{Check}_{\mathrm{exp}}$. La tabella dei simboli per le variabili è data dal parametro *vtable*, mentre la tabella dei simboli per le funzioni è data dal parametro *ftable*. La funzione *error* segnala un errore di tipo. Per consentire al controllore dei tipi di continuare e segnalare più di un errore, consentiamo alla funzione di segnalazione degli errori di restituire. Dopo aver segnalato un errore di tipo, il controllore dei tipi può fare una congettura su quale sarebbe dovuto essere il tipo e restituire questa congettura, consentendo così al type checking di continuare per il resto del programma. Tuttavia, questa congettura potrebbe essere sbagliata, il che potrebbe causare la segnalazione di errori di tipo spurii in seguito. Pertanto, tutti gli errori di tipo successivi al primo dovrebbero essere presi con cautela.

Spiegheremo brevemente ciascuno dei casi gestiti da $\text{Check}_{\text{exp}}$.

- Un numero ha tipo $\int ¿$.
- Il tipo di una variabile viene trovato cercando il suo nome nella tabella dei simboli delle variabili. Se la variabile non è presente nella tabella dei simboli, la funzione di ricerca restituisce il valore speciale *unbound*. Quando ciò accade, viene segnalato un errore e il controllore dei tipi ipotizza arbitrariamente che il tipo sia $\int ¿$. In caso contrario, viene restituito il tipo restituito da *lookup*.
- Un'espressione di addizione richiede che entrambi gli argomenti siano interi e ha come risultato un intero.
- La comparazione richiede che gli argomenti abbiano lo stesso tipo. In entrambi i casi, il risultato è un booleano.
- In un'espressione condizionale, la condizione deve essere di tipo *bool* e i due rami devono avere tipi identici. Il risultato di una condizione è il valore di uno dei rami, quindi ha lo stesso tipo di questi. Se i rami hanno tipi diversi, il controllore dei tipi segnala un errore e sceglie arbitrariamente il tipo del ramo *then* come sua congettura per il tipo dell'intera espressione.
- In una chiamata di funzione, il nome della funzione viene cercato nell'ambiente delle funzioni per trovare il numero e i tipi degli argomenti e il tipo di ritorno. Il numero di argomenti della chiamata deve coincidere con il numero previsto e i loro tipi devono corrispondere ai tipi dichiarati. Il tipo risultante è il tipo di ritorno della funzione. Se il nome della funzione non viene trovato in *ftable*, viene segnalato un errore e il controllore dei tipi ipotizza arbitrariamente il tipo di risultato come $\int ¿$.
- Un'espressione *let* dichiara una nuova variabile, il cui tipo è quello dell'espressione che definisce il valore della variabile. La tabella dei simboli per le variabili viene estesa utilizzando la funzione *bind*, e la tabella estesa viene utilizzata per controllare l'espressione del corpo e trovare il suo tipo, che a sua volta è il tipo dell'intera espressione. Un'espressione *let* non può di per sé causare un errore di tipo (anche se le sue parti possono farlo), quindi non viene effettuato alcun test.

Poiché $\text{Check}_{\text{exp}}$ menziona il non-terminale *Exps* e la sua funzione di type checking correlata Check_{Exps}, abbiamo incluso Check_{Exps} nella figura.
Check_{Exps} costruisce una lista dei tipi delle espressioni nell'elenco delle espressioni. La notazione è presa da SML: Una lista è scritta tra parentesi quadre con virgole tra gli elementi. L'operatore :: aggiunge un elemento all'inizio di una lista.

Type checking delle dichiarazioni di funzioni

Una dichiarazione di funzione esplicita dichiara i tipi degli argomenti. Questa informazione viene utilizzata per costruire una tabella dei simboli per le variabili, che viene utilizzata durante il type checking del corpo della funzione. Il tipo del corpo deve corrispondere al tipo di risultato dichiarato della funzione. La funzione di type checking per le funzioni, Check_{Fun}, ha come

attributo ereditato la tabella dei simboli per le funzioni, che viene passata alla funzione di type checking per le espressioni. Check_{Fun} non restituisce informazioni, verifica solo eventuali errori interni. Check_{Fun} è mostrata nella figura

$Check_{Fun}(Fun, ftable) =$ case Fun of	
$TypeId\ (\ TypeIds\ )\ =\ Exp$	$(f, t_0) = Get_{TypeId}(TypeId)$ $vtable = Check_{TypeIds}(TypeIds)$ $t_1 = Check_{Exp}(Exp, vtable, ftable)$ *if* $t_0 \neq t_1$ *then* **error**()

$Get_{TypeId}(TypeId) =$ case $TypeId$ of	
`int` **id**	$(getname(\textbf{id}), \texttt{int})$
`bool` **id**	$(getname(\textbf{id}), \texttt{bool})$

$Check_{TypeIds}(TypeIds) =$ case $TypeIds$ of	
$TypeId$	$(x,t) = Get_{TypeId}(TypeId)$ $bind(emptytable, x, t)$
$TypeId\ ,\ TypeIds$	$(x,t) = Get_{TypeId}(TypeId)$ $vtable = Check_{TypeIds}(TypeIds)$ *if* $lookup(vtable, x) = unbound$ *then* $bind(vtable, x, t)$ *else* **error**(); $vtable$

insieme alle funzioni per *TypeId* e *TypeIds*, che essa utilizza. La funzione Get_{TypeId} restituisce semplicemente una coppia con il nome e il tipo dichiarato, mentre $\text{Check}_{TypeIds}$ costruisce una tabella dei simboli da tali coppie. $\text{Check}_{TypeIds}$ verifica anche se tutti i parametri hanno nomi diversi. *emptytable* è una tabella dei simboli vuota. La ricerca di qualsiasi nome nella tabella dei simboli vuota restituisce *unbound*.

Il type checking di un programma

Un programma è una lista di funzioni ed è considerato corretto dal punto di vista dei tipi se tutte le funzioni sono corrette dal punto di vista dei tipi, e non ci sono due definizioni di funzione che definiscono lo stesso nome di funzione. Inoltre, deve esserci una funzione chiamata *main* con un argomento di tipo intero e un risultato di tipo intero.
Poiché tutte le funzioni sono mutuamente ricorsive, ciascuna di esse deve essere controllata dal punto di vista dei tipi utilizzando una tabella dei simboli in cui tutte le funzioni sono vincolate al loro tipo. Questo richiede due passaggi sulla lista delle funzioni: uno per costruire la tabella dei simboli e uno per controllare le definizioni delle funzioni utilizzando questa tabella. Di conseguenza, abbiamo bisogno di due funzioni che operano su $Funs$ e due funzioni che operano su Fun. Abbiamo già visto una delle ultime, Check_{Fun}. L'altra, Get_{Fun}, restituisce la coppia del nome dichiarato della funzione e del tipo, che consiste nei tipi degli argomenti e nel tipo del risultato. Essa utilizza una funzione ausiliaria, Get_{Types}, per trovare i tipi degli argomenti. Le due funzioni per la categoria sintattica $Funs$ sono Get_{Funs}, che costruisce la tabella dei simboli e controlla le definizioni duplicate, e Check_{Funs}, che chiama Check_{Fun} per tutte le funzioni. Queste funzioni e la funzione principale $\text{Check}_{Program}$, che unisce tutti gli elementi, sono mostrate nella figura

$Check_{Program}(Program) = \texttt{case}\ Program\ \texttt{of}$	
$Funs$	$ftable = Get_{Funs}(Funs)$ $Check_{Funs}(Funs, ftable)$ $if\ lookup(ftable, \texttt{main}) \neq (\texttt{int}) \rightarrow \texttt{int}$ $then\ \textbf{error}()$

$Get_{Funs}(Funs) = \texttt{case}\ Funs\ \texttt{of}$	
Fun	$(f,t) = Get_{Fun}(Fun)$ $bind(emptytable, f, t)$
$Fun\ Funs$	$(f,t) = Get_{Fun}(Fun)$ $ftable = Get_{Funs}(Funs)$ $if\ lookup(ftable, f) = unbound$ $then\ bind(ftable, f, t)$ $else\ \textbf{error}();\ ftable$

$Get_{Fun}(Fun) = \texttt{case}\ Fun\ \texttt{of}$	
$TypeId\ (\ TypeIds\ )\ =\ Exp$	$(f, t_0) = Get_{TypeId}(TypeId)$ $[t_1, \ldots, t_n] = Get_{Types}(TypeIds)$ $(f, (t_1, \ldots, t_n) \rightarrow t_0)$

$Get_{Types}(TypeIds) = \texttt{case}\ TypeIds\ \texttt{of}$	
$TypeId$	$(x,t) = Get_{TypeId}(TypeId)$ $[t]$
$TypeId\ TypeIds$	$(x_1, t_1) = Get_{TypeId}(TypeId)$ $[t_2, \ldots, t_n] = Get_{Types}(TypeIds)$ $[t_1, t_2, \ldots, t_n]$

$Check_{Funs}(Funs, ftable) = \texttt{case}\ Funs\ \texttt{of}$	
Fun	$Check_{Fun}(Fun, ftable)$
$Fun\ Funs$	$Check_{Fun}(Fun, ftable)$ $Check_{Funs}(Funs, ftable)$

Questo completa il type checking del nostro piccolo linguaggio di esempio.

Type checking avanzato

Il nostro linguaggio di esempio è molto semplice e ovviamente non copre tutti gli aspetti del type checking. Di seguito sono elencati alcuni esempi di altre funzionalità e brevi spiegazioni su come possono essere gestiti.

Assegnazioni. Quando una variabile riceve un valore tramite un'assegnazione, è necessario verificare che il tipo del valore sia lo stesso del tipo dichiarato della variabile. Alcuni compilatori possono controllare se una variabile viene potenzialmente utilizzata prima che le venga assegnato un valore, o se una variabile non viene utilizzata dopo la sua assegnazione. Anche se non sono esattamente errori di tipo, tale comportamento è probabilmente indesiderato. Tuttavia, testare tale comportamento richiede un'analisi leggermente più complicata rispetto alla semplice verifica dei tipi presentata in questo capitolo, poiché si basa su informazioni non strutturali.

Strutture dati. Una struttura dati può definire un valore con diversi componenti (ad esempio, una struct, una tupla o un record) o un valore che può essere di tipi diversi in momenti diversi (ad esempio, un'unione, una variante o una somma). Per controllare i tipi di tali strutture, il controllore dei tipi deve essere in grado di rappresentarne i tipi. Di conseguenza, il controllore dei tipi può aver bisogno di una struttura dati che descriva tipi complessi. Questo può essere simile alla struttura dati utilizzata per gli abstract syntax tree delle dichiarazioni. Le operazioni che costruiscono o smontano dati strutturati devono essere testate per correttezza. Se ogni

operazione su dati strutturati ha tipi ben definiti per i suoi argomenti e un tipo per il risultato, ciò può essere fatto in modo simile alla verifica delle chiamate di funzione.

Overloading. L'overloading significa che lo stesso nome viene utilizzato per diverse operazioni su diversi tipi di dati. Abbiamo visto un esempio semplice di questo nel linguaggio di esempio, dove = era utilizzato sia per confrontare interi che booleani. In molti linguaggi, operatori aritmetici come + e - sono definiti sia per interi che per numeri in virgola mobile, e possibilmente anche per altri tipi. Se questi operatori sono predefiniti e coprono solo un numero finito di casi, tutti i possibili casi possono essere provati a turno, proprio come nel nostro esempio.
Questo, tuttavia, richiede che le diverse istanze dell'operatore abbiano tipi di argomenti disgiunti. Se, ad esempio, c'è una funzione *read* che legge un valore da uno stream di testo e questa è definita per leggere sia interi che numeri in virgola mobile, l'argomento (lo stream di testo) da solo non può essere utilizzato per selezionare l'operatore corretto. Pertanto, il controllore dei tipi deve passare il tipo atteso di ciascuna espressione come attributo ereditato, in modo che questo (eventualmente in combinazione con i tipi degli argomenti) possa essere utilizzato per scegliere la corretta istanza dell'operatore sovraccaricato.
Potrebbe non sempre essere possibile trasmettere un tipo atteso a causa della mancanza di informazioni. Nel nostro linguaggio di esempio, questo è il caso degli argomenti per = (poiché questi possono essere sia $\int$ ¿ che *bool*) e della prima espressione in un'espressione *let* (poiché la variabile legata nell'espressione *let* non è dichiarata come un tipo specifico). Se il controllore dei tipi, per questo o per un'altra ragione, non è in grado di selezionare un operatore univoco, potrebbe segnalare "overloading non risolto" come errore di tipo, oppure potrebbe selezionare un'istanza predefinita.

Conversione di tipo. Un linguaggio può avere operatori per convertire un valore di un tipo in un valore di un altro tipo, ad esempio un intero in un numero in virgola mobile. A volte questi operatori sono espliciti nel programma e quindi facili da controllare. Tuttavia, molti linguaggi consentono la conversione implicita degli interi in float, in modo tale che, ad esempio, $3+3.12$ sia ben tipizzato con l'assunzione implicita che l'intero 3 venga convertito in float prima dell'addizione. Questo può essere gestito nel seguente modo: se il controllore dei tipi scopre che gli argomenti di un operatore non hanno il tipo corretto, può provare a convertire uno o entrambi gli argomenti per vedere se questo aiuta. Se ci sono un piccolo numero di conversioni legali predefinite, questo non è un problema grave. Tuttavia, una combinazione di operatori overloaded definiti dall'utente e tipi definiti dall'utente con conversioni può rendere piuttosto difficile il processo di type checking, poiché le informazioni necessarie per scegliere correttamente potrebbero non essere disponibili a tempo di compilazione. Questo è tipicamente il caso nei linguaggi orientati agli oggetti, dove la selezione del metodo è spesso fatta a tempo di esecuzione. Non entreremo nei dettagli su come ciò possa essere fatto.

Il Polimorfismo / Tipi Generici. Alcuni linguaggi consentono a una funzione di essere polimorfica o generica, ovvero definita su una vasta gamma di tipi simili, ad esempio su tutti gli array indipendentemente dai tipi degli elementi. Una funzione può dichiarare esplicitamente quali parti del tipo sono generiche/polimorfiche oppure questo può essere implicito (vedi sotto).

Il controllore dei tipi può inserire i tipi effettivi in ogni uso della funzione generica/polimorfica per creare istanze del tipo generico/polimorfico.
Questo meccanismo è diverso dall'overloading poiché le istanze saranno correlate da un tipo generico comune e perché una funzione polimorfica/generica può essere istanziata da qualsiasi tipo, non solo da una lista limitata di alternative dichiarate come nel caso dell'overloading.

Tipi impliciti. Alcuni linguaggi (come Standard ML e Haskell) richiedono che i programmi siano ben tipizzati, ma non richiedono dichiarazioni esplicite di tipo per variabili o funzioni. Per far sì che ciò funzioni, si utilizza un algoritmo di inferenza di tipo. Questo algoritmo raccoglie informazioni sull'uso di funzioni e variabili e utilizza tali informazioni per inferire i loro tipi. Se ci sono usi inconsistevoli di una variabile, viene segnalato un errore di tipo.

Analisi semantica

La fase di analisi semantica è un passaggio cruciale nel processo di compilazione. Una volta identificata la struttura sintattica del programma, l'analisi semantica aggiunge informazioni aggiuntive che vanno oltre ciò che può essere catturato dalle regole delle grammatiche context-free (ad esempio il fatto che una variabile venga dichiarata prima di essere utilizzata).

Dunque, successivamente a una delle due tipologie di analisi sintattica (sia top-down che bottom-up), le quali danno in output le produzioni della grammatica che sono state impiegate durante la derivazione (se esiste), si ricostruisce l’albero di derivazione per poi procedere con l’analisi semantica.

Le attività principali di questa fase includono la creazione e la popolazione delle **tabelle dei simboli** dopo le dichiarazioni per tenere traccia dei tipi delle variabili e delle funzioni. Inoltre, si esegue l'**inferenza** e il **type checking** sulle espressioni e sulle istruzioni presenti nel codice.

Tutte queste analisi (lessicale, sintattica, semantica) vengono solitamente eseguite in sequenza, ma quando si sviluppa un compilatore, è possibile eseguirle (almeno in parte) in parallelo.

Questa fase si suddivide in due categorie di analisi: quella necessaria per garantire la correttezza del programma e quella che mira a migliorare l'efficienza del programma tradotto. La prima si concentra sulla correttezza dei tipi e delle strutture dati, mentre la seconda può riguardare ottimizzazioni che rendono il programma più veloce o efficiente.

L'analisi semantica può essere descritta in modo più pratico identificando le proprietà (**attributi**) dei simboli della grammatica e scrivendo regole (**regole semantiche**) che spiegano come il calcolo di queste proprietà è correlato alle produzioni della grammatica.

Questa metodologia (che è la più semplice ma non risulta poi essere la più ottimizzata) è definita **grammatica attribuita** o **definizione sintattico-direzionale**.

Al posto dell'albero di derivazione, una base migliore per le computazioni semantiche è rappresentata dall'**abstract syntax tree**, ovvero una rappresentazione più compatta dell'albero di derivazione.

La funzione primaria dell'analisi semantica, infatti, consiste nel costruire l'abstract syntax tree, che è un'astrazione dell'albero di derivazione. L'obiettivo è descrivere un insieme di operazioni necessarie per elaborare le informazioni durante il processo di compilazione.

Ecco cosa vedremo ora:

- Le **Definizioni Sintattico-Direzionali** (SDDs), ovvero tecniche e metodologie mirate a calcolare le specifiche proprietà associate ai simboli all'interno di una grammatica.
- Le **Tecniche per il calcolo degli attributi specificati**, ovvero una comprensione dettagliata di come gli attributi vengano definiti e assegnati.

Le Syntax-Directed Definitions (SDDs) sono delle grammatiche context-free arricchite, a cui sono stati aggiunti attributi e regole specifiche per calcolarli:

- Gli **attributi** sono informazioni associate a simboli della grammatica (gli elementi del vocabolario), ad esempio numeri, tipi di dati, riferimenti a tabelle dei simboli o posizioni di variabili in memoria.
- Le **regole semantiche**, invece, sono legate alle produzioni della grammatica e servono a calcolare gli attributi dei simboli in funzione degli attributi di altri simboli presenti nella stessa produzione.

Questi strumenti sono fondamentali per condurre l'analisi semantica del codice e svolgere altre operazioni correlate.

Gli **attributi dei non-terminali** possono essere classificati come sintetizzati o ereditati:

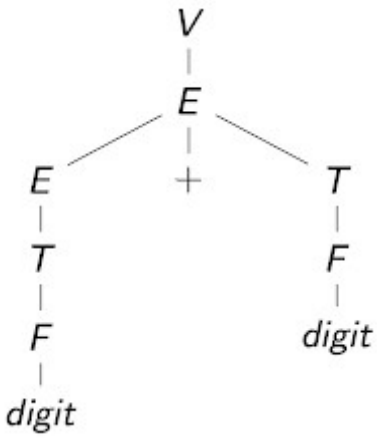
- Gli **attributi sintetizzati** sono quelli del driver definiti in funzione degli attributi dei simboli presenti nel body della produzione.
- Gli **attributi ereditati** sono invece quelli dei non-terminali nel corpo, definiti in funzione degli attributi degli altri simboli della produzione.

Gli **attributi dei terminali** possono solo essere sintetizzati in quanto sono forniti dal lexer e non c'è una regola per calcolarli, sono sempre noti.

Vediamo subito un esempio per capire in cosa consiste una SDD per un'espressione aritmetica. Prendiamo in considerazione la seguente grammatica LALR(1) per le espressioni aritmetiche:

$$
\begin{aligned}
V &\to E \\
E &\to E + T \mid T \\
T &\to T * F \mid F \\
F &\to (E) \mid digit
\end{aligned}
$$

e valutiamo l'espressione rappresentata dall'albero di parsing seguente:



Se un *digit* (cifra) ha valore lessicale 3 e l'altro *digit* ha valore lessicale 4, allora il valore dell'espressione dovrebbe essere 7.

La SDD per valutare le espressioni aritmetiche potrebbe essere organizzata nella seguente maniera, settando i seguenti attributi:

$V \to E$	$\{V.val = E.val\}$
$E \to E_1 + T$	$\{E.val = E_1.val + T.val\}$
$E \to T$	$\{E.val = T.val\}$
$T \to T_1 * F$	$\{T.val = T_1.val * F.val\}$
$T \to F$	$\{T.val = F.val\}$
$F \to (E)$	$\{F.val = E.val\}$
$F \to digit$	$\{F.val = digit.lexval\}$

Nota che i pedici numerici usati in alcune produzioni sono usati solo per differenziare il body dal driver, non hanno nessuna valenza semantica (di fatto E_1 ed E sono lo stesso non-terminale). Questo perché bisogna essere in grado di fare riferimento agli attributi di ciascun elemento delle produzioni; quindi, è essenziale distinguere, ad esempio, le due E nella seconda produzione. Sulla sinistra ci sono tutte le produzioni della grammatica. Quelle che abbiamo elencato sulla destra delle produzioni tra parentesi graffe sono le **regole semantiche** che compongono la grammatica attribuita. Tutto ciò che si trova all'interno delle regole sono gli attributi (nota però che i simboli + e $*$ in questo caso non sono attributi, ma indicano proprio le operazioni matematiche rispettive).

Quindi, la regola associata alla produzione $F \to digit$ digit afferma che l'attributo *val* del non-terminale F è equivalente a $digit.lexval$, dove si suppone che *lexval* indichi il valore ricordato dall'analizzatore lessicale per quella specifica occorrenza di cifra durante la lettura.

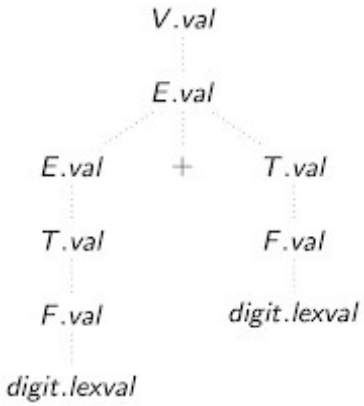
Proviamo ora a dare una spiegazione del significato delle varie regole così da capire meglio qual è lo scopo delle grammatiche attribuite:

- la prima produzione è una produzione aggiuntiva che abbiamo inserito noi (un po' come $S' \to S$) e ci dice che quando incontriamo produzioni $V \to E$ il valore di V è $E.val$;
- per produzioni come $E \to E_1 + T$ avremo che il valore di E corrisponde alla somma del sottoalbero E_1 più il valore di T (nota che E_1 è usato solo per differenziare da E);
- se ho una produzione $E \to T$ il valore di E è uguale al valore di T;
- se ho una produzione $T \to T_1 * F$ il valore di T corrisponde al valore del sottoalbero T_1 moltiplicato per il valore di F (nota che T_1 è usato solo per differenziare da T);
- se ho una produzione del tipo $T \to F$ il valore di T è uguale a $F.val$;
- se ho una produzione del tipo $F \to (E)$ il valore che assegno a F corrisponde a $E.val$;
- se ho una produzione del tipo $F \to digit$ mi aspetto che il valore di F sia il valore riportato nella tabella dei simboli per tale $digit$.

Notiamo la presenza di un nodo extra V e della relativa produzione aggiuntiva, comunemente utilizzati per eseguire azioni come "$print\ \ V.val$". È pertanto vantaggioso avere una produzione che viene impiegata una sola volta, così da poter eseguire azioni specifiche di questo tipo.

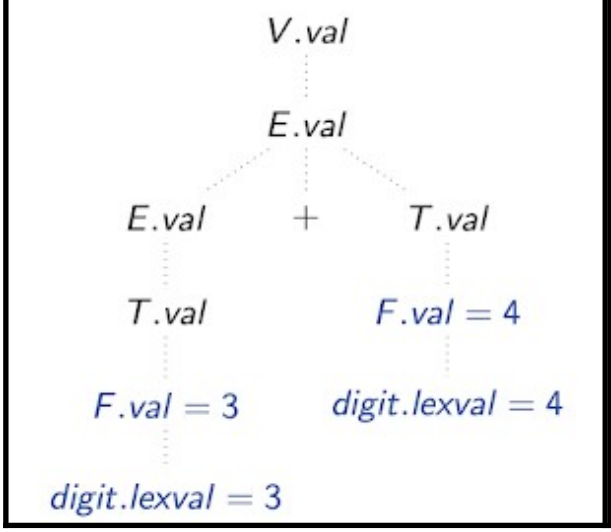
Con questo sistema arricchiamo l'albero di prima partendo dagli attributi $lexval$ delle $digit$, dunque viene naturale andare dal basso verso l'alto.

Vediamo ora come utilizzare questa SDD. Ora che abbiamo la grammatica attribuita infatti possiamo utilizzarla appunto per dare un valore al parse tree della nostra somma.
Prima di tutto prendiamo in considerazione l'**albero di parsing annotato**, ovvero l'albero di parsing arricchito con i valori degli attributi nei suoi nodi.

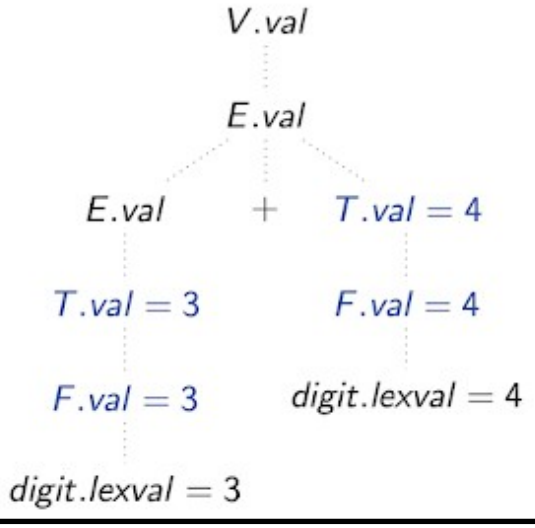


Il parse tree annotato viene letto dal basso verso l'alto (**bottom-up**) seguendo le regole di attribuzione che sono segnate sui vari nodi per ottenere la valutazione dell'albero stesso.
Vediamo ora questo procedimento passo per passo.

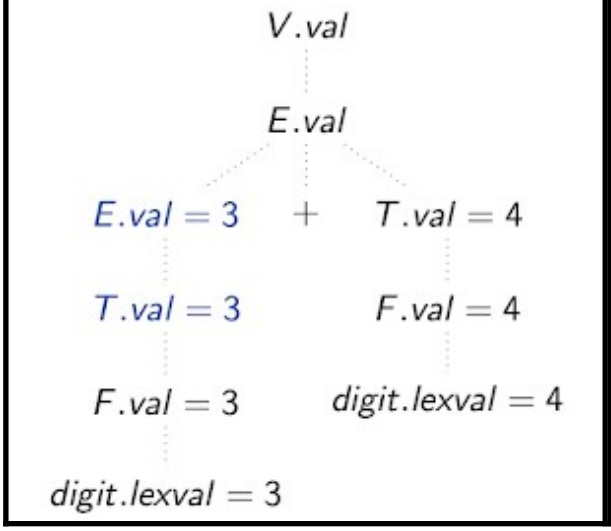
$F \to digit \qquad \{F.val = digit.lexval\}$



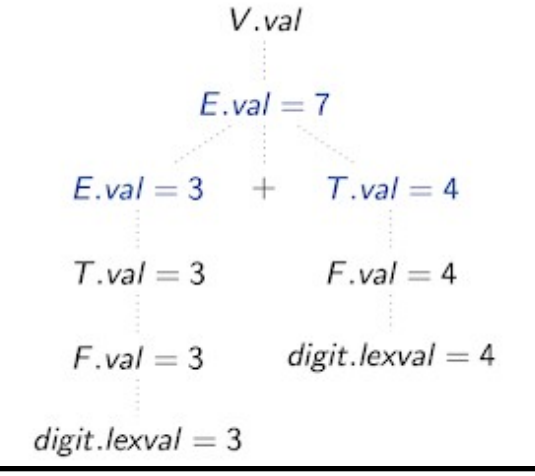
$T \to F \qquad \{T.val = F.val\}$

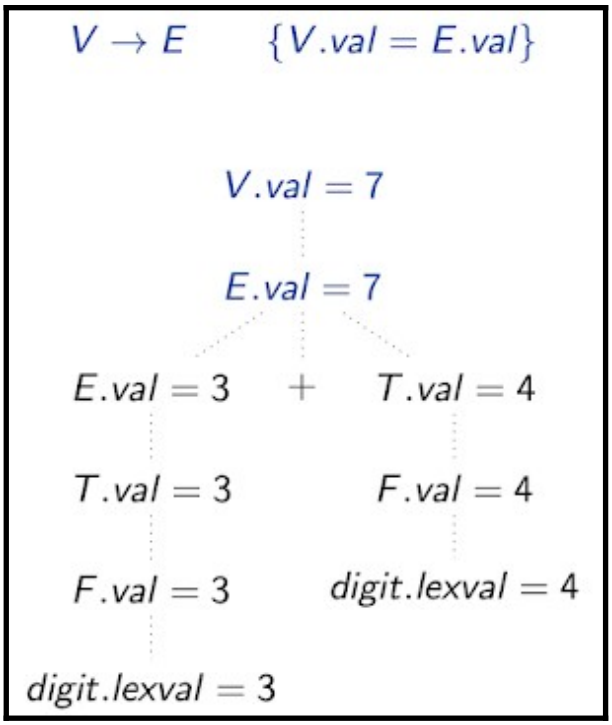


$E \to T \qquad \{E.val = T.val\}$



$E \to E_1 + T \qquad \{E.val = E_1.val + T.val\}$





A questo punto, diventa evidente come si possa utilizzare una grammatica attribuita per determinare il valore di una specifica stringa all'interno di una data grammatica:

1. Si crea un albero di parsing annotato per la stringa in questione.
2. Si impiega l'albero di parsing annotato in combinazione con le regole definite dalla grammatica attribuita per attribuire valore, un nodo alla volta, a tutti i nodi presenti nell'albero.
3. Una volta che la radice dell'albero di parsing annotato è stata valutata, si ottiene il valore desiderato per la stringa.

Un aspetto interessante, già accennato prima ma evidenziato dall'albero di parsing annotato in questo esempio, è che gli attributi presenti nei nodi (i terminali), essendo valori esterni, sono tutti sintetizzati.
Questo perché l'attributo sintetizzato è calcolato in base agli attributi dei figli; poiché il terminale non ha figli, l'attributo si considera già sintetizzato.

Il metodo che abbiamo appena visto è fatto un po' ad intuizione (ed infatti vale solo quando abbiamo esclusivamente attributi sintetizzati), vediamo quindi un metodo più generale, che ci permette anche di capire quando un parse tree annotato può essere valutato.
Si inizia creando un **grafo delle dipendenze**:

1. Crea un nodo del grafo di dipendenza per ogni attributo associato a qualche nodo dell'albero di parsing.
2. Per ogni attributo $X.x$ usato per definire l'attributo $Y.y$, crea un arco da $X.x$ a $Y.y$ nel grafo. Questo arco rappresenta che "$X.x$ **è necessario per definire** $Y.y$".
3. Dopo aver costruito il grafo, è possibile valutare le SDDs seguendo qualsiasi **ordinamento topologico** di questo grafo delle dipendenze.

Questo grafo quindi mette in relazione quali valori devono essere conosciuti per calcolare il valore dell'attributo preso in considerazione. La dipendenza viene descritta da un arco che va

dal nodo necessario per calcolare il nodo che deve essere calcolato al nodo che deve essere calcolato.

Per ottenere un ordinamento topologico, numeriamo i nodi $N_1,\dots,N_k$ del grafo delle dipendenze in modo che se c'è un collegamento da N_i a N_j, allora il nodo N_i sia valutato prima del nodo N_j. Questo metodo di numerazione permette di garantire un ordine sequenziale e corretto nella valutazione delle SDDs basato sulle loro dipendenze.

Quando la SDD ha sia attributi sintetizzati che ereditati, non è sempre garantito che esista un ordinamento topologico (e quindi un ordine di valutazione). In questo caso non sarà possibile calcolare il valore di tutti gli attributi.
In particolare, la presenza di cicli nel grafo delle dipendenze può impedire un ordinamento topologico. Ad esempio:

$$A \to B \qquad \{A.s = B.i;\ B.i = A.s + 7\}$$



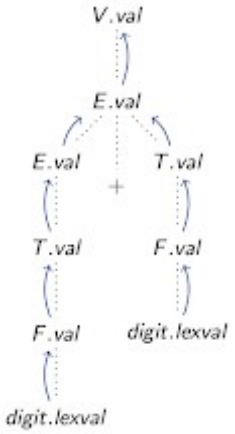
Esistono però classi di SDD per le quali la presenza di un ordinamento topologico nel grafo delle dipendenze è garantita:

- **S-attributed SDDs**: Queste SDDs hanno solo attributi sintetizzati, il che significa che possono essere valutate con un semplice attraversamento post-order dell'albero di parsing. Non ci sono attributi ereditati, quindi non esistono cicli di dipendenze tra gli attributi stessi. Parliamo dunque di alberi ed è sufficiente una visita post-ordine.
- **L-attributed SDDs**: Queste SDDs possono avere sia attributi sintetizzati che ereditati, ma gli attributi ereditati devono seguire delle regole specifiche. In particolare, quando definiamo attributi ereditati per una produzione, diciamo $A \to X_1 \dots X_n$, la definizione dell'attributo di ciascun X_j (ovvero $X_j.i$) deve utilizzare al massimo:
    - Gli attributi ereditati di A
    - Gli attributi ereditati o sintetizzati dei fratelli a sinistra $X_1,\dots,X_{j-1}$

  Questo vincolo limita le dipendenze tra attributi ereditati, assicurando che non ci siano cicli nel grafo delle dipendenze e quindi consentendo un ordinamento topologico.

Le S-attributed SDD risultano ideali per il parsing bottom-up poiché consentono di creare e valutare il parse tree attribuito simultaneamente all'analisi della stringa.
Per lo stesso motivo, le L-attributed SDD sono ottimali per il parsing top-down.

Ecco il grafo di dipendenza per l’esempio visto prima, che era una S-attributed SSD:



Calcolare un ordinamento topologico dei nodi diventa superfluo per valutare la SDD per questo grafo di dipendenza. Infatti, come già detto, basta una visita post-order dell'albero di parsing per valutare la SDD.
Questo è sempre vero per le S-attributed SDD, in quanto non hanno attributi ereditati.
L'algoritmo post-order per l'attraversamento dell'albero è il seguente:

```
post-order(N) {
    foreach child C of N do post-order(C);
    evaluate attributes of N;
}
```

Questa procedura visita prima tutti i figli di un nodo, valuta i loro attributi, e infine valuta gli attributi del nodo stesso. Dunque, questo approccio funziona efficacemente per le SDD senza attributi ereditati, rendendo l'ordinamento topologico dei nodi un passo superfluo.

La procedura per calcolare il valore di un'espressione tramite un parse tree è la seguente:

```
procedure expEval(N)
    foreach child C of N do
        expEval(C) ;
    if N is a Fnode then
        if N has only one child then
            assign to N.val the attribute digit.lexval of its child
        else
            assign to N.val the attribute of its second child
    else if N is a Enode then
        if N has only one child then
            assign to N.val the attribute of its child
        else
            assign to N.val the sum of the attributes of its first
            and third children
    ...
```

In alcuni casi, gli attributi sintetizzati potrebbero non essere sufficienti per svolgere determinate operazioni. Ad esempio, consideriamo una grammatica per la **dichiarazione di variabili**

tipizzate, in cui le regole definiscono la struttura per associare tipi come $\int ¿$ o $float$ a identificatori in una lista separata da virgole o a un singolo identificatore.

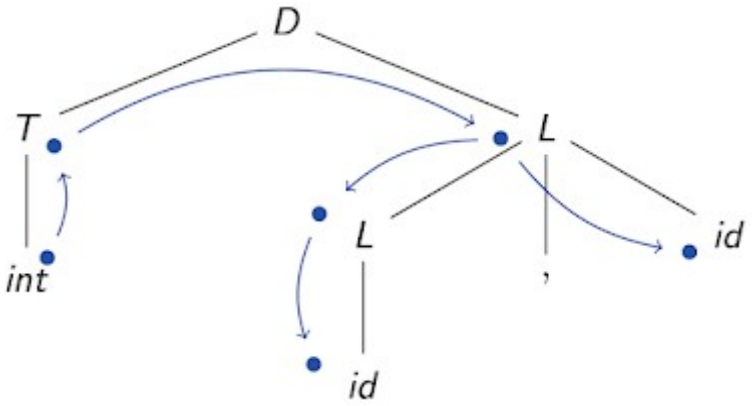
$$
\begin{aligned}
D &\rightarrow TL \\
T &\rightarrow int \\
T &\rightarrow float \\
L &\rightarrow L, id \\
L &\rightarrow id
\end{aligned}
$$

Notiamo che gli identificatori stanno nel sottoalbero destro di D mentre i tipi nel sottoalbero sinistro di D. Dunque se si vuole associare il tipo agli identificatori, bisogna farlo tramite una dipendenza tra nodi fratelli.

Per assegnare il tipo dichiarato ($\int ¿$ o $float$) all'attributo $id.type$, potremmo progettare azioni semantiche specifiche. Queste operazioni sarebbero configurate per estrarre e assegnare il tipo in base alle regole di grammatica. Ad esempio, potremmo definire operazioni che, durante l'analisi della dichiarazione di variabili, identificano e memorizzano il tipo associato a ciascun identificatore.

Nel caso specifico della grammatica fornita, si potrebbe utilizzare un unico attributo ereditato per la gestione del tipo.

$D \rightarrow TL$	$\{L.type = T.type\}$
$T \rightarrow int$	$\{T.type = integer\}$
$T \rightarrow float$	$\{T.type = float\}$
$L \rightarrow L_1, id$	$\{L_1.type = L.type; id.type = L.type\}$
$L \rightarrow id$	$\{id.type = L.type\}$



Dunque alla L associo un attributo di tipo ereditario, perchè pesco un'informazione da T (suo fratello).

Questa è una L-attributed SDDs, infatti gli attributi ereditati di L e di id vengono “pescati” o dal fratello sinistro o dal padre.
Per l'effettiva valutazione dell'albero sintattico, dato che non si può usare il post-ordine, si può scegliere tra:

- la precomputazione di un ordinamento topologico
- l'applicazione di una combinazione ad hoc di visite in-order e pre-order sull'albero.

La procedura per calcolare gli attributi è la seguente:

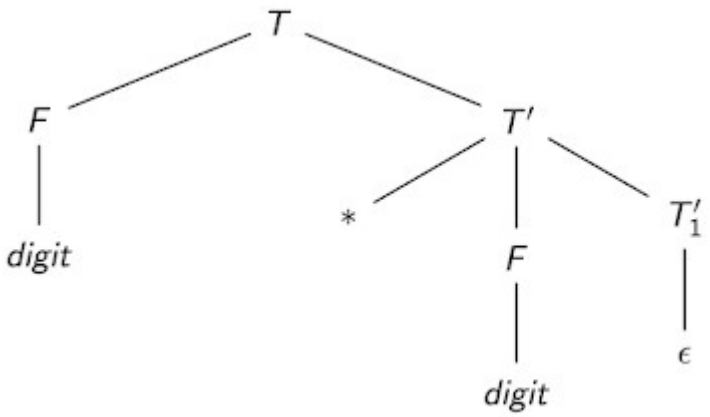
```
procedure typeEval(N)
    if N is a Dnode then
        typeEval(Tnode child of N) ;
        assign the type of the Tnode child to the Lnode child ;
        typeEval(Lnode child of N);
    else if N is a Lnode then
        assign N.type to the IDnode child ;
        if N has a Lnode child then
            assign N.type to the Lnode child ;
            typeEval (Lnode child of N) ;
    else if N is a Tnode then
        assign integer to N.type if child of N is int, assign float to
        N.type otherwise
```

Dunque osserviamo che diverse grammatiche presentano diverse sfide nel progettare la SDD.
Analizziamo ora un caso specifico: le espressioni aritmetiche.
In particolare consideriamo la seguente grammatica LL(1) per le espressioni aritmetiche:

$$
\begin{aligned}
V &\rightarrow E \\
E &\rightarrow TE' \\
E' &\rightarrow +TE' \mid \epsilon \\
T &\rightarrow FT' \\
T' &\rightarrow *FT' \mid \epsilon \\
F &\rightarrow (E) \mid digit
\end{aligned}
$$

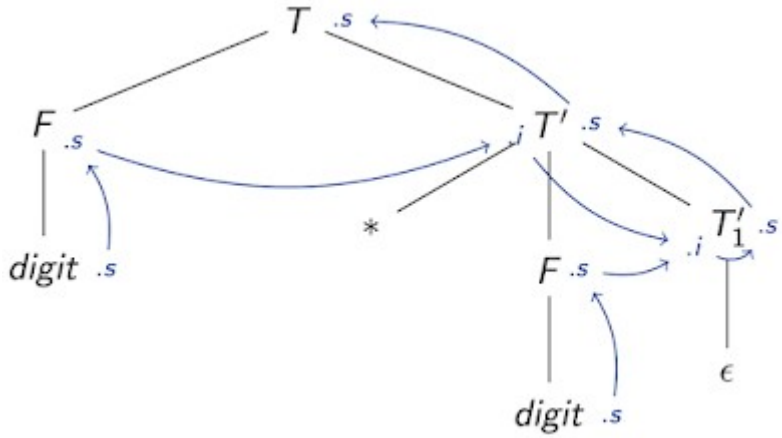
Dunque non analizziamo più una grammatica bottom-up, ma una top-down, in particolare LL(1).
Come sempre, l'obiettivo è quello di sistemare gli attributi in modo tale che andando a valutare l'albero di derivazione annotato, si è in grado di determinare il valore della radice.

Prendiamo come esempio $3 * 5$, il cui parse tree ha il seguente sottoalbero (analizziamo solamente il nucleo centrale):



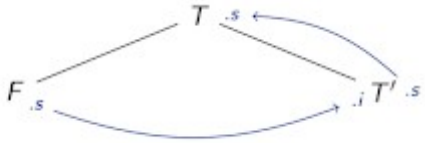
In modo analogo a quello che succedeva nelle dichiarazioni tipizzate, abbiamo una $digit$ che è nel sottoalbero sinistro della radice, mentre l'operatore $*$ e l'altra $digit$ sono nel sottoalbero destro della radice.
La cosa importante da considerare è che se avessimo, ad esempio, una stringa più lunga di moltiplicazioni rispetto a $digit * digit$, avremmo un sottoalbero destro molto più ampio rispetto a quello mostrato nella figura. Devo quindi muovermi nell'albero in modo parametrico che dipenda dalla grammatica, ovvero muovermi sull'albero in base alle istanze che posso aver letto.
Quindi, non possiamo associare a T il valore di 15 a meno che non riusciamo a trasferire il valore del $digit$ dal sottoalbero di sinistra al sottoalbero di destra.

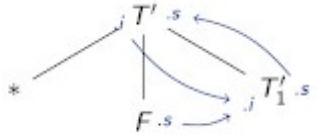
Per costruire l'albero attribuito ci comportiamo similmente al caso precedente.
Associamo a T' un attributo ereditato nel quale andiamo a copiare il valore della F che sta a sinistra. Dobbiamo però capire come far arrivare il valore 15 alla T. La soluzione è la seguente:



Dove ogni $.s$ rappresenta un attributo sintetizzato del nodo, e ogni $.i$ rappresenta un attributo ereditato del nodo.
Questo grafo mostra come "viaggiano" le cose nell'albero annotato. Dunque i nodi di tipo T' oltre agli attributi sintetizzati anche hanno degli attributi ereditati.
Vediamo quindi come, anche in questo caso, sia necessario utilizzare gli attributi ereditati, che però aggiungono un livello di complessità.

Prendiamo ora in considerazione varie porzioni di albero e vediamo le produzioni con le loro azioni semantiche associate, ricordando che quando studiamo la regola per una produzione dobbiamo usare solo le informazioni che abbiamo a quel livello, in quella produzione:





$T \to FT' \qquad \{T.s = T'.s;\ \ T'.i = F.s\}$ $\qquad\qquad$ $T' \to *FT'_1 \qquad \{T'.s = T'_1.s;\ \ T'_1.i = T'.i * F.s\}$

Il trucco qui è usare attributi ereditati per trasferire informazioni dai sottoalberi sulla sinistra a quelli sulla destra. Questi dati vengono trasportati finché non sono necessari per i calcoli che ci interessano di più, ossia quelli più a destra. Solo a quel punto, queste informazioni possono risalire verso la radice dell'albero.

Ora che abbiamo studiato i due punti critici dell'SDD per questa grammatica riportiamo la L-attributed SDD nella sua completezza:

$$
\begin{array}{ll}
V \to E & \{V.s = E.s\} \\
E \to TE' & \{E.s = E'.s;\ \ E'.i = T.s\} \\
E' \to +TE'_1 & \{E'.s = E'_1.s;\ \ E'_1.i = E'.i + T.s\} \\
E' \to \epsilon & \{E'.s = E'.i\} \\
T \to FT' & \{T.s = T'.s;\ \ T'.i = F.s\} \\
T' \to *FT'_1 & \{T'.s = T'_1.s;\ \ T'_1.i = T'.i * F.s\} \\
T' \to \epsilon & \{T'.s = T'.i\} \\
F \to (E) & \{F.s = E.s\} \\
F \to digit & \{F.s = digit.lexval\}
\end{array}
$$

Il vantaggio di utilizzare il parsing bottom-up è che è possibile effettuare la valutazione dell'albero di derivazione durante la sua costruzione.
La sfida dunque consiste nell'implementare la traduzione durante il parsing (rispetto all'ottenere l'albero di parsing, quindi annotarlo, e poi valutare l'albero di parsing annotato).
L'implementazione più semplice avviene quando la grammatica può essere analizzata dall'algoritmo *shift-reduce* e la SDD sottostante è S-attributed:

- Insieme allo stack di stato $stSt$ e allo stack di simboli $symSt$, si utilizza uno stack semantico $semSt$ per conservare (record di) attributi.
- Si esegue il codice associato a $A \to \beta$ quando la produzione viene ridotta.
  Gli attributi necessari dei simboli in cima a $symSt$ si trovano nelle posizioni corrispondenti in cima a $semSt$.

Le SDDs (Syntax-Directed Definitions) sono alla base degli schemi **Syntax-Directed Translation** (SDT), in cui il calcolo degli attributi avviene insieme a frammenti di codice che li utilizzano.
Ad esempio, se interpretiamo il segno = come un'assegnazione, allora

$$
\begin{array}{ll}
V \to E & \{print(E.val)\} \\
E \to E_1 + T & \{E.val = E_1.val + T.val\} \\
E \to T & \{E.val = T.val\} \\
T \to T_1 * F & \{T.val = T_1.val * F.val\} \\
T \to F & \{T.val = F.val\} \\
F \to (E) & \{F.val = E.val\} \\
F \to digit & \{F.val = digit.lexval\}
\end{array}
$$

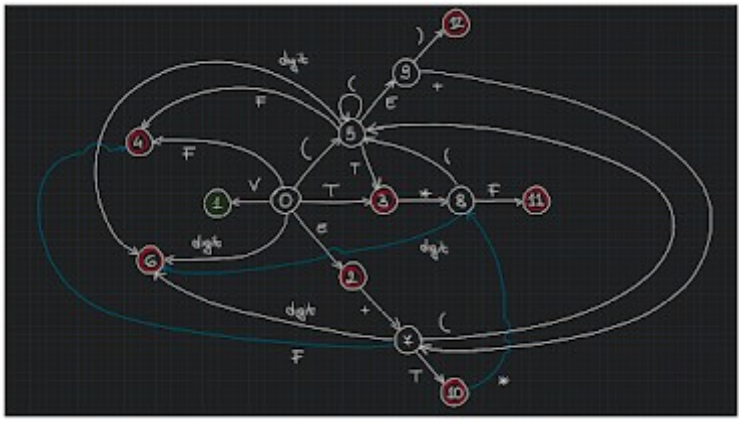
È una Syntax-Directed Translation (SDT).

Le Syntax-Directed Definitions (SDDs) e le Syntax-Directed Translations (SDTs) sono concetti correlati ma con differenze chiave:

1. Syntax-Directed Definitions (SDDs): Queste definiscono il modo in cui vengono associate azioni o attributi ai nodi dell'albero di sintassi durante il parsing. Le SDDs specificano le regole di produzione della grammatica e le azioni o attributi associati ad esse.
2. Syntax-Directed Translations (SDTs): Sono un tipo specifico di SDD. Le SDTs sono regole o specifiche che guidano la traduzione da una forma di input (tipicamente un linguaggio di programmazione) a una forma di output (ad esempio, un codice intermedio o un'altra forma del linguaggio). Le SDTs sono un modo per eseguire la traduzione durante il processo di parsing stesso.

In breve, le SDDs definiscono le azioni associate alla sintassi, mentre le SDTs si concentrano sulla traduzione della sintassi in un altro linguaggio o forma durante il parsing. Le SDTs sono una sottoclasse delle SDDs, poiché forniscono una specifica di come tradurre durante il processo di analisi sintattica.

Vediamo un esempio di esecuzione dell'algoritmo *shift-reduce* con valutazione sull'input $digit + digit$ per la grammatica LALR(1) delle espressioni aritmetiche.
Ecco il DFA e la relativa tabella di parsing:



	$+$	$*$	$($	$)$	$digit$	\$	V	E	T	F
0			s5		s6		1	2	3	4
1						Acc				
2	s7					r1				
3	r3	s8		r3		r3				
4	r5	r5		r5		r5				
5			s5		s6			9	3	4
6	r7	r7		r7		r7				
7			s5						10	
8			s5		s6					11
9	s7			s12						
10	r2	s8		r2		r2				
11	r4	r4		r4		r4				
12	r6	r6		r6		r6				

Tabella 10.1: Tabella di parsing LALR(1) Espressioni Aritmetiche

Supponiamo che il primo digit abbia valore lessicale pari a 3, e il secondo pari a 4.

$$
\begin{aligned}
w &= digit + digit\$ \\
stSt &= 0 \\
symSt &= \\
semSt &=
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit} + digit\$ \\
stSt &= 0\ 6 \\
symSt &= digit \\
semSt &= 3 \qquad\qquad \texttt{push}\ digit.lexval
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit} + digit\$ \\
stSt &= 0\ 4 \\
symSt &= F \qquad\qquad \texttt{reduce}\ F \to digit \\
semSt &= 3 \qquad\qquad \texttt{pop}\ digit.lexval;\ \texttt{push}\ F.val = digit.lexval
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit} + digit\$ \\
stSt &= 0\ 3 \\
symSt &= T \qquad\qquad \texttt{reduce}\ T \to F \\
semSt &= 3 \qquad\qquad \texttt{pop}\ F.val;\ \texttt{push}\ T.val = F.val
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit} + digit\$ \\
stSt &= 0\ 2 \\
symSt &= E \qquad\qquad \texttt{reduce}\ E \to T \\
semSt &= 3 \qquad\qquad \texttt{pop}\ T.val;\ \texttt{push}\ E.val = T.val
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit + digit}\$ \\
stSt &= 0\ 2\ 7 \\
symSt &= E + \\
semSt &= 3\ 0 \qquad\qquad \textbf{push}\ dummy
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit + digit}\$ \\
stSt &= 0\ 2\ 7\ 6 \\
symSt &= E + digit \\
semSt &= 3\ 0\ 4 \qquad\qquad \textbf{push}\ digit.lexval
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit + digit}\$ \\
stSt &= 0\ 2\ 7\ 4 \\
symSt &= E + F \qquad\qquad \textbf{reduce}\ F \to digit \\
semSt &= 3\ 0\ 4 \qquad \textbf{pop}\ digit.lexval;\ \textbf{push}\ F.val = digit.lexval
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit + digit}\$ \\
stSt &= 0\ 2\ 7\ 10 \\
symSt &= E + T \qquad\qquad \textbf{reduce}\ T \to F \\
semSt &= 3\ 0\ 4 \qquad \textbf{pop}\ F.val;\ \textbf{push}\ T.val = F.val
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit + digit}\$ \\
stSt &= 0\ 2 \\
symSt &= E \qquad\qquad \textbf{reduce}\ E \to E + T \\
semSt &= 7 \qquad \textbf{pop}\ T.val;\ \textbf{pop}\ dummy;\ \textbf{pop}\ E_1.val; \\
&\qquad\qquad\qquad \textbf{push}\ E = E_1.val + T.val
\end{aligned}
$$

$$
\begin{aligned}
w &= \underline{digit + digit}\$ \\
stSt &= 0\ 1 \\
symSt &= V \qquad\qquad \textbf{reduce}\ V \to E;\ Acc \\
semSt &= 7 \qquad \textbf{pop}\ E.val;\ print(E.val)
\end{aligned}
$$

Siccome desidero che la pila semantica aumenti in modo congruente alla pila dei simboli per mantenere l'allineamento tra i simboli e i valori degli attributi, quando si legge il simbolo "+", si inserisce un valore “dummy”, come 0.

Vediamo ora alcuni esempi di SDD che ci permettono di acquisire maggiore consapevolezza e confidenza.

Primo esempio: L’obiettivo è tradurre stringhe di cifre nei loro valori numerici decimali.
Prendiamo in considerazione la seguente SDD:

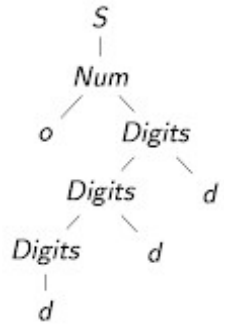
$S \to Digits$	$\{print(Digits.v)\}$
$Digits \to Digits_1\ d$	$\{Digits.v = Digits_1.v * 10 + d.lexval\}$
$Digits \to d$	$\{Digits.v = d.lexval\}$

La grammatica è LALR(1), l'SDD è S-attributed e dunque gli attributi possono essere valutati facilmente.

Secondo esempio: L'obiettivo è fare in modo che se la stringa di cifre è preceduta dal terminale o, di tradurla al suo valore numerico in ottale; altrimenti, di tradurla al suo valore numerico decimale.
Come primo tentativo proviamo semplicemente ad aggiungere una produzione alla grammatica di prima per generare stringhe di cifre precedute da o:

$$S \to Num$$
$$Num \to o\ Digits \mid Digits$$
$$Digits \to Digits\ d \mid d$$

La grammatica è ancora LALR(1), però durante il calcolo dei $Digits$, non ho accesso alla o a sinistra.



In sintesi, le informazioni necessarie per ottenere la traduzione corretta non sono disponibili da sotto e non possono essere sintetizzate.
Pertanto, non posso affidarmi esclusivamente agli attributi sintetizzati. Questa grammatica, quindi, non è adatta per noi (supponendo di voler utilizzare solamente attributi sintetizzati).

Come secondo tentativo proviamo a clonare le produzioni per separare la gestione dei numeri ottali da quella dei numeri decimali.
Utilizziamo quindi un non-terminale specifico (O) per facilitare il calcolo dei numeri ottali.

$$\begin{array}{ll} S \to Num & \{print(Num.v)\} \\ Num \to o\ O & \{Num.v = O.v\} \\ Num \to D & \{Num.v = D.v\} \\ O \to O_1\ d & \{O.v = O_1.v * 8 + d.lexval\} \\ O \to d & \{O.v = d.lexval\} \\ D \to D_1\ d & \{D.v = D_1.v * 10 + d.lexval\} \\ D \to d & \{D.v = d.lexval\} \end{array}$$

La grammatica è LALR(1), è ottima per la sintetizzazione però è ridondante.
La grammatica risulta ridondante perché le produzioni di O e di D sono identiche, differenziandosi solo per l'operazione.

Come terzo tentativo proviamo ad introdurre produzioni unitarie aggiuntive usate eslusivamente per forzare azioni semantiche.

$$
\begin{array}{ll}
S \to Num & \{print(Num.v)\} \\
Num \to Octal\ Digits & \{Num.v = Digits.v\} \\
Num \to Decimal\ Digits & \{Num.v = Digits.v\} \\
Octal \to o & \{base = 8\} \\
Decimal \to \epsilon & \{base = 10\} \\
Digits \to Digits_1\ d & \{D.v = D_1.v * base + d.lexval\} \\
Digits \to d & \{Digits.v = d.lexval\}
\end{array}
$$

È evidente l'utilizzo di una variabile globale denominata $base$. Il problema principale risiede appunto nell'impiego di variabili globali, in quanto se combiniamo questa grammatica con altre che fanno uso di variabili globali con lo stesso nome, si generano conflitti e problemi di gestione.

Come quarto tentativo facciamo in modo che la base appropriata venga catturata da un attributo sintetizzato.

$$
\begin{array}{ll}
S \to D & \{print(D.v)\} \\
D \to D\ d & \{D.v = D_1.v * D_1.base + d.lexval; \\
 & \ D.base = D_1.base\} \\
D \to B\ d & \{D.v = d.lexval; \\
 & \ D.base = B.val\} \\
B \to o & \{B.val = 8\} \\
B \to \epsilon & \{B.val = 10\}
\end{array}
$$

Con questo approccio, notiamo che il terminale o viene ridotto prima di qualsiasi altra cosa. Modifico dunque il valore della base $B.val$: quando effettuo la produzione $D \to Bd$, indico che il valore di D è uguale a $d.lexval$, ma copio anche il valore della base in D, in modo che sia visibile anche agli elementi superiori. Infatti poi tramite la produzione $D \to Dd$ utilizzo proprio questo attributo sintetizzato come base.
In questo modo, in ogni istante, ho accesso alla base senza la necessità di utilizzare variabili globali.

In sostanza, il punto principale è che le grammatiche e le azioni semantiche sono strettamente interdipendenti. Modifichiamo la grammatica non solo per ottenere un certo linguaggio, ma anche per catturare specifiche proprietà necessarie per l'elaborazione.

Vediamo ora un altro problema: La conversione da integer a float.
Prendiamo in considerazione la seguente grammatica:

$$E \to E\ dop\ T \mid T$$
$$T \to num \mid num.num$$

L'obiettivo è quello di aggiungere azioni semantiche per calcolare il valore dell'espressione in modo che:

- Se l'espressione contiene solo numeri interi (terminale num), allora dop (division operator) viene tradotto nell'operatore di divisione intera div.

- Se l'espressione contiene almeno un numero reale (terminale $num.num$), l'intera espressione viene convertita in un'espressione in virgola mobile, con i numeri interi castati in numeri in virgola mobile. Ad esempio, $5\ \ dop\ \ 2\ \ dop\ \ 2.0$ viene convertito in $float(5)\ \ dop\ \ float(2)\ \ dop\ \ 2.0$.

Ci sono diverse sfide qui rispetto a quello che abbiamo visto precedentemente: probabilmente sarà necessario esaminare più roba rispetto a solamente il sottoalbero. Ad esempio, in $5\ \ dop\ \ 2\ \ dop\ \ 2.0$, fino a quando non si raggiunge 2.0, non è possibile prendere una decisione. In sostanza, è necessario esaminare l'intero insieme di nodi foglia prima di determinare se eseguire la conversione o meno. Senza attributi ereditati o qualche tipo di contesto aggiuntivo, è impossibile procedere in questo caso.

Abbiamo bisogno di sapere se l'albero di parsing contiene delle foglie $num.num$ e possiamo calcolare il valore dell'espressione solo quando sappiamo se gli operandi devono essere promossi a numeri in virgola mobile o no.
Quindi utilizziamo un attributo booleano $float$ che fluisce dalle foglie alla radice (attributo sintetizzato).

$$E \to E_1\ dop\ T \qquad E.float = E_1.float\ \ OR\ \ T.float; \ldots$$

$$E \to T \qquad E.float = T.float; \ldots$$

$$T \to num \qquad T.float = false; \ldots$$

$$T \to num.num \qquad T.float = true; \ldots$$

Notiamo che impostiamo questo attributo a true quando abbiamo $num.num$ e che quando dobbiamo eseguire l'operazione, impostiamo l'attributo dell'operazione come OR degli attributi degli operandi.

Se l'albero ha una qualsiasi foglia $num.num$, allora l'attributo booleano $float$ della radice è vero.
Una volta conosciuto l'attributo $float$ della radice, possiamo decidere il tipo dell'intera espressione e farlo fluire verso le foglie attraverso l'attributo $type$ (attributo ereditato).
Per essere certi di decidere il tipo dell'espressione esclusivamente alla radice dell'albero di parsing (piuttosto che in qualche nodo E che non è radice, e dunque non può accedere a tutte le foglie dell'albero), aggiungiamo la produzione extra $S \to E$ per distinguere il nodo E che funge da radice. Questo lo facciamo in quanto E è impiegata nella grammatica anche nel body di qualche produzione, e dunque posso trovarla sia come radice sia come nodo intermedio.
Questo ragionamento vale anche quando vogliamo fare l'operazione di $print$ quando arriviamo alla radice.
L'attributo $type$ per E viene impostato in questa produzione extra.
Facciamo dunque fluire le informazioni in maniera top-down in questo caso:

$$
\begin{array}{ll}
S \to E & E.type = IF\ E.float\ THEN\ float\ ELSE\ int; \ldots \\
E \to E_1\ dop\ T & E_1.type = E.type;\ T.type = E.type; \ldots \\
E \to T & T.type = E.type; \ldots \\
T \to num & \ldots \\
T \to num.num & \ldots
\end{array}
$$

Il valore dell'espressione, che dipende dal suo tipo, viene calcolato nell'attributo *val* (attributo sintetizzato).
L'attributo *val* fluisce le informazioni in maniera bottom-up:

$$
\begin{array}{ll}
S \to E & S.val = E.val; \ldots \\
E \to E_1\ dop\ T & E.val = IF\ E.type = int \\
 & \quad THEN\ E_1.val\ div\ T.val \\
 & \quad ELSE\ E_1.val / T.val; \ldots \\
E \to T & E.val = T.val; \ldots \\
T \to num & T.val = IF\ T.type = int \\
 & \quad THEN\ num.lexval \\
 & \quad ELSE\ float(num.lexval); \ldots \\
T \to num.num & T.val = num.num.lexval; \ldots
\end{array}
$$

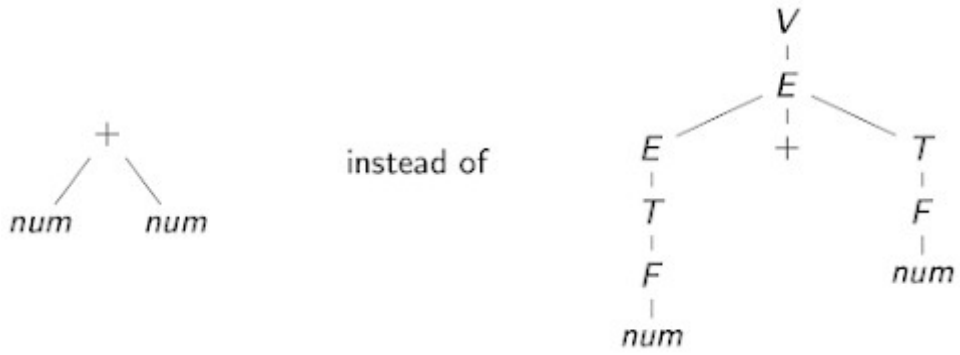
Ora possiamo:
-Raccogliere tutte le azioni semantiche e ottenere la SDD
-Ottenere l'albero di parsing annotato
-Valutare

Alberi di sintassi astratta

Gli **alberi di sintassi astratta** sono una specie di riassunto più compatto degli alberi di analisi grammaticale. Sono una sorta di rappresentazione intermedia che non segue regole fisse, ma varia in base alla grammatica specifica che si sta utilizzando e alle scelte fatte nell'implementazione del linguaggio (il tipo di analisi). L'importante è che questi alberi contengano tutte le informazioni necessarie per continuare l'analisi del linguaggio o del testo in questione.

L'albero di sintassi astratta è una versione compatta derivata dall'albero di analisi. Si definiscono "astratti" perché contengono solo le informazioni cruciali per il compimento di un'operazione specifica. In pratica, conservano solo ciò che è veramente rilevante.

Ecco un esempio:



Dunque le uniche informazioni “essenziali”, in questo caso, sono gli operandi e il tipo di operazione.

Vediamo subito un esempio di costruzione, costruendo un albero di sintassi astratta per le espressioni aritmetiche.
Dobbiamo dunque fornire una grammatica S-attribuita che permetta la costruzione dell'albero di sintassi astratta durante il processo di parsing (in maniera bottom-up dunque).
Assumiamo che esistano due funzioni (senza interessarci su come sono implementate):

- La funzione $newLeaf(label, val)$ crea un oggetto foglia con due campi, la sua etichetta (label) e il suo valore (val).
- La funzione $newNode(label, c_1, \ldots, c_k)$ crea un nodo interno con un'etichetta (label) e k figli, dove $c_1, \ldots, c_k$ sono riferimenti ai nodi figli.

Queste funzioni vengono utilizzate per creare l'albero di sintassi astratta mentre si analizza il testo, consentendo di associare informazioni specifiche a ciascun nodo dell'albero durante il processo di parsing.

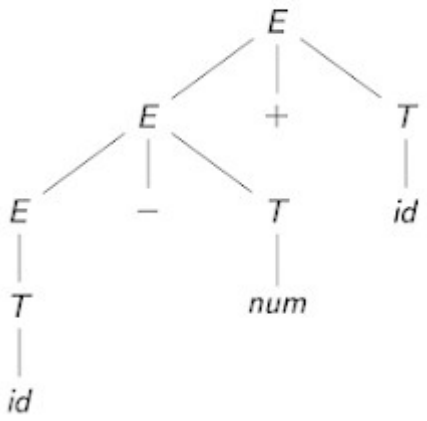
Possiamo organizzare la SDD nella seguente maniera:

$E \to E_1 + T$	$\{E.node = newNode('+', E_1.node, T.node)\}$
$E \to E_1 - T$	$\{E.node = newNode('-', E_1.node, T.node)\}$
$E \to T$	$\{E.node = T.node\}$
$T \to (E)$	$\{T.node = E.node\}$
$T \to id$	$\{T.node = newLeaf(id, id.entry)\}$
$T \to num$	$\{T.node = newLeaf(num, num.lexval)\}$

Dunque quando affrontiamo la produzione $T \to num$, creiamo un nodo foglia e manteniamo il riferimento a questo nodo, il quale conterrà direttamente il valore di num. Nel caso di $T \to id$, $id.entry$ è il puntatore alla tabella dei simboli che raccoglie tutte le informazioni rilevanti che abbiamo raccolto fino a quel punto.
Per le altre regole il significato, abbastanza intuitivo, è quello della creazione di un nodo intermedio che mantiene come informazioni l’etichetta dell’operazione da compiere e i riferimenti ai nodi degli operandi necessari.

Vediamo ora come funziona la costruzione durante il parsing.

Prendiamo in considerazione l'analisi di $id - num + id$, ottenuta dal processo di analisi lessicale di $a - 4 + c$.



Reductions:

1. $T \rightarrow id$
2. $E \rightarrow T$
3. $T \rightarrow num$
4. $E \rightarrow E - T$
5. $T \rightarrow id$
6. $E \rightarrow E + T$

L’ordine di parsing è quello riportato qui sopra.

First reduction:

$T \rightarrow id \qquad \{T.node = newLeaf(id, id.entry)\}$



Quindi, il primo evento che si verifica è $T \rightarrow id$. L'azione semantica corrispondente è la creazione di un nuovo nodo foglia. L'indirizzo di questa foglia è identificato come $T.node$. Essendo una variabile (identificatore), il nodo contiene un puntatore alla tabella dei simboli.

Second reduction:

$$E \to T \qquad \{E.node = T.node\}$$



Successivamente, copiamo $T.node$ nell'attributo di E, facendo riferimento allo stesso nodo (è equivalente ad una rinominazione).

Third reduction:

$$T \to num \qquad \{T.node = newLeaf(num, num.lexval)\}$$



Durante la riduzione per *num*, inseriamo una nuova foglia senza puntatori, poiché 4 è statico.

Fourth reduction:

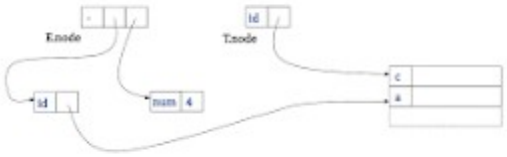
$$E \to E_1 - T \qquad \{E.node = newNode('-', E_1.node, T.node)\}$$



Successivamente, quando eseguiamo la riduzione $E \rightarrow E\text{-}T$, creiamo un nuovo nodo in cui il campo valore è -, e i figli sono i riferimenti ai nodi di E e di T.

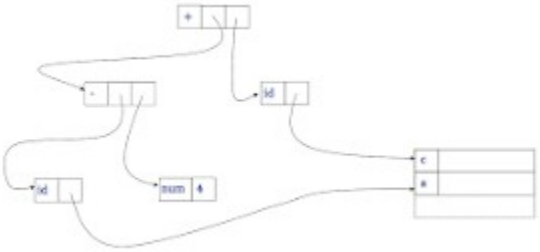
Fifth reduction:

$$T \rightarrow id \qquad \{T.node = newLeaf(id, id.entry)\}$$



Last reduction:

$$E \rightarrow E_1 + T \qquad \{E.node = newNode(`+', E_1.node, T.node)\}$$



Possiamo dunque creare un albero di sintassi astratta durante il parsing utilizzando proprio un approccio del genere. Se volessimo invece costruire un albero di derivazione classico, dovremmo creare un nodo per ogni produzione che osserviamo durante il processo.

Vediamo ora come costruire l'albero di sintassi astratta per una grammatica top-down. Prendiamo in considerazione la seguente grammatica LL:

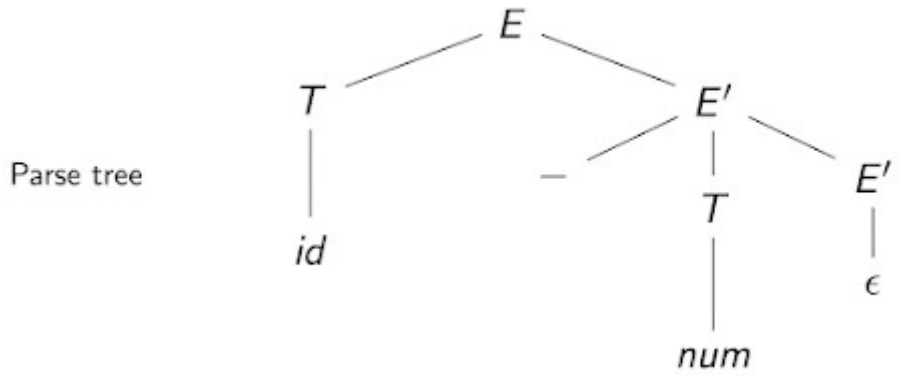
$$\begin{aligned}
&E \rightarrow TE' \\
&E' \rightarrow +TE'_1 \\
&E' \rightarrow -TE'_1 \\
&E' \rightarrow \epsilon \\
&T \rightarrow (E) \\
&T \rightarrow id \\
&T \rightarrow num
\end{aligned}$$

Utilizziamo attributi sintetizzati e ereditati seguendo una strategia simile a quella impiegata per calcolare il valore dell'espressione di un'espressione aritmetica. In questo caso, invece di valori, passiamo riferimenti ai nodi tra loro. Otteniamo così l'albero di parsing annotato.
Successivamente, definiamo il grafo delle dipendenze e ne troviamo un ordinamento topologico tra i suoi nodi. Infine, durante l'operazione di valutazione, costruiamo l'albero di sintassi astratta. In questo caso infatti non possiamo costruire l'albero di sintassi astratta durante il parsing, visto che ci sono attributi ereditati.

La SDD è la seguente:

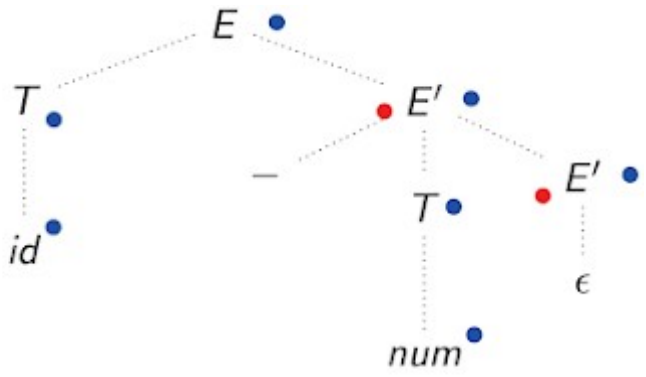
$E \to TE'$	$\{E.node = E'.node;$ $E'.i = T.node\}$
$E' \to +TE'_1$	$\{E'.node = E'_1.node;$ $E'_1.i = newNode('+', E'.i, T.node)\}$
$E' \to -TE'_1$	$\{E'.node = E'_1.node;$ $E'_1.i = newNode('-', E'.i, T.node)\}$
$E' \to \epsilon$	$\{E'.node = E'.i\}$
$T \to (E)$	$\{T.node = E.node\}$
$T \to id$	$\{T.node = newLeaf(id, id.entry)\}$
$T \to num$	$\{T.node = newLeaf(num, num.lexval)\}$

Vediamo ora come funziona la costruzione durante il parsing.
Prendiamo in considerazione l'analisi di id - num, ottenuta dal processo di analisi lessicale di a - 4.



L'albero di parsing annotato è il seguente:

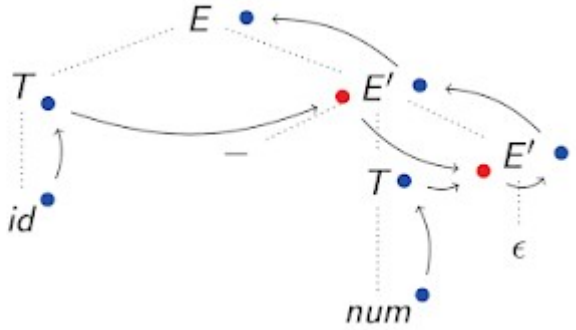
Annotated parse tree



Where • stands for the inherited attribute of the tree node and • stands for its synthesized attribute

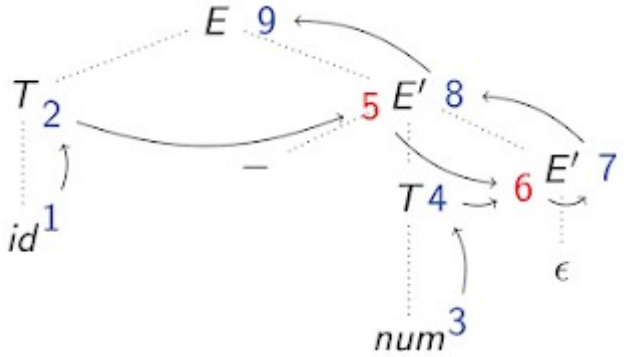
Il grafo di dipendenza è il seguente:

Dependency graph



Dunque un ordinamento topologico possibile è:

Topological sort



La valutazione viene fatta nel seguente modo:

1: *id.entry* (reference to the entry for *a* in the symbol table)

2: $T.node = newLeaf(id, 1)$

3: 4 (by $num.lexval = 4$)

4: $T.node = newLeaf(num, 3)$

5: $E'.i = 2$

6: $E'.i = newNode('-', 5, 4)$

7: reference to $Node('-', 5, 4)$
$By\ E'.node = E'.i\ for\ E' \to \epsilon$

8: reference to $Node('-', 5, 4)$
$By\ E'.node = E'_1.node\ for\ E' \to -TE'_1$

9: reference to $Node('-', 5, 4)$
$By\ E.node = E'.node\ for\ E \to TE'$

Complessivamente:
$E.node$ è un riferimento a $Node('-', 5, 4)$, dove:

- 5: reference to $Leaf(id, \text{entry for } a)$

- 4: reference to $Leaf(num, 4)$

Tabella dei simboli

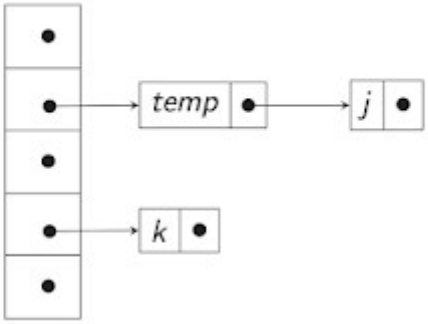
Le **tabelle dei simboli** sono strutture dati cruciali all'interno di un compilatore, poste dopo i syntax tree in ordine di importanza.
Rappresentano un punto chiave per gli attributi ereditati e solitamente utilizzano strutture dati di tipo dizionario. Le operazioni principali includono l'inserimento, la ricerca e la cancellazione.
Le implementazioni tipiche comprendono liste lineari, alberi di ricerca e tabelle hash. Spesso, le tabelle hash sono la scelta preferita in quanto consentono di eseguire quasi tutte le operazioni in tempo quasi lineare, rendendole particolarmente efficienti.

Le **tabelle hash** gestiscono le collisioni tra elementi attraverso due principali strategie.

- Con il metodo dell'**indirizzamento aperto** (open addressing), ogni "secchio" ha spazio sufficiente per un solo elemento; in caso di collisione, gli elementi collidenti vengono inseriti in secchi successivi. Tuttavia, le dimensioni della tabella sono limitate dalla grandezza dell'array dei secchi, il che può portare a prestazioni scadenti in termini di efficienza.
- D'altra parte, con l'approccio del **chaining separato** (separate chaining), ogni secchio è una lista lineare e le collisioni di hash vengono risolte inserendo il nuovo elemento nella lista corrispondente. Questo metodo permette una gestione più flessibile delle collisioni senza limitazioni legate alle dimensioni degli array, consentendo di gestire efficacemente le collisioni senza compromettere le prestazioni.

Ecco un esempio di chaining separato:

For example, if *temp* and *j* have the same hash value, then:



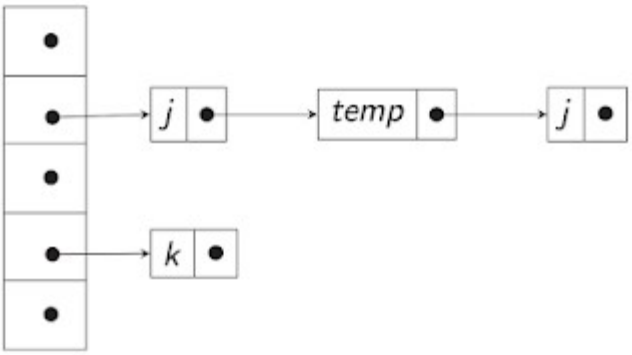
Le funzioni di hash mirano a trasformare una stringa di caratteri, come ad esempio il nome di un identificatore, in un numero intero compreso tra 0 e ¿ 1, dove size rappresenta la dimensione della tabella e dell'array dei secchi. Questo processo è guidato da diverse tattiche:

- Conversione dei singoli caratteri in numeri non negativi tramite i meccanismi di conversione integrati nel linguaggio di programmazione utilizzato per implementare il compilatore.
- Applicazione di una funzione di hash adeguata, questo perché spesso i nomi di variabili vengono assegnati a gruppi (ad esempio $temp\,1$, $temp\,2$, …). Nella generazione delle funzioni di hash, si considerano tutti i caratteri di questi nomi per evitare un'eccessiva quantità di collisioni.
- Scelta tipica e efficiente: $h =$ ¿. Dove c_i è il valore numerico dell'i-esimo carattere e ϕ rappresenta una potenza di 2, permettendo così di utilizzare una moltiplicazione che può essere eseguita come uno shift.

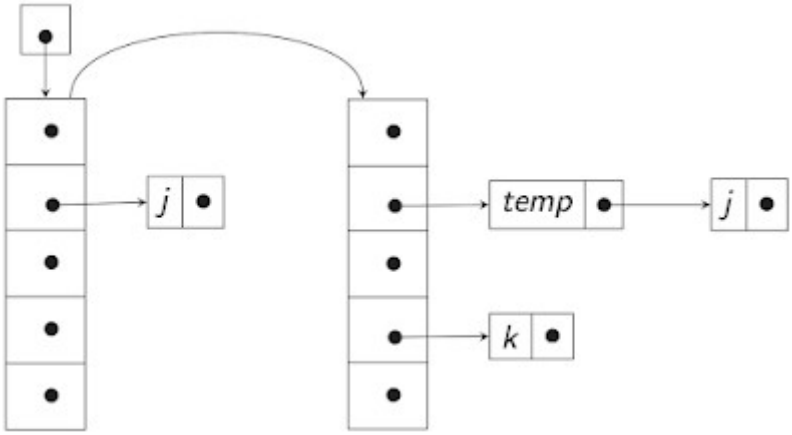
In sostanza, le funzioni di hash sono progettate per convertire in modo efficiente stringhe di caratteri in numeri interi, consentendo una distribuzione uniforme degli identificatori all'interno della tabella, riducendo al minimo le collisioni e migliorando le prestazioni del compilatore nel gestire grandi quantità di dati.

Nel contesto della programmazione, i nomi sono associati a dichiarazioni, che possono riguardare costanti, variabili o funzioni. Questi nomi possono avere una portata locale o globale. Le dichiarazioni locali limitano la visibilità dei nomi a una porzione specifica del programma, mentre le dichiarazioni globali rendono i nomi visibili in tutto il programma.
È possibile dichiarare lo stesso nome più volte; in tal caso, l'uso del nome fa riferimento alla sua dichiarazione più vicina.
Lo **scope** di una dichiarazione corrisponde a un sotto-albero dell'albero di sintassi. Le dichiarazioni nidificate generano portate che sono sotto-alberi annidati della struttura di sintassi. Questo implica che la visibilità e l'utilizzo dei nomi sono definiti dalla struttura gerarchica dell'albero di sintassi, dove le dichiarazioni più interne (o "annidate") possono avere un'area di influenza limitata, specifica a quella porzione di codice.

Una soluzione consiste nel gestire la tabella hash in modo simile a uno stack.



Un'altra soluzione è quella di costruire una nuova tabella dei simboli per ciascun ambito e collegare le tabelle dai contesti interni a quelli esterni.



Vediamo ora un ultimo esercizio, che consiste nell' aggiungere i tipi nella tabella dei simboli.

Dobbiamo progettare delle azioni semantiche per ottenere una SDD che permetta di aggiungere il tipo degli identificatori dichiarati (int o float) alle loro voci nella tabella dei simboli.
La grammatica è la seguente:

$$D \rightarrow TL$$
$$T \rightarrow int$$
$$T \rightarrow float$$
$$L \rightarrow L_1, id$$
$$L \rightarrow id$$

Per aggiornare la tabella dei simboli, si utilizza la funzione $addtype(table\ \ entry, type\ \ instance)$ dove $type\ \ instance$ può essere sia $\int$ ¿ che $float$. Questa funzione è utilizzata per associare i tipi (int o float) alle voci della tabella dei simboli, in base alle azioni semantiche definite dalla grammatica.

La SDD è la seguente:

$$
\begin{array}{ll}
D \to TL & \{L.i = T.type\} \\
T \to int & \{T.type = int\} \\
T \to float & \{T.type = float\} \\
L \to L_1, id & \{L_1.i = L.i;\ addtype(id.entry, L.i)\} \\
L \to id & \{addtype(id.entry, L.i)\}
\end{array}
$$

Generazione codice intermedio

L'obiettivo finale di un compilatore è far eseguire programmi scritti in un linguaggio ad alto livello su un computer. Ciò significa che, alla fine, il programma dovrà essere espresso come codice macchina che può essere eseguito sul computer. Questo non significa che dobbiamo tradurre direttamente dalla sintassi astratta ad alto livello al codice macchina. Molti compilatori utilizzano un **linguaggio di livello intermedio** come ponte tra il linguaggio ad alto livello e il codice macchina molto basso. Tali linguaggi intermedi sono chiamati **codice intermedio**.

Oltre a strutturare il compilatore in lavori più piccoli, l'utilizzo di un linguaggio intermedio ha altri vantaggi:

- Se il compilatore deve generare codice per diverse architetture di macchine diverse, è necessaria una sola traduzione in codice intermedio. Solo la traduzione da codice intermedio a linguaggio macchina (cioè, il back-end) deve essere scritta in diverse versioni.
- Se diversi linguaggi ad alto livello devono essere compilati, è necessario scrivere solo la traduzione in codice intermedio per ciascun linguaggio. Possono tutti condividere il back-end, cioè la traduzione da codice intermedio a codice macchina.
- Invece di tradurre il linguaggio intermedio in codice macchina, può essere interpretato da un piccolo programma scritto in codice macchina o in un linguaggio per il quale esista già un compilatore o un interprete.

Il vantaggio nell'utilizzare un linguaggio intermedio è più evidente se molti linguaggi devono essere compilati per molte macchine. Se la traduzione viene fatta direttamente, il numero di compilatori è pari al prodotto tra il numero di linguaggi e il numero di macchine. Se viene utilizzato un linguaggio intermedio comune, è necessario un front-end (ossia, un compilatore per il codice intermedio) per ogni linguaggio e un back-end (interprete o generatore di codice) per ogni macchina, rendendo il numero totale di front-end e back-end pari alla somma del numero di linguaggi e il numero di macchine.

Se un interprete per un linguaggio intermedio è scritto in un linguaggio per il quale esistono già implementazioni per le macchine di destinazione, lo stesso interprete può essere interpretato o compilato per ogni macchina. In questo modo, non è necessario scrivere un back-end separato per ogni macchina. I vantaggi di questo approccio sono:

- Non è necessario scrivere effettivamente un back-end per ogni nuova macchina, a patto che la macchina sia dotata di un interprete o compilatore per il linguaggio di implementazione dell'interprete per il linguaggio intermedio.
- Un programma compilato può essere distribuito in una singola forma intermedia per tutte le macchine, invece di spedire binari separati per ogni macchina.
- La forma intermedia può essere più compatta del codice macchina. Questo risparmia spazio sia nella distribuzione che sulla macchina che esegue i programmi (anche se quest'ultimo è in parte compensato dalla necessità di mantenere l'interprete in memoria durante l'esecuzione).

Lo svantaggio è la velocità: interpretare la forma intermedia sarà nella maggior parte dei casi molto più lento rispetto all'esecuzione diretta del codice tradotto. Tuttavia, questo approccio ha visto alcuni successi, ad esempio con Java.
Parte della penalità di velocità può essere eliminata traducendo il codice intermedio in codice macchina immediatamente prima o durante l'esecuzione del programma. Questa forma ibrida è chiamata compilazione just-in-time ed è spesso utilizzata per eseguire il codice intermedio per Java.
In questo libro, tuttavia, ci concentreremo principalmente sull'utilizzo del codice intermedio per la compilazione tradizionale, dove la forma intermedia sarà tradotta in codice macchina dal back-end del compilatore.

Con la generazione del codice intermedio completiamo la fasi di front-end di un compilatore. Riassumendo, le fasi sono:

- Analisi lessicale (distinzione dei lessemi e restituzione di una stringa di tokens; inoltre ha il compito di popolare la tabella dei simboli)
- Analisi sintattica (verifica che la stringa ottenuta dall’analisi lessicale appartiene al linguaggio e restituzione dell’albero di derivazione)
- Analisi semantica (si occupa di vari controlli statici tra cui: la compatibilità di operandi e operatori nei vari statements; la validità di istruzioni condizionali e di controllo)
- Generazione del codice intermedio

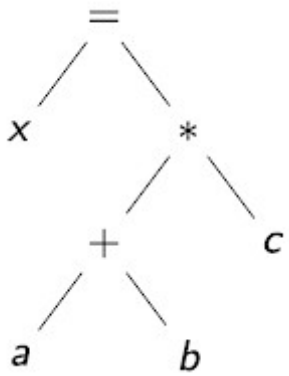
L'analisi semantica e la generazione del codice intermedio possono essere eseguite tramite **syntax-directed translations**, ovvero possono essere integrate direttamente nell’analisi sintattica.
Durante l'analisi sintattica possono essere implementati molti schemi di traduzione, tutti invece possono essere realizzati creando un albero di sintassi astratta e poi attraversandolo.

Il codice intermedio è una rappresentazione del nostro codice di partenza, abbastanza astratta da nascondere alcune specifiche che sono tipiche del codice macchina e che ne rendono la lettura poco agevole (come ad esempio i movimenti di informazioni tra registri o tra memoria e registri), ma comunque più specifica del linguaggio originale da cui siamo partiti.

Esistono diverse modalità per rappresentare il codice intermedio, ognuna con le proprie peculiarità. Di seguito, esploreremo le più comuni:

- **Strutture a grafo**: Una prima opzione è l'utilizzo di strutture a grafo derivate dai parse trees, come gli AST (Abstract Syntax Trees) o grafi diretti aciclici (DAG). Questi ultimi risultano spesso più compatti degli AST: considerando un AST con due nodi foglia che condividono lo stesso identificatore 'a', in un DAG avremmo un singolo nodo con più connessioni in ingresso. Gli AST, a loro volta, risultano più compatti dei parse trees, infatti i parse trees spesso richiedono percorsi più lunghi per rappresentare specifiche derivazioni, rallentando le operazioni successive.
- **Codice a tre indirizzi**: Un'alternativa leggermente migliore è l'utilizzo del cosiddetto "codice a tre indirizzi", così denominato perché limita ogni istruzione a tre valori, ad esempio nella forma $x = y \ \ op \ \ z$. Questo approccio riduce notevolmente la complessità delle istruzioni stesse.
- **Codice in un altro linguaggio**: Una terza opzione è adottare un altro linguaggio di programmazione come codice intermedio effettivo. Questo metodo evita la necessità di implementare un backend per la compilazione, poiché si può utilizzare la catena di compilazione del linguaggio selezionato come codice intermedio. In queste circostanze, la scelta comune è il linguaggio C, per la sua straordinaria flessibilità e l'eccellente compilatore gcc.

Noi vedremo il **codice a tre indirizzi**.
Il "codice a tre indirizzi" è una rappresentazione lineare (testuale) dei syntax trees.
Per capire di cosa stiamo parlando, andiamo subito a vedere un esempio di traduzione da AST a codice a tre indirizzi, prendiamo in considerazione un'operazione come $x = (a + b) * c$.



L'equivalente rappresentazione dell'AST nel codice a tre indirizzi può essere ottenuta salvando l'esito delle operazione intermedie in dei registri temporanei, come segue:

$$\begin{aligned} t_1 &= a + b \\ t_2 &= t_1 * c \\ x &= t_2 \end{aligned}$$

Questa rappresentazione è utile per generare il codice finale e per ottimizzarlo.
Avere una rappresentazione testuale di un AST, facendo uso del codice a tre indirizzi, è comodo perché semplifica la generazione di linguaggio macchina e la sua ottimizzazione; tuttavia questa tecnica non è molto frequente, perché è dipendente dalla macchina per cui stiamo scrivendo il backend (machine dependent).

Il "codice a tre indirizzi" permette diversi tipi di istruzioni:

- $a1 = a2 \ \ op \ \ a3$ (assegnazioni tramite operazioni)
- $a1 = op \ \ a2$ (operazioni unarie)
- $a1 = a2$ (assegnazioni semplici)
- $a1 = a2[a3]$ (accesso agli elementi di array)
- $a1[a2] = a3$ (assegnazione agli elementi di array)
- …
- $goto \ \ L$ (istruzioni di salto)
- $if \ \ a \ \ goto \ \ L$ (istruzioni di salto condizionali)
- …

Questa struttura offre una vasta gamma di istruzioni per esprimere operazioni e controlli nei programmi.

L'uso del codice intermedio aiuta a rappresentare in modo più semplice istruzioni complesse. Ad esempio prendiamo in considerazione l'istruzione

```
if ( x < 100 || x > 200 && x != y ) x=0;
```

Essa può essere tradotta in una serie di istruzioni più semplici utilizzando etichette di destinazione ($L1, L2, L3, L4$) e istruzioni di salto condizionate ($IF \dots GOTO$).

```
      IF x < 100 GOTO L2
      GOTO L3
L3:   IF x > 200 GOTO L4
      GOTO L1
L4:   IF x != y GOTO L2
      GOTO L1
L2:   x = 0
L1:
```

Questa traduzione spezza la complessità della condizione in una sequenza di istruzioni più gestibili.
Tuttavia, in questa traduzione abbiamo numerosi $GOTO$ ridondanti; vediamo dunque di presentare una traduzione leggermente migliore:

```
      IF x < 100 GOTO L2
      IFFALSE x > 200 GOTO L1
      IFFALSE x != y GOTO L1
L2:   x = 0
L1:
```

Il controllo sulla falsità di x ci permette di ridurre il numero di righe e labels utilizzate, nonché sfruttare meglio la cortocircuitazione, caratteristica fondamentale per un buon compilatore. Si tenga sempre a mente che non sempre le condizioni vengono valutate nello stesso esatto ordine in cui sono state scritte dal programmatore.

Scegliere un linguaggio intermedio

QUESTO CAPITOLO È FUORI DAL PROGRAMMA

Un linguaggio intermedio dovrebbe idealmente avere le seguenti proprietà:

- Dovrebbe essere facile tradurre da un linguaggio ad alto livello al linguaggio intermedio. Questo dovrebbe essere valido per una vasta gamma di diversi linguaggi sorgente.
- Dovrebbe essere facile tradurre dal linguaggio intermedio al codice macchina. Questo dovrebbe essere vero per una vasta gamma di diverse architetture di destinazione.
- Il formato intermedio dovrebbe essere adatto per ottimizzazioni.

Le prime due di queste proprietà possono essere difficili da conciliare. Un linguaggio destinato come obiettivo per la traduzione da un linguaggio ad alto livello dovrebbe essere piuttosto vicino a questo. Tuttavia, questo potrebbe essere difficile da raggiungere per più di un piccolo numero di linguaggi simili. Inoltre, un linguaggio intermedio ad alto livello mette più peso sui back-end. Un linguaggio intermedio a basso livello potrebbe rendere più facile scrivere back-end, ma mette più peso sui front-end. Un linguaggio intermedio a basso livello potrebbe inoltre non adattarsi bene a tutte le macchine, anche se questo è di solito meno problematico rispetto al problema simile per i front-end, poiché le macchine sono tipicamente più simili rispetto ai linguaggi ad alto livello.

Una soluzione che potrebbe ridurre il peso della traduzione, sebbene non affronti gli altri problemi, è avere due livelli intermedi: uno, piuttosto alto, utilizzato per i front-end, e l'altro, piuttosto basso, utilizzato per i back-end. Un singolo traduttore condiviso viene quindi utilizzato per tradurre tra questi due formati intermedi.

Quando il formato intermedio è condiviso tra molti compilatori, ha senso fare il maggior numero possibile di ottimizzazioni sul formato intermedio. In questo modo, lo sforzo (spesso sostanziale) di scrivere buone ottimizzazioni viene fatto solo una volta invece che in ogni compilatore.

Un'altra cosa da considerare nella scelta di un linguaggio intermedio è la "granularità": un'operazione nel linguaggio intermedio dovrebbe corrispondere a una grande quantità di lavoro o a una piccola quantità di lavoro?
Il primo di questi approcci è spesso utilizzato quando il linguaggio intermedio viene interpretato, poiché l'overhead di decodifica delle istruzioni viene ammortizzato su un maggior lavoro effettivo, ma può anche essere utilizzato per la compilazione. In questo caso, ciascuna operazione del codice intermedio viene tipicamente tradotta in una sequenza di istruzioni del codice macchina. Quando si utilizza un codice intermedio a “grossa granularità”, di solito vi è un numero piuttosto grande di diverse operazioni del codice intermedio.
L'approccio opposto è quello di rendere ogni operazione del codice intermedio il più piccola possibile. Ciò significa che ciascuna operazione del codice intermedio viene tipicamente tradotta in una singola istruzione del codice macchina o che diverse operazioni del codice intermedio possono essere combinate in un'unica operazione del codice macchina. Quest'ultimo può, in qualche misura, essere automatizzato poiché ciascuna istruzione del codice macchina può essere descritta come una sequenza di istruzioni del codice intermedio. Quando il codice intermedio viene tradotto in codice macchina, il generatore di codice può cercare sequenze che corrispondano a operazioni del codice macchina. Assegnando un costo a

ciascuna operazione del codice macchina, questo può essere trasformato in un problema di ottimizzazione combinatoria, dove si trova la soluzione con il costo minimo.

Il linguaggio intermedio

QUESTO CAPITOLO È FUORI DAL PROGRAMMA

In questo capitolo abbiamo scelto un linguaggio intermedio piuttosto a basso livello e dettagliato, poiché è il più adatto per illustrare le tecniche che vogliamo coprire.
Non tratteremo la traduzione delle chiamate di funzione, quindi un "programma" nel nostro linguaggio intermedio, per il momento, corrisponderà al corpo di una funzione o procedura in un programma reale. Per lo stesso motivo, le chiamate di funzione sono inizialmente trattate come operazioni primitive nel linguaggio intermedio.

La grammatica per il linguaggio intermedio è mostrata nella seguente figura.

$$
\begin{array}{lcl}
\textit{Program} & \rightarrow & [\,\textit{Instructions}\,] \\
 & & \\
\textit{Instructions} & \rightarrow & \textit{Instruction} \\
\textit{Instructions} & \rightarrow & \textit{Instruction}\ ,\ \textit{Instructions} \\
 & & \\
\textit{Instruction} & \rightarrow & \texttt{LABEL}\ \textbf{labelid} \\
\textit{Instruction} & \rightarrow & \textbf{id} := \textit{Atom} \\
\textit{Instruction} & \rightarrow & \textbf{id} := \textbf{unop}\ \textit{Atom} \\
\textit{Instruction} & \rightarrow & \textbf{id} := \textbf{id}\ \textbf{binop}\ \textit{Atom} \\
\textit{Instruction} & \rightarrow & \textbf{id} := M[\textit{Atom}] \\
\textit{Instruction} & \rightarrow & M[\textit{Atom}] := \textbf{id} \\
\textit{Instruction} & \rightarrow & \texttt{GOTO}\ \textbf{labelid} \\
\textit{Instruction} & \rightarrow & \texttt{IF}\ \textbf{id}\ \textbf{relop}\ \textit{Atom}\ \texttt{THEN}\ \textbf{labelid}\ \texttt{ELSE}\ \textbf{labelid} \\
\textit{Instruction} & \rightarrow & \textbf{id} := \texttt{CALL}\ \textbf{functionid}(\textit{Args}) \\
 & & \\
\textit{Atom} & \rightarrow & \textbf{id} \\
\textit{Atom} & \rightarrow & \textbf{num} \\
 & & \\
\textit{Args} & \rightarrow & \textbf{id} \\
\textit{Args} & \rightarrow & \textbf{id}\ ,\ \textit{Args}
\end{array}
$$

Un programma è una sequenza di istruzioni. Le istruzioni sono:

- Un'etichetta. Questo non ha effetto, ma serve solo a marcare la posizione nel programma come destinazione per i salti.
- Un'assegnazione di un'espressione atomica (costante o variabile) a una variabile.
- Un operatore unario applicato a un'espressione atomica, con il risultato memorizzato in una variabile.
- Un operatore binario applicato a una variabile e a un'espressione atomica, con il risultato memorizzato in una variabile.
- Un trasferimento dalla memoria a una variabile. La posizione in memoria è un'espressione atomica.
- Un trasferimento da una variabile alla memoria. La posizione in memoria è un'espressione atomica.
- Un salto a un'etichetta.
- Una selezione condizionale tra salti a due etichette. La condizione è determinata confrontando una variabile con un'espressione atomica utilizzando un operatore relazionale ($=, \neq, <, >, \leq, \geq$).

- Una chiamata di funzione. Gli argomenti per la chiamata di funzione sono variabili e il risultato è assegnato a una variabile. Questa istruzione è utilizzata anche se non c'è un risultato effettivo (cioè se viene chiamata una procedura anziché una funzione), in questo caso la variabile risultato è una variabile dummy.

Un'espressione atomica è o una variabile o una costante.
Non abbiamo specificato l'insieme di operazioni unarie e binarie, ma ci aspettiamo che queste includano le normali operazioni aritmetiche intere e le operazioni logiche bitwise.
Assumiamo che tutti i valori siano interi. Aggiungere numeri in virgola mobile e altri tipi primitivi non è difficile, però.

Syntax-directed translation

Genereremo codice utilizzando funzioni di traduzione per ogni categoria sintattica. Generiamo codice per una costruzione sintattica indipendentemente dalle costruzioni circostanti, eccetto che i parametri di una funzione di traduzione possono contenere informazioni sul contesto (come tabelle dei simboli) e il risultato di una funzione di traduzione può (oltre al codice generato) contenere informazioni su come il codice generato interagisce con il suo contesto (come quali variabili utilizza). Poiché la traduzione segue strettamente la struttura sintattica del programma, è chiamata **syntax-directed translation** (come abbiamo già visto).

Dato che la traduzione di una costruzione sintattica è per lo più indipendente dalle costruzioni sintattiche circostanti e racchiuse, potremmo perdere opportunità di sfruttare sinergie tra queste e, di conseguenza, generare codice meno ottimale. Si può porre rimedio a questo utilizzando varie tecniche di ottimizzazione.

Esempi

Iniziamo vedendo la Syntax-directed translation delle **espressioni aritmetiche.**
La grammatica è la seguente:

$$S \to id = E$$

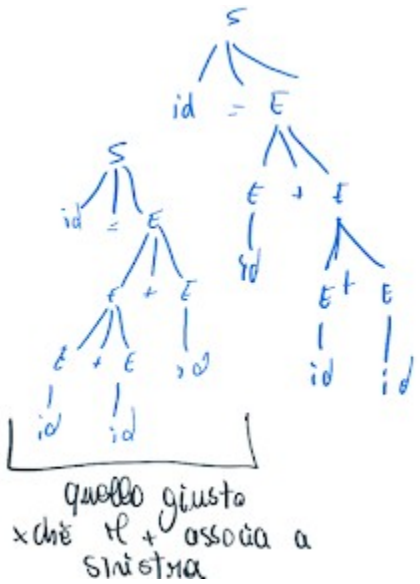
$$E \to E_1 + E_2$$

$$E \to -E_1$$

$$E \to (E_1)$$

$$E \to id$$

La grammatica è ambigua e non è SLR.
Ecco un esempio che dimostra la sua ambiguità:



La SDT delle espressioni mira dunque a generare codice a tre indirizzi.
Per farlo, si utilizzano attributi e funzioni ausiliarie:

- $E.addr$ indica l'indirizzo temporaneo che contiene il valore di E.
- $S.code$, $E.code$ indicano il codice emesso per S ed E.
- $gen(str)$ emette la stringa str.
- $newtemp()$ genera un nuovo indirizzo temporaneo
- Il simbolo ▷ rappresenta la concatenazione di frammenti di codice intermedio (overloaded anche per gestire le etichette).

In sostanza, questo approccio si basa sull'utilizzo di attributi e funzioni per generare il codice a tre indirizzi dalle espressioni, semplificando così il processo di traduzione guidata dalla sintassi.

La SDT è la seguente:

$$S \to id = E \qquad S.code = E.code \triangleright gen(table.get(id)\ '='\ E.addr)$$

$$E \to E_1 + E_2 \qquad E.addr = newtemp()$$
$$E.code = E_1.code \triangleright E_2.code \triangleright gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr)$$

$E \to -E_1$ $\quad E.addr = newtemp()$

$E.code = E_1.code \triangleright$

$gen(E.addr\ '='\ '-'\ E_1.addr)$

$E \to (E_1)$ $\quad E.addr = E_1.addr$

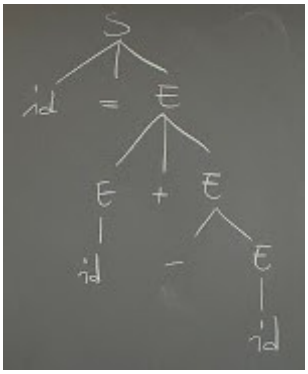
$E.code = E_1.code$

$E \to id$ $\quad E.addr = table.get(id)$

$E.code = ''$

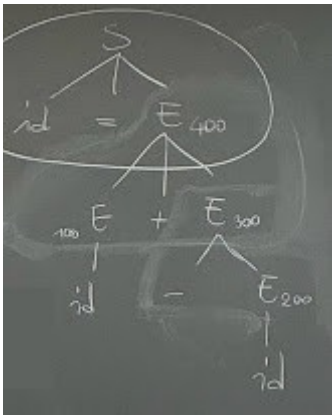
Usiamo gli apici per specificare un simbolo particolare, come il simbolo di uguaglianza, indicando così proprio quel segno in particolare.

La grammatica è ambigua ma ci va bene perché vogliamo mantenerla sintetica e concentrarci sulla generazione del codice.
L'albero per la stringa $a = b + - c$ è il seguente:



Vediamo quale è il codice generato secondo il SDT precedente.

Per distinguere le etichette rinominiamo i nodi E dell'albero nella seguente maniera:



Iniziamo da E_{100}.
Il codice di E_{100} sarà vuoto, l'indirizzo di E_{100} è uguale a b.

- $E_{100}.addr = b$

- $E_{100}.code = ''$

Stessa cosa per E_{200}.

- $E_{200}.addr = c$
- $E_{200}.code = ''$

Vediamo ora cosa fare per E_{300}, notando che per valutarlo abbiamo bisogno di sapere il valore degli attributi di E_{200}.
Per esso devo generare un indirizzo temporaneo $t1$. Il codice di E_{300} è uguale al codice del figlio, in questo caso $E_{200}.code$ (che è vuoto), concatenato con una stringa che è data dall'indirizzo di sé stesso, in questo caso $E_{300}.addr$ (che è $t1$), seguito dal simbolo di uguaglianza $'='$, seguito dal simbolo di sottrazione $'-'$, seguito dall'indirizzo del figlio, in questo caso $E_{200}.addr$ (che è c).

- $E_{300}.addr = t1$
- $E_{300}.code = 't1 = -c'$

Vediamo invece cosa fare per E_{400}, notando che per valutarlo abbiamo bisogno di sapere il valore degli attributi di E_{100} e E_{300}.
Per esso devo generare un indirizzo temporaneo $t2$. Il codice di E_{400} è uguale al codice del figlio sinistro, in questo caso $E_{100}.code$ (che è vuoto), concatenato con il codice del figlio destro, in questo caso $E_{300}.code$ (che è $'t1 = -c'$), concatenato con una stringa che è data dall'indirizzo di se stesso, in questo caso $E_{400}.address$ (che è $t2$), seguito dal simbolo di uguaglianza $'='$, seguito dall'indirizzo del figlio sinistro, in questo caso $E_{100}.address$ (che è b), seguito dal simbolo di somma $'+'$, seguito dall'indirizzo del figlio destro, in questo caso $E_{200}.address$ (che è $t1$).

- $E_{400}.addr = t2$
- $E_{400}.code = 't1 = -c;\ t2 = b + t1'$

Passiamo dunque alla valutazione di S.
Di S non ci interessa l'indirizzo, ma solamente il suo codice.
Il codice è dato da il codice del figlio, in questo caso $E_{400}.code$ (che è $'t1 = -c;\ t2 = b + t1'$), concatenato con una stringa che è data da il nome dell'identificatore, che è a, seguito dal simbolo di uguaglianza $'='$, seguito dall'indirizzo del figlio, in questo caso $E_{400}.address$ (che è $t2$).

- $S.code = 't1 = -c;\ t2 = b + t1;\ a = t2'$

Il codice generato è quello contenuto in S.

Prendiamo ora in considerazione i **control flow statements**.
Le istruzioni di controllo del flusso (control flow statements) sono comandi utilizzati nella programmazione per gestire l'esecuzione sequenziale delle istruzioni nel codice. Queste

istruzioni consentono di controllare l'ordine di esecuzione delle istruzioni, consentendo alla logica del programma di prendere decisioni basate su condizioni o di eseguire azioni ripetute.

I control flow statements, come *if-then*, *if-then-else*, *while-do*, *repeat-until* e *for*, utilizzano valori booleani per determinare quale percorso di esecuzione seguire all'interno di un programma. Se una condizione è vera (true), il programma esegue un blocco di istruzioni specifico; altrimenti, può passare a un altro blocco di istruzioni o continuare il ciclo fino a quando la condizione cambia e diventa vera. In sostanza, questi statement consentono al programma di prendere decisioni e di iterare attraverso il codice in base alle condizioni specificate.

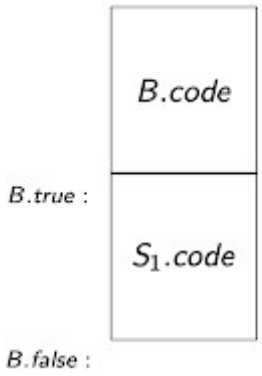
Prendiamo in considerazione la seguente grammatica:

$$
\begin{aligned}
&P \rightarrow S \\
&S \rightarrow S_1 S_2 \\
&S \rightarrow \textit{if } (B) S_1 \\
&S \rightarrow \textit{while } (B)\ S_1 \\
&\ldots \\
&B \rightarrow \textit{true} \\
&B \rightarrow \textit{false} \\
&B \rightarrow \textit{not } B \\
&B \rightarrow E_1 \textit{ rel } E_2 \\
&B \rightarrow B_1 \parallel B_2 \\
&B \rightarrow B_1 \,\&\&\, B_2 \\
&\ldots
\end{aligned}
$$

Dove S indica gli statements e B le espressioni booleane (che vengono utilizzate per decidere il flusso).

I control flow statement vengono convertiti in codice intermedio utilizzando due attributi fondamentali: $B.true$ e $B.false$. Questi agiscono come punti di riferimento (etichette), indicando dove il programma deve dirigere il flusso quando la condizione è rispettivamente vera o falsa.

Prendiamo in considerazione lo statement *if-then*.
Innanzitutto si inserisce tutto il codice necessario per valutare B ($B.code$).
Nella linea successiva a questo blocco, si inserisce l'etichetta $B.true$, punto di partenza per il codice relativo agli statement da eseguire se la condizione è verificata, che chiamiamo $S_1.code$.
Subito dopo, nella linea successiva alla fine del blocco $S_1.code$ viene inserita l'etichetta $B.false$.
Questo consente di saltare il blocco di codice relativo al vero dello statement ($S_1.code$) e proseguire direttamente con l’esecuzione del blocco successivo.



Vediamo ora gli attributi che utilizziamo per i control flow statements:

- **S.next (ereditato)**: Etichetta l'inizio del codice che deve essere eseguito dopo che lo statement S è completato.
  Conservare l'etichetta è utile, perché S potrebbe avere degli altri statements di controllo annidati (ad esempio un ciclo while o una catena di if), per cui è importante poter sempre sapere qual è l'istruzione che deve essere eseguita dopo lo statement padre
- **S.code (sintetizzato)**: È la sequenza di passaggi del codice intermedio che implementa lo statement S e termina con un salto a $S.next$.
- **B.true (ereditato)**: Etichetta l'inizio del codice che deve essere eseguito se la condizione B è vera.
- **B.false (ereditato)**: Etichetta l'inizio del codice che deve essere eseguito se la condizione B è falsa.
- **B.code (sintetizzato)**: È la sequenza di passaggi del codice intermedio (istruzioni a tre indirizzi) che implementa la condizione booleana B e salta a $B.true$ se B è vera, altrimenti a $B.false$.

È da notare che i booleani nelle grammatiche svolgono due ruoli distinti:

- Modificare il flusso di controllo: Vengono utilizzati come condizioni booleane per determinare quale ramo di esecuzione seguire in un programma.
- Calcolare valori logici: Possono essere espressioni booleane, ad esempio, nel lato destro delle assegnazioni.

Ciò che esaminiamo qui è come vengono tradotti nel contesto degli statement di controllo del flusso. Se li vediamo come espressioni, vengono tradotti in modo analogo alle espressioni aritmetiche, ma utilizzando operatori logici invece che aritmetici.
La grammatica può avere non terminali distinti per distinguere questi due ruoli, permettendo una traduzione specifica a seconda dell'uso che ne facciamo in un dato contesto.

Vediamo la SDT per gli statement in questa grammatica:

$$P \to S \qquad S.next = newlabel();$$

$$P.code = S.code \triangleright label(S.next)$$

Dove:

- $newlabel()$: Genera una nuova etichetta ogni volta che viene chiamata.
- $label(L)$: Collega l'etichetta L alla prossima istruzione a tre indirizzi generata.

$S.next$ è l'etichetta che denota la prima istruzione da eseguire quando S è completato.
Se dopo l'esecuzione di alcune istruzioni in S abbiamo bisogno di saltare in un'altra posizione, dobbiamo configurare esplicitamente un'istruzione $GOTO$.
Dunque, il codice relativo a P sarà la conversione dello statement S ($S.code$) seguito dall'etichetta $S.next$.
Notiamo subito che $.next$ è un attributo ereditato (lo stiamo settando durante la produzione riguardante il padre), mentre $.code$ è un attributo sintetizzato.

$$S \rightarrow S_1 S_2 \qquad S_1.next = newlabel()$$
$$S_2.next = S.next$$
$$S.code = S_1.code \triangleright label(S_1.next) \triangleright S_2.code$$

Quando ho la produzione $S \rightarrow S_1 S_2$, allora $S_2.next$ è uguale a $S.next$.
Dunque, sappiamo esattamente dove dirigerci una volta che l'esecuzione di S_2 è terminata perché anche l'esecuzione di S è terminata.
Non sappiamo, però, dove dirigerci al termine dell'esecuzione di S_1. Perciò generiamo una nuova etichetta per $S_1.next$. Questa etichetta la inseriamo alla fine del codice di S_1, cioè nella prima riga del codice di S_2.

$$S \rightarrow if\ (B)\ S_1 \qquad B.true = newlabel()$$
$$B.false = S.next$$
$$S_1.next = S.next$$
$$S.code = B.code \triangleright label(B.true) \triangleright S_1.code$$

Bisogna sapere dove dirigersi quando B è vero e quando B è falso.
Innanzitutto bisogna valutare $B.code$, poi inseriamo subito in cascata $S_1.code$.
Ma ho bisogno di alcuni puntatori per gestire i "goto" per entrambi i casi, vero e falso.
Come osservato precedentemente, sappiamo esattamente dove dirigerci una volta che l'esecuzione di S_1 è terminata perché anche l'esecuzione di S è terminata.

$B.false$ è l'etichetta che utilizzo per dire dove devo andare se B è falso. Poiché non devo eseguire S_1 in questo caso, mi dirigo direttamente dopo la fine del blocco S_1, ovvero $S.next$.
Se B è vero, invece, devo eseguire il codice di S. Quindi, devo creare un'etichetta che inserisco alla prima linea del blocco S_1.

$$
\begin{aligned}
S \rightarrow \textit{while}\ (B)\ S_1 \qquad & B.\textit{true} = \textit{newlabel}() \\
& B.\textit{false} = S.\textit{next} \\
& S_1.\textit{next} = \textit{newlabel}() \\
& S.\textit{code} = \textit{label}(S_1.\textit{next}) \triangleright B.\textit{code} \triangleright \\
& \qquad \textit{label}(B.\textit{true}) \triangleright S_1.\textit{code} \triangleright \\
& \qquad \textit{gen}(\textit{GOTO}\ S_1.\textit{next})
\end{aligned}
$$

Esaminiamo ora il ciclo while, seguendo una strategia simile.
La situazione è un po' più complessa però, perché devo ritornare al controllo della condizione dopo l'esecuzione dello statement.
Iniziamo traducendo il codice del booleano con $B.code$ e il codice dello statement con $S_1.code$.
Posso assumere di avere $S.next$, ovvero l'etichetta della prima riga successiva al blocco S.
Il corpo del while è rappresentato da $S_1.code$. Questo codice lo raggiungo solo se il booleano è vero. Dopo aver completato questo blocco, devo tornare a valutare il booleano.
Dobbiamo definire $S_1.next$ (poiché deve essere definito per ogni statement), quindi inventiamo una nuova etichetta e la posizioniamo all'inizio di $B.code$. Questo è il punto in cui ci reindirizzeremo una volta completato il codice di S_1.
Quando il booleano è falso, ci dirigiamo a $S.next$ (uscendo dal ciclo).
Quando il booleano è vero, devo passare al codice di S_1. Tuttavia, non ho quella specifica etichetta, quindi la creo e la colloco come prima istruzione in $S_1.code$.
Aggiungiamo dunque alla fine di $S_1.code$ l'istruzione $GOTO$ per tornare alla valutazione dell'espressione booleana. Questo $GOTO$ non ha niente a che vedere con i salti condizionali, ma riguarda esclusivamente il ritorno alla verifica della condizione del while una volta finita l'esecuzione dello statement.

Vediamo ora gli SDT per i booleani:

$B \to true$ $\quad B.code = gen(GOTO\ B.true)$

$B \to false$ $\quad B.code = gen(GOTO\ B.false)$

$B \to not\ B_1$ $\quad B_1.true = B.false$
$B_1.false = B.true$
$B.code = B_1.code$

$B \to E_1\ rel\ E_2$ $\quad B.code = E_1.code \triangleright E_2.code \triangleright$
$gen(IF\ E_1.addr\ relop\ E_2.addr\ GOTO\ B.true) \triangleright$
$gen(GOTO\ B.false)$

Per concludere, diamo un'occhiata rapida alla traduzione delle espressioni logiche. Lo faremo andando a confrontare la diversa traduzione di due semplici operazioni di OR e AND:

$B \to B_1 \parallel B_2$ $\quad B_1.true = B.true$
$B_1.false = newlabel()$
$B_2.true = B.true$
$B_2.false = B.false$
$B.code = B_1.code \triangleright$
$label(B_1.false) \triangleright B_2.code$

$B \to B_1 \,\&\&\, B_2$ $\quad B_1.true = newlabel()$
$B_1.false = B.false$
$B_2.true = B.true$
$B_2.false = B.false$
$B.code = B_1.code \triangleright$
$label(B_1.true) \triangleright B_2.code$

Esaminiamo attentamente il codice intermedio di queste due espressioni, analizzandone le similitudini e il modo in cui sfruttano la logica della cortocircuitazione:

- Nell'operatore logico OR, l'etichetta $B_1.true$ punta direttamente all'etichetta $B.true$ dell'intera istruzione, poiché basta che uno degli elementi sia true per rendere vera l'intera espressione. Nel caso dell'AND, invece, viene semplicemente creata una nuova etichetta.
- Similmente, ma in direzione opposta, l'etichetta $B_1.false$ opera in maniera speculare: nel caso dell'AND punta all'etichetta $B.false$ dell'istruzione, mentre nell'OR viene creata una nuova etichetta.
- Procedendo, definiamo le etichette per il secondo elemento delle nostre espressioni nelle righe successive. In entrambi i casi, queste etichette si collegano alle etichette di uscita dell'intera istruzione. Questo avviene perché sono gli ultimi elementi dell'espressione e il loro valore determina il risultato dell'intera espressione.
- La differenza nel codice di $B.code$ risiede quindi nell'etichetta interposta tra il codice del primo e del secondo elemento: nell'OR, infatti, valutiamo il secondo elemento solo se il primo è falso (da qui l'etichetta $B_1.false$); viceversa, nell'AND, consideriamo il secondo elemento solo se il primo è vero, e quindi l'etichetta interposta è $B_1.true$.

Vediamo ora un esempio più pratico della generazione di codice intermedio.
In particolare vediamo la generazione del codice per la seguente stringa

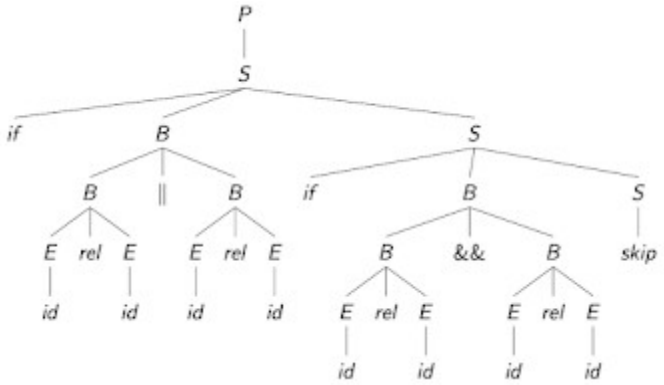
$$if\ ((a = b)\ \|\ (c = d))\ if\ ((e = f)\ \&\&\ (g = h))\ skip$$

Dove lo statement *skip* (che significa "non fare nulla") è stato aggiunto alla grammatica per semplificare la generazione.
La grammatica è quindi estesa con:

$$S \to skip \qquad S.code = ''$$

Che appunto indica che l'azione associata a *skip* è nessuna istruzione a tre indirizzi.
Lo statement *skip* è comunemente utilizzato quando lavoriamo con un linguaggio che comprende solo *if-then-else* (ovvero senza problemi di ambiguità) e immaginiamo di non aver bisogno dell'istruzione *else*. In questo caso, possiamo aggiungere un *else skip*.

Omettendo le foglie per le parentesi, l'albero di derivazione è il seguente:



Vediamo dunque che il programma diventa uno statement, in particolare $if\ \ B\ \ S$: la condizione booleana B è un OR e il body S è a sua volta un if dove il booleano è una congiunzione.

$S.next$, $B.true$ e $B.false$ sono attributi ereditati; $S.code$ e $B.code$ sono attributi sintetizzati.
Le produzioni come $E \to id$ si traducono, come abbiamo visto precedentemente, così

$$E \to id \qquad E.addr = table.get(id)$$

$$E.code = ''$$

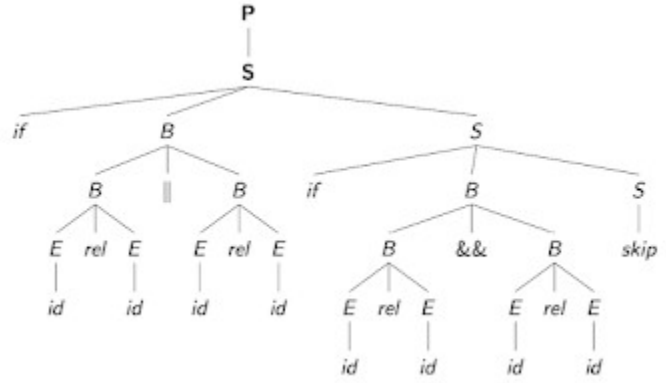
Questo significa che definisco gli indirizzi in base a quanto trovo nella tabella dei simboli. Di fatto, gli indirizzi sono attributi sintetizzati.

Abbiamo quindi sia attributi sintetizzati che ereditati. Abbiamo visto che è facile calcolare quelli sintetizzati attraverso una visita mirata dell'albero. Se la grammatica non è S-attribuita, però, è richiesto un po' più di sforzo, ma se è L-attribuita (come questa), possiamo determinare un ordine topologico per calcolare gli attributi.
Pertanto, intendiamo attraversare l'albero due volte: dall'alto verso il basso per calcolare gli attributi ereditati e dal basso verso l'alto per calcolare quelli sintetizzati.
Proviamo a farlo per

$$if\ ((a = b)\ ||\ (c = d))\ if\ ((e = f)\ \&\&\ (g = h))\ skip$$

Eseguiamo una prima visita dell'albero per calcolare gli attributi ereditati.

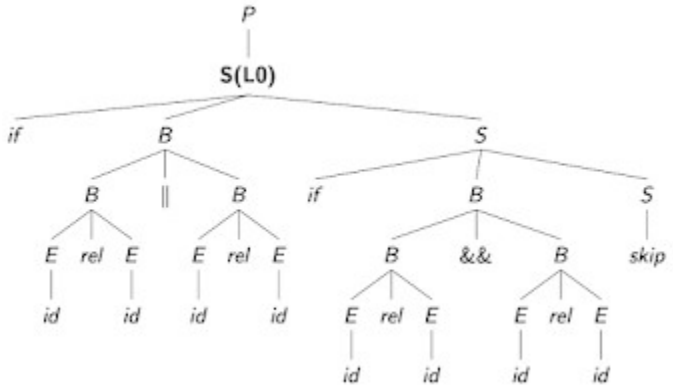
Nel livello più alto dell'albero, c'è la produzione $P \rightarrow S$. Stiamo focalizzando l'attenzione su quei due nodi dell'albero che dipendono dal fatto che la grammatica utilizzi la produzione $P \rightarrow S$.



Per quanto riguarda l'attributo ereditato, la regola afferma

$$S.next = newlabel()$$

richiamando quindi la procedura che genera nuove etichette. Supponiamo che quando applico questa regola, il generatore di etichette restituisca, per esempio, $L0$.
Dopo questa operazione, per quel particolare nodo S in quell'albero specifico, $S.next$ è $L0$.
Manteniamo le informazioni nei nodi e continuiamo la valutazione:



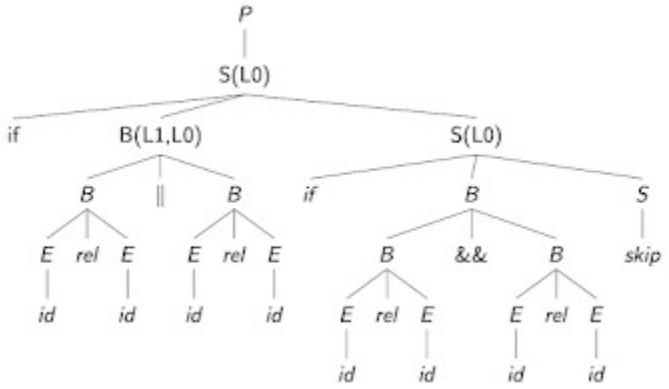
Nel branch di S, la produzione specifica è $S \rightarrow if\ (B)\ \ S_1$.
Qui le regole sono le seguenti:

$$B.true = newlabel();\ B.false = S.next;\ S_1.next = S.next$$

La cui valutazione supponiamo sia (anche in base a quello che abbiamo assegnato a $S.next$):

$$B.true = L1;\ B.false = L0;\ S_1.next = L0$$

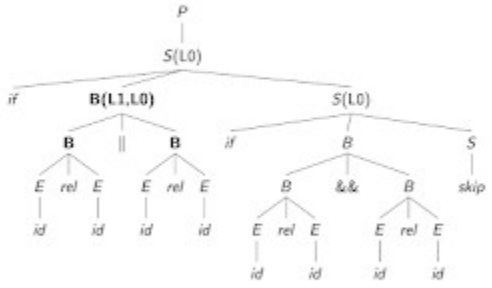
In questo modo otteniamo l'albero seguente



E si continua con questo procedimento fino a che non si arriva in fondo all'albero:

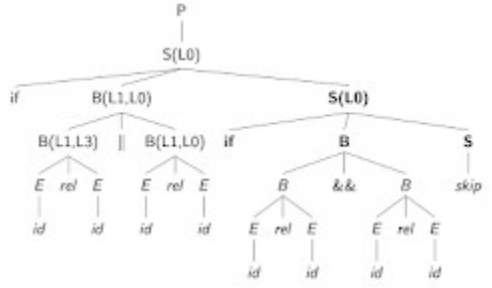
Next production: $B \rightarrow B_1 \parallel B_2$

Relevant rules: $B_1.true = B.true$; $B_1.false = newlabel()$;
$B_2.true = B.true$; $B_2.false = B.false$;



Evaluation:
$B_1.true = L1$; $B_1.false = L3$; $B_2.true = L1$; $B_2.false = L0$
Next production: $S \rightarrow if\ (B)\ S_1$

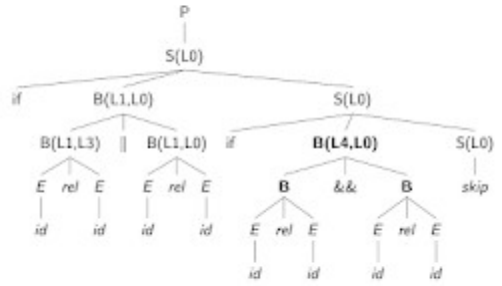
Relevant rules:
$B.true = newlabel()$; $B.false = S.next$; $S_1.next = S.next$



Evaluation:
$B.true = L4$; $B.false = L0$; $S_1.next = L0$

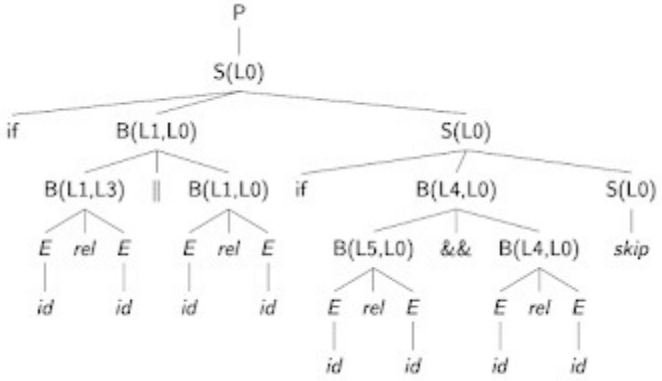
Next production: $B \rightarrow B_1 \;\&\&\; B_2$

Relevant rules: $B_1.true = newlabel()$; $B_1.false = B.false$;
$B_2.true = B.true$; $B_2.false = B.false$;



Evaluation:
$B_1.true = L5$; $B_1.false = L0$; $B_2.true = L4$; $B_2.false = L0$;

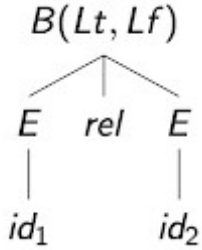
Ottenendo alla fine il seguente risultato per gli attributi ereditati:



NOTA: Nel complesso della grammatica, $B.true$ e $B.false$ sono attributi ereditati, poiché la loro natura come sintetizzati o ereditati dipende dalla specifica situazione, come nel caso di operatori logici come "or" e "and". Affermare che siano sintetizzati richiede una verifica preliminare: è necessario confermare che in tutte le produzioni semantiche, questi attributi siano sempre trattati come attributi sintetizzati.

Passiamo ora alla valutazione (generazione) del codice, tramite una visita bottom-up e utilizzando un approccio divide-et-impera.

Per ogni sotto-albero che segue questa struttura



Secondo la regola associata alla produzione $B \rightarrow E \;\; rel \;\; E$, ovvero

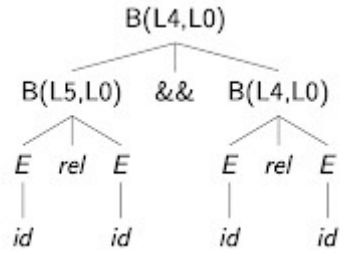
$$B \rightarrow E_1 \; rel \; E_2 \qquad B.code = E_1.code \triangleright E_2.code \triangleright gen(IF \; E_1.addr \; relop \; E_2.addr \; GOTO \; B.true) \triangleright gen(GOTO \; B.false)$$

otteniamo

$$IF \; id_1 \; relop \; id_2 \; GOTO \; Lt$$
$$GOTO \; Lf$$

Ricordando quanto detto in precedenza, ovvero che quando un'espressione generica E è un semplice identificatore, il codice relativo a E è vuoto.

Nel caso dell'albero rappresentato qui sotto



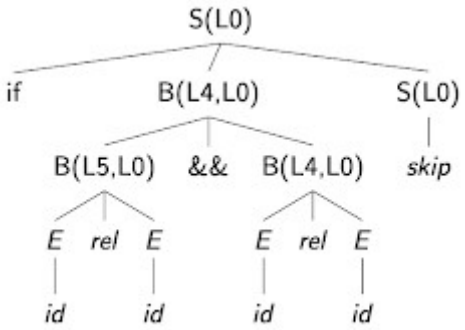
esaminiamo come è definito l'attributo $B.code$ considerando la regola semantica legata alla congiunzione, ossia

$$B.code = B_1.code \triangleright label(B_1.true) \triangleright B_2.code$$

Secondo questa regola, il codice generato per l'espressione B che si trova alla radice del sottoalbero illustrato precedentemente è il codice di B_1, ossia il codice relativo al sottoalbero sinistro di $B(L5, L0)$, seguito dall'etichetta $B_1.true$ e infine dal codice di B_2.
Otteniamo quindi

$$IF \; e = f \; GOTO \; L5$$
$$GOTO \; L0$$
$$L5 : IF \; g = h \; GOTO \; L4$$
$$GOTO \; L0$$

Salendo verso l'alto da questo insieme di nodi abbiamo il seguente sottoalbero



dunque esaminiamo le regole semantiche per generare il codice in un'istruzione di "if":

$$S.code = B.code \triangleright label(B.true) \triangleright S_1.code$$

Otteniamo quindi

$$\begin{aligned} & IF\ e = f\ GOTO\ L5 \\ & GOTO\ L0 \\ L5 : & IF\ g = h\ GOTO\ L4 \\ & GOTO\ L0 \\ L4 : & \end{aligned}$$

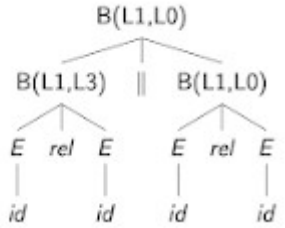
È da notare che, in questa situazione, $S_1.code$ corrisponde a uno "skip", quindi è nullo.

Continuiamo così:

For

$$(a = b) \parallel (c = d)$$

By the disjunction-rule, we generate
$B_1.code \triangleright label(B_1.false) \triangleright B_2.code$

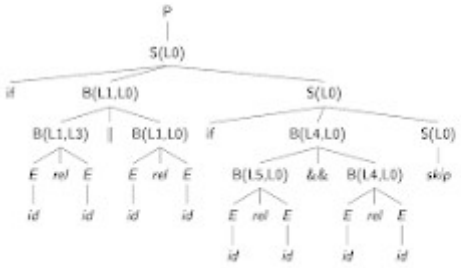


$$\begin{aligned} & IF\ a = b\ GOTO\ L1 \\ & GOTO\ L3 \\ L3 : & IF\ c = d\ GOTO\ L1 \\ & GOTO\ L0 \end{aligned}$$

E alla fine otteniamo:

For all of it

$$if\ ((a = b) \parallel (c = d))\ if\ ((e = f)\ \&\&\ (g = h))\ skip$$



$$\begin{aligned} & IF\ a = b\ GOTO\ L1 \\ & GOTO\ L3 \\ L3 : & IF\ c = d\ GOTO\ L1 \\ & GOTO\ L0 \\ L1 : & IF\ e = f\ GOTO\ L5 \\ & GOTO\ L0 \\ L5 : & IF\ g = h\ GOTO\ L4 \\ & GOTO\ L0 \\ L0, L4 : & \end{aligned}$$

Però, quando scriviamo codice, è importante considerare che, a causa dell'ottimizzazione eseguita dal compilatore, la sequenza di condizioni che scriviamo non garantisce necessariamente che vengano controllate nell'ordine in cui le abbiamo scritte. Il compilatore potrebbe ottimizzare il codice valutando prima altri parametri che sono più rapidi da verificare, indipendentemente dall'ordine in cui sono stati inseriti nel codice sorgente.

L'aspetto cruciale è questo: qualsiasi percorso seguiamo nell'albero, arriviamo inevitabilmente a $L0$ e $L4$, dove non c'è codice da eseguire. Questo è il punto in cui dovrebbe iniziare il codice di un'altra istruzione, successiva ai nostri "if". Ciò che rende interessante questo fatto è che tutti questi punti di $GOTO$ che abbiamo generato puntano sempre a $L0$ e $L4$, quindi un ottimizzatore di solito li elimina completamente. In altre parole, utilizzare un "if" o evitarlo ci conduce sempre allo stesso punto, senza influire sul programma.

L'approccio di creare un ordinamento topologico, calcolare gli attributi ereditati e quindi eseguire le azioni semantiche è esattamente ciò che abbiamo fatto in questo esercizio (in cui non abbiamo effettivamente calcolato l'ordinamento topologico, poiché le azioni lo permettevano). Questo approccio ci ha comunque condotto a L0 e L4. È il metodo più efficiente per associare gli attributi all'albero di derivazione ottenuto durante l'analisi sintattica.

Esistono varie ottimizzazioni che possono essere applicate al tipo di codice generato.
Prendendo in considerazione il nostro codice generato, "GOTO L3" con L3 che rappresenta l'etichetta della riga successiva è un'azione inutile (ridondante).
La semantica arricchita che abbiamo usato tende a generare questi $GOTO$, che in realtà non vogliamo.

Un'ottimizzazione semplice si ottiene togliendo i $GOTO$ ridondanti, per fare questo bisogna invertire i controlli sul vero con quelli sul falso, come nel seguente esempio:

```
     IF x < 100 GOTO L2
     GOTO L3
L3:  IF x > 200 GOTO L4
     GOTO L1
L4:  IF x != y GOTO L2
     GOTO L1
L2:  x = 0
L1:
```

```
     IF x < 100 GOTO L2
     IFFALSE x > 200 GOTO L1
     IFFALSE x != y GOTO L1
L2:  x = 0
L1:
```

Come si può notare, questo nuovo approccio genera un codice molto più efficiente, riducendo sia il numero di istruzioni $GOTO$ che il numero totale di righe di codice (possiamo organizzare la traduzione per ottenere direttamente il codice a destra).

Per rendere il codice più compatto, si adotta una tecnica chiamata **tecnica di fall**.
Quest'ultima mira ad evitare l'uso di istruzioni $GOTO$ per raggiungere la linea successiva, dove il program counter si sposta di default.
Questa tecnica comporta una variazione nella tipologia delle regole semantiche che abbiamo precedentemente esaminato: sotto l'ipotesi che se la condizione booleana è vera, allora l'istruzione successiva da eseguire è sempre quella successiva nella sequenza, possiamo riformulare la SDD per

$$P \to S$$

$$S \to \mathit{if}\ (B) S_1 \mid \mathit{while}\ (B)\ S_1$$

...

$$B \to \mathit{true} \mid \mathit{false} \mid \mathit{not}\ B \mid E_1\ \mathit{rel}\ E_2 \mid B_1 \parallel B_2 \mid B_1\ \&\&\ B_2$$

...

utilizzando un'etichetta speciale chiamata $fall$, la quale non genera $GOTO$ (ma passa direttamente all'istruzione successiva nella sequenza).

L'impiego del "fall" è chiaro sin dalle prime produzioni. Se B è vero, attuiamo un "fall" (ossia passiamo (cadiamo) alla riga successiva), altrimenti, con B falso, andiamo al "next". Per il resto, tutto rimane invariato come precedentemente.

$P \to S$	$S.next = newlabel();$ $P.code = S.code \triangleright label(S.next)$
$S \to \mathit{if}\ (B)\ S_1$	$B.true = fall$ $B.false = S.next$ $S_1.next = S.next$ $S.code = B.code \triangleright S_1.code$
$S \to \mathit{while}\ (B)\ S_1$	$B.true = fall$ $B.false = S.next$ $S_1.next = newlabel()$ $S.code =$ $label(S_1.next) \triangleright B.code \triangleright$ $S_1.code \triangleright gen(GOTO\ S_1.next)$
$B \to \mathit{true}$	$\mathit{if}\ B.true \neq fall$ $\mathit{then}\ gen(GOTO\ B.true)$
$B \to \mathit{false}$	$gen(GOTO\ B.false)$
$B \to \mathit{not}\ B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \to E_1\ \mathit{rel}\ E_2$	$B.code = E_1.code \triangleright E_2.code \triangleright st$

Where, for $test = E_1.addr\ relop\ E_2.addr$

st = $\mathit{if}\ B.true \neq fall\ \mathit{and}\ B.false \neq fall$
$\mathit{then}\ gen(IF\ test\ GOTO\ B.true) \triangleright gen(GOTO\ B.false)$
$\mathit{elseif}\ B.true \neq fall$
$\mathit{then}\ gen(IF\ test\ GOTO\ B.true)$
$\mathit{elseif}\ B.false \neq fall$
$\mathit{then}\ gen(IFFALSE\ test\ GOTO\ B.false)$
$\mathit{else}\ skip$

$B \rightarrow B_1 \parallel B_2$

$B_1.true = \mathit{if}\ B.true \neq \mathit{fall}$
$\quad \mathit{then}\ B.true$
$\quad \mathit{else}\ \mathit{newlabel}()$
$B_1.false = \mathit{fall}$
$B_2.true = B.true$
$B_2.false = B.false$
$B.code =$
$\quad \mathit{if}\ B.true \neq \mathit{fall}$
$\quad \mathit{then}\ B_1.code \triangleright B_2.code$
$\quad \mathit{else}\ B_1.code \triangleright B_2.code \triangleright \mathit{label}(B_1.true)$

$B \rightarrow B_1 \,\&\&\, B_2$

$B_1.true = \mathit{fall}$
$B_1.false = B.false$
$B_2.true = B.true$
$B_2.false = B.false$
$B.code = B_1.code \triangleright B_2.code$